

# Sistemi Operativi

Guida alla prova di teoria

Teoria, esercizi svolti e colloquio orale

Corso di Laurea Triennale in Informatica — Università di Catania

Canale A–L, a.a. 2025–2026

*Questa guida copre la prova di **teoria** del corso di Sistemi Operativi: i venti argomenti del programma d'esame, il compito tipo dell'appello svolto per intero, gli esercizi dello stesso tipo con i numeri già calcolati, e le domande del colloquio orale con la risposta già impaginata come discorso.*

**Com'è organizzata:**

- **Parte I — Il sistema operativo:** che cos'è un sistema operativo, l'hardware su cui poggia, come è strutturato al suo interno.
- **Parte II — Processi e thread:** il concetto di processo, gli stati e le transizioni, i modelli di thread.
- **Parte III — Concorrenza e sincronizzazione:** sezioni critiche, Peterson, TSL, semafori, monitor, e i due problemi classici dei filosofi e dei lettori-scrittori.
- **Parte IV — Scheduling della CPU:** scheduler e dispatcher, tutti gli algoritmi del programma, thread e multi-processore.
- **Parte V — Gestione della memoria:** swapping, rilocalizzazione, paginazione, memoria virtuale, algoritmi di sostituzione, working set.
- **Parte VI — File system e dischi:** file e directory, allocazione, consistenza e journaling, scheduling del disco, RAID, SSD.
- **Parte VII — Esercizi e colloquio:** il compito tipo svolto, le varianti, le domande a risposta multipla, le domande d'orale.

**Da dove vengono i contenuti.** I capitoli di teoria seguono le proiezioni del corso. Le figure sono ritagliate dalle proiezioni stesse a 300 dpi e ogni didascalia dichiara la slide di provenienza. Gli esercizi sono di due tipi, e ognuno lo dice: quelli marcati come *domanda reale* vengono dal compito tipo pubblicato dal docente, quelli marcati come *costruiti* seguono lo stesso modello ma con numeri diversi, scelti per coprire i casi che il compito tipo non toccava. I numeri degli esercizi sono prodotti e verificati da uno script, non ricopiati a mano.

# Indice

|           |                                                                          |           |
|-----------|--------------------------------------------------------------------------|-----------|
| <b>I</b>  | <b>Il sistema operativo</b>                                              | <b>1</b>  |
|           | Come usare questa guida                                                  | 2         |
|           | Com'e' fatta la prova di teoria                                          | 3         |
| <b>1</b>  | <b>Introduzione al concetto di sistema operativo</b>                     | <b>5</b>  |
| 1.1       | Perché serve un sistema operativo . . . . .                              | 5         |
| 1.2       | La doppia modalità di esecuzione . . . . .                               | 5         |
| 1.3       | L'evoluzione dei sistemi operativi . . . . .                             | 6         |
| 1.4       | Lo zoo dei sistemi operativi . . . . .                                   | 6         |
| <b>2</b>  | <b>Richiami sull'architettura degli elaboratori</b>                      | <b>8</b>  |
| 2.1       | Architettura di un calcolatore . . . . .                                 | 8         |
| 2.2       | Il processore . . . . .                                                  | 8         |
| 2.3       | Modalità di esecuzione e chiamate di sistema . . . . .                   | 8         |
| 2.4       | Più processori . . . . .                                                 | 9         |
| 2.5       | Le memorie . . . . .                                                     | 9         |
| 2.6       | I dispositivi di I/O . . . . .                                           | 10        |
| 2.6.1     | Le tre modalità di I/O . . . . .                                         | 10        |
| <b>3</b>  | <b>Struttura di un sistema operativo</b>                                 | <b>12</b> |
| 3.1       | Struttura monolitica . . . . .                                           | 12        |
| 3.2       | Struttura a livelli (o a strati) . . . . .                               | 12        |
| 3.3       | Microkernel . . . . .                                                    | 13        |
| 3.4       | Struttura a moduli . . . . .                                             | 13        |
| 3.5       | Macchine virtuali . . . . .                                              | 13        |
| <b>II</b> | <b>Processi e thread</b>                                                 | <b>15</b> |
| <b>4</b>  | <b>I processi: definizione, multiprogrammazione, stati e transizioni</b> | <b>16</b> |
| 4.1       | Che cos'è un processo . . . . .                                          | 16        |
| 4.2       | Il modello dei processi . . . . .                                        | 16        |
| 4.3       | Creazione e terminazione . . . . .                                       | 17        |
| 4.3.1     | Come nasce un processo . . . . .                                         | 17        |
| 4.3.2     | Come muore un processo . . . . .                                         | 17        |
| 4.4       | Gli stati di un processo . . . . .                                       | 18        |
| 4.5       | Il passaggio da un processo all'altro . . . . .                          | 18        |
| 4.6       | Code e accodamento . . . . .                                             | 19        |
| <b>5</b>  | <b>I thread: definizione, modelli, programmazione multicore</b>          | <b>20</b> |
| 5.1       | Perché servono i thread . . . . .                                        | 20        |
| 5.2       | Le operazioni sui thread . . . . .                                       | 20        |

|            |                                                            |           |
|------------|------------------------------------------------------------|-----------|
| 5.3        | Programmazione multicore . . . . .                         | 21        |
| 5.3.1      | I cinque principi della programmazione multicore . . . . . | 21        |
| 5.4        | I modelli di thread . . . . .                              | 21        |
| 5.4.1      | Thread a livello utente ( <i>1-a-molti</i> ) . . . . .     | 22        |
| 5.4.2      | Thread a livello kernel ( <i>1-a-1</i> ) . . . . .         | 22        |
| 5.4.3      | Modello ibrido ( <i>molti-a-molti</i> ) . . . . .          | 22        |
| 5.5        | I thread nei sistemi operativi reali . . . . .             | 22        |
| <b>III</b> | <b>Concorrenza e sincronizzazione</b>                      | <b>24</b> |
| <b>6</b>   | <b>Sezioni critiche e mutua esclusione</b>                 | <b>25</b> |
| 6.1        | Perché i processi devono cooperare . . . . .               | 25        |
| 6.2        | Sezioni critiche e le quattro condizioni . . . . .         | 25        |
| 6.3        | Soluzioni con attesa attiva . . . . .                      | 26        |
| 6.3.1      | Disabilitare gli interrupt . . . . .                       | 26        |
| 6.3.2      | Variabili di lock . . . . .                                | 26        |
| 6.3.3      | Alternanza stretta . . . . .                               | 26        |
| 6.3.4      | La soluzione di Peterson . . . . .                         | 27        |
| 6.3.5      | Le istruzioni TSL e XCHG . . . . .                         | 27        |
| 6.4        | Bloccare invece di girare a vuoto . . . . .                | 28        |
| 6.4.1      | Sleep e wakeup . . . . .                                   | 28        |
| 6.4.2      | I semafori . . . . .                                       | 29        |
| 6.4.3      | Produttore-consumatore con i semafori . . . . .            | 29        |
| 6.4.4      | Mutex e thread utente . . . . .                            | 30        |
| 6.4.5      | I futex . . . . .                                          | 30        |
| 6.5        | I monitor . . . . .                                        | 31        |
| 6.5.1      | Le tre semantiche della signal . . . . .                   | 31        |
| 6.5.2      | Produttore-consumatore con i monitor . . . . .             | 32        |
| 6.6        | Scambio di messaggi tra processi . . . . .                 | 32        |
| <b>7</b>   | <b>I problemi classici: filosofi e lettori-scrittori</b>   | <b>34</b> |
| 7.1        | Il problema dei cinque filosofi . . . . .                  | 34        |
| 7.1.1      | Il problema . . . . .                                      | 34        |
| 7.1.2      | Le quattro soluzioni discusse nelle slides . . . . .       | 34        |
| 7.1.3      | La soluzione con i semafori . . . . .                      | 35        |
| 7.1.4      | La soluzione con i monitor . . . . .                       | 36        |
| 7.2        | Il problema dei lettori e scrittori . . . . .              | 37        |
| 7.2.1      | Il problema . . . . .                                      | 37        |
| 7.2.2      | La soluzione con i semafori . . . . .                      | 37        |
| 7.2.3      | Le tre soluzioni con i monitor . . . . .                   | 38        |
| <b>IV</b>  | <b>Scheduling della CPU</b>                                | <b>41</b> |
| <b>8</b>   | <b>Scheduler e dispatcher</b>                              | <b>42</b> |
| 8.1        | Che cosa fa lo scheduler . . . . .                         | 42        |
| 8.1.1      | Due tipi di processo . . . . .                             | 42        |
| 8.1.2      | Quando viene attivato lo scheduler . . . . .               | 42        |
| 8.2        | Il dispatcher . . . . .                                    | 43        |
| 8.3        | Gli obiettivi di un algoritmo di scheduling . . . . .      | 43        |
| 8.3.1      | Gli obiettivi comuni . . . . .                             | 43        |

|           |                                                            |           |
|-----------|------------------------------------------------------------|-----------|
| 8.3.2     | Gli obiettivi tipici dei sistemi batch . . . . .           | 43        |
| 8.3.3     | Gli obiettivi tipici dei sistemi interattivi . . . . .     | 44        |
| 8.3.4     | Gli obiettivi tipici dei sistemi real-time . . . . .       | 44        |
| <b>9</b>  | <b>Gli algoritmi di scheduling</b>                         | <b>46</b> |
| 9.1       | Gli algoritmi dei sistemi batch . . . . .                  | 46        |
| 9.1.1     | FCFS — First-Come First-Served . . . . .                   | 46        |
| 9.1.2     | SJF — Shortest Job First . . . . .                         | 46        |
| 9.1.3     | SRTN — Shortest Remaining Time Next . . . . .              | 47        |
| 9.1.4     | Il confronto completo sulle slides . . . . .               | 47        |
| 9.2       | Gli algoritmi dei sistemi interattivi . . . . .            | 49        |
| 9.2.1     | RR — Round-Robin . . . . .                                 | 49        |
| 9.2.2     | Scheduling a priorità . . . . .                            | 50        |
| 9.2.3     | Variante: lo scheduling a code multiple . . . . .          | 50        |
| 9.2.4     | SPN — Shortest Process Next . . . . .                      | 51        |
| 9.2.5     | Scheduling garantito . . . . .                             | 52        |
| 9.2.6     | Scheduling a lotteria . . . . .                            | 52        |
| 9.2.7     | Scheduling fair-share . . . . .                            | 52        |
| <b>10</b> | <b>Scheduling dei thread e su sistemi multi-processore</b> | <b>54</b> |
| 10.1      | Lo scheduling dei thread . . . . .                         | 54        |
| 10.1.1    | I thread utente . . . . .                                  | 54        |
| 10.1.2    | I thread del kernel . . . . .                              | 54        |
| 10.2      | Lo scheduling su sistemi multi-processore . . . . .        | 55        |
| 10.2.1    | I due approcci di base . . . . .                           | 55        |
| 10.2.2    | Le politiche di scheduling . . . . .                       | 55        |
| 10.3      | Che cosa usano i sistemi operativi reali . . . . .         | 56        |
| <b>V</b>  | <b>Gestione della memoria</b>                              | <b>58</b> |
| <b>11</b> | <b>Gestione della memoria per la multi-programmazione</b>  | <b>59</b> |
| 11.1      | Memoria centrale e processi . . . . .                      | 59        |
| 11.2      | Senza alcuna astrazione . . . . .                          | 59        |
| 11.3      | Multiprogrammazione senza astrazione . . . . .             | 59        |
| 11.3.1    | Il problema della rilocazione . . . . .                    | 59        |
| 11.3.2    | Il problema della protezione . . . . .                     | 60        |
| 11.4      | Lo spazio degli indirizzi . . . . .                        | 60        |
| 11.5      | Lo swapping . . . . .                                      | 61        |
| 11.5.1    | Le strategie di allocazione e la frammentazione . . . . .  | 61        |
| 11.6      | La gestione dell'allocazione . . . . .                     | 62        |
| 11.6.1    | La bitmap . . . . .                                        | 62        |
| 11.6.2    | Le liste dei blocchi liberi e occupati . . . . .           | 62        |
| 11.6.3    | Gli algoritmi di ricerca . . . . .                         | 62        |
| <b>12</b> | <b>La memoria virtuale</b>                                 | <b>64</b> |
| 12.1      | Da contigua a non contigua . . . . .                       | 64        |
| 12.2      | La paginazione . . . . .                                   | 65        |
| 12.2.1    | La traduzione degli indirizzi . . . . .                    | 65        |
| 12.2.2    | La voce della tabella delle pagine . . . . .               | 65        |
| 12.2.3    | La tabella dei frame . . . . .                             | 66        |
| 12.3      | Progettare una tabella delle pagine . . . . .              | 66        |

|           |                                                   |           |
|-----------|---------------------------------------------------|-----------|
| 12.3.1    | La memoria associativa (TLB)                      | 67        |
| 12.3.2    | Effective Access Time (EAT)                       | 67        |
| 12.3.3    | La tabella delle pagine multilivello              | 68        |
| 12.3.4    | La tabella delle pagine invertita                 | 69        |
| 12.3.5    | Cache della memoria e memoria virtuale            | 69        |
| <b>13</b> | <b>Gli algoritmi di sostituzione delle pagine</b> | <b>71</b> |
| 13.1      | Il problema                                       | 71        |
| 13.1.1    | La soluzione ottimale (OPT)                       | 71        |
| 13.2      | Algoritmo NRU (Not Recently Used)                 | 71        |
| 13.3      | FIFO e seconda chance                             | 72        |
| 13.4      | Algoritmo Clock                                   | 73        |
| 13.5      | Algoritmo LRU                                     | 73        |
| 13.6      | Algoritmo NFU                                     | 74        |
| 13.7      | L'algoritmo di Aging                              | 74        |
| 13.8      | Il confronto delle prestazioni                    | 75        |
| 13.8.1    | L'anomalia di Belady                              | 75        |
| 13.8.2    | Chi soffre dell'anomalia                          | 76        |
| 13.9      | Riepilogo sugli algoritmi                         | 76        |
| <b>14</b> | <b>Altri aspetti della gestione della memoria</b> | <b>78</b> |
| 14.1      | L'allocazione dei frame                           | 78        |
| 14.1.1    | Le strategie di allocazione                       | 78        |
| 14.1.2    | Allocazione locale e globale                      | 79        |
| 14.1.3    | Thrashing e sovraccarico                          | 79        |
| 14.2      | Il working set                                    | 80        |
| 14.2.1    | Il calcolo in pratica                             | 81        |
| 14.3      | La Page Fault Frequency                           | 81        |
| 14.4      | La politica di pulitura                           | 82        |
| 14.5      | La dimensione della pagina                        | 83        |
| 14.6      | Le pagine condivise                               | 84        |
| 14.6.1    | Le difficoltà: la cache                           | 85        |
| 14.6.2    | Le difficoltà: la tabella delle pagine invertita  | 86        |
| 14.7      | Copy-on-write e zero-fill-on-demand               | 87        |
| 14.8      | Librerie condivise e file mappati                 | 89        |
| 14.9      | L'allocazione della memoria per il kernel         | 90        |
| 14.9.1    | Lo slab allocator                                 | 90        |
| <b>VI</b> | <b>File system e dischi</b>                       | <b>93</b> |
| <b>15</b> | <b>I file system</b>                              | <b>94</b> |
| 15.1      | Il problema di base                               | 94        |
| 15.1.1    | L'astrazione: file e directory                    | 94        |
| 15.1.2    | I lock sui file                                   | 95        |
| 15.2      | La struttura di un file system                    | 96        |
| <b>16</b> | <b>Progettazione di un file system</b>            | <b>98</b> |
| 16.1      | L'allocazione dei file                            | 98        |
| 16.1.1    | Allocazione contigua                              | 98        |
| 16.1.2    | Allocazione concatenata                           | 99        |
| 16.1.3    | Allocazione tabellare: la FAT                     | 99        |

|            |                                                                      |            |
|------------|----------------------------------------------------------------------|------------|
| 16.1.4     | Allocazione indicizzata: gli i-node . . . . .                        | 101        |
| 16.2       | L'implementazione delle directory . . . . .                          | 102        |
| 16.3       | La condivisione di file . . . . .                                    | 105        |
| 16.4       | La gestione dei blocchi liberi . . . . .                             | 106        |
| <b>17</b>  | <b>Consistenza, journaling e cache</b>                               | <b>110</b> |
| 17.1       | I controlli di consistenza . . . . .                                 | 110        |
| 17.2       | Il journaling . . . . .                                              | 111        |
| 17.3       | La cache del disco . . . . .                                         | 112        |
| 17.4       | Cosa usano i nostri sistemi operativi . . . . .                      | 114        |
| <b>18</b>  | <b>Scheduling del disco</b>                                          | <b>116</b> |
| 18.1       | Il problema . . . . .                                                | 116        |
| 18.2       | L'esempio di riferimento . . . . .                                   | 118        |
| 18.3       | First Come First Served . . . . .                                    | 118        |
| 18.4       | Shortest Seek Time First . . . . .                                   | 120        |
| 18.5       | Scheduling per scansione (algoritmo dell'ascensore) . . . . .        | 121        |
| 18.6       | Scheduling per scansione circolare . . . . .                         | 122        |
| 18.7       | Che cosa scegliere . . . . .                                         | 124        |
| 18.8       | La verifica: i numeri delle slides . . . . .                         | 125        |
| 18.9       | Riepilogo . . . . .                                                  | 126        |
| <b>19</b>  | <b>I sistemi RAID</b>                                                | <b>128</b> |
| 19.1       | L'idea: più dischi, visti come uno . . . . .                         | 128        |
| 19.2       | Il problema che lo striping crea . . . . .                           | 129        |
| 19.3       | RAID 0: striping puro . . . . .                                      | 130        |
| 19.4       | RAID 1: mirroring . . . . .                                          | 131        |
| 19.5       | RAID 2: striping a livello di bit con ECC . . . . .                  | 132        |
| 19.6       | RAID 3: striping a livello di bit con parità . . . . .               | 133        |
| 19.7       | RAID 4: striping a blocchi con XOR sull'ultimo disco . . . . .       | 133        |
| 19.8       | RAID 5: parità distribuita . . . . .                                 | 135        |
| 19.9       | Riepilogo dei livelli . . . . .                                      | 136        |
| 19.10      | La verifica . . . . .                                                | 136        |
| <b>20</b>  | <b>Memorie flash e dischi a stato solido</b>                         | <b>138</b> |
| 20.1       | Che cosa sono . . . . .                                              | 138        |
| 20.2       | Perché i file system classici non vanno bene . . . . .               | 139        |
| 20.3       | Garbage collection e TRIM . . . . .                                  | 140        |
| <b>VII</b> | <b>Esercizi e colloquio orale</b>                                    | <b>142</b> |
| <b>21</b>  | <b>Il compito tipo svolto</b>                                        | <b>143</b> |
| 21.1       | Domanda 1 — l'aggiornamento del contatore di Aging . . . . .         | 143        |
| 21.2       | Domanda 2 — l'affermazione falsa sull'allocazione dei file . . . . . | 145        |
| 21.3       | Domanda 3 — quanti frame fisici . . . . .                            | 147        |
| 21.4       | Domanda 4 — la traccia dei semafori . . . . .                        | 148        |
| 21.5       | Domanda 5 — un offset dentro un file su FAT . . . . .                | 150        |
| 21.6       | Domanda 6 — gli offset dentro un i-node . . . . .                    | 151        |
| 21.7       | Domanda 7 — lo scheduling del disco . . . . .                        | 154        |
| 21.8       | Che cosa insegna questo compito . . . . .                            | 155        |
| <b>22</b>  | <b>Varianti: gli stessi esercizi con numeri nuovi</b>                | <b>157</b> |

|           |                                                                             |            |
|-----------|-----------------------------------------------------------------------------|------------|
| 22.1      | Memoria virtuale . . . . .                                                  | 157        |
| 22.2      | Effective Access Time . . . . .                                             | 158        |
| 22.3      | Trace di semafori . . . . .                                                 | 159        |
| 22.4      | FAT . . . . .                                                               | 160        |
| 22.5      | i-node . . . . .                                                            | 161        |
| 22.6      | Scheduling del disco . . . . .                                              | 162        |
| 22.7      | Sostituzione delle pagine . . . . .                                         | 163        |
| 22.8      | RAID . . . . .                                                              | 164        |
| 22.9      | Come usare queste varianti . . . . .                                        | 165        |
| <b>23</b> | <b>Domande a risposta multipla</b>                                          | <b>166</b> |
| 23.1      | Parte I — Il sistema operativo e l'hardware . . . . .                       | 166        |
| 23.1.1    | Capitolo 1 — Introduzione al concetto di sistema operativo . . . . .        | 166        |
| 23.1.2    | Capitolo 2 — Richiami sull'architettura degli elaboratori . . . . .         | 167        |
| 23.1.3    | Capitolo 3 — Struttura di un sistema operativo . . . . .                    | 168        |
| 23.1.4    | Capitolo 4 — I processi . . . . .                                           | 168        |
| 23.1.5    | Capitolo 5 — I thread . . . . .                                             | 169        |
| 23.2      | Parte II — Processi, thread e sincronizzazione . . . . .                    | 170        |
| 23.2.1    | Capitolo 6 — Sezioni critiche e mutua esclusione . . . . .                  | 170        |
| 23.2.2    | Capitolo 7 — I problemi classici . . . . .                                  | 171        |
| 23.3      | Parte III — Scheduling . . . . .                                            | 172        |
| 23.3.1    | Capitolo 8 — Scheduler e dispatcher . . . . .                               | 172        |
| 23.3.2    | Capitolo 9 — Gli algoritmi di scheduling . . . . .                          | 172        |
| 23.3.3    | Capitolo 10 — Scheduling dei thread e su sistemi multi-processore . . . . . | 173        |
| 23.4      | Parte IV — La memoria . . . . .                                             | 174        |
| 23.4.1    | Capitolo 11 — Gestione della memoria . . . . .                              | 174        |
| 23.4.2    | Capitolo 12 — La memoria virtuale . . . . .                                 | 175        |
| 23.4.3    | Capitolo 13 — Gli algoritmi di sostituzione delle pagine . . . . .          | 175        |
| 23.4.4    | Capitolo 14 — Altri aspetti della gestione della memoria . . . . .          | 176        |
| 23.5      | Parte V — I file system . . . . .                                           | 177        |
| 23.5.1    | Capitolo 15 — I file system . . . . .                                       | 177        |
| 23.5.2    | Capitolo 16 — Progettazione di un file system . . . . .                     | 177        |
| 23.5.3    | Capitolo 17 — Consistenza, journaling e cache . . . . .                     | 178        |
| 23.6      | Parte VI — Dispositivi e affidabilità . . . . .                             | 179        |
| 23.6.1    | Capitolo 18 — Scheduling del disco . . . . .                                | 179        |
| 23.6.2    | Capitolo 19 — I sistemi RAID . . . . .                                      | 179        |
| 23.6.3    | Capitolo 20 — Memorie flash e dischi a stato solido . . . . .               | 180        |
| 23.7      | Due domande nel formato del compito . . . . .                               | 181        |
| <b>24</b> | <b>Il colloquio orale</b>                                                   | <b>183</b> |
| 24.1      | Come si risponde . . . . .                                                  | 183        |
| 24.2      | L'indice delle domande . . . . .                                            | 183        |
| 24.3      | Le domande che attraversano i capitoli . . . . .                            | 186        |
| 24.4      | Le domande-trappola . . . . .                                               | 187        |
| 24.5      | Gli errori da non fare . . . . .                                            | 188        |
| 24.6      | Gli esempi da saper rifare . . . . .                                        | 189        |



## Parte I

# Il sistema operativo

## Come usare questa guida

---

Questa guida copre la **prova di teoria** di Sistemi Operativi. Non copre il laboratorio, che è una prova separata e si prepara in un altro modo: qui non c'è una riga di C.

### Da dove viene quello che leggi

Ogni affermazione di questa guida ha una fonte, e la fonte è sempre dichiarata. In pratica:

- i **venti capitoli di teoria** seguono le proiezioni del corso. Quando una figura è presa dalle slide, la didascalia dice da quale pagina. Quando un numero viene da una slide, accanto c'è la citazione fra parentesi quadre;
- il **compito tipo** è l'unico esempio ufficiale di prova di teoria pubblicato dal corso: si trova nella pagina del docente come "*Stralcio di un compito d'esame*". Le sue sette domande sono riportate e svolte per intero;
- gli **esercizi** sono di due tipi, e ognuno lo dichiara. Quelli marcati **Domanda reale** vengono dal compito tipo appena detto; quelli marcati **Esercizio costruito** li ho scritti io sullo stesso modello, con numeri diversi, per coprire i casi che il compito tipo non tocca;
- i **numeri** di tutti gli esercizi non sono ricopiati né calcolati a mano: sono prodotti da uno script e verificati eseguendolo. Dove un conto non torna con quello che dice la slide, la guida lo scrive invece di aggiustare il numero.

### Che cosa non troverai

Nessun contenuto inventato per riempire un buco. Dove il corso non dice qualcosa — e capita, per esempio sui livelli RAID o sull'implementazione del TRIM — la guida lo dichiara e spiega il perché invece di inventare un dettaglio plausibile. Un dettaglio plausibile sbagliato, il giorno dell'esame, vale zero come una lacuna, e in più non si sa di non saperlo.

### Come è organizzata

La numerazione dei capitoli segue l'elenco degli argomenti del programma d'esame: il capitolo  $N$  è l'argomento  $N$ . Se stai ripassando l'argomento 12 del programma, apri il capitolo 12 e sei nel posto giusto.

Ogni capitolo di teoria chiude con un riquadro viola, *Come lo dici all'orale*: è la risposta già impaginata come discorso, da leggere ad alta voce. Il voto lo dà il colloquio, e un colloquio non si prepara con un elenco di parole chiave.

La parte finale raccoglie il compito tipo svolto, gli esercizi dello stesso tipo, le domande a risposta multipla e le domande d'orale.

# Com'è fatta la prova di teoria

---

## Il corso e l'esame

### Il corso, come lo presenta la prima lezione

[SO-01 p2]

- **9 CFU** (72 ore), strutturato in **teoria** e **laboratorio**;
- **propedeuticità**: Architettura degli Elaboratori e Programmazione I;
- **esame**: prova di **teoria** (65%), fatta di **test** e **orale**; prova **pratica in laboratorio** (35%).

Il 65% della teoria si divide a sua volta in due momenti: un **test** scritto e un **colloquio orale**. Sono due prove di natura diversa, e si preparano in modo diverso: il test chiede di riconoscere e di calcolare, l'orale chiede di *spiegare*. Questa guida le affronta entrambe, e i riquadri viola *Come lo dici all'orale* in fondo a ogni capitolo sono la parte pensata per il secondo momento.

### Il laboratorio resta fuori

La prova pratica (35%) **non** è coperta da questa guida. È una scelta, non una dimenticanza: si tratta di scrivere programmi in C con processi, thread, mutex e variabili di condizione, e si prepara esercitandosi sui compiti passati di laboratorio, che il corso pubblica in numero abbondante. Studiare il laboratorio su una guida di teoria sarebbe tempo speso male.

## Che aspetto ha il compito di teoria

Il corso pubblica **un solo** esempio di prova di teoria, intitolato "*Stralcio di un compito d'esame*": è il file **compito-tipo.pdf** nella pagina del docente. È poco, ma è molto meglio di niente, perché mostra con precisione *come* vengono chieste le cose.

Il compito tipo è fatto di **sette domande** e si divide in due metà nette:

- le prime **due** sono a **risposta multipla**: cinque opzioni da A a E, una sola corretta. Una delle due non chiede nemmeno di indicare la risposta giusta, ma *quale affermazione è falsa*. Sono domande di riconoscimento: o si sa, o non si sa, e non c'è modo di ricavare la risposta dal foglio;
- le altre **cinque** sono **esercizi di calcolo**: memoria virtuale, tracce di semafori, FAT, i-node, scheduling del disco. Qui il risultato è un numero, e il procedimento va scritto.

## Le cinque famiglie di esercizi

Vale la pena fissarle subito, perché sono il 70% del compito scritto e si riconoscono a colpo d'occhio:

1. **Memoria virtuale**. Dati lo spazio di indirizzamento, i bit di pagina virtuale e i bit dell'indirizzo fisico, determinare quanti frame ci sono. È la madre di tutte le domande di indirizzamento: dal capitolo 12 in poi è tutta aritmetica di potenze di due.
2. **Tracce di semafori**. Dato lo pseudo-codice di due o tre processi che condividono una variabile e usano **wait** e **signal**, determinare l'output. Si risolve *esplorando gli interleaving*, non indovinandone uno (capitolo 6).

3. **FAT.** Data una tabella FAT, la voce della cartella radice e un offset in byte, dire in quale blocco cade quell'offset. Capitolo 16.
4. **i-node.** Dato l'i-node di un file e il contenuto di alcuni blocchi, dire in quali blocchi del disco risiedono i byte di un certo offset, distinguendo la zona diretta da quella indiretta semplice, doppia e tripla. Capitolo 16.
5. **Scheduling del disco.** Data una coda di richieste, la posizione della testina e una politica, calcolare il tempo di ricerca complessivo e indicare la sequenza di servizio. Capitolo 18.

#### Che cosa significa per come si studia

Il compito tipo dice una cosa che vale la pena mettere a fuoco subito: le cinque famiglie di calcolo sono **tutte** riconducibili a un procedimento meccanico, che una volta capito si applica sempre allo stesso modo. Non c'è creatività da mettere: c'è da fare bene i conti. Questo cambia la strategia di preparazione. Le due domande a risposta multipla si vincono con la **lettura** della teoria — sono definizioni e regole, e non si ricavano. I cinque esercizi si vincono con l'**allenamento**: farne dieci di ogni famiglia, con numeri diversi, finché il procedimento non diventa automatico. La parte III di questa guida è fatta esattamente per questo.

#### Una nota sul compito tipo, e sul suo “duplicato”

Trascrivendo il compito tipo si incontrano due dettagli che vale la pena conoscere, perché sono nel file originale.

Il primo: il documento si chiama *stralcio* di un compito d'esame, e in effetti è incompleto per costruzione — è un estratto, non una prova intera. Serve a mostrare la forma delle domande, non a essere una simulazione completa.

Il secondo: la domanda 6 comincia con la parola **[duplicato]** stampata nel testo. È un residuo dell'impaginazione del docente, rimasto nel PDF. Nella parte esercizi di questa guida la domanda 6 è riportata senza quel marcatore, che non fa parte del testo della domanda.

# Capitolo 1

## Introduzione al concetto di sistema operativo

---

### 1.1 Perché serve un sistema operativo

Un moderno calcolatore è formato da uno o più processori, dalla memoria centrale, da dischi e da altre periferiche di I/O. I dettagli di basso livello di queste componenti sono molto complessi, e gestirle tutte quante richiede uno strato intermedio software: il **sistema operativo**. [SO-01 p5]

Da questa definizione discendono subito le due facce del sistema operativo, che sono anche le due domande d'apertura più frequenti all'orale.

#### Il sistema operativo come macchina estesa

Il sistema operativo è uno **strato intermedio** fra l'hardware e i programmi. Nasconde la complessità dell'hardware e offre ai programmi un'interfaccia più semplice da usare: il concetto che riassume questo ruolo è l'**astrazione**. [SO-01 p7]

#### Il sistema operativo come gestore delle risorse

Da un moderno sistema operativo ci si aspetta che gestisca **più programmi in esecuzione** e **più utenti**. Questo richiede un'allocazione ordinata e controllata di risorse: processori, memoria, unità di I/O, e non solo hardware (anche file, database, ...). [SO-01 p8]

Il meccanismo con cui un solo sistema operativo serve più programmi e più utenti si chiama **multiplexing**, e si presenta in due forme distinte che l'esame chiede esplicitamente di saper separare:

#### Multiplexing nel tempo e nello spazio

- **Nel tempo**: la risorsa viene assegnata a turno a più richiedenti. Esempi: la **CPU**, la **stampante**.
- **Nello spazio**: la risorsa viene suddivisa fisicamente fra più richiedenti. Esempi: la **memoria centrale**, il **disco**.

[SO-01 p8]

#### Errore tipico da evitare

L'errore tipico è classificare la memoria come multiplexing *nel tempo*. Non lo è: la memoria si divide in porzioni diverse che esistono contemporaneamente, quindi è multiplexing *nello spazio*. Viceversa la CPU non viene divisa in porzioni, ma assegnata a turno: multiplexing nel tempo. Il criterio per distinguerli: se i richiedenti usano la risorsa *contemporaneamente* su porzioni distinte, è spazio; se la usano *a turno*, è tempo.

### 1.2 La doppia modalità di esecuzione

L'hardware dei processori moderni supporta **due modalità** di esecuzione:

- la **modalità kernel** (detta anche *supervisor*);
- la **modalità utente**.

[SO-01 p6]

È questa distinzione hardware a rendere possibile il ruolo del sistema operativo: il codice del kernel gira in modalità kernel e può eseguire tutte le istruzioni, comprese quelle privilegiate; il codice applicativo gira in modalità utente e non può toccare direttamente le risorse protette. Il passaggio da una modalità all'altra avviene tramite **chiamate di sistema** (*system call*) e **interrupt hardware**, ed è descritto in dettaglio nel capitolo 2.

#### Perché questo dettaglio conta

La doppia modalità è il meccanismo hardware su cui poggia quasi tutto il resto del corso: la **mutua esclusione** (capitolo 6) esiste perché le istruzioni di sincronizzazione devono essere privilegiate o atomiche; la **rilocazione** e la **memoria virtuale** (capitoli 11 e 12) servono anche a impedire che un processo in modalità utente legga la memoria di un altro. Se ti chiedono “perché il sistema operativo può fare cose che un programma utente non può fare?”, la risposta parte da qui.

### 1.3 L'evoluzione dei sistemi operativi

Le proiezioni presentano l'evoluzione dei sistemi operativi come una linea del tempo, dalla prima generazione fino ai sistemi attuali. [SO-01 p9]

Il filo conduttore dell'evoluzione è sempre lo stesso: aumentare il numero di programmi che possono stare in esecuzione contemporaneamente, passando dai sistemi a lotti (un programma alla volta, in sequenza) alla **multiprogrammazione**, che è il tema dei capitoli 4 e 11.

### 1.4 Lo zoo dei sistemi operativi

Non esiste *un* tipo di sistema operativo: le esigenze sono diverse a seconda del contesto d'uso, e il corso ne distingue cinque famiglie. [SO-01 p18]

| Famiglia                     | Caratteristiche                                                                                        |
|------------------------------|--------------------------------------------------------------------------------------------------------|
| Mainframe / server           | Molti utenti e molti programmi insieme; grande throughput, condivisione di risorse costose.            |
| Personal computer            | Un singolo utente, ma multiprogrammazione per l'interattività; interfaccia grafica.                    |
| Palmari / smartphone         | Risorse limitate, forte risparmio energetico, interfaccia touch, connettività.                         |
| Sistemi integrati (embedded) | Vincoli di tempo e di spazio molto stretti, funzionalità fisse, spesso senza utente umano.             |
| Real-time                    | Esiste un tempo massimo di risposta che <b>deve</b> essere rispettato, pena il fallimento del sistema. |

#### Cosa chiedono all'esame

La distinzione importante, e la più richiesta, è quella fra **embedded** e **real-time**: un sistema embedded non è necessariamente real-time (una lavatrice con logica fissa non ha scadenze temporali critiche), e un sistema real-time non è necessariamente embedded (il sistema di controllo di un treno può girare su hardware da rack). I sistemi real-time si dividono ulteriormente in *hard* real-time, dove perdere una scadenza è un guasto, e *soft* real-time, dove perdere una scadenza degrada la qualità ma non è un guasto.

**Come lo dici all'orale**

“Che cos'è un sistema operativo?” va risposto su **due piani**, non su uno solo. Primo piano: è uno strato di software intermedio fra hardware e applicazioni, che presenta all'utente una *macchina estesa*, cioè un insieme di astrazioni (file, processo, memoria virtuale) più comode dell'hardware sottostante. Secondo piano: è il *gestore delle risorse*, che assegna in modo ordinato e controllato CPU, memoria e dispositivi di I/O a più programmi e più utenti, tipicamente con multiplexing nel tempo e nello spazio. Chi si ferma al primo piano prende metà del voto su questa domanda.

## Capitolo 2

### Richiami sull'architettura degli elaboratori

---

Questo capitolo non è architettura degli elaboratori fine a sé: serve a capire **quali meccanismi hardware il sistema operativo può usare** e quali vincoli deve rispettare. All'orale le domande su questa parte sono sempre del tipo “come fa il sistema operativo a...?”, e la risposta è un meccanismo hardware.

#### 2.1 Architettura di un calcolatore

Lo schema semplificato di un calcolatore mette in relazione il processore, la memoria, i dispositivi di I/O e il bus che li collega. [SO-01 p10, p17]

Il **bus** è il canale condiviso su cui viaggiano gli indirizzi, i dati e i segnali di controllo. È una risorsa condivisa, e come tutte le risorse condivise del corso diventa un possibile *collo di bottiglia*: è uno dei motivi per cui si usano cache locali e DMA. [SO-01 p17]

#### 2.2 Il processore

Il processore esegue un **ciclo di base** in tre fasi: **prelevamento** (*fetch*), **decodifica**, **esecuzione**. [SO-01 p11]

##### I tre registri che il sistema operativo deve salvare

- **PC** — *Program Counter*: l'indirizzo della prossima istruzione da eseguire.
- **SP** — *Stack Pointer*: la posizione corrente dello stack.
- **PSW** — *Program Status Word*: le informazioni di stato del processore, fra cui la modalità di esecuzione corrente (kernel o utente) e i flag dei risultati.

[SO-01 p11]

Questi tre registri rivestono un'importanza particolare per il sistema operativo: quando si passa da un processo a un altro bisogna salvare *il contesto*, e il contesto comprende almeno PC, SP e PSW. È il meccanismo del *context switch*, che ritroverai nello scheduling (capitolo 8).

Le progettazioni avanzate — **pipeline** e CPU **superscalare** — non sono del tutto trasparenti al sistema operativo: esistono istruzioni che si comportano diversamente da quanto ci si aspetterebbe guardando il solo flusso sequenziale, e il sistema operativo deve tenerne conto. [SO-01 p11]

#### 2.3 Modalità di esecuzione e chiamate di sistema

Il meccanismo cardine è la **doppia modalità di esecuzione** (kernel/utente), già introdotta nel capitolo 1. Qui interessa *come* avviene il passaggio.

Lo schema della slide mostra il passaggio in quattro punti: [SO-01 p12]

1. il **processo utente** sta eseguendo in modalità utente;
2. il processo **chiama una system call**: si genera un **TRAP**, che è un'istruzione particolare;
3. il processore passa in **modalità kernel** ed esegue la system call;
4. al termine, l'istruzione di **return** riporta il processo in modalità utente.



**TRAP e interrupt**

Oltre alle system call, il passaggio in modalità kernel può avvenire per **interrupt hardware**: un segnale proveniente da un dispositivo che interrompe il flusso di esecuzione corrente. Il TRAP è la via “volontaria” (il programma chiede un servizio), l’interrupt è la via “involontaria” (l’hardware segnala un evento). [SO-01 p12]

**Errore tipico da evitare**

Non confondere **chiamata di sistema** e **chiamata di funzione**. Una chiamata di funzione resta in modalità utente e non dà alcun privilegio in più; una chiamata di sistema provoca un TRAP, cambia modalità, ed è l’*unico* modo con cui un programma utente può chiedere al kernel di fare qualcosa che da solo non potrebbe fare (leggere un file, allocare memoria, creare un processo). Se ti chiedono “un programma utente può accedere direttamente al disco?”, la risposta è no: deve passare da una system call.

**2.4 Più processori**

Il corso distingue nettamente tre situazioni diverse, che all’esame vengono spesso confuse fra loro. [SO-01 p13]

| Tecnologia                                | Cosa è e cosa comporta per il SO                                                                                                                                                                                                                                                                                              |
|-------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Multithreading (o <i>hyperthreading</i> ) | La CPU tiene al suo interno lo stato di <b>due thread</b> . Non c’è una esecuzione parallela vera e propria, ma il sistema operativo deve tenerne conto: dal punto di vista del SO le unità logiche eseguibili sembrano due.                                                                                                  |
| Multiprocessori                           | Più processori veri nello stesso sistema. I vantaggi sono tre: <b>throughput</b> (più lavoro completato per unità di tempo), <b>economia di scala</b> (più processori costano meno di più sistemi completi) e <b>affidabilità</b> (la degradazione è graduale: se un processore si guasta, il sistema continua a funzionare). |
| Multicore                                 | Più nuclei integrati nello stesso chip.                                                                                                                                                                                                                                                                                       |
| GPU                                       | Molti nuclei specializzati per il calcolo parallelo su dati.                                                                                                                                                                                                                                                                  |

**Cosa chiedono all’esame**

La domanda classica è: “l’hyperthreading raddoppia le prestazioni?”. La risposta è **no**, e il motivo sta nella slide: *non c’è una esecuzione parallela vera e propria*. Si tiene lo stato di due thread dentro la stessa CPU per riempire i vuoti (gli *stalli*) della pipeline, non perché si eseguano davvero due istruzioni nello stesso istante. Il guadagno è reale ma parziale, e dipende dal carico.

**2.5 Le memorie**

La memoria di un calcolatore non è un blocco unico, ma una **gerarchia**: si va da memorie piccolissime, velocissime e costose, vicine al processore, a memorie enormi, lente ed economiche, lontane dal processore. Il principio è che conviene tenere “vicino” ciò che si usa spesso. [SO-01 p14]

**Le unità di misura, e la trappola**

La slide richiama esplicitamente la differenza fra due sistemi di unità:

- **kB, MB, GB** — multipli in **base 10**: 1 kB =  $10^3$  byte, 1 MB =  $10^6$  byte, 1 GB

$= 10^9$  byte.

- **KiB, MiB, GiB** — multipli in **base 2**:  $1 \text{ KiB} = 2^{10} = 1024$  byte,  $1 \text{ MiB} = 2^{20}$  byte,  $1 \text{ GiB} = 2^{30}$  byte.

[SO-01 p14]

### Errore tipico da evitare

Questa distinzione non è pignoleria: è la fonte di errore numero uno negli esercizi su memoria virtuale e file system. Un disco “da 1 TB” ha  $10^{12}$  byte, che divisi per  $2^{30}$  fanno 931,32 GiB, non 1024 GiB. Negli esercizi del compito tipo la dimensione dei blocchi è sempre indicata in kB e vale 4096 byte, quindi il conto torna comodo, ma la domanda “quanti frame ci sono?” va fatta con le potenze di 2. Nel capitolo ?? trovi l'esercizio svolto per esteso.

## 2.6 I dispositivi di I/O

In un dispositivo di I/O si individuano **due componenti** distinte: [SO-01 p15]

- il **controller**: più semplice da usare per il sistema operativo;
- il **dispositivo in sé**: interfaccia elementare ma complicata da pilotare. Esempio: i dischi SATA.

Ogni controller ha bisogno di un **driver** per il sistema operativo. Il driver interagisce con il controller attraverso le **porte di I/O**, in due modi alternativi:

- con istruzioni dedicate tipo **IN / OUT**;
- con la **mappatura in memoria** (*memory-mapped I/O*), dove i registri del controller appaiono come normali indirizzi di memoria.

### Perché il driver sta nel kernel

Il driver deve parlare direttamente con l'hardware, quindi deve poter eseguire istruzioni privilegiate e accedere alle porte di I/O: deve girare in modalità kernel. È una delle ragioni per cui un driver difettoso può mandare in crash l'intero sistema, e una delle motivazioni storiche del **microkernel** (capitolo 3), che sposta i driver fuori dal kernel.

### 2.6.1 Le tre modalità di I/O

La slide successiva presenta le tre modalità con cui il processore può gestire un trasferimento di I/O, in ordine crescente di efficienza. [SO-01 p16]

| Modalità                        | Come funziona                                                                                                                                                                                                              |
|---------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Busy waiting                    | Il processore interroga ripetutamente il dispositivo finché non è pronto, restando occupato in un ciclo di attesa. Semplice, ma spreca completamente la CPU.                                                               |
| Con programmazione di interrupt | Il processore avvia il trasferimento e prosegue con altro lavoro; quando il dispositivo ha finito, genera un <b>interrupt</b> che sospende il lavoro corrente per gestire il completamento. La CPU non resta bloccata.     |
| Con uso del DMA                 | <i>Direct Memory Access</i> : un controllore dedicato trasferisce i dati direttamente fra dispositivo e memoria centrale, senza far passare ogni byte dal processore. La CPU viene avvisata solo a trasferimento concluso. |

Lo schema della slide mostra il confronto temporale fra processore e dispositivo: con l'interrupt la CPU appare *idle* (o impegnata su altro) nel momento in cui il dispositivo sta trasferendo, e viene risvegliata dagli eventi **I/O request**, **transfer done**, **I/O request**, **transfer done**, ...[SO-01 p16]

#### La filastrocca delle tre modalità

Busy waiting: la CPU chiede “sei pronto? sei pronto? sei pronto?”. Interrupt: la CPU dice “fammi sapere quando hai finito” e va a fare altro. DMA: la CPU dice “sbrigatela fra voi due” e non si occupa nemmeno del trasferimento.

#### Come lo dici all'orale

“Perché si usa il DMA?” La risposta che vogliono sentire non è “perché è più veloce”, ma: perché senza DMA ogni byte trasferito dovrebbe passare attraverso il processore, occupandolo per tutta la durata del trasferimento; con il DMA il processore si limita ad avviare l'operazione e a ricevere un interrupt alla fine, quindi resta disponibile per altro lavoro. È lo stesso principio del multiplexing nel tempo: non rendere più veloce la singola operazione, ma non sprecare la risorsa condivisa.

## Capitolo 3

### Struttura di un sistema operativo

---

Il corso elenca cinque strutture possibili per un sistema operativo: **monolitica**, **a livelli**, **micro-kernel**, **a moduli** e a **macchine virtuali**. [SO-01 p19]

Due avvertenze che la slide stessa mette in evidenza, e che conviene ripetere all'orale prima che lo faccia il docente:

- queste categorie hanno **intersezioni**: i sistemi reali sono spesso **ibridi**;
- la tassonomia **non è per forza completa né condivisa**: è una classificazione didattica, non un teorema.

[SO-01 p19]

#### 3.1 Struttura monolitica

Nella forma più antica non c'è alcun supporto hardware: il sistema operativo è un unico programma senza barriere interne, e questo crea problemi. Gli esempi sono MS-DOS e UNIX. [SO-01 p20]

Con l'arrivo del **supporto hardware alla modalità kernel/utente** si arriva alla forma classica: **un unico kernel con tutto dentro**, in cui **ogni componente può richiamare direttamente tutti gli altri**. [SO-01 p20]

| Vantaggi                                                                                | Svantaggi                                                                                                                             |
|-----------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------|
| Chiamate interne dirette: nessun overhead di comunicazione, quindi prestazioni elevate. | Poco gestibile nel tempo: con milioni di righe di codice le dipendenze incrociate rendono difficile capire l'impatto di una modifica. |
| Tutto il codice gira in modalità kernel con accesso completo.                           | Un errore in un punto qualunque — per esempio in un driver — può compromettere l'intero sistema.                                      |

#### 3.2 Struttura a livelli (o a strati)

Si utilizza una **gerarchia di livelli**: ogni livello implementa le proprie funzionalità impiegando *soltanto* quelle fornite dal livello inferiore. [SO-01 p21]

Il vantaggio dichiarato dalla slide è di progettazione: migliore progettazione, più semplice da sviluppare e da controllare, con un *incapsulamento tipo OOP* e una suddivisione a livello progettuale. Il difetto sono i **problemi di prestazioni** dovuti alle **chiamate nidificate** e al relativo overhead: per arrivare al livello più basso una richiesta deve attraversare tutti i livelli intermedi. [SO-01 p21]

##### La variante ad anelli concentrici: MULTICS

Esiste una variante in cui i livelli diventano **anelli concentrici**, con una **separazione forzata dall'hardware**: il processore stesso impedisce a un anello esterno di accedere alle risorse di un anello interno. L'esempio citato è MULTICS. [SO-01 p21]

##### Errore tipico da evitare

L'errore classico è dire che la struttura a livelli è “la più efficiente perché è ordinata”. È esattamente il contrario: è la *meno* efficiente delle strutture presentate, proprio per l'overhead delle chiamate nidificate. È la più *pulita* da progettare, non la più veloce. Se la domanda è “quale struttura sceglierebbe per prestazioni?”, la risposta non è questa.

### 3.3 Microkernel

L'idea è ridurre il kernel al minimo indispensabile: un **microkernel** che si occupa soltanto di **scheduling, memoria e IPC** (comunicazione fra processi). Tutto il resto — filesystem, driver di dispositivi — viene spostato a **livello utente**, come moduli. [SO-01 p22]

La comunicazione fra i componenti avviene **attraverso messaggi**. I pregi dichiarati sono un **miglior design** (componenti piccoli) e una **migliore stabilità**: se un driver va in crash, è un processo utente che muore, non il kernel. [SO-01 p22]

Gli esempi citati: MINIX 3, Mach, QNX, Mac OS (Darwin), Windows NT.

#### Il prezzo e il guadagno

Il microkernel scambia **prestazioni** con **robustezza**. Ogni operazione che prima era una chiamata di funzione interna al kernel diventa uno scambio di messaggi fra processi, con il costo di due passaggi di contesto. Il guadagno è che il kernel è piccolo e verificabile, e che un guasto in un componente non privilegiato non travolge il sistema.

### 3.4 Struttura a moduli

È l'idea della programmazione orientata agli oggetti applicata al kernel: si hanno **moduli** che implementano ciascuno un aspetto specifico (filesystem, driver, ...), con un **kernel principale a funzionalità ridotte** e moduli **caricabili dinamicamente**. [SO-01 p23]

La differenza decisiva rispetto al microkernel sta in una proprietà sola, e la slide la mette in evidenza: qui **ogni modulo può invocare qualunque altro modulo direttamente**, e **non ci sono messaggi**. Il risultato è un design pulito (a oggetti) ma anche **efficiente**. [SO-01 p23]

Gli esempi: Solaris, Linux, Mac OS (ibrido).

#### Cosa chiedono all'esame

La coppia microkernel / a moduli è il confronto più gettonato su questa parte. Il modo sicuro di esporlo è in termini di *come comunicano i componenti*:

- **microkernel**: componenti in spazio utente, comunicazione **a messaggi**, quindi più robusto ma più lento;
- **a moduli**: moduli nel kernel, chiamate **dirette** fra moduli, quindi più veloce ma con la stessa esposizione ai guasti della struttura monolitica.

Nota che Linux viene citato come esempio di struttura *a moduli*, non microkernel, anche se è spesso descritto come “monolitico con moduli”: entrambe le descrizioni si sentono, e la seconda è quella che il corso adotta quando parla di kernel monolitico modulare.

### 3.5 Macchine virtuali

L'estremizzazione del concetto di **astrazione** porta alla **virtualizzazione**: il sistema operativo non si limita a nascondere l'hardware, ma ne crea una copia virtuale su cui può girare un altro sistema operativo. [SO-01 p24]

Le motivazioni elencate dalla slide sono: usare più sistemi operativi sulla stessa macchina, realizzare **VPS** (server privati virtuali) e **isolare i servizi** l'uno dall'altro. [SO-01 p24]

Si distinguono due approcci: la **simulazione** e la **paravirtualizzazione**. Tutto viene gestito dall'**hypervisor**, di cui la slide distingue due tipi: [SO-01 p24]

| Tipo   | Dove gira                                             | Esempi                                                      |
|--------|-------------------------------------------------------|-------------------------------------------------------------|
| Tipo 1 | Direttamente sull'hardware                            | VMware ESX/ESXi, KVM su Linux, Microsoft Hyper-V hypervisor |
| Tipo 2 | È un processo dentro un sistema operativo <i>host</i> | VMware Workstation, VirtualBox                              |

La slide chiude osservando che esiste **supporto hardware** dedicato alla virtualizzazione, che serve a renderla efficiente: senza di esso ogni istruzione privilegiata del sistema operativo ospite deve essere intercettata e simulata via software, con un costo altissimo. [SO-01 p24]

#### Come lo dici all'orale

“Mi parli delle macchine virtuali.” Una risposta ordinata: definizione (astrazione portata all'estremo: si virtualizza l'hardware per farci girare sopra uno o più sistemi operativi ospiti); a cosa serve (più sistemi operativi sulla stessa macchina, VPS, isolamento dei servizi); chi lo fa (l'hypervisor); i due tipi, di tipo 1 direttamente sull'hardware e di tipo 2 come processo di un SO host, con un esempio ciascuno; e infine che esiste supporto hardware dedicato, perché senza di esso la virtualizzazione puramente software è troppo costosa. Chi arriva in fondo a questo elenco ha risposto in modo completo.

## Parte II

# Processi e thread

## Capitolo 4

### I processi: definizione, multiprogrammazione, stati e transizioni

---

#### 4.1 Che cos'è un processo

##### Processo

Un **processo** è una **istanza di esecuzione di un programma**. [SO-02 p2]

La differenza fra *programma* e *processo* è la stessa che c'è fra una ricetta e il cucinare quella ricetta: il programma è un file di codice su disco, immutabile e potenzialmente usato da molti; il processo è l'attività che si svolge mentre quel codice viene eseguito, e ne esiste una copia distinta per ogni esecuzione.

A un processo si associano le seguenti informazioni: [SO-02 p2]

- lo **spazio degli indirizzi**, composto da **codice**, **dati** e **stack**;
- una **copia dei registri della CPU**;
- i **file aperti**;
- gli **allarmi pendenti**;
- i **processi imparentati**.

##### Le cinque voci, e perché sono quelle

Il criterio con cui è costruito questo elenco: un processo deve poter essere *sospeso e ripreso* senza che se ne accorga. Tutto ciò che serve a riprenderlo dov'è essere salvato — ed è esattamente questo elenco. Lo spazio degli indirizzi dice *dove* sta lavorando; i registri dicono *a che punto* è arrivato; i file aperti dicono *su cosa* stava operando; gli allarmi pendenti dicono quali *eventi futuri* ha programmato; i processi imparentati dicono *chi* dipende da lui.

Per tenere traccia di tutto questo il sistema operativo mantiene una **tabella dei processi** con un **Process Control Block (PCB)** per ogni processo. [SO-02 p2]

#### 4.2 Il modello dei processi

Il sistema operativo dà l'illusione che ogni processo abbia una CPU tutta per sé. In realtà la CPU è una sola e i processi si alternano: questo si chiama **multiprogrammazione** e produce uno **pseudo-parallelismo**, cioè un parallelismo solo apparente. [SO-02 p3]

La slide aggiunge una nota metodologica che vale la pena ripetere: è **più semplice ragionare** pensando a processi *sequenziali*, a ciascuno dei quali sia dedicata una **CPU virtuale**. [SO-02 p3]

##### Il modello mentale da usare negli esercizi

Quando devi risolvere un esercizio su processi concorrenti, non provare a immaginare l'esecuzione reale intrecciata: è troppo complicato e si sbaglia. Ragiona invece su *un processo alla volta*, come se ciascuno avesse la sua CPU, e usa poi le primitive di sincronizzazione per decidere chi può proseguire. È esattamente il modello con cui è costruito il risolutore di tracce del capitolo ??, che esplora tutti gli intrecci possibili invece di indovinarne uno.



## 4.3 Creazione e terminazione

### 4.3.1 Come nasce un processo

Un processo può essere creato: [SO-02 p4]

- in fase di **inizializzazione del sistema**;
- da parte di **un altro processo** (il padre), oppure per un'azione dell'utente.

Le metodologie sono due, e la differenza fra le due famiglie di sistemi operativi è netta: [SO-02 p4]

| Metodologia                                                                    | Come funziona                                                                                                                                                                                                                                                |
|--------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>Sdoppiamento del padre</i><br><b>fork</b> e <b>exec</b> (UNIX)              | Il processo padre si <b>duplica</b> con <b>fork</b> , ottenendo un processo figlio che è una copia del padre; se il figlio deve eseguire un programma diverso, chiama <b>exec</b> , che <b>sostituisce</b> il contenuto del processo con il nuovo programma. |
| <i>Nuovo processo per nuovo programma</i><br><b>CreateProcess</b><br>(Windows) | Si crea direttamente un processo per il nuovo programma, in un solo passo.                                                                                                                                                                                   |

#### Errore tipico da evitare

L'errore da evitare è credere che **fork** faccia partire un nuovo programma. **No**: dopo **fork** il figlio sta eseguendo *lo stesso* codice del padre, dalla stessa riga. È **exec** a cambiare il programma, e **exec** *non* crea un processo nuovo: sostituisce l'immagine di quello corrente. Le due operazioni sono separate proprio per permettere al figlio di preparare l'ambiente (ridirezionare i file, cambiare directory) *prima* di diventare un altro programma. È il meccanismo su cui è costruita la shell.

### 4.3.2 Come muore un processo

La terminazione può essere volontaria o involontaria: [SO-02 p4]

| Tipo                           | Volontaria? | Esempio                                                            |
|--------------------------------|-------------|--------------------------------------------------------------------|
| Uscita normale                 | sì          | <b>exit</b> (UNIX), <b>ExitProcess</b> (Windows)                   |
| Uscita su errore               | sì          | il programma rileva una condizione di errore e decide di terminare |
| Errore critico                 | no          | un errore non gestibile che il processo non può recuperare         |
| Terminato da un altro processo | no          | <b>kill</b> (UNIX), <b>TerminateProcess</b> (Windows)              |

#### Cosa chiedono all'esame

La distinzione che chiedono è fra “uscita su errore” e “errore critico”: la prima è **volontaria** (il processo *decide* di uscire perché ha rilevato un errore), la seconda è **involontaria** (il processo viene abbattuto da una condizione che non controlla). Un programma che controlla se un file esiste e, non trovandolo, esce con codice di errore fa un'uscita su errore volontaria; un programma che divide per zero viene terminato da un errore critico involontario.

## 4.4 Gli stati di un processo

Un processo non è sempre in esecuzione: il corso distingue **3 stati principali** più **2 stati aggiuntivi**, collegati da transizioni. [SO-02 p5]

| Stato                               | Che cosa significa                                                                                                                              |
|-------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| In esecuzione<br>( <i>running</i> ) | Sta effettivamente usando la CPU. Su un sistema con una sola CPU, al più un processo per volta può stare qui.                                   |
| Pronto ( <i>ready</i> )             | È pronto a eseguire e aspetta solo di ottenere la CPU. Da qui si passa a <i>in esecuzione</i> quando lo scheduler lo sceglie.                   |
| In attesa<br>( <i>blocked</i> )     | Aspetta un evento esterno (il completamento di un I/O, un segnale, un dato). Non basta che la CPU si liberi: deve verificarsi <i>l'evento</i> . |

I due stati aggiuntivi sono quelli in cui il processo non è ancora “vivo” o non lo è più: **Nuovo**, subito dopo la creazione e prima di essere ammesso fra i pronti, e **Terminato**, dopo la fine dell'esecuzione e prima che le risorse vengano recuperate.

### La differenza che decide le domande d'esame

**Pronto** e **In attesa** si distinguono per *che cosa* manca al processo:

- **Pronto** aspetta la **CPU**: l'evento che lo sblocca è una *decisione dello scheduler*.
- **In attesa** aspetta un **evento esterno**: la CPU non c'entra, e per questo da *In attesa* non si passa mai direttamente a *In esecuzione*. Prima si torna a *Pronto*, e solo dopo lo scheduler potrà sceglierlo.

È l'errore più frequente su questo argomento: disegnare la freccia diretta da *In attesa* a *In esecuzione*. Non esiste.

## 4.5 Il passaggio da un processo all'altro

Quando arriva un interrupt, il sistema operativo deve sospendere il processo corrente e decidere chi eseguirà dopo. La slide elenca i passi esatti, e all'orale è utile saperli in ordine: [SO-02 p6]

1. salvataggio nello stack del **PC** e del **PSW** nello stack attuale;
2. caricamento dal **vettore degli interrupt** dell'indirizzo della procedura associata;
3. salvataggio dei **registri** e impostazione di un **nuovo stack**;
4. esecuzione della **procedura di servizio** per l'interrupt;
5. interrogazione dello **scheduler** per sapere con quale processo proseguire;
6. ripristino dal **PCB** dello stato di tale processo (registri, mappa della memoria);
7. ripresa nel processo corrente.

### Perché servono due stack

Il punto 3 (“impostazione di un nuovo stack”) esiste perché il codice che gestisce l'interrupt non può usare lo stack del processo interrotto: quel processo è sospeso a metà di un'operazione, e il suo stack contiene dati incoerenti. Il gestore dell'interrupt gira quindi su uno stack proprio, tipicamente quello del kernel.

## 4.6 Code e accodamento

I processi in attesa di essere eseguiti vengono tenuti in **strutture collegate sui PCB**: [SO-02 p7]

- la **coda dei processi pronti**, unica;
- le **code dei dispositivi**, una per ciascun dispositivo di I/O.

### Come lo dici all'orale

“Mi descriva gli stati di un processo.” La risposta che funziona: prima i **tre stati principali** con il criterio che li distingue — *in esecuzione* usa la CPU, *pronto* aspetta la CPU, *in attesa* aspetta un evento esterno — poi i **due addizionali**, *nuovo* e *terminato*, con la precisazione che servono a coprire la vita del processo prima che sia ammesso alle code e dopo la fine. Infine le transizioni, insistendo su quella che *non* esiste: da *in attesa* non si passa direttamente a *in esecuzione*, ma sempre attraverso *pronto*. Se ti chiedono “perché?”, la risposta è che lo sbloccarsi di un evento esterno non dà diritto alla CPU: dà solo il diritto di *competere* per essa, e chi vince lo decide lo scheduler.

## Capitolo 5

### I thread: definizione, modelli, programmazione multicore

#### 5.1 Perché servono i thread

Il modello dei processi presentato nel capitolo 4 tratteggia “entità indipendenti che raggruppano risorse e con un flusso di esecuzione”. Può però essere utile far **condividere a più flussi di esecuzione lo stesso spazio di indirizzi**: questi flussi sono i **thread**. [SO-02 p8]

##### Thread

Un **thread** è un flusso di esecuzione all'interno di un processo. È caratterizzato da **PC, registri, stack e stato**, e **condivide tutto il resto** con gli altri thread dello stesso processo. [SO-02 p9]

La slide aggiunge una conseguenza importantissima, che va detta esplicitamente: fra thread dello stesso processo **non c'è protezione di memoria**. [SO-02 p9]

##### Processo e thread, in una riga

Il processo è l'unità di **possesso delle risorse**; il thread è l'unità di **esecuzione**. Cambiare thread costa meno che cambiare processo, perché le risorse condivise non vanno riconfigurate.

Quando conviene usare più thread in un unico processo? Gli esempi delle slide sono quelli in cui il programma deve fare più cose “contemporaneamente”: un **web-browser** (rendere la pagina mentre scarica le immagini), un programma di **videoscrittura** (gestire l'interfaccia mentre riformatta), un **web-server** (servire più richieste insieme). [SO-02 p8]

#### 5.2 Le operazioni sui thread

Le operazioni tipiche sono quattro: [SO-02 p9]

| Operazione                 | Effetto                                                                  |
|----------------------------|--------------------------------------------------------------------------|
| <code>thread_create</code> | un thread ne crea un altro                                               |
| <code>thread_exit</code>   | il thread chiamante termina                                              |
| <code>thread_join</code>   | un thread si sincronizza con la fine di un altro thread, cioè lo aspetta |
| <code>thread_yield</code>  | il thread chiamante rilascia <b>volontariamente</b> la CPU               |

##### Errore tipico da evitare

`thread_yield` e `thread_join` vengono confusi spesso. **yield** è un “passo indietro”: il thread è ancora vivo e riprenderà appena lo scheduler lo risceglierà. **join** è un’**attesa**: il thread si blocca finché l'altro non è *terminato*. Chi scrive “**yield** aspetta che l'altro finisca” sbaglia la domanda su tutta la linea.

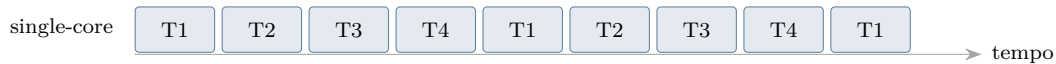
Rispetto ai processi, i thread portano anche un **cambio di contesto più veloce** e hanno propri **cambiamenti di stato** (gli stessi tre stati principali visti per i processi). [SO-02 p9]

Nelle proiezioni compare la voce *scheduling dei thread* fra le caratteristiche elencate: significa che anche i thread vengono scelti da uno scheduler, e che questo può avvenire a due livelli diversi — quello del kernel e quello di una libreria in spazio utente. È il tema della sezione 5.4.

## 5.3 Programmazione multicore

I thread permettono una migliore **scalabilità**, sia con core che fanno hyperthreading sia, soprattutto, con sistemi multicore. [SO-02 p10]

La differenza fra i due casi si vede bene in un diagramma di interleaving. Con quattro thread T1, T2, T3, T4 su un sistema a un solo core, l'esecuzione è **intrecciata** (*interleaved*): un solo thread per volta occupa la CPU, e si alternano nel tempo.



Su un sistema **multi-core**, invece, abbiamo **parallelismo puro**: core diversi eseguono thread diversi nello stesso istante.



I due diagrammi sono ridisegnati da [SO-02 p10, p11], dove compaiono come schemi; qui sono tradotti in TikZ perché le etichette restino leggibili e coerenti con il resto della guida.

### 5.3.1 I cinque principi della programmazione multicore

Progettare programmi che sfruttino davvero le moderne architetture multicore **non è banale**. La slide elenca cinque principi di base: [SO-02 p11]

1. **separazione dei task**: individuare attività indipendenti da mettere su core diversi;
2. **bilanciamento**: distribuire il lavoro in modo che nessun core resti inattivo mentre altri sono sovraccarichi;
3. **suddivisione dei dati**: ripartire i dati fra i core;
4. **dipendenze dei dati**: verificare che le parti non dipendano l'una dai risultati dell'altra, altrimenti il parallelismo non c'è;
5. **test e debugging**: le condizioni di gara si manifestano in modo non deterministico, quindi vanno cercate apposta.

#### Cosa chiedono all'esame

I punti 1 e 4 sono lo stesso problema visto da due lati. Non basta *volere* il parallelismo: bisogna che i task siano **effettivamente indipendenti**. Se due attività scrivono la stessa variabile, non si possono parallelizzare senza sincronizzazione, e la sincronizzazione riporta la parte contesa in serie. All'orale, se ti chiedono “quali difficoltà porta la programmazione multicore?”, la risposta forte è: le dipendenze fra i dati, che limitano il parallelismo *realmente* ottenibile rispetto a quello teorico.

## 5.4 I modelli di thread

Esistono tre modelli, distinti da **chi gestisce la tabella dei thread**: una libreria in spazio utente, il kernel, o entrambi.

### 5.4.1 Thread a livello utente (*1-a-molti*)

Una **libreria** implementa un sistema *run-time* che gestisce una **tabella dei thread del processo**. È utile quando il kernel non offre alcun supporto ai thread. [SO-02 p12]

| A favore                                                                                                    | Contro                                                                                                                                                                           |
|-------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Il <i>dispatching</i> non richiede un trap nel kernel: cambiare thread è una semplice chiamata di libreria. | Una <b>chiamata bloccante</b> blocca l'intero processo: se un thread fa <b>select</b> o provoca un <i>page-fault</i> , si fermano tutti, perché il kernel vede un solo processo. |
| Si può usare uno <b>scheduling personalizzato</b> , scelto dall'applicazione.                               | <b>Possibilità di non rilascio della CPU</b> : un thread che non cede volontariamente la CPU può impedire agli altri dello stesso processo di girare.                            |

### 5.4.2 Thread a livello kernel (*1-a-1*)

Il kernel gestisce direttamente i thread in un'unica **tabella dei thread del kernel**. Richiede supporto specifico, che praticamente tutti i moderni sistemi operativi forniscono. [SO-02 p13]

| A favore                                                                                                                                            | Contro                                                                                                                                                         |
|-----------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Un thread che fa una <b>chiamata bloccante non intralcia gli altri</b> : il kernel sa che sono flussi distinti e può mettere in attesa solo quello. | Il <b>cambio di contesto è più lento</b> , perché richiede un trap nel kernel.                                                                                 |
|                                                                                                                                                     | <b>Creazione e distruzione sono più costose</b> : il numero di thread del kernel è tipicamente limitato, e si ricorre al <i>riciclo</i> dei thread già creati. |

### 5.4.3 Modello ibrido (*molti-a-molti*)

Prende il meglio dei due modelli precedenti: si prevede un certo numero di thread del kernel e **ognuno di essi viene assegnato a un certo numero di thread utente** (eventualmente uno solo). L'assegnazione è decisa dal **programmatore**. [SO-02 p14]

#### Cosa chiedono all'esame

Lo schema da tenere a mente per non confondersi:

- **1-a-molti** = molti thread utente su *un* thread del kernel. Veloce ma fragile (una chiamata bloccante ferma tutto).
- **1-a-1** = un thread del kernel per ogni thread utente. Robusto ma costoso.
- **molti-a-molti** =  $n$  thread utente su  $m$  thread del kernel, con  $m < n$ . Compromesso, con l'assegnazione decisa dal programmatore.

Il nome del modello si legge come "thread utente a thread kernel".

## 5.5 I thread nei sistemi operativi reali

Quasi tutti i sistemi operativi supportano i **thread a livello kernel**: Windows, Linux, Solaris, Mac OS. [SO-02 p15]

Il supporto ai **thread utente** avviene attraverso apposite librerie: *green threads* su Solaris, *GNU portable thread* su UNIX, *fiber* su Windows. [SO-02 p15]

Le librerie di accesso ai thread, a prescindere dal modello, sono invece: [SO-02 p15]

- **Pthreads** di POSIX (Solaris, Linux, Mac OS, anche Windows): è una *specifica* da implementare sui vari sistemi, non un'implementazione;
- i **thread di Windows**;
- i **thread in Java**, che sono un *wrapper* sulle API sottostanti.

#### Pthreads è una specifica, non una libreria

La slide lo mette in evidenza e la distinzione conta: POSIX definisce *come* devono chiamarsi e comportarsi le funzioni, non fornisce il codice. Ogni sistema operativo ne dà la propria implementazione, e questo è ciò che permette a un programma scritto con Pthreads di essere ricompilato su sistemi diversi. All'orale è un buon esempio di *portabilità* ottenuta tramite standard, non tramite codice condiviso.

#### Come lo dici all'orale

“Differenza fra processo e thread?” La risposta che regge: il processo è l'unità di **possesso delle risorse** — ha uno spazio di indirizzi proprio, file aperti, allarmi; il thread è l'unità di **esecuzione** — ha solo PC, registri, stack e stato. I thread dello stesso processo condividono tutto il resto, e quindi *non c'è protezione di memoria* fra loro: è il vantaggio (comunicare è immediato, cambiare contesto costa poco) e insieme il rischio (un thread può corrompere i dati di un altro). Se ti chiedono “quando conviene l'uno e quando l'altro?”, la risposta è: conviene il thread quando i flussi devono condividere molti dati, conviene il processo quando serve isolamento.

## Parte III

# Concorrenza e sincronizzazione



## Capitolo 6

### Sezioni critiche e mutua esclusione

---

Questo è il capitolo più denso del programma, e quello in cui si concentrano più domande all'orale. Conviene studiarlo come una *scala*: si parte dal problema, si scrivono le condizioni che una buona soluzione deve rispettare, e poi si sale di livello di astrazione — soluzioni con attesa attiva, poi primitive che bloccano il processo, poi costrutti di alto livello.

#### 6.1 Perché i processi devono cooperare

Spesso i processi hanno bisogno di cooperare. I meccanismi citati dalle slide sono il collegamento di I/O tra processi (**pipe**) e la **InterProcess Communication** (IPC). [SO-02 p16]

La cooperazione porta con sé tre problemi distinti, che vale la pena elencare separatamente perché corrispondono a domande diverse: [SO-02 p16]

1. **come scambiarsi i dati**;
2. **l'accavallamento delle operazioni su dati comuni**;
3. il **coordinamento** tra le operazioni (la sincronizzazione).

Il secondo problema ha un nome preciso.

##### Corsa critica (*race condition*)

Si ha una **corsa critica** quando il risultato di un'operazione dipende dall'**ordine** in cui vengono eseguite le istruzioni di processi concorrenti, e quell'ordine non è controllato. [SO-02 p16]

L'esempio delle slide è quello dei **versamenti su un conto corrente**: due processi leggono il saldo, lo incrementano, lo riscrivono. Se le due letture avvengono prima delle due scritture, uno dei due versamenti sparisce. [SO-02 p16]

##### La corsa critica spiegata in tre righe

Il saldo vale 100. Il processo A legge 100. Il processo B legge 100. A scrive  $100+50 = 150$ . B scrive  $100+30 = 130$ . Risultato: 130, come se il versamento di A non fosse mai avvenuto. Non è un errore di calcolo: è che “leggi, modifica, scrivi” **non è un'operazione atomica**, e nel mezzo si è infilato qualcun altro.

Le slide segnalano un punto che spesso si dimentica: le corse critiche esistono anche **nel codice del kernel**, non solo nei programmi utente. La soluzione generale è la **mutua esclusione** nell'accesso ai dati condivisi. [SO-02 p16]

#### 6.2 Sezioni critiche e le quattro condizioni

L'astrazione del problema consiste nel dividere il codice di ogni processo in due parti: una **sezione critica**, in cui si accede ai dati condivisi, e una **sezione non critica**, in cui no. [SO-02 p17]

##### Le quattro condizioni di una buona soluzione

[SO-02 p17]

1. **mutua esclusione** nell'accesso alle rispettive sezioni critiche;

2. **nessuna assunzione** sulla velocità di esecuzione o sul numero di CPU;
3. nessun processo **fuori** dalla propria sezione critica può **bloccare** un altro processo;
4. nessun processo dovrebbe restare **all'infinito in attesa** di entrare nella propria sezione critica.

#### Cosa chiedono all'esame

Queste quattro condizioni sono la **griglia di valutazione** di ogni soluzione che incontrerai. Quando il docente ti chiede “questa soluzione va bene?”, non vuole un sì o un no: vuole che tu passi in rassegna le quattro condizioni e dica *quale* fallisce. Le risposte tipiche sono:

- l'alternanza stretta viola la **condizione 3** (un processo fuori dalla sezione critica blocca l'altro: se tocca a me e io non voglio entrare, tu resti fuori);
- le soluzioni con attesa attiva non violano le quattro condizioni, ma sprecono CPU;
- una soluzione che assume “due processi, su una sola CPU, a velocità simile” viola la **condizione 2**.

### 6.3 Soluzioni con attesa attiva

#### 6.3.1 Disabilitare gli interrupt

È la soluzione più semplice da enunciare: si disabilitano gli interrupt, così nessuno può interrompere il processo mentre è nella sezione critica. [SO-02 p19]

#### Errore tipico da evitare

Disabilitare gli interrupt è una soluzione **pessima**, e l'esame chiede di saper dire perché: funziona solo su un sistema con una sola CPU (con più CPU, disabilitare gli interrupt sulla propria non ferma le altre), e soprattutto è un'**istruzione privilegiata**: se un programma utente potesse disabilitare gli interrupt, potrebbe impedire al sistema operativo di riprendere il controllo e tenersi la CPU per sempre. Va bene solo *dentro il kernel*, e solo per pochissime istruzioni.

#### 6.3.2 Variabili di lock

Si usa una variabile condivisa che il processo controlla prima di entrare nella sezione critica. È il tentativo più ingenuo, e non funziona: fra il momento in cui il processo *controlla* la variabile e il momento in cui la *imposta*, può essere interrotto, e un altro processo può fare altrettanto. Il difetto è lo stesso del conto corrente: controllare e impostare non è atomico. [SO-02 p19]

#### 6.3.3 Alternanza stretta

Si introduce una variabile **turn** che indica di chi è il turno: [SO-02 p19]

```
int N = 2
int turn

function enter_region(int process)
    while (turn != process) do nothing

function leave_region(int process)
    turn = 1 - process
```

Le proprietà che le slide attribuiscono a questa soluzione: [SO-02 p19]

- può essere facilmente generalizzata al caso  $N$ ;
- fa **busy waiting** (si parla di *spin lock*);
- implica **rigidi turni** tra le parti, e questo **viola la condizione 3**.

#### Errore tipico da evitare

Il difetto dell'alternanza stretta è il più sottile della lista, e va capito bene. Immagina che tocchi al processo A e che A non abbia bisogno di entrare nella sezione critica: A resta nella sua sezione *non* critica, e B — che invece vorrebbe entrare — resta bloccato ad aspettare il proprio turno. Ma B è **fuori** dalla sezione critica, quindi per la condizione 3 non dovrebbe poter essere bloccato da nessuno. Da qui la conclusione: non basta la mutua esclusione, serve anche che l'accesso non dipenda dalla buona volontà di chi non partecipa.

#### 6.3.4 La soluzione di Peterson

È la soluzione classica per due processi, e combina l'idea del turno con quella dell'interesse dichiarato: [SO-02 p20]

```
int N = 2
int turn
int interested[N]

function enter_region(int process)
    other = 1 - process
    interested[process] = true
    turn = process
    while (interested[other] == true and turn == process) do nothing

function leave_region(int process)
    interested[process] = false
```

I limiti dichiarati dalle slide: [SO-02 p20]

- è ancora **busy waiting**;
- può essere generalizzato al caso  $N$ ;
- può avere problemi sui moderni **multi-processori** a causa del **riordino degli accessi alla memoria centrale**.

#### Perché il riordino rompe Peterson

L'ultimo punto merita una spiegazione, perché è la domanda “cattiva” su questo argomento. Per motivi di prestazioni, il processore e il compilatore possono *riordinare* le scritture in memoria, purché il risultato sia identico *per quel singolo flusso di esecuzione*. Ma qui il risultato deve essere identico *anche per gli altri processori*, che leggono quelle stesse variabili. Se la scrittura di `turn` viene spostata prima di quella di `interested[process]`, l'altro processo può osservare uno stato intermedio e il ragionamento salta. Serve una **barriera di memoria** (*memory barrier*) che imponga l'ordine, oppure — ed è la soluzione che i sistemi reali adottano — un'istruzione hardware atomica, come vediamo adesso.

#### 6.3.5 Le istruzioni TSL e XCHG

Molte architetture, soprattutto multi-processore, offrono istruzioni specifiche che risolvono il problema alla radice: **TSL** (*Test and Set Lock*) e **XCHG** (*eXCHAnGe*). [SO-02 p21]

```

enter_region:          leave_region:
    TSL REGISTER, LOCK    MOVE  LOCK, #0
    CMP REGISTER, #0      RET
    JNE enter_region
    RET

```

### Che cosa rende speciale TSL

L'istruzione `TSL registro, lock` legge il valore di `lock` nel registro e **contemporaneamente** vi scrive 1. La proprietà decisiva, che la slide mette in evidenza, è che l'operazione è **atomica** e **blocca il bus di memoria**: nessun altro processore può accedere alla memoria fra la lettura e la scrittura. [SO-02 p21]

`XCHG` fa la stessa cosa scambiando il contenuto di due locazioni, ed è disponibile in tutte le CPU Intel x86. [SO-02 p21]

| Soluzione              | Meccanismo                  | Problema residuo                  |
|------------------------|-----------------------------|-----------------------------------|
| Disabilitare interrupt | privilegiata, solo kernel   | non funziona con più CPU          |
| Variabili di lock      | controllo di una variabile  | controllo e scrittura non atomici |
| Alternanza stretta     | variabile <code>turn</code> | <b>viola la condizione 3</b>      |
| Peterson               | interesse + turno           | riordino della memoria            |
| TSL / XCHG             | istruzione atomica hardware | <b>busy waiting</b>               |

### Cosa chiedono all'esame

Il filo logico di questa successione è ciò che il docente vuole sentire: si parte da soluzioni software, che falliscono perché non hanno un'operazione *atomica* su cui appoggiarsi; si arriva a una soluzione *hardware* (TSL), che l'atomicità ce l'ha per costruzione. Nota bene però cosa *non* cambia: TSL risolve la mutua esclusione ma **non** il busy waiting. Tutte le soluzioni viste finora fanno attendere il processo *consumando CPU*. È il problema che apre la sezione successiva.

## 6.4 Bloccare invece di girare a vuoto

### 6.4.1 Sleep e wakeup

Tutte le soluzioni viste fanno *spin lock*, e questo porta con sé un problema specifico che le slide nominano: **l'inversione di priorità**. [SO-02 p22]

La soluzione è dare al processo la possibilità di **bloccarsi in modo passivo**, cioè di essere rimosso dai processi pronti, attraverso due primitive: `sleep` e `wakeup`. [SO-02 p22]

### Inversione di priorità

Un processo a bassa priorità tiene occupata la CPU girando su uno spin lock in attesa di entrare in sezione critica; un processo ad alta priorità, che dovrebbe avere la precedenza, viene *prelazato* da quello a bassa priorità proprio a causa del suo busy waiting. Il processo "importante" aspetta quello "meno importante". È il problema che Windows, secondo la slide conclusiva del deck, attenua con apposite euristiche. [SO-02 p22, p47]

Il primo tentativo di soluzione si applica al **problema del produttore-consumatore** con **buffer limitato** di dimensione  $N$ : il produttore deposita elementi nel buffer, il consumatore li

preleva, e una variabile condivisa `count` tiene il numero di elementi presenti, inizialmente posta a 0. [SO-02 p22]

```
function producer()                function consumer()
  while (true) do                  while (true) do
    item = produce_item()          if (count == 0) sleep()
    if (count == N) sleep()         item = remove_item()
    insert_item(item)              count = count - 1
    count = count + 1              if (count == N - 1)
    if (count == 1)                wakeup(producer)
      wakeup(consumer)              consume_item(item)
```

La slide commenta questa soluzione dicendo che **non funziona bene**, e che serve un *bit di attesa del wakeup*. Vale la pena capire perché, perché è una domanda d'orale frequente.

#### La corsa fra `sleep` e `wakeup`

Il consumatore legge `count`, lo trova uguale a 0, e sta per chiamare `sleep`. Ma **prima** che ci arrivi, scatta un cambio di contesto: il produttore inserisce un elemento, porta `count` a 1, e chiama `wakeup(consumer)`. Il consumatore non è ancora “addormentato”, quindi la `wakeup` non ha effetto: è **persa**. Poi il consumatore riprende e chiama `sleep`. Adesso è addormentato davvero, ma nessuno lo sveglierà più: il produttore ha già mandato la sua `wakeup`, e nel frattempo `count` non tornerà a 1. Il consumatore dorme per sempre. La radice del problema è che **segnalare un evento non è la stessa cosa che ricordarsene** quando l'evento arriva troppo presto. Il *bit di attesa* delle slide serve esattamente a ricordarsene.

### 6.4.2 I semafori

Il concetto di `sleep` e `wakeup` viene generalizzato nel **semaforo**: [SO-02 p23]

- è una **variabile intera condivisa** `S`;
- le operazioni sono **down** e **up**, dette anche **wait** e **signal**;
- queste operazioni sono **atomiche**, realizzate con la disabilitazione degli interrupt oppure con spin lock TSL/XCHG;
- è tipicamente implementato **senza busy waiting**, con una **lista di processi bloccati**.

#### I due usi del semaforo

[SO-02 p23]

- **semaforo mutex**: serve alla **mutua esclusione**, inizializzato a 1;
- **conteggio risorse**: serve alla **sincronizzazione**, inizializzato al numero di risorse disponibili.

Inizializzare un semaforo a 1 e usarlo come se fosse un contatore è l'errore che fa fallire gli esercizi: il valore iniziale non è un dettaglio, è ciò che dà al semaforo il suo significato.

### 6.4.3 Produttore-consumatore con i semafori

È la soluzione corretta del problema, e va saputa a memoria: [SO-02 p24]

```
int N = 100
semaphore mutex = 1
semaphore empty = N
semaphore full = 0
```

```

function producer()
    while (true) do
        item = produce_item()
        down(empty)
        down(mutex)
        insert_item(item)
        up(mutex)
        up(full)

function consumer()
    while (true) do
        down(full)
        down(mutex)
        item = remove_item()
        up(mutex)
        up(empty)
        consume_item(item)

```

La slide chiude con un avvertimento che vale da solo una domanda d'esame: *l'ordine delle operazioni sui semafori è fondamentale*. [SO-02 p24]

#### Cosa chiedono all'esame

Perché l'ordine conta, e in che modo. Nel produttore `down(empty)` viene *prima* di `down(mutex)`. Se li invertissi, il produttore prenderebbe il mutex e poi si bloccherebbe su `empty` aspettando che il buffer si svuoti; ma per svuotarlo il consumatore deve prendere il mutex, che è in mano al produttore bloccato. **Deadlock**. La regola generale: prima si acquisisce il semaforo che rappresenta la *disponibilità della risorsa*, poi quello che protegge la sezione critica. Il semaforo “di risorsa” può bloccare; quello “di mutua esclusione” no, se l'ordine è giusto.

#### 6.4.4 Mutex e thread utente

Quando i thread sono tutti interni a un unico processo (modello 1-a-molti, capitolo 5), i mutex si possono implementare *efficientemente* in spazio utente usando TSL o XCHG: [SO-02 p25]

```

mutex_lock:                mutex_unlock:
    TSL REGISTER,MUTEX      MOVE MUTEX,#0
    CMP REGISTER,#0         RET
    JZE ok
    CALL thread_yield
    JMP mutex_lock
ok: RET

```

Il codice è simile a `enter_region/leave_region`, ma con una differenza decisiva: al posto dello spin lock c'è `thread_yield`. Il motivo è che il **busy waiting sarebbe problematico con i thread utente**: se tutti i thread dello stesso processo girassero a vuoto, nessuno di loro potrebbe mai rilasciare il lock, perché il kernel vede un solo processo e non ha motivo di togliere loro la CPU. Cedere volontariamente il controllo risolve. [SO-02 p25]

#### Due livelli di “blocco”, da non confondere

`thread_yield` restituisce il controllo *al sistema run-time della libreria*, in spazio utente: il thread resta “pronto” e verrà rieseguito. `sleep` invece è una primitiva del kernel e rimuove il processo dai pronti. Sono due cose diverse, e il motivo per cui qui basta la prima è che si sta parlando di thread utente dello stesso processo.

#### 6.4.5 I futex

L'osservazione da cui parte il meccanismo: i mutex in spazio utente sono molto efficienti, ma lo spin lock può essere lungo. Da qui i **futex** (*fast user space mutex*) di Linux. [SO-02 p26]

Un futex ha due componenti: [SO-02 p26]

- un **servizio del kernel**, con la coda dei thread bloccati;

- una **libreria utente**, con la variabile di lock, la gestione della contesa in modalità utente (tipo TSL/XCHG) e il richiamo al kernel **solo in caso di bloccaggio**.

#### L'idea dei futex in una frase

Nella stragrande maggioranza dei casi il lock è libero e si acquisisce in spazio utente, senza entrare nel kernel. Si paga il costo della system call solo nel caso raro di contesa. È lo stesso principio della paginazione (capitolo 12): ottimizzare per il caso comune, e pagare il costo pieno solo per quello raro.

## 6.5 I monitor

Il **monitor** è un costrutto di **alto livello**, disponibile in alcuni linguaggi di programmazione. È un **tipo astratto di dato** (variabili più procedure) con due garanzie: [SO-02 p27]

- **garanzia di mutua esclusione**: solo un processo per volta può essere attivo dentro il monitor;
- **vincolo di accesso ai dati**: i dati interni sono accessibili solo attraverso le procedure del monitor.

Esiste inoltre una **coda di attesa interna** al monitor.

Il meccanismo di sincronizzazione sono le **variabili condizione**, con le operazioni **wait** e **signal**; esiste una **coda di attesa per ogni variabile**. [SO-02 p27]

### 6.5.1 Le tre semantiche della signal

Questo è un punto tecnico che distingue chi ha capito il monitor da chi l'ha solo letto. Dopo una **signal**, il processo che l'ha eseguita e il processo risvegliato sono *entrambi* attivi dentro il monitor — ma la mutua esclusione ne ammette uno solo. Le tre semantiche possibili sono: [SO-02 p27]

| Semantica                    | Dove si trova                      | Cosa fa chi ha segnalato                                                                                                 |
|------------------------------|------------------------------------|--------------------------------------------------------------------------------------------------------------------------|
| <b>signal &amp; wait</b>     | monitor di Hoare<br>(teorico)      | Sospende sé stesso e cede il monitor al processo risvegliato; riprenderà dopo.                                           |
| <b>signal &amp; continue</b> | monitor di Mesa (Java)             | Continua a eseguire; il risvegliato dovrà riacquisire il monitor e nel frattempo la condizione potrebbe essere cambiata. |
| <b>signal &amp; return</b>   | compromesso<br>(Concurrent Pascal) | Considera conclusa la propria attività nel monitor.                                                                      |

#### Errore tipico da evitare

La conseguenza pratica della semantica di Mesa (**signal & continue**), quella di Java, è che **la condizione va ricontrollata in un ciclo**:

```
while (condizione) wait();    // corretto
if (condizione) wait();      // sbagliato con signal & continue
```

Chi scrive **if** invece di **while** sta assumendo la semantica di Hoare, dove nessuno può toccare lo stato fra la **signal** e la ripresa. In Java quel **if** produce errori che si manifestano raramente e in modo non deterministico: esattamente il tipo di bug che la guida vuole farti riconoscere.

### 6.5.2 Produttore-consumatore con i monitor

[SO-02 p28]

```
monitor pc_monitor
  condition full, empty;
  integer count = 0;

  function insert(item)
    if count == N then wait(full);
    insert_item(item);
    count = count + 1;
    if count = 1 then signal(empty)

  function remove()
    if count = 0 then wait(empty);
    remove_item();
    count = count - 1;
    if count = N-1 then signal(full)
```

Le funzioni producer e consumer diventano banali: chiamano `pc_monitor.insert(item)` e `pc_monitor.remove()`, e tutta la sincronizzazione è *dentro* il monitor. [SO-02 p28]

#### Cosa chiedono all'esame

Il confronto semafori / monitor, se te lo chiedono, si fa su due assi:

- **chi gestisce la mutua esclusione:** con i semafori è il programmatore a ricordarsi di chiamare **down** e **up** nell'ordine giusto (e può sbagliare — vedi il deadlock dell'ordine dei semafori); con i monitor è **il compilatore** a garantire che un solo processo per volta sia attivo dentro, quindi non ci si può dimenticare.
- **dove sta la logica di sincronizzazione:** con i semafori è *sparsa* nel codice di produttore e consumatore; con i monitor è *incapsulata* nel monitor.

È il motivo per cui i monitor nascono: ridurre la probabilità di errore spostando la correttezza dal programmatore al linguaggio.

## 6.6 Scambio di messaggi tra processi

Le primitive di livello più alto sono due: [SO-02 p29]

- `send(destinazione, messaggio);`
- `receive(sorgente, messaggio)`, **bloccante per il chiamante** (oppure può restituire un errore).

Le proprietà che le rendono utili: [SO-02 p29]

- sono **estendibili al caso di più macchine** (per esempio con la libreria MPI) — cosa che semafori e monitor non sono, perché presuppongono memoria condivisa;
- i metodi di indirizzamento possono essere **diretti** oppure **tramite mailbox**;
- assumendo realisticamente l'esistenza di un buffer per i messaggi, di **capienza finita**  $N$ , anche la **send** può essere bloccante.



### La ragione profonda dello scambio di messaggi

I semafori e i monitor si appoggiano su **memoria condivisa**: presuppongono che due processi possano leggere la stessa variabile. Su macchine diverse questa possibilità non esiste, e allora la sincronizzazione deve passare per la comunicazione esplicita di messaggi. È questa la ragione per cui esistono: non sono “un’alternativa più elegante”, sono l’*unica* alternativa quando la memoria non è condivisa.

### Come lo dici all’orale

“Mi parli della mutua esclusione.” La scaletta che funziona, e che mostra di aver capito la struttura dell’argomento: (1) il problema — la corsa critica, con l’esempio del conto corrente, e il fatto che un’operazione non atomica si può inframmezzare; (2) le **quattro condizioni** di una buona soluzione; (3) la successione dei tentativi, dicendo per ciascuno *quale* condizione o quale proprietà fallisce: disabilitare gli interrupt (solo kernel, non scala a più CPU), variabili di lock (non atomico), alternanza stretta (viola la condizione 3), Peterson (riordino in memoria), TSL (atomico ma busy waiting); (4) il salto di qualità: **sleep/wakeup**, con il problema della corsa del wakeup perso, e i **semafori** che lo risolvono; (5) i **monitor** come costrutto di alto livello che sposta la correttezza dal programmatore al linguaggio. Chi racconta questa successione come una *storia* ha già preso il massimo su questa domanda.

## Capitolo 7

### I problemi classici: filosofi e lettori-scrittori

---

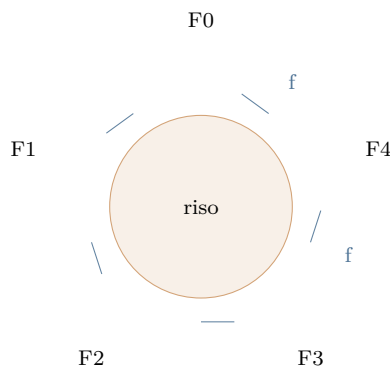
I due problemi di questo capitolo sono classici non perché siano difficili, ma perché sono **modelli**: il primo rappresenta l'accesso esclusivo a un numero *limitato* di risorse da parte di processi in concorrenza, il secondo rappresenta l'accesso a un database con letture e scritture. [SO-02 p30, p33]

Sono anche i due esercizi su cui l'esame insiste di più, perché si prestano a far ragionare sui *difetti* delle soluzioni: si mostra una soluzione che sembra corretta, e si chiede che cosa può andare storto.

#### 7.1 Il problema dei cinque filosofi

##### 7.1.1 Il problema

Cinque filosofi siedono attorno a un tavolo rotondo. Nel mezzo c'è una portata di riso, e attorno al tavolo ci sono **cinque forchette**, una fra un filosofo e l'altro. Ogni filosofo passa la vita alternando due attività: **pensare** e **mangiare**. Per mangiare gli servono *due* forchette: quella alla sua sinistra e quella alla sua destra. [SO-02 p30]



Il numero di forchette è pari al numero di filosofi, e ogni forchetta è condivisa da due filosofi adiacenti: è questo che rende il problema interessante. Non c'è abbastanza per tutti, e chi ne ha una sola non può mangiare.

##### 7.1.2 Le quattro soluzioni discusse nelle slides

[SO-02 p30]

**Soluzione 1: prendi la prima, poi la seconda.** È il tentativo ovvio:

```
int N = 5
function philosopher(int i)
    think()
    take_fork(i)
    take_fork((i+1) mod N)
    eat()
    put_fork(i)
    put_fork((i+1) mod N)
```

**Errore tipico da evitare**

Questa soluzione si blocca. Se tutti e cinque i filosofi afferrano *contemporaneamente* la forchetta alla propria sinistra, ognuno tiene una forchetta e aspetta quella del vicino, che è tenuta da qualcun altro. È un **deadlock**: nessuno può proseguire, e la condizione non si sblocca da sola. Nota che il difetto *non* è nella mutua esclusione — le forchette sono correttamente inaccessibili a due filosofi insieme — ma nell'**ordine di acquisizione** delle risorse. È lo stesso difetto che abbiamo visto nel capitolo 6 a proposito dell'ordine dei semafori nel produttore-consumatore. Attenzione: la soluzione 1 non “prende le forchette in modo non atomico”, come si sente dire spesso; il problema è che ogni filosofo **trattiene** ciò che ha già preso.

**Soluzione 2: controlla con rilascio, riprova.** Si prende la prima forchetta; se la seconda non è disponibile, si **rilascia la prima** e si riprova. [SO-02 p30]

**Soluzione 3: come la 2, ma aspettando un tempo casuale prima di riprovare.** L'attesa casuale serve a **rompere la simmetria**: se tutti riprovano nello stesso istante, possono continuare a collidere all'infinito (*livelock*). Aspettare tempi diversi fa sì che i turni si diversifichino. [SO-02 p30]

**Soluzione 4: un semaforo mutex.** Si introduce un semaforo di mutua esclusione che serializza una parte dell'operazione. [SO-02 p30]

**7.1.3 La soluzione con i semafori**

È la soluzione “buona” del problema, e va saputa ricostruire: [SO-02 p31]

```
int N = 5;
int THINKING = 0; int HUNGRY = 1; int EATING = 2;
int state[N]
semaphore mutex = 1
semaphore s[N] = {0,...,0}

function philosopher(int i)
    while (true) do
        think()
        take_forks(i)
        eat()
        put_forks(i)

function take_forks(int i)
    down(mutex)
    state[i] = HUNGRY
    test(i)
    up(mutex)
    down(s[i])

function put_forks(int i)
    down(mutex)
    state[i] = THINKING
    test(left(i))
    test(right(i))
    up(mutex)

function left(int i) = i-1 mod N
function right(int i) = i+1 mod N

function test(int i)
    if state[i] = HUNGRY and state[left(i)] != EATING and state[right(i)] != EATING
```

```
state[i] = EATING
up(s[i])
```

#### Che cosa fa funzionare questa soluzione

Tre cose, e tutte e tre vanno dette.

1. Il filosofo **non prende le forchette**. Non c'è nessuna `take_fork`: c'è un **invito** a mangiare. La variabile `state[i]` registra l'*intenzione*, e il filosofo si sospende su `down(s[i])`.
2. Il filosofo **mangia solo quando può davvero mangiare**, e la decisione la prende `test()`, che controlla *entrambi* i vicini. Poiché la decisione è presa dentro una sezione critica protetta da `mutex`, non può accadere che due filosofi vicini si vedano entrambi "autorizzati".
3. Il filosofo che ha finito **prova a svegliare i vicini** (`test(left(i))` e `test(right(i))`), invece di rilasciare semplicemente delle risorse: sono i vicini, e solo loro, a poter essere diventati "mangiabili".

Il "trucco" è che il blocco della soluzione 1 sparisce perché l'acquisizione delle due forchette è diventata **atomica**: avviene tutta dentro la sezione critica su `mutex`, e chi non le ottiene *non ne trattiene nessuna*.

#### Perché non basta prendere le forchette dentro il mutex

La domanda che segue naturalmente: "e allora perché non si mettono semplicemente `take_fork` e `take_fork` dentro un `down(mutex)/up(mutex)`?" Risposta: perché il filosofo resterebbe **bloccato dentro la sezione critica** aspettando la seconda forchetta, impedendo a tutti gli altri di entrare e di rilasciare la propria. Si trasformerebbe un deadlock in una serializzazione completa: un solo filosofo per volta potrebbe *tentare*, e per di più tenendo fermo il mutex mentre aspetta. La soluzione corretta è quella sopra, in cui la sezione critica **non contiene attese**: si entra, si decide, si esce.

#### 7.1.4 La soluzione con i monitor

[SO-02 p32]

```
int N = 5; int THINKING = 0; int HUNGRY = 1; int EATING = 2

monitor dp_monitor
    int state[N]
    condition self[N]

    function take_forks(int i)
        state[i] = HUNGRY
        test(i)
        if state[i] != EATING
            wait(self[i])

    function put_forks(int i)
        state[i] = THINKING;
        test(left(i));
        test(right(i));

    function test(int i)
        if ( state[left(i)] != EATING and state[i] == HUNGRY
```

```

        and state[right(i)] != EATING )
    state[i] = EATING
    signal(self[i])

```

Il programma di ogni filosofo è ormai una sequenza di tre chiamate:

```
dp_monitor.take_forks(i) → eat() → dp_monitor.put_forks(i)
```

La mutua esclusione sul monitor è garantita dal costrutto, quindi il semaforo **mutex** non serve più, e **non compare nemmeno una down**. [SO-02 p32]

#### Il confronto fra le due soluzioni

Le due versioni sono *strutturalmente identiche*: stesso vettore **state**, stessa funzione **test**, stesso schema “decidi e poi eventualmente sospenditi”. Cambia solo il meccanismo:

- la versione a semafori protegge **state** con **mutex** (esplicito) e sospende su **down(s[i])** (uno per filosofo);
- la versione a monitor ha la mutua esclusione implicita nel costrutto e sospende su **wait(self[i])** (una variabile condizione per filosofo).

All’orale, dire *questo* — che è lo stesso algoritmo con due meccanismi diversi — vale più che recitare due codici a memoria.

## 7.2 Il problema dei lettori e scrittori

### 7.2.1 Il problema

Un **database** è accessibile a più processi: i **lettori** leggono, gli **scrittori** modificano. Le regole sono due: [SO-02 p33]

- più lettori possono accedere **contemporaneamente** (leggere non disturba nessuno);
- se c’è uno scrittore, **nessun altro** può accedere — né un altro scrittore, né un lettore.

### 7.2.2 La soluzione con i semafori

[SO-02 p33]

```

semaphore mutex = 1
semaphore db    = 1
int rc = 0

function reader()
    while true do
        down(mutex)
        rc = rc + 1
        if (rc == 1)
            down(db)
        up(mutex)
        read_database()
        down(mutex)
        rc = rc - 1
        if (rc == 0) up(db)
        up(mutex)
        use_data_read()

function writer()
    while true do
        think_up_data()
        down(db)
        write_database()
        up(db)

```

### Come si legge questo codice

Il ruolo dei due semafori è ben distinto, e confonderli è l'errore classico:

- **db** è il semaforo *del database*: protegge l'accesso vero e proprio. Il *primo* lettore lo prende (`if (rc == 1) down(db)`), l'*ultimo* lo rilascia (`if (rc == 0) up(db)`). Nel mezzo, gli altri lettori passano senza toccarlo — ed è così che più lettori stanno dentro insieme.
- **mutex** serve solo a rendere **atomico** il contatore `rc`. Senza di esso, due lettori potrebbero leggere `rc` entrambi uguali a 0, entrambi concludere di essere “il primo”, e il secondo `down(db)` bloccherebbe l'uno sull'altro — oppure entrambi incrementarlo e registrare un lettore in meno del reale, lasciando `db` acquisito per sempre.

Il campo `rc` è l'esempio perfetto di **risorsa condivisa da proteggere**: non contiene i dati del database, ma è esso stesso un dato condiviso, e quindi ha bisogno della sua mutua esclusione.

La slide segnala però un difetto della soluzione, ed è il punto più importante di tutta la sezione: **lo scrittore potrebbe attendere per un tempo indefinito**. [SO-02 p33]

### Starvation dello scrittore

Il meccanismo “il primo lettore prende `db`, l'ultimo lo rilascia” non dà alcuna garanzia sull'ordine di arrivo. Se i lettori arrivano di continuo, `rc` non torna mai a zero, `db` non viene mai rilasciato, e uno scrittore che aspetta può aspettare **all'infinito**. Non è un deadlock (i lettori proseguono, il sistema non si blocca) ma una **starvation**, che è un problema diverso e va nominata con il termine giusto: nessun processo è in attesa circolare, è semplicemente *sfortunato* nella selezione.

### 7.2.3 Le tre soluzioni con i monitor

Le slides presentano *tre* versioni della stessa soluzione a monitor, e la cosa interessante è che **differiscono per pochissime righe**. Le riportiamo prima di commentarle, perché il confronto è esattamente ciò che l'esame vuole.

Il blocco comune a tutte e tre è: [SO-02 p34]

```
monitor rw_monitor
    int rc = 0; boolean busy_on_write = false
    condition read, write

    function end_read()
        rc = rc-1
        if (rc == 0) signal(write)

    function start_write()
        if (rc > 0 OR busy_on_write) wait(write)
        busy_on_write = true
```

### Soluzione n. 1. [SO-02 p34]

```
function start_read()
    if (busy_on_write) wait(read)
    rc = rc+1
    signal(read)

function end_write()
```

```

busy_on_write = false
if (in_queue(read)) signal(read)
else signal(write)

```

**Soluzione n. 2.** Cambia una sola riga, in `start_read`: [SO-02 p35]

```

function start_read()
  if (busy_on_write OR in_queue(write)) wait(read)
  rc = rc+1
  signal(read)

```

**Soluzione n. 3.** Rispetto alla n. 2, cambia una sola riga, in `end_write`: [SO-02 p36]

```

function end_write()
  busy_on_write = false
  if (in_queue(write)) signal(write)
  else signal(read)

```

|             | Che cosa cambia                                                                                                                  | Effetto                                                                                                                                                                                                      |
|-------------|----------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| n. 1 → n. 2 | In <code>start_read</code> , la condizione di attesa diventa <code>busy_on_write OR in_queue(write)</code>                       | Un lettore che arriva mentre c'è già uno scrittore in coda si mette in attesa <i>anche se</i> nessuno sta scrivendo. Serve a non far passare i lettori “saltando la fila” davanti a uno scrittore in attesa. |
| n. 2 → n. 3 | In <code>end_write</code> , si preferisce risvegliare uno <b>scrittore</b> ( <code>in_queue(write)</code> ) invece di un lettore | Dà la precedenza agli scrittori.                                                                                                                                                                             |

#### Il significato complessivo delle tre versioni

È una **scala di priorità crescente agli scrittori**, e va letta come tale:

- la **n. 1** è la più sbilanciata verso i lettori: se ci sono lettori in coda, si sveglia un lettore, e lo scrittore può attendere a lungo (la stessa starvation della versione a semafori);
- la **n. 2** impedisce ai lettori *nuovi* di infilersi davanti a uno scrittore già in attesa;
- la **n. 3** al momento del rilascio preferisce esplicitamente lo scrittore.

Il termine tecnico per ciò che si guadagna passando da 1 a 3 è **fairness** (equità): si accetta di far attendere i lettori per non lasciare gli scrittori in attesa indefinita. Nota che *nessuna* delle tre è “sbagliata”: sono tre scelte di politica diverse, e la domanda d’esame è saper dire *quale* preferisce chi.

#### La funzione `in_queue`

Le tre soluzioni usano `in_queue(read)` e `in_queue(write)`, cioè “c’è qualcuno in attesa su questa variabile condizione?”. È una funzione di *ispezione* della coda del monitor, che non esiste in tutti i linguaggi: in Java, per esempio, si usa `isEmpty()` della coda equivalente, oppure si tiene un contatore esplicito. Il concetto da ricordare è che i monitor espongono, oltre a `wait` e `signal`, anche un modo per **guardare** le code di attesa — e che le soluzioni

di fairness si costruiscono quasi sempre così: rendendo la decisione *dipendente da chi sta aspettando*, non solo dallo stato.

#### Come lo dici all'orale

I due problemi di questo capitolo si presentano bene con la stessa scaletta, e conviene usarla per entrambi:

1. **che cosa modella** il problema, in una frase (filosofi: accesso esclusivo a risorse limitate; lettori-scrittori: accesso a un database);
2. il **tentativo ingenuo** e il suo difetto, nominato con il termine giusto (filosofi: deadlock, tutti prendono la forchetta sinistra; lettori-scrittori: starvation dello scrittore);
3. la **soluzione corretta** con lo schema “decidi dentro la sezione critica, poi sospenditi fuori” — e il fatto che la sezione critica non contenga attese;
4. se te lo chiedono, la **versione a monitor**, dicendo che è lo stesso algoritmo con la mutua esclusione implicita.

Su lettori-scrittori aggiungi sempre il confronto fra le tre versioni: è lì che si vede se hai capito la differenza fra “corretto” e “equo”.



## Parte IV

# Scheduling della CPU

## Capitolo 8

### Scheduler e dispatcher

---

#### 8.1 Che cosa fa lo scheduler

Quando in un sistema ci sono più processi pronti e una sola CPU, qualcuno deve decidere chi va avanti. Quel qualcuno è lo **scheduler**, e la regola con cui decide è l'**algoritmo di scheduling**. [SO-02 p37]

Prima di entrare negli algoritmi conviene fissare un vocabolario, perché è su questo che si perdono i punti facili.

##### 8.1.1 Due tipi di processo

[SO-02 p37]

###### Processi CPU-bounded e I/O-bounded

- un processo è **CPU-bounded** quando passa la maggior parte del tempo a *calcolare*, e quindi tende a tenere occupata la CPU per lunghi tratti;
- un processo è **I/O-bounded** quando passa la maggior parte del tempo *in attesa di I/O*, e quindi usa la CPU in brevi periodi intervallati da lunghe attese.

[SO-02 p37]

###### Perché questa distinzione conta

Un processo I/O-bounded lascia la CPU quasi subito (perché va a fare I/O) e quindi, in un buon scheduler, non è un problema: è anzi *utile* servirlo *subito*, perché libera in fretta la CPU e le periferiche lavorano in parallelo. Un processo CPU-bounded invece occupa la CPU a lungo, e penalizza tutti gli altri.

È la ragione per cui quasi tutti gli scheduler reali “prediligono” i processi I/O-bounded (lo dice anche la slide conclusiva su Windows e Linux, [SO-02 p47]) e per cui lo SJF, che dà la precedenza ai lavori brevi, funziona bene in pratica: i lavori brevi sono spesso proprio quelli I/O-bounded.

##### 8.1.2 Quando viene attivato lo scheduler

Le occasioni in cui lo scheduler può essere chiamato a decidere sono: [SO-02 p37]

1. la **terminazione** (e la **creazione**) di un processo;
2. una **chiamata bloccante** (per esempio un'I/O) e l'arrivo del relativo interrupt;
3. gli **interrupt periodici**.

È l'ultimo punto a introdurre la distinzione più importante del capitolo.

###### Sistemi con e senza prelazione

- nei sistemi **non-preemptive** (senza prelazione) il processo in esecuzione **non viene mai interrotto**: lo scheduler decide solo quando il processo si blocca o termina;
- nei sistemi **preemptive** (con prelazione) lo scheduler può togliere la CPU a un processo ancora in esecuzione, tipicamente allo scadere di un interrupt periodico.

[SO-02 p37]

### Cosa chiedono all'esame

La prelazione è un *costo* che si paga in cambio di una *reattività*. Un sistema con prelazione deve saper sospendere un processo in qualunque punto della sua esecuzione e riprenderlo esattamente da lì: serve salvare e ripristinare tutto il contesto (registri, PC, stato della memoria). Senza prelazione questo non serve, ma un processo che calcola per un'ora tiene la CPU per un'ora, e in un sistema interattivo questo è inaccettabile. Da qui la corrispondenza che vale la pena ricordare: **i sistemi batch tollerano l'assenza di prelazione, i sistemi interattivi no.**

## 8.2 Il dispatcher

Lo scheduler decide *chi*; il **dispatcher** è colui che *esegue la decisione*. Lo scheduler **collabora** con il dispatcher, e il tempo che passa fra la decisione e l'effettivo inizio dell'esecuzione si chiama **latenza di dispatch**. [SO-02 p37]

### Latenza di dispatch

È il tempo necessario a **effettuare il passaggio di contesto** da un processo all'altro: salvare lo stato del processo uscente, caricare quello del processo entrante, e — nei sistemi con memoria virtuale — riprogrammare la MMU. È tempo **speso dal sistema e non dal programma**, quindi va tenuto piccolo. [SO-02 p37]

### Scheduler e dispatcher in una riga

Lo **scheduler** è il *decisore* (una politica); il **dispatcher** è l'*esecutore* (un meccanismo). È la stessa separazione concettuale che si ritrova in altre parti del corso: la politica di sostituzione delle pagine (capitolo 13) decide *quale* pagina togliere, mentre il meccanismo che poi la scrive su disco è un'altra cosa.

## 8.3 Gli obiettivi di un algoritmo di scheduling

Non esiste “il migliore” algoritmo di scheduling in assoluto: esiste il migliore *per un certo ambiente*. È il punto da cui parte la slide. [SO-02 p38]

Gli ambienti sono tre: **batch**, **interattivi** e **real-time**. [SO-02 p38]

### 8.3.1 Gli obiettivi comuni

[SO-02 p38]

- **equità** nell'assegnazione della CPU;
- **bilanciamento** nell'uso delle risorse.

### 8.3.2 Gli obiettivi tipici dei sistemi batch

[SO-02 p38]

- massimizzare il **throughput** (o produttività);
- minimizzare il **tempo di turnaround** (o tempo di completamento);
- minimizzare il **tempo di attesa**.

### 8.3.3 Gli obiettivi tipici dei sistemi interattivi

[SO-02 p38]

- minimizzare il **tempo di risposta**.

### 8.3.4 Gli obiettivi tipici dei sistemi real-time

[SO-02 p38]

- rispetto delle **scadenze** (*deadline*);
- **prevedibilità**.

| Ambiente           | Obiettivo principale           | Perché                                                                                                           |
|--------------------|--------------------------------|------------------------------------------------------------------------------------------------------------------|
| <b>batch</b>       | throughput, turnaround, attesa | Non c'è nessuno che aspetta davanti a uno schermo: si ottimizza il lavoro complessivo svolto.                    |
| <b>interattivo</b> | tempo di risposta              | L'utente è lì: conta il tempo che passa fra il suo comando e la prima reazione del sistema, non il tempo totale. |
| <b>real-time</b>   | scadenze, prevedibilità        | Un risultato giusto ma in ritardo è un risultato <i>sbagliato</i> . Non conta la media, conta il caso peggiore.  |

#### Le tre misure di tempo, da non confondere

Sono le grandezze su cui si fanno TUTTI gli esercizi, e vanno sapute distinguere:

- il **tempo di turnaround** (tempo di completamento) è l'intervallo fra l'**arrivo** del processo e la sua **conclusione**: quanto ha dovuto aspettare *in tutto*, coda compresa;
- il **tempo di attesa** è il tempo passato **in coda senza essere eseguito**: si ottiene come attesa = turnaround – durata del burst (l'unica formula di questo capitolo che vale la pena mandare a memoria);
- il **tempo di risposta** è l'intervallo fra l'arrivo e il **primo istante** in cui il processo ottiene la CPU.

I tre valori coincidono per un processo che non viene mai interrotto e non aspetta nulla; differiscono in tutti gli altri casi. Il tempo di attesa è esattamente il turnaround meno il tempo in cui il processo ha effettivamente lavorato; il tempo di risposta è l'unico dei tre che non dipende da quanto dura il processo.

#### Turnaround e tempo di risposta non sono la stessa cosa

Un processo lungo che ottiene la CPU subito e la tiene fino alla fine ha un tempo di **risposta** piccolissimo (parte immediatamente) ma un turnaround grande (dura molto). Un processo breve che arriva quando la CPU è occupata da un lavoro lungo ha un turnaround piccolo ma un tempo di risposta grande. Ecco perché un sistema interattivo ottimizza il *tempo di risposta*: vuole che *tutto* cominci subito, anche se poi finirà dopo.

#### Come lo dici all'orale

“Differenza fra scheduler e dispatcher?” La risposta breve: lo scheduler è il **decisore**, il dispatcher è l'**esecutore**; il tempo che il dispatcher impiega a fare il passaggio di contesto

è la **latenza di dispatch**. Se la domanda si allarga a “quali obiettivi ha uno scheduler?”, la risposta che fa la differenza è dire che **gli obiettivi dipendono dall’ambiente**: batch, interattivo, real-time hanno metriche diverse, e chiedere un algoritmo “ottimo” senza specificare l’ambiente è una domanda mal posta. Portare questa risposta a una domanda generica sugli algoritmi mostra che hai capito la struttura dell’argomento, non solo le formule.

## Capitolo 9

### Gli algoritmi di scheduling

Questo è il capitolo con più cose da ricordare e meno da capire: gli algoritmi sono semplici, il rischio è confonderli fra loro. Conviene tenerli divisi per *ambiente*, che è il criterio con cui le slide stesse li presentano.

| Ambiente    | Algoritmi                                                         | Idea comune                                                                                                     |
|-------------|-------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|
| batch       | FCFS, SJF, SRTN                                                   | Non c'è nessuno che aspetta: si massimizza il lavoro svolto e si minimizzano i tempi medi.                      |
| interattivo | RR, priorità, code multiple, SPN, garantito, lotteria, fair-share | C'è un utente davanti allo schermo: conta la reattività, e si accetta di sacrificare l'efficienza per l'equità. |

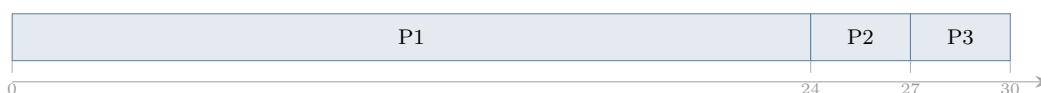
#### 9.1 Gli algoritmi dei sistemi batch

##### 9.1.1 FCFS — First-Come First-Served

È l'algoritmo più semplice: chi arriva prima viene servito prima (*per ordine di arrivo*). È **non-preemptive** e si realizza con una semplice coda **FIFO**. [SO-02 p39]

**Esempio delle slides.** Tre processi tutti pronti all'istante 0: [SO-02 p39]

| Processo | Arrivo | Durata |
|----------|--------|--------|
| P1       | 0      | 24     |
| P2       | 0      | 3      |
| P3       | 0      | 3      |



Le medie della slide: tempo medio di attesa  $\frac{0+24+27}{3} = 17$ ; tempo medio di completamento  $\frac{24+27+30}{3} = 27$ . [SO-02 p39]

#### L'effetto convoglio

L'esempio è costruito apposta per mostrare il difetto del FCFS: un processo lungo (P1, 24 unità) blocca due processi cortissimi (P2 e P3, 3 unità ciascuno). Il tempo medio di attesa è 17, eppure il lavoro "utile" dei tre processi è solo 30 unità: quasi tutto il tempo è attesa. Si chiama **effetto convoglio**, ed è esattamente ciò che lo SJF elimina.

##### 9.1.2 SJF — Shortest Job First

Si sceglie il lavoro con la **durata minore**. È **non-preemptive** e **presuppone la conoscenza del tempo impiegato da ogni lavoro**. Il punto decisivo, che la slide mette in evidenza, è che è **ottimale solo se i lavori sono tutti disponibili subito**. [SO-02 p39]

**Esempio delle slides: SJF non è ottimale.** [SO-02 p39]

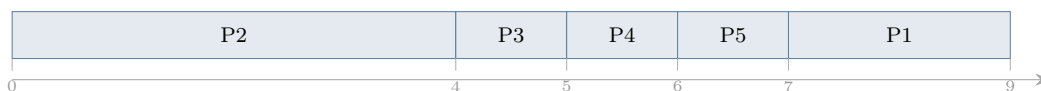
| Processo | Arrivo | Durata |
|----------|--------|--------|
| P1       | 0      | 2      |
| P2       | 0      | 4      |
| P3       | 3      | 1      |
| P4       | 3      | 1      |
| P5       | 3      | 1      |

Con lo SJF, all'istante 0 sono pronti solo P1 (durata 2) e P2 (durata 4): si sceglie P1. A P1, che è non-preemptive, si applica la regola una volta sola, quindi si prende comunque il più corto dei due.



Tempo medio di attesa =  $\frac{0+2+3+4+5}{5} = 2,8$ . [SO-02 p39]

Consideriamo invece la sequenza alternativa in cui si serve *subito* P2, che dura di più:



Tempo medio di attesa =  $\frac{0+1+2+3+7}{5} = 2,6$ . [SO-02 p39]

#### Cosa chiedono all'esame

$2,6 < 2,8$ : la seconda sequenza è **migliore**, e non è quella prodotta dallo SJF. Sembra un paradosso — “il più corto per primo” che non dà il risultato migliore — e la spiegazione è tutta nell’*istante* in cui la scelta viene fatta: all’istante 0 lo SJF non può sapere che alle 3 arriveranno tre lavori da 1 unità, perché non sono ancora arrivati. Lo SJF è ottimale **solo se tutti i lavori sono disponibili da subito**, e questa è precisamente la condizione che nel secondo esempio manca. È il motivo per cui serve la versione con prelazione.

### 9.1.3 SRTN — Shortest Remaining Time Next

È la **versione preemptive dello SJF**: si sceglie il processo a cui *resta* meno tempo da eseguire, e ogni volta che arriva un processo nuovo si riconsidera la scelta. [SO-02 p39]

### 9.1.4 Il confronto completo sulle slides

Le tre slides successive usano la **stessa tabella** per FCFS, SJF e SRTN, il che le rende il modo migliore per studiare le differenze. [SO-02 p40]

| Processo | Arrivo | Durata |
|----------|--------|--------|
| P1       | 0      | 7      |
| P2       | 2      | 4      |
| P3       | 4      | 1      |
| P4       | 5      | 4      |

**FCFS.**

Tempi di attesa:  $P1 = 0$ ;  $P2 = 5$ ;  $P3 = 7$ ;  $P4 = 7$  (media 4,75). Tempi di completamento:  $P1 = 7$ ;  $P2 = 9$ ;  $P3 = 8$ ;  $P4 = 11$  (media 8,75). [SO-02 p40]

**SJF.**

Tempi di attesa:  $P1 = 0$ ;  $P2 = 6$ ;  $P3 = 3$ ;  $P4 = 7$  (media 4). Tempi di completamento:  $P1 = 7$ ;  $P2 = 10$ ;  $P3 = 4$ ;  $P4 = 11$  (media 8). [SO-02 p40]

**SRTN.**

Tempi di attesa:  $P1 = 9$ ;  $P2 = 1$ ;  $P3 = 0$ ;  $P4 = 2$  (media 3). Tempi di completamento:  $P1 = 16$ ;  $P2 = 5$ ;  $P3 = 1$ ;  $P4 = 6$  (media 7). [SO-02 p40]

| Algoritmo | Attesa media | Completamento medio | Prelazione |
|-----------|--------------|---------------------|------------|
| FCFS      | 4,75         | 8,75                | no         |
| SJF       | 4,00         | 8,00                | no         |
| SRTN      | <b>3,00</b>  | <b>7,00</b>         | sì         |

**Che cosa mostrano questi tre diagrammi**

Tre cose, e sono tutte domande possibili.

1. **Il SRTN vince su entrambe le medie**, ed è il motivo per cui si introduce la prelazione: la scelta “a posteriori” di chi ha meno tempo residuo è sempre quella localmente migliore.
2. **P1 peggiora passando da FCFS a SRTN**: il suo tempo di completamento passa da 7 a 16, e la sua attesa da 0 a 9. È il processo lungo, e viene continuamente interrotto dai lavori brevi che arrivano. Questo mostra che **ottimizzare la media non significa migliorare tutti**: c'è sempre qualcuno che paga.
3. **P3, che arriva alle 4 e dura 1, con SRTN viene servito subito** (attesa 0): il processo che arriva *ultimo* viene servito *per primo*. È controintuitivo e va notato esplicitamente in un esercizio, perché è la prova che hai applicato la regola giusta e non una FIFO travestita.

**Una trappola nel calcolo dell'attesa con prelazione**

Con FCFS e SJF il processo viene eseguito in un blocco unico, quindi la formula attesa = completamento – durata si applica “a occhio”. Con SRTN no: l'attesa di P1 è la **somma di tutti i pezzi in cui P1 ha aspettato** (da 2 a 11, più il primo istante), e non un intervallo singolo. Per questo conviene sempre calcolare attesa = (istante di fine – istante di arrivo) –



durata, che vale in tutti i casi: è l'unica formula che regge anche con la prelazione.

## 9.2 Gli algoritmi dei sistemi interattivi

### 9.2.1 RR — Round-Robin

È la **versione con prelazione del FCFS**, basata su un **quanto di tempo** (*timeslice*). [SO-02 p41]

Il funzionamento: ogni processo riceve la CPU per un quanto; se non ha finito quando il quanto scade, viene **messo in fondo alla coda** e si aspetta il suo prossimo turno.

**Quanto deve essere lungo il timeslice?** Valori tipici: **20–50 ms**. [SO-02 p41]

Con  $n$  processi e un quanto di  $q$  ms, ogni processo avrà diritto a circa  $\frac{1}{n}$  della CPU e **attenderà al più**  $(n-1)q$  ms. [SO-02 p41]

#### Il compromesso sul quanto di tempo

Queste due formule vanno sapute, perché riassumono tutto il dilemma del Round-Robin:

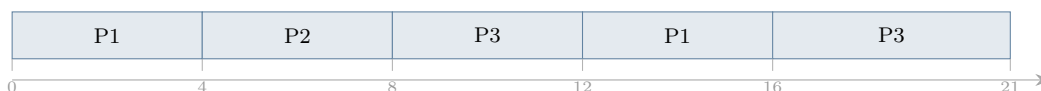
- un quanto **troppo piccolo** rende il sistema reattivo (tutti partono in fretta) ma il **costo del cambio di contesto** diventa una frazione enorme del tempo: se il quanto è 1 ms e il passaggio di contesto costa 0,1 ms, il 10% della CPU se ne va in contabilità;
- un quanto **troppo grande** rende il cambio di contesto trascurabile, ma il comportamento degenera verso il FCFS: chi arriva prima tiene la CPU a lungo, e i processi interattivi “a scatti” (che usano la CPU per pochi millisecondi per volta) aspettano comunque.

La formula  $(n-1)q$  quantifica il primo aspetto: il tempo massimo di attesa *cresce linearmente con il numero di processi*. Con 100 processi e un quanto di 50 ms, un processo può aspettare quasi 5 secondi: da qui la necessità di tenere  $q$  piccolo quando  $n$  è grande.

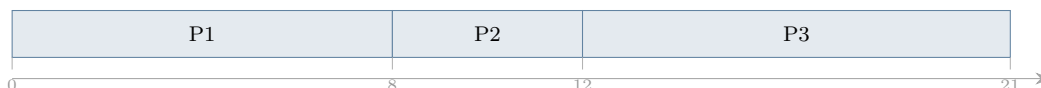
**Un esempio svolto.** **Esercizio costruito sul modello del compito tipo.** Le slide non svolgono nessun esempio di Round-Robin: quelle a disposizione trattano solo i sistemi batch. Questo esempio segue il modello del compito tipo, e i numeri sono calcolati dallo stesso script che verifica gli esempi delle slides.

Confrontiamo RR (quanto  $q = 4$ ) e FCFS sulla stessa tabella, che è scelta apposta per mettere in difficoltà il FCFS: P1 è lungo e arriva subito, mentre P3 arriva dopo, quando la CPU è già occupata.

| Processo | Arrivo | Durata |
|----------|--------|--------|
| P1       | 0      | 8      |
| P2       | 1      | 4      |
| P3       | 2      | 9      |



*Round-Robin:* attese P1 = 8, P2 = 3, P3 = 10, media **7,00**; completamenti P1 = 16, P2 = 7, P3 = 19, media 14,00; tempi di risposta P1 = 0, P2 = 3, P3 = 6, media **3,00**.



*FCFS*: attese  $P1 = 0$ ,  $P2 = 7$ ,  $P3 = 10$ , media **5,67**; completamenti  $P1 = 8$ ,  $P2 = 11$ ,  $P3 = 19$ , media 12,67; tempi di risposta  $P1 = 0$ ,  $P2 = 7$ ,  $P3 = 10$ , media **5,67**.

#### Il risultato controintuitivo, e perché non lo è

Qui il FCFS ha attesa media *migliore* (5,67 contro 7,00) e completamento medio migliore (12,67 contro 14,00), eppure il Round-Robin è considerato l'algoritmo giusto per i sistemi interattivi. Il motivo sta nella terza colonna: il **tempo di risposta** medio passa da 5,67 a 3,00, e soprattutto il caso peggiore crolla — con FCFS P3 aspetta **10** unità prima di cominciare, con RR ne aspetta **6**. In un sistema interattivo conta questo: che ogni comando *cominci* a essere servito in fretta, anche se finirà più tardi. Se all'orale ti chiedono “perché il RR se peggiora le medie?”, la risposta è: perché le medie che peggiora sono quelle dei sistemi batch (attesa e completamento), mentre la media che migliora — il tempo di risposta — è l'obiettivo dichiarato dei sistemi interattivi (capitolo 8).

### 9.2.2 Scheduling a priorità

[SO-02 p42]

- **regola di base**: si assegna la CPU al processo con **più alta priorità**;
- **assegnamento delle priorità**: possono essere **statiche** (fissate una volta) o **dinamiche** (che cambiano nel tempo);
- si tende a **favorire i processi I/O-bounded**;
- lo **SJF** si può vedere come un sistema a priorità, dove la priorità è l'*inverso* della durata: più il lavoro è corto, più è prioritario;
- vale sia con **prelazione** sia **senza**;
- il problema tipico sono la **starvation** e l'**aging**.

#### Starvation e aging

Si ha **starvation** (*attesa indefinita*) quando un processo a bassa priorità non ottiene mai la CPU, perché c'è sempre qualche processo a priorità più alta pronto a eseguire. L'**aging** è la soluzione: si **aumenta progressivamente la priorità** dei processi che attendono da troppo tempo, così che prima o poi ottengano la CPU. [SO-02 p42]

#### Starvation non è deadlock

Sono due cose diverse e l'esame ci insiste:

- nel **deadlock** c'è un'*attesa circolare*: A aspetta B, B aspetta A, e nessuno dei due può proseguire. Il sistema è fermo.
- nella **starvation** il sistema *prosegue*: i processi a priorità alta vanno avanti, il processo basso è semplicemente *sfortunato* nella selezione. Non c'è nessun ciclo di attese.

L'abbiamo già incontrata due volte: con lo scrittore nel problema dei lettori-scrittori (capitolo 7) e qui. La cura è l'aging.

### 9.2.3 Variante: lo scheduling a code multiple

Invece di un'unica coda si usano **più code, una per ogni classe di priorità**. Le due varianti sono: [SO-02 p42]

- **con priorità fisse:** un processo resta nella sua classe per tutta la vita;
- **con feedback** (o retroazione): un processo può **cambiare classe**. È il meccanismo con cui si realizza l'aging in modo strutturato — e anche quello con cui si puniscono i processi CPU-bounded, spostandoli verso il basso man mano che consumano quanto di tempo.

### 9.2.4 SPN — Shortest Process Next

L'idea è **applicare lo SJF ai processi interattivi**. Il problema è immediato: come si fa a **identificare la durata del prossimo burst di CPU**? Nei sistemi interattivi i processi alternano continue richieste, e la durata del prossimo tratto di calcolo non è nota. [SO-02 p43]

La soluzione sono **stime basate sui burst precedenti**, con una **media esponenziale**: [SO-02 p43]

$$S_{n+1} = S_n (1 - a) + T_n a$$

dove  $T_n$  è la durata dell' $n$ -esimo burst osservato,  $S_n$  la stima precedente e  $a$  un parametro fra 0 e 1. [SO-02 p43]

#### Come si legge la formula della media esponenziale

$S_{n+1}$  è una **media pesata** fra la vecchia stima  $S_n$  e l'ultima osservazione  $T_n$ :

- con  $a \rightarrow 0$  prevale la **storia**: la stima è stabile ma lenta a reagire ai cambiamenti;
- con  $a \rightarrow 1$  prevale l'**ultima osservazione**: la stima reagisce subito, ma è nervosa e “insegue” ogni fluttuazione.

La scelta di  $a$  è quindi un compromesso fra reattività e stabilità, ed è lo stesso compromesso che si ritrova nell'**aging** del capitolo sulla memoria virtuale (capitolo 13). Non è una coincidenza: in entrambi i casi si sta stimando, a partire da un passato rumoroso, una quantità che serve a prendere una decisione.

**L'esempio delle slides** ( $a = 1/2$ ). La stima iniziale è  $S_1 = 10$ . [SO-02 p43]

|       |    |   |   |   |    |    |    |
|-------|----|---|---|---|----|----|----|
| $T_n$ | 6  | 4 | 6 | 4 | 13 | 13 | 13 |
| $S_n$ | 10 | 8 | 6 | 6 | 5  | 9  | 11 |

Verifichiamo il meccanismo sui primi passi, con  $a = \frac{1}{2}$ :

$$S_2 = 10 \cdot \frac{1}{2} + 6 \cdot \frac{1}{2} = 8$$

$$S_3 = 8 \cdot \frac{1}{2} + 4 \cdot \frac{1}{2} = 6$$

$$S_4 = 6 \cdot \frac{1}{2} + 6 \cdot \frac{1}{2} = 6$$

$$S_5 = 6 \cdot \frac{1}{2} + 4 \cdot \frac{1}{2} = 5$$

$$S_6 = 5 \cdot \frac{1}{2} + 13 \cdot \frac{1}{2} = 9$$

$$S_7 = 9 \cdot \frac{1}{2} + 13 \cdot \frac{1}{2} = 11$$

#### Il senso della tabella

Si osservi l'ultima riga: quando il processo cambia comportamento (passa a burst da 13), la stima **non salta subito** a 13 — passa a 9, poi a 11, e solo dopo diversi burst si avvicinerà. È precisamente l'effetto di  $a = \frac{1}{2}$ : la stima *insegue* il cambiamento senza inseguirlo istantaneamente. Nota anche che  $S_n$  è sempre un valore *passato*: la stima per il

burst successivo, con questi dati, sarebbe 12.

### 9.2.5 Scheduling garantito

Si stabilisce una **percentuale di utilizzo** della CPU (per esempio: “questo processo avrà il 30% della CPU”) e la si **fa rispettare**. [SO-02 p44]

### 9.2.6 Scheduling a lotteria

Si distribuiscono **biglietti** ai processi e si fanno **estrazioni a caso**: chi vince l'estrazione ottiene la CPU. Le proprietà che le slide evidenziano: [SO-02 p44]

- il **criterio è semplice e chiaro**;
- c'è la possibilità di avere **processi cooperanti** — basta che si scambino i biglietti.

#### Perché la lotteria funziona bene in teoria

La percentuale di CPU che un processo ottiene è proporzionale al numero di biglietti che possiede, ma la decisione è *probabilistica*. Il vantaggio è la semplicità con cui si esprimono le politiche: “dai il doppio della CPU a questo processo” diventa “dagli il doppio dei biglietti”. Il prezzo è che non c'è alcuna garanzia *deterministica*: solo in media il processo otterrà la sua quota. Ed è per questo che questo schema non va bene per i sistemi **real-time**, dove serve la prevedibilità (capitolo 8).

### 9.2.7 Scheduling fair-share

Realizza un **equo uso tra gli utenti del sistema**. [SO-02 p44]

#### La differenza rispetto a tutti gli altri

È l'unico algoritmo di questa sezione in cui l'unità di misura *non è il processo, ma l'utente*. Se un utente lancia un solo processo e un altro ne lancia venti, uno scheduler “equo fra i processi” dà  $\frac{1}{21}$  al primo e  $\frac{20}{21}$  al secondo; uno scheduler fair-share dà metà della CPU a ciascun *utente*, quindi  $\frac{1}{2}$  al primo e  $\frac{1}{40}$  a ciascun processo del secondo. L'equità si sposta quindi dal piano dei processi a quello degli utenti — ed è ciò che serve in un sistema multiutente, dove sarebbe altrimenti sufficiente lanciare molti processi per accaparrarsi la macchina.

| Algoritmo     | Criterio di scelta                                                  |
|---------------|---------------------------------------------------------------------|
| FCFS          | chi è in coda da più tempo                                          |
| SJF           | durata minore (non-preemptive)                                      |
| SRTN          | minor tempo <i>residuo</i> (preemptive)                             |
| RR            | a turno, un quanto di tempo ciascuno                                |
| a priorità    | priorità più alta; statiche o dinamiche; aging contro la starvation |
| code multiple | classi di priorità, con o senza feedback                            |
| SPN           | durata minore, ma <i>stimata</i> con una media esponenziale         |
| garantito     | percentuale di utilizzo dichiarata e fatta rispettare               |
| a lotteria    | estrazione casuale, proporzionale ai biglietti posseduti            |
| fair-share    | equità fra gli <i>utenti</i> , non fra i processi                   |

**Come lo dici all'orale**

“Mi illustri gli algoritmi di scheduling.” Non elencarli tutti in fila: dai la **struttura**, che è quella delle slide stesse — *batch* contro *interattivi*.

1. **Batch:** FCFS (semplice, ma soffre l'effetto convoglio), SJF (ottimo solo se tutti i lavori sono disponibili subito), SRTN (la versione con prelazione, la migliore delle tre sulle medie).
2. **Interattivi:** RR (risponde al problema del tempo di risposta, con il compromesso sul quanto), a priorità (con starvation e aging), code multiple con feedback, SPN (SJF con stime), garantito, lotteria, fair-share.

Poi scegli *due* algoritmi e mostra di saperli confrontare con i numeri — il confronto FCFS/SJF/SRTN della slide a tre colonne è perfetto per questo, perché permette di dire che “SRTN vince sulle medie ma P1, il processo lungo, peggiora”. Chi porta i numeri e non solo le definizioni dimostra di aver capito, non di aver memorizzato. E se ti chiedono “qual è il migliore?”, la risposta è: dipende dall'ambiente e dall'obiettivo, e le tre metriche (attesa, completamento, risposta) possono andare in direzioni opposte — con l'esempio RR/FCFS appena visto a dimostrarlo.

## Capitolo 10

### Scheduling dei thread e su sistemi multi-processore

---

Finora abbiamo parlato di “processi” come se fossero le uniche entità da mettere in esecuzione. Ma il capitolo 5 ha mostrato che l’unità di esecuzione è il **thread**, e il capitolo 2 che una macchina può avere **più processori**. Questo capitolo chiude le due questioni.

#### 10.1 Lo scheduling dei thread

La distinzione fondamentale è quella già vista: chi gestisce la tabella dei thread. [SO-02 p45]

##### 10.1.1 I thread utente

[SO-02 p45]

- sono **ignorati dallo scheduler del kernel**: il kernel vede un solo processo e non sa che dentro ce ne sono molti;
- per lo scheduler del **sistema run-time** della libreria vanno bene **tutti gli algoritmi non-preemptive** visti nel capitolo 9;
- c’è la possibilità di usare uno **scheduling personalizzato**, scelto dall’applicazione.

##### Perché solo algoritmi non-preemptive

È una conseguenza diretta del modello 1-a-molti, e la domanda “perché proprio non-preemptive?” capita spesso. Il motivo è che la **prelazione richiede un interrupt**, e un interrupt arriva al *processo*, cioè al kernel: il kernel non ha modo di sapere che il processo contiene più thread e che bisognerebbe sospenderne uno in particolare. La libreria in spazio utente, dal canto suo, **non riceve mai il controllo** se non quando un thread glielo cede volontariamente (con `thread_yield`). Quindi la libreria può cambiare thread solo nei punti in cui un thread decide di farsi da parte: esattamente la definizione di scheduling non-preemptive.

È anche il motivo per cui, con i thread utente, **un thread che non cede mai la CPU blocca tutti gli altri dello stesso processo**: il kernel continua a vedere un processo che lavora, e non ha motivo di intervenire.

##### 10.1.2 I thread del kernel

Qui il kernel *sa* che i thread esistono, e ci sono due possibilità: [SO-02 p45]

- si considerano **tutti i thread uguali** — l’appartenenza a un processo non conta;
- oppure si **pesa l’appartenenza al processo**: il thread di un processo a cui appartengono molti thread viene penalizzato, per non far “vincere” il processo che ne ha di più.

Le slide aggiungono una conseguenza tecnica che vale la pena sapere: **lo switch su un thread di un processo diverso** implica anche la **riprogrammazione della MMU** e, in alcuni casi, **l’azzeramento della cache** della CPU. [SO-02 p45]

### Che cosa cambia, in concreto

Questo è il punto in cui i capitoli si collegano:

- cambiare thread *dentro lo stesso processo* è **economico**: lo spazio di indirizzi resta lo stesso, quindi la MMU non si tocca;
- cambiare thread *tra processi diversi* è **costoso**: va riprogrammata la MMU (la tabella delle pagine attiva cambia) e la cache si “sporca”, perché conteneva dati del processo precedente che ora non servono più.

È il motivo per cui il *processor affinity* — cercare di rieseguire un thread sullo stesso core su cui era già — è una tecnica utile, e non un’ottimizzazione oziosa.

## 10.2 Lo scheduling su sistemi multi-processore

### 10.2.1 I due approcci di base

[SO-02 p46]

- **multielaborazione asimmetrica**: uno dei processori assume il ruolo di **master server**. È il modello più semplice e si estende poco: con molti processori, il master diventa il collo di bottiglia.
- **multielaborazione simmetrica (SMP)**: ogni processore può schedare da sé. Si realizza in due modi:
  - con una **coda unificata** dei processi pronti, condivisa da tutti i processori;
  - con **code separate per ogni processore/core**.

### Coda unica o code separate

È la scelta che determina tutto il resto, e va saputa giustificare con il *trade-off*:

- **coda unificata**: il bilanciamento del carico è *automatico* — un processore libero prende il primo processo disponibile, e nessuno resta ozioso se c’è lavoro. Il difetto è che la coda è una struttura **condivisa**, quindi serve sincronizzare ogni accesso (e con molti processori questo diventa un punto di contesa).
- **code separate**: nessuna contesa sulla coda, ma il carico *non* si bilancia da solo: un processore può avere una coda piena mentre un altro è inattivo. Serve un **bilanciamento del carico** esplicito.

La slide lo dice esplicitamente: il bilanciamento del carico è **necessario solo in presenza di code distinte**. È una di quelle frasi da ricordare parola per parola, perché mette in relazione causa ed effetto.

### 10.2.2 Le politiche di scheduling

[SO-02 p46]

- **predilezione per i processori** (*processor affinity*): presenza o assenza, e nel caso di presenza si distingue fra
  - **predilezione debole**: si preferisce lo stesso processore, ma si migra se serve;
  - **predilezione forte**: il processo è *vincolato* al suo processore.

- **bilanciamento del carico:**
  - necessario solo con code distinte;
  - **migrazione guidata:** un processore inattivo *tira* verso di sé il lavoro da un altro;
  - **migrazione spontanea:** chi ha troppo lavoro lo *spinge* verso un processore inattivo;
  - esistono **approcci misti** (Linux e FreeBSD).

| Tecnica              | Chi prende l'iniziativa              | Effetto                                                  |
|----------------------|--------------------------------------|----------------------------------------------------------|
| predilezione debole  | il processo                          | migra solo se serve                                      |
| predilezione forte   | —                                    | nessuna migrazione, ma carico potenzialmente sbilanciato |
| migrazione guidata   | il processore <b>inattivo</b> (pull) | sfrutta le CPU ferme                                     |
| migrazione spontanea | il processore <b>carico</b> (push)   | scarica il lavoro in eccesso                             |

#### Predilezione contro bilanciamento: il conflitto

[SO-02 p46]

Le due politiche **si contraddicono**, ed è l'ultimo punto della slide: “bilanciamento del carico *vs.* predilezione del processore”.

- la **predilezione** dice: lascia il thread sullo stesso core, altrimenti paghi la riprogrammazione della MMU e l'azzeramento della cache;
- il **bilanciamento del carico** dice: sposta il thread dove la CPU è libera, altrimenti quella CPU sta ferma.

Non si possono soddisfare entrambe. Un sistema reale sceglie caso per caso: se la migrazione è “quasi gratuita” (thread dello stesso processo, o cache già fredda) si bilancia; se costa cara (thread con working set grande) si preferisce tenerlo dove sta. È per questo che le slide parlano di approcci **misti**.

### 10.3 Che cosa usano i sistemi operativi reali

La slide conclusiva del deck riassume ciò che i tre sistemi principali fanno davvero. [SO-02 p47]

**Elementi comuni a tutti.** [SO-02 p47]

- i **thread**;
- l'**SMP** (multielaborazione simmetrica);
- la **gestione delle priorità**;
- la **predilezione per i processi I/O-bounded**.

**Windows.** [SO-02 p47]

- scheduler basato su **code di priorità**;
- **euristiche** per migliorare il servizio dei processi interattivi, e in particolare di quelli *in foreground*;
- **euristiche** per evitare il problema dell'inversione di priorità.



**Linux.** [SO-02 p47]

- scheduling basato sui **task**, intesi come **generalizzazione di processi e thread**;
- **Completely Fair Scheduler** (CFS): un *moderno scheduler garantito*.

**MacOS.** [SO-02 p47]

- **Mach scheduler**, basato su **code di priorità** con euristiche.

**Inversione di priorità: il cerchio si chiude**

L'inversione di priorità l'avevamo incontrata nel capitolo 6 parlando dello *spin lock*: un processo a bassa priorità tiene la CPU girando su un lock, e un processo ad alta priorità — che dovrebbe avere la precedenza — resta in attesa. Le slide si chiudono dicendo che **Windows usa euristiche per evitarla**, ed è significativo che questo problema compaia sia nel capitolo sulla sincronizzazione sia in quello sullo scheduling dei sistemi reali: è il punto in cui le due aree si toccano.

**Che cosa ricordare della slide conclusiva**

Non serve imparare a memoria le tre liste: serve ricordare la **forma** della risposta. Tutti i sistemi reali fanno le stesse quattro cose (thread, SMP, priorità, predilezione I/O), e poi si differenziano per il *meccanismo*:

- Windows e MacOS usano **code di priorità con euristiche**;
- Linux usa un algoritmo **garantito** (il CFS) e ragiona in termini di *task* invece che di processi e thread separati.

Se il docente chiede “come fanno i sistemi reali?”, la risposta forte parte dagli elementi comuni e poi distingue i meccanismi. Elencare tre sistemi senza dire che cosa hanno in comune non dimostra nulla.

**Come lo dici all'orale**

“Scheduling dei thread e su multiprocessore” è in realtà *due* domande, e conviene trattarle separate.

1. **Thread**: la distinzione è la solita — thread utente contro thread del kernel. Per i primi, il kernel non sa che esistono, quindi si possono usare solo algoritmi **non preemptive** (la prelazione richiede un interrupt, e l'interrupt arriva al processo, non al thread); per i secondi, o si trattano tutti uguali, o si pesa l'appartenenza al processo. Aggiungi la conseguenza pratica: cambiare thread dello stesso processo è economico, cambiare processo costa la MMU e la cache.
2. **Multiprocessore**: i due approcci (asimmetrico con master, simmetrico SMP), la scelta fra coda unica e code separate — che è la scelta da cui dipende se il bilanciamento del carico serve o no — e il conflitto fra predilezione del processore e bilanciamento del carico, con la sua risoluzione “mista”.

Il collegamento che vale la pena mettere in evidenza in entrambe: la **riprogrammazione della MMU** e l'**azzeramento della cache** sono il costo che rende la predilezione del processore utile invece che oziosa — e sono anche il motivo per cui questo capitolo appartiene più alla gestione della memoria che allo scheduling puro.

## Parte V

# Gestione della memoria

## Capitolo 11

### Gestione della memoria per la multi-programmazione

---

Con questo capitolo comincia la seconda metà del programma. Il filo conduttore è sempre lo stesso: *come si fa a far convivere più processi su una macchina che ha una memoria sola*. La risposta cambia a seconda di quanta astrazione il sistema decide di offrire, e il capitolo racconta proprio questa progressione.

#### 11.1 Memoria centrale e processi

Il punto di partenza è la **gerarchia di memoria**. La **memoria centrale**, formata da RAM, rappresenta il **livello più basso direttamente utilizzabile dalla CPU**. [SO-03 p2]

##### Perché “livello più basso utilizzabile”

Sotto la RAM ci sono i dischi, che sono *persistenti* ma non indirizzabili direttamente dalla CPU: per leggere un byte dal disco bisogna prima portarlo in memoria. Sopra la RAM ci sono le cache e i registri, che sono *più veloci* ma gestiti in modo automatico dall’hardware, non dal programma. La RAM è quindi l’unico livello che la CPU può indirizzare direttamente e che il programmatore può decidere di usare. È questa posizione intermedia che rende la gestione della memoria un problema del sistema operativo.

Il tema del capitolo è l’**astrazione della memoria a favore dei processi**, che si realizza in **vari livelli di complessità via via maggiore**. [SO-03 p2]

#### 11.2 Senza alcuna astrazione

[SO-03 p3]

- è il modello usato sui **primi mainframe** (anni '60) e sui **primi PC** (primi anni '80);
- i programmi utilizzano **direttamente gli indirizzi fisici**;
- è **difficile eseguire due programmi contemporaneamente**.

##### L'esempio da fare all'orale

Il modo migliore per spiegare perché l’assenza di astrazione sia inaccettabile è un esempio concreto. Un programma compilato per essere caricato all’indirizzo fisico 0 contiene al suo interno dei *salti a indirizzi assoluti* (“vai all’istruzione che sta all’indirizzo 1000”). Se lo si carica all’indirizzo 0, tutto funziona; se lo si carica all’indirizzo 5000, tutti quei salti puntano al posto sbagliato. Da qui nascono i due problemi del paragrafo successivo.

#### 11.3 Multiprogrammazione senza astrazione

Si prova a tenere **più processi in memoria** contemporaneamente. Questo fa emergere due problemi. [SO-03 p4]

##### 11.3.1 Il problema della rilocalizzazione

[SO-03 p4]

- **rilocazione a compile-time**: si compila il programma per un indirizzo preciso. Funziona, ma il programma può girare solo da quell'indirizzo, e non si può spostare;
- **rilocazione statica in fase di caricamento**: si carica il programma dove c'è spazio e si *correggono* tutti gli indirizzi assoluti mentre lo si carica. Il costo dichiarato dalla slide è il **rallentamento del loader**: ogni caricamento richiede un passaggio su tutto il codice.

#### Cosa chiedono all'esame

I due modi sbagliati di risolvere la rilocazione hanno lo stesso difetto di fondo, ed è quello da dire all'orale: **risolvono il problema una volta per tutte, mentre il problema si ripresenta ogni volta che il processo si sposta**. Un processo che viene messo in swap e ricaricato altrove richiede che la rilocazione sia rifatta, e con questi schemi non si può fare. È la ragione per cui serve la **rilocazione dinamica** del paragrafo successivo.

### 11.3.2 Il problema della protezione

Come si impedisce a un processo di scrivere nella memoria di un altro? La soluzione dei mainframe era **lock & key**: blocchi di memoria con delle **chiavi di protezione**, e una **PSW** (Program Status Word) contenente la chiave del processo in esecuzione. L'accesso riesce solo se la chiave del processo corrisponde a quella del blocco. [SO-03 p4]

### 11.4 Lo spazio degli indirizzi

#### Spazio degli indirizzi

Lo **spazio degli indirizzi** è una **astrazione per la memoria utilizzabile da un processo**: il processo vede un insieme di indirizzi tutti suoi, e non sa (né deve sapere) dove si trovino *fisicamente* i dati. [SO-03 p5]

Si realizza con la **rilocazione dinamica**, usando due registri: [SO-03 p5]

- il **registro base**: dove comincia lo spazio del processo;
- il **registro limite**: quanto è grande.

La CPU controlla gli accessi alla memoria in base a questi registri; nei sistemi più evoluti questo compito è svolto dalla **Memory Management Unit (MMU)**. [SO-03 p5]

#### Perché bastano due registri

La trasformazione è semplicissima:  $\text{indirizzo fisico} = \text{indirizzo virtuale} + \text{base}$ , e l'accesso è negato se l'indirizzo virtuale supera il limite. I due registri svolgono due funzioni *diverse* e vanno tenuti distinti:

- il **base** realizza la *rilocazione* (trasforma gli indirizzi);
- il **limite** realizza la *protezione* (impedisce al processo di uscire dal proprio spazio).

Questo è già il cuore di tutto ciò che verrà dopo: la paginazione (capitolo 12) è la stessa idea, con la memoria spezzata in pezzi invece che presa in un blocco unico.

#### Un dettaglio che sembra banale e non lo è

La slide lo dice esplicitamente: **i registri vanno aggiornati ad ogni context-switch**. È ovvio — ogni processo ha il suo spazio — ma è anche un *costo*: il cambio di contesto ora include la scrittura di due registri della MMU. Quando nel capitolo 10 si parlava di

“riprogrammazione della MMU” come costo dello switch fra processi diversi, si parlava esattamente di questo, in versione più complessa (con la paginazione non sono due registri ma un puntatore a una tabella).

I sistemi citati dalla slide che usavano questo schema sono il **CDC 6600** e l'**Intel 8088**.

## 11.5 Lo swapping

### Swapping

Il livello di multiprogrammazione è **seriamente limitato dalla dimensione della memoria centrale**: se i processi non stanno tutti in RAM, qualcuno deve restare fuori. La prima soluzione è lo **swapping**: si sposta un processo *intero* dalla memoria al disco e viceversa.

[SO-03 p6]

Lo swapping introduce una figura nuova: l'**swapper**, detto anche **scheduler di medio termine**, che decide quale processo portare fuori e quale riportare dentro. [SO-03 p6]



### Il problema dei processi con I/O pendenti

La slide segnala una difficoltà che è facile dimenticare: **non si può fare swapping su un processo che ha operazioni di I/O in corso**. Se il processo P3 ha chiesto al disco di leggere dei dati e nel frattempo viene spostato fuori, il controller del disco scriverà i dati letti nell'indirizzo fisico che P3 occupava — che ora magari è di un altro processo. Il risultato è la corruzione dei dati di un processo innocente. Le soluzioni sono due e vanno entrambe nominate: non fare swapping su processi con I/O pendenti, oppure **bufferizzare** l'I/O nel kernel e completare il trasferimento al rientro del processo.

### 11.5.1 Le strategie di allocazione e la frammentazione

Le strategie di allocazione della memoria sono due: [SO-03 p6]

- **dimensione fissa**: lo spazio è diviso in partizioni di dimensione prestabilita;
- **dimensione dinamica**: le partizioni vengono create su misura per i processi che arrivano.

Da qui nasce la distinzione fra due tipi di frammentazione che è fondamentale per tutto il resto del corso:

### Frammentazione interna ed esterna

[SO-03 p6]

- **frammentazione interna**: lo spazio sprecato *dentro* la partizione assegnata a un processo. Tipica delle partizioni a **dimensione fissa**: se la partizione è da 8 kB e il processo ne usa 5, i 3 kB restanti sono sprecati e *inutilizzabili da altri*, perché formalmente appartengono a quel processo.
- **frammentazione esterna**: lo spazio sprecato *fuori* dalle partizioni — tanti buchi liberi che, sommati, sarebbero sufficienti, ma sono *sparsi* e nessuno è grande abbastanza per il processo che deve entrare. Tipica delle partizioni a **dimensione dinamica**.

| Frammentazione                          |                                                       |
|-----------------------------------------|-------------------------------------------------------|
| partizioni a <b>dimensione fissa</b>    | <b>interna:</b> il processo non riempie la partizione |
| partizioni a <b>dimensione dinamica</b> | <b>esterna:</b> i buchi liberi sono tanti e sparsi    |

La cura per la frammentazione esterna è la **memory compaction**: si spostano i processi in memoria per far confluire in un unico blocco tutti i buchi liberi. [SO-03 p6]

#### Cosa chiedono all'esame

La compaction ha un costo che va detto: **spostare un processo significa copiare tutta la sua memoria**, e con la rilocalizzazione *statica* non sarebbe nemmeno possibile, perché gli indirizzi dentro il processo puntano ancora alle vecchie posizioni. La compaction funziona solo perché la rilocalizzazione è **dinamica** (sezione precedente): il processo usa indirizzi virtuali, e basta cambiare il registro base. È un buon esempio di come le scelte fatte in un punto del progetto condizionino ciò che si può fare dopo.

## 11.6 La gestione dell'allocazione

Per tenere traccia di quali parti di memoria sono libere e quali occupate esistono due metodologie. [SO-03 p7]

### 11.6.1 La bitmap

Si usa un **bit per unità di allocazione**: 0 se libera, 1 se occupata (o viceversa). È il metodo più compatto, ma il problema è scegliere la **dimensione del blocco**: blocchi piccoli danno una mappa grande ma precisa, blocchi grandi una mappa piccola ma approssimata. [SO-03 p7]

### 11.6.2 Le liste dei blocchi liberi e occupati

Si mantiene una **lista** dei blocchi, **ordinata per indirizzo**. Le due proprietà che la rendono utile sono: [SO-03 p7]

- la **coalescenza**: quando un processo libera un blocco, se i blocchi adiacenti sono a loro volta liberi i tre vengono fusi in uno solo. Senza coalescenza, la lista si riempirebbe di frammenti adiacenti che nessuno può usare;
- la lista è **doppiamente collegata**: serve a poter togliere un blocco dalla lista in tempo costante.

### 11.6.3 Gli algoritmi di ricerca

[SO-03 p7]

| Algoritmo        | Criterio                                                                        |
|------------------|---------------------------------------------------------------------------------|
| <b>first fit</b> | il <i>primo</i> blocco abbastanza grande che si incontra                        |
| <b>next fit</b>  | come il first fit, ma la ricerca riprende da dove si era fermata l'ultima volta |
| <b>best fit</b>  | il blocco <i>più piccolo</i> fra quelli abbastanza grandi                       |
| <b>worst fit</b> | il blocco <i>più grande</i>                                                     |

**L'idea dietro best fit e worst fit**

Sono opposti e nessuno dei due è “giusto”:

- il **best fit** cerca di sprecare il meno possibile *subito*, ma lascia dietro di sé tanti frammenti piccolissimi, che difficilmente serviranno ancora;
- il **worst fit** prende il blocco più grande sperando che quello che resta sia ancora abbastanza grande da servire. Anche qui, però, il blocco grande è quello che servirebbe a un processo grande che arriva dopo.

È un esempio da manuale del fatto che le euristiche locali non garantiscono l'ottimo globale.

Infine, una **lista separata**: al posto di un'unica lista ordinata per indirizzo, si tiene una **lista dei blocchi liberi ordinata per dimensione**, che permette di trovare rapidamente il blocco della taglia giusta. [SO-03 p7]

**Come lo dici all'orale**

“Mi parli della gestione della memoria.” La scaletta che funziona è la **progressione**, non l'elenco:

1. si parte dal **modello senza astrazione**: indirizzi fisici diretti, e i due problemi che rendono impossibile la multiprogrammazione (rilocazione e protezione);
2. i due tentativi di soluzione e il loro limite comune: la rilocazione a compile-time e quella statica in fase di caricamento risolvono il problema *una volta sola*, mentre serve ogni volta che il processo si sposta;
3. lo **spazio degli indirizzi** e la **rilocazione dinamica** con registro **base** (rilocazione) e **limite** (protezione);
4. lo **swapping** con lo swapper (scheduler di medio termine), il problema dell'I/O pendente, e la distinzione fra frammentazione **interna** (partizioni fisse) ed **esterna** (partizioni dinamiche), con la **compaction** come rimedio;
5. la **gestione dell'allocazione**: bitmap contro liste, la coalescenza, e i quattro algoritmi di ricerca con i loro compromessi.

Il valore aggiunto è mostrare che è tutto un crescendo: ogni soluzione introduce un problema che la successiva risolve — e la paginazione, che viene dopo, è il punto in cui si smette di allocare blocchi contigui.

## Capitolo 12

### La memoria virtuale

---

Il capitolo precedente si chiudeva con un'allocazione fatta di blocchi *contigui*: un processo occupa una porzione di memoria tutta di seguito. Questo capitolo abbandona quell'ipotesi, ed è il salto concettuale più importante di tutta la parte sulla memoria.

#### 12.1 Da contigua a non contigua

Si passa all'**allocazione non contigua** della memoria fisica, con due spazi distinti: [SO-03 p8]

- lo **spazio di indirizzamento virtuale** è diviso in **pagine**;
- lo **spazio della memoria fisica** è diviso in **frame**.

Le pagine e i frame hanno la **stessa dimensione**, e le conseguenze elencate dalla slide sono quattro: [SO-03 p8]

1. la dimensione uguale comporta **frammentazione interna**; in alcuni casi la dimensione è **variabile** (Solaris);
2. **alcune pagine possono anche non risiedere in memoria**: all'occorrenza vengono caricate dalla memoria secondaria;
3. si adatta **perfettamente a un sistema multiprogrammato**;
4. c'è **protezione implicita tra processi**.

#### Il punto 2 è la memoria virtuale, il resto è paginazione

La distinzione fra le due cose si perde spesso, e vale la pena fissarla:

- la **paginazione** è la tecnica che risolve il problema dell'allocazione: invece di cercare un blocco contiguo abbastanza grande, si spezza il processo in pezzi uguali e li si sparpaglia dove c'è posto. Elimina la **frammentazione esterna** — non ci sono più “buchi troppo piccoli”, perché ogni buco è fatto di frame tutti uguali.
- la **memoria virtuale** è la conseguenza del fatto che *non tutte le pagine devono essere in memoria*: un processo può essere più grande della memoria fisica, e in generale può usarne *meno*, perché le pagine che non servono in questo momento stanno su disco.

La pagina 8 delle slide mette le due cose nella stessa slide, ma l'ordine è significativo: la paginazione viene *prima*. Non si può avere memoria virtuale senza paginazione; si può avere paginazione senza memoria virtuale (ed è ciò che fanno i sistemi che non mettono mai le pagine su disco).

#### La protezione diventa implicita

Nel capitolo precedente la protezione era un registro **limite** controllato ad ogni accesso. Qui diventa **implicita**: un processo non può accedere alla memoria di un altro semplicemente perché *quella memoria non esiste nel suo spazio*, e non compaiono nella sua tabella delle pagine gli indirizzi per raggiungerla. È una protezione più robusta, perché non dipende dal controllo di un confronto numerico ma dall'assenza stessa della traduzione.



## 12.2 La paginazione

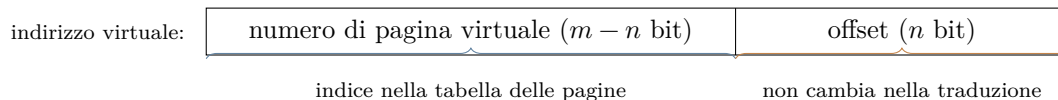
La paginazione è **gestita dalla MMU**, e si appoggia a due strumenti: [SO-03 p9]

- il **bit di presenza** (nella voce della tabella delle pagine), che dice se quella pagina è attualmente in memoria;
- l'**errore di pagina** (*page fault*): l'evento che si verifica quando si accede a una pagina che non è presente.

### 12.2.1 La traduzione degli indirizzi

È la parte da saper fare con i numeri, ed è anche l'esercizio più frequente del compito. Sia lo spazio degli indirizzi virtuali di  $2^m$  byte e la dimensione di pagina  $2^n$  byte. [SO-03 p10]

|                         |                                                                                               |
|-------------------------|-----------------------------------------------------------------------------------------------|
| <b>Numero di pagina</b> | i $(m - n)$ bit <b>più significativi</b> dell'indirizzo virtuale $\rightarrow 2^{m-n}$ pagine |
| <b>Offset</b>           | gli $n$ bit <b>meno significativi</b>                                                         |
| <b>Frame</b>            | se lo spazio degli indirizzi fisici è $2^t$ , allora ci sono $2^{t-n}$ frame, e vale $m > t$  |



#### La cosa da capire, non da imparare a memoria

L'**offset non viene tradotto**. È il concetto chiave: la pagina virtuale  $k$  finisce nel frame  $j$ , ma il byte che sta all'interno della pagina resta *allo stesso posto* dentro il frame. Se la pagina è da 4 kB e l'indirizzo virtuale è 10100, si ha numero di pagina =  $\lfloor 10100/4096 \rfloor = 2$  e offset =  $10100 \bmod 4096 = 1908$ . L'indirizzo fisico sarà  $\text{frame} \times 4096 + 1908$ . Quindi la traduzione è: si sostituisce la parte alta, si lascia intatta quella bassa.

### 12.2.2 La voce della tabella delle pagine

Ogni voce contiene molto più di un semplice numero di frame: [SO-03 p11]

| Campo                                  | Significato                                                        |
|----------------------------------------|--------------------------------------------------------------------|
| numero del <b>frame</b>                | dove sta la pagina in memoria fisica                               |
| bit <b>presente/assente</b>            | questa pagina è in memoria?                                        |
| <b>protezione</b>                      | lettura/scrittura ed eventualmente esecuzione                      |
| bit <b>modificato</b> ( <i>dirty</i> ) | indica se la pagina è stata modificata                             |
| bit <b>referenziato</b>                | impostato quando si fa un riferimento <i>qualunque</i> alla pagina |
| bit per <b>disabilitare la cache</b>   | per le pagine che non vanno messe in cache                         |

La slide aggiunge che nella pratica esiste anche un **bit di validità** (o di allocazione). [SO-03 p11]

#### Dirty e referenziato: due bit, due usi diversi

È la coppia di bit più importante della tabella, perché è su di essi che si basano *tutti* gli algoritmi di sostituzione del capitolo 13:

- il bit **modificato (dirty)** dice se la pagina è stata *scritta*. Serve a sapere se, per

liberare il frame, bisogna **salvare** la pagina su disco o si può semplicemente buttarla via. È quindi un'informazione sul *costo* della sostituzione.

- il bit **referenziato** dice se la pagina è stata *usata* (letto o scritto). Serve a stimare se servirà ancora. È quindi un'informazione sul *valore* della pagina.

Il primo è aggiornato dall'hardware quando si scrive, il secondo quando si accede; entrambi sono **azzerati dal sistema operativo**, ed è questa asimmetria (hardware li alza, il sistema li abbassa) che permette di usare il bit referenziato come “finestra temporale”: se lo azzerò ogni tanto, il fatto che sia a 1 significa “usata dopo l'ultimo azzeramento”.

### 12.2.3 La tabella dei frame

Il sistema operativo tiene traccia dello stato di occupazione di ogni frame fisico attraverso la **tabella dei frame**: [SO-03 p12]

- lo **stato**: occupato oppure libero;
- se è occupato, **da quale processo**.

Viene consultata in due occasioni: [SO-03 p12]

- ogni volta che viene **creato un nuovo processo**, per creare la relativa tabella delle pagine di quel processo;
- ogni volta che un processo **chiede di allocare nuove pagine**.

#### Due tabelle, due direzioni opposte

Vale la pena non confonderle, perché l'esame le chiede insieme:

- la **tabella delle pagine** è *una per processo* e va dal *virtuale al fisico*: “dove sta la mia pagina *k*?”.
- la **tabella dei frame** è *una sola per sistema* e va dal *fisico al proprietario*: “chi sta usando il frame *j*?”.

La prima serve alla traduzione degli indirizzi (cioè a ogni accesso alla memoria), la seconda serve alla *gestione* (allocare e liberare frame), ed è il motivo per cui la tabella dei frame si consulta solo alla creazione di un processo o all'allocazione di una pagina.

## 12.3 Progettare una tabella delle pagine

Gli aspetti da curare sono due, e sono in conflitto: [SO-03 p13]

- la **velocità nella consultazione**;
- la **dimensione**.

Le due idee di implementazione per la velocità: [SO-03 p13]

1. avere un numero sufficiente di **registri** su cui caricare l'intera tabella. È velocissimo ma ovviamente poco scalabile.
2. la tabella **interamente residente in memoria**, con un registro **PTBR** (*Page-Table Base Register*) che punta all'inizio della tabella. Il costo è che **servono due accessi alla memoria** per prelevare un dato: uno per leggere la voce della tabella delle pagine, uno per leggere il dato vero.

Il vantaggio del secondo schema, dice la slide, è un **context switch molto veloce**: basta cambiare un registro. [SO-03 p13]

#### Cosa chiedono all'esame

Il “doppio accesso” è il problema che genera *tutto* il resto del capitolo. Vale la pena enunciarlo come un conto: se ogni accesso a memoria costa  $\alpha$ , con la tabella in memoria ogni accesso *logico* del programma costa  $2\alpha$ . Il programma rallenta della metà, e questo è inaccettabile. Le tre soluzioni che seguono — memoria associativa, tabelle multilivello, tabella invertita — attaccano rispettivamente il *tempo*, la *dimensione* e di nuovo il tempo. Tenere a mente questo filo rende il capitolo molto più semplice da ricordare.

### 12.3.1 La memoria associativa (TLB)

La soluzione al problema del doppio accesso è il **Translation Lookaside Buffer (TLB)**, una memoria dentro la MMU detta anche **memoria (o registri) associativa**. [SO-03 p14]

L'**osservazione di partenza** è fondamentale e va citata: *in genere un programma usa un gran numero di riferimenti a un piccolo numero di pagine*. È il principio di località, e senza di esso la TLB non funzionerebbe. [SO-03 p14]

Le caratteristiche: [SO-03 p14]

- un numero ridotto di registri, **tra 64 e 1024**;
- ogni voce contiene: numero di **pagina virtuale**, un **bit di validità** della voce, il **codice di protezione**, il **dirty bit** e il numero di **frame**;
- la ricerca è **parallelizzata in hardware**: è questa la proprietà che la rende veloce. Una memoria normale richiede di scandire le voci una per una; una memoria associativa confronta il valore cercato con *tutte* le voci contemporaneamente;
- possibilità di **voci vincolate** (“*wired*”): voci che non possono essere sostituite;
- si distingue fra **TLB hit** e **TLB miss**;
- uso di **address-space identifiers (ASID)** in alternativa al **flush della TLB**.

#### Il problema degli ASID, che è la domanda “cattiva”

La TLB contiene voci di *un* processo. Quando avviene un context switch, le voci del processo precedente sono sbagliate per quello nuovo — ma corrispondono a numeri di pagina virtuale che il nuovo processo userà per tutt'altro. Serve quindi o **svuotare la TLB** ad ogni cambio di contesto, o **etichettare** ogni voce con l'identificativo dello spazio di indirizzi a cui appartiene (l'ASID). Con gli ASID, una voce di un altro processo semplicemente *non corrisponde*, e non serve svuotare nulla. Il costo del flush — che si paga ad *ogni* context switch, anche se i due processi usano le stesse pagine — è la ragione per cui gli ASID esistono.

### 12.3.2 Effective Access Time (EAT)

È l'esempio numerico delle slides, e va saputo rifare. [SO-03 p15]

|                                      |                             |
|--------------------------------------|-----------------------------|
| tempo di accesso alla <b>memoria</b> | $\alpha = 100 \text{ nsec}$ |
| tempo di accesso alla <b>TLB</b>     | $\beta = 20 \text{ nsec}$   |

I due casi: [SO-03 p15]

- **TLB hit**:  $\alpha + \beta = 120 \text{ nsec}$  (si consulta la TLB e poi si accede direttamente al dato);

- **TLB miss:**  $2\alpha + \beta = 220$  nsec (si consulta la TLB senza successo, si legge la voce nella tabella delle pagine, e poi si accede al dato).

Ipotizzando un **TLB ratio** del 80%: [SO-03 p15]

$$0,8 \times 120 + 0,2 \times 220 = 140 \text{ nsec}$$

In generale, con TLB ratio  $\varepsilon$ : [SO-03 p15]

$$\text{EAT} = \varepsilon (\alpha + \beta) + (1 - \varepsilon) (2\alpha + \beta)$$

#### Che cosa dice davvero questa formula

Sembra una media banale, ma contiene una lezione sulla **differenza fra hit e miss**. Il tempo di un hit è 120, quello di un miss 220: il miss costa **quasi il doppio**. Da qui due conseguenze:

- l'EAT è dominato dal tasso di successi, e per questo le TLB sono piccole ma molto efficienti: basta un hit ratio alto, non serve azzeccarci sempre;
- esiste un **minimo assoluto** per l'EAT, che è  $\alpha + \beta$  — il tempo dell'hit con rapporto di successo pari al 100%. Con questi numeri è 120 nsec, e nessuna ottimizzazione della TLB può scendere sotto. Chiedere “quanto deve valere il TLB ratio per scendere a 110 nsec?” significa chiedere qualcosa di *impossibile*, non di difficile: sarebbe  $\varepsilon = 110\%$ .

Con i numeri della slide: per scendere a 130 nsec servirebbe un TLB ratio del 90%; per 125 nsec, del 95%.

### 12.3.3 La tabella delle pagine multilivello

È la soluzione al problema della **dimensione**, che finora abbiamo ignorato. [SO-03 p16]

Il conto delle slides: [SO-03 p16]

- con **indirizzi virtuali a 32 bit** (Pentium) e pagine da 4 kB si hanno circa **1 milione di pagine**, e una tabella grande circa **4 MB** (supponendo voci da 32 bit), con la necessità che sia **contigua** in memoria;
- con **indirizzi a 64 bit** si arriva a  $2^{52}$  pagine da 4 kB.

La soluzione: **non mantenere l'intera tabella in memoria**, ma usare **tabelle multilivello** (paginazione gerarchica). Nel caso di due livelli, dice la slide, possono essere necessari 3 accessi. [SO-03 p16]

#### Perché il multilivello fa risparmiare spazio

È la cosa che si sbaglia più spesso. Il multilivello *non* riduce la dimensione *totale* della tabella se tutte le pagine fossero in uso: la riduce perché **permette di non allocare le parti che non servono**. Un processo usa tipicamente una parte piccola del suo spazio di indirizzamento (lo stack in alto, il codice in basso, i dati in mezzo): con una tabella a un livello tutte le voci intermedie esistono comunque, con una a due livelli le tabelle di secondo livello delle zone vuote **semplicemente non esistono**.

Il costo lo dice la slide: **un accesso in più** per ogni livello. Con due livelli servono 3 accessi in totale (tabella di primo livello, tabella di secondo livello, dato). Ed è esattamente qui che la TLB torna a servire: poiché l'accesso alla TLB costa  $\beta$  e non  $\alpha$ , un hit nella TLB *salta tutti i livelli*. Le due soluzioni — TLB e multilivello — lavorano insieme, ed è per questo che la slide successiva parla di cache.

### 12.3.4 La tabella delle pagine invertita

È una soluzione alternativa, con una filosofia opposta: invece di una tabella per processo, **una voce per ogni frame fisico**, e ogni voce riporta la coppia (**id processo, pagina virtuale**). [SO-03 p17]

Le proprietà: [SO-03 p17]

- la tabella è **alquanto piccola** a paragone delle tabelle delle pagine tradizionali — c'è una voce per frame fisico, indipendentemente dal numero di processi;
- però una ricerca sarebbe **molto lenta**: si risolve con una **tabella hash indicizzata sull'indirizzo virtuale, accoppiata con una TLB**;
- servono comunque delle **tabelle per processo**, per le informazioni necessarie a **gestire i page fault**;
- queste ultime possono però essere **paginate a loro volta**.

#### La direzione della ricerca

Nelle tabelle tradizionali la ricerca è *diretta*: si conosce il numero di pagina virtuale e si vuole il frame. Nella tabella invertita il verso è opposto — si *scandirebbe* la tabella per trovare quale pagina di quale processo sta in un certo frame — ed è questo che la rende lenta. La tabella hash risolve proprio questo: riporta la ricerca al caso diretto, indicizzandola sull'indirizzo virtuale. Ma è una struttura che gestisce *collisioni*, e quindi serve la TLB per non pagarne il costo ad ogni accesso.

### 12.3.5 Cache della memoria e memoria virtuale

L'ultima slide del capitolo mette in relazione due cose che si studiano separatamente, e che in realtà interagiscono. [SO-03 p18]

La **cache della memoria** può essere: [SO-03 p18]

- **basata sugli indirizzi fisici**:
  - *non serve invalidarla* sul context switch;
  - ma il *caching* risulta *poco efficace*.
- **basata sugli indirizzi virtuali**:
  - servono gli **ASID** per non invalidarla ogni volta;
  - *scala male su L2*;
  - ma ha *maggiore efficacia*.

**Cosa si usa in pratica:** [SO-03 p18]

- la cache **L1 basata su indirizzi virtuali**;
- le cache **L2 e successive basate su indirizzi fisici**.

#### Perché si sceglie così — e che cosa c'entra la TLB

Il diagramma della slide mostra la catena: CPU → MMU → cache → memoria DRAM, con la TLB accanto alla cache L1.

Il ragionamento, che vale la pena ricostruire:

- una cache **a indirizzi virtuali** permette di cercare *contemporaneamente* alla traduzione: non bisogna aspettare che la TLB dia l'indirizzo fisico. È ciò che serve a L1, dove ogni nanosecondo conta. Il prezzo sono i problemi di coerenza quando due processi

condividono una pagina o quando lo spazio di indirizzi cambia;

- una cache **a indirizzi fisici** non ha quei problemi, ma *deve* aspettare la traduzione. Va bene per L2 e L3, dove la latenza è già alta e la coerenza conta di più.

Il “scala male su L2” della slide si riferisce proprio a questo: i problemi di coerenza delle cache virtuali peggiorano al crescere della cache — e L2 è molto più grande di L1.

### Come lo dici all’orale

“Mi parli della memoria virtuale.” Questo è un capitolo dove la **struttura** vale più dei dettagli, e la struttura è quella dei due problemi (tempo e dimensione) con le loro soluzioni:

1. **paginazione**: pagine virtuali e frame fisici della stessa dimensione, e la traduzione (numero di pagina nei bit alti, offset nei bit bassi, e l’**offset non si traduce**);
2. il problema del **doppio accesso**: tabella in memoria con PTBR,  $\Rightarrow$  **TLB** come soluzione al tempo, con l’EAT e l’esempio numerico ( $100/20/80\% \rightarrow 140$  nsec);
3. il problema della **dimensione**: 4 MB per processo a 32 bit  $\Rightarrow$  **tabelle multilivello**, che non riducono il totale ma permettono di *non allocare* ciò che non serve;
4. l’alternativa: **tabella invertita**, una voce per frame, con hash e TLB per compensare la lentezza della ricerca;
5. la voce della tabella delle pagine, con i due bit che contano davvero — **dirty** (costo della sostituzione) e **referenziato** (valore della pagina) — che sono il ponte verso il capitolo successivo.

Se ti chiedono “a cosa serve la memoria virtuale?”, la risposta forte non è “a usare più memoria del fisico”, ma è: a dare a ogni processo uno spazio di indirizzi *suo, contiguo e isolato*, indipendentemente da come la memoria fisica è fatta — con la possibilità di usare più memoria della fisica come conseguenza, non come scopo.

## Capitolo 13

### Gli algoritmi di sostituzione delle pagine

---

#### 13.1 Il problema

##### La pagina vittima

In caso di **page fault** e in **assenza di frame liberi** è necessario scegliere una **pagina vittima** da rimuovere per far posto alla nuova. L'**obiettivo** è **minimizzare il numero di page fault in futuro**. [SO-03 p19]

La slide osserva che il problema è **simile alla gestione delle cache**: in entrambi i casi si deve decidere cosa buttare via senza sapere cosa servirà. [SO-03 p19]

##### 13.1.1 La soluzione ottimale (OPT)

##### Algoritmo OPT

Si sceglie la pagina che verrà **referenziata in un futuro più lontano**. È **ottimale** ma **difficilmente realizzabile**, perché richiede di conoscere il futuro; rappresenta comunque un **termine di paragone** per valutare gli altri algoritmi. [SO-03 p19]

##### Perché OPT è ottimale — e perché non si può usare

Il ragionamento è: se *devo* buttare via una pagina, quella che non servirà per più tempo è quella il cui ritorno costerà di meno. Non serve buttare la pagina “meno utile”, serve buttare quella la cui assenza si farà sentire più tardi.

Non è realizzabile perché il sistema operativo non conosce il futuro del programma. Non è nemmeno ricavabile “guardando avanti” nel codice, perché il flusso di esecuzione dipende dai dati. Resta comunque utilissimo: eseguendo OPT su una traccia si ottiene il **minimo teorico**, e ogni altro algoritmo si giudica *quanto gli si avvicina*. È un ruolo che vale la pena ricordare, perché è lo stesso che nella sezione 13.9 spiega come mai OPT compaia in tutte le tabelle di confronto pur non essendo usabile.

Nella simulazione della slide, con 3 frame e la sequenza di 20 riferimenti, OPT fa **9 page fault**.

#### 13.2 Algoritmo NRU (Not Recently Used)

L'idea è usare le statistiche che l'hardware già raccoglie. [SO-03 p20]

- si usano il bit di **referenziamento (R)** e di **modifica (M)**;
- sono **aggiornati tipicamente in hardware** e **azzerati dal sistema operativo**;
- il bit di referenziamento viene azzerato **periodicamente**.

##### NRU: le quattro classi

Si distinguono **quattro classi** di pagine: [SO-03 p20]

| Classe | R | M |                                         |
|--------|---|---|-----------------------------------------|
| 0      | 0 | 0 | <i>non referenziata, non modificata</i> |
| 1      | 0 | 1 | <i>non referenziata, modificata</i>     |
| 2      | 1 | 0 | <i>referenziata, non modificata</i>     |
| 3      | 1 | 1 | <i>referenziata, modificata</i>         |

Viene scelta una pagina dalla **classe non vuota di numero più basso**.

#### L'ordine delle classi non è arbitrario

È la domanda che smaschera chi ha memorizzato la tabella senza capirla. L'ordine dice, in sostanza: si butta per prima la pagina che non è stata usata di recente (“non serve”) e che inoltre non è stata modificata (“buttarla non costa nulla”). Si sale di classe solo quando le classi più basse sono vuote. La gerarchia mette insieme le due informazioni:

- **R** dice il *valore*: se è stata usata di recente, probabilmente servirà ancora;
- **M** dice il *costo*: se è stata modificata, va salvata su disco prima di essere buttata, e questo costa un trasferimento.

Da notare che la classe **1** (R=0, M=1) sta *prima* della **2** (R=1, M=0): una pagina modificata ma non recente è considerata più sacrificabile di una non modificata ma recente. È una scelta precisa: il costo del salvataggio conta meno del valore d'uso.

### 13.3 FIFO e seconda chance

[SO-03 p21]

#### FIFO

Viene rimossa la **pagina più vecchia**, cioè quella caricata per prima.

La slide commenta che la scelta **non è sempre felice**: “può rimuovere pagine, sì vecchie, ma magari molto usate”. [SO-03 p21]

#### Seconda chance

Si tiene conto dell'**attuale stato del bit R**: viene rimossa la pagina più vecchia **se non usata di recente**. Se la più vecchia ha R = 1, le si dà una seconda possibilità.

#### Cosa chiedono all'esame

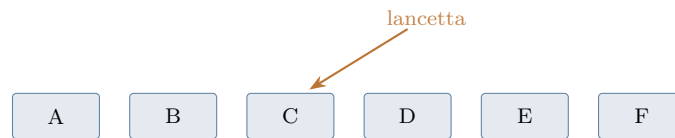
La seconda chance è il FIFO “corretto” da un'osservazione semplice: nel FIFO l'età di caricamento è usata come *stima* dell'inutilità, ma la stima è sbagliata quando una pagina vecchia è anche molto usata. La seconda chance sostituisce alla domanda “quanto tempo è in memoria?” la domanda “è stata usata di recente?”. Nota che il bit R viene **azzerato** quando si concede la seconda possibilità: così una pagina può essere “risparmiata” al massimo una volta per ciclo.

Il costo però è importante: quando molte pagine hanno R = 1, si percorre la coda quasi interamente prima di trovarne una da buttare, e se tutte hanno R = 1 la ricerca può essere lunga. È esattamente ciò che l'algoritmo successivo risolve.



## 13.4 Algoritmo Clock

È la seconda chance implementata in modo più efficiente. [SO-03 p22]



### Clock

Le pagine sono disposte **in cerchio**, come i numeri di un orologio, e una **lancetta** indica la pagina considerata. Quando serve una vittima si guarda la pagina puntata: se ha  $R=0$  è la vittima; se ha  $R=1$  le si azzerava  $R$  e la lancetta **avanza**. L'idea è quella della seconda chance, ma l'**implementazione è più efficiente** perché non richiede di spostare elementi in una coda: si muove solo un indice. [SO-03 p22]

### Che cosa si guadagna davvero

La seconda chance, per “rimandare in fondo” una pagina, deve *spostarla* nella lista. Il clock ottiene lo stesso risultato *senza spostare nulla*: le pagine restano dove sono e si sposta solo il punto di osservazione. È un esempio da manuale di come due algoritmi equivalenti per comportamento possano avere costi implementativi molto diversi — ed è il motivo per cui il clock, e non la seconda chance, è ciò che si trova nei sistemi reali.

## 13.5 Algoritmo LRU

[SO-03 p23]

Il ragionamento di partenza è un'**ipotesi**: *probabilmente le pagine più usate di recente lo saranno anche in futuro*. Da qui l'idea di rimuovere le pagine **meno usate di recente**.

### LRU — Least Recently Used

Si rimuove la pagina che **non viene usata da più tempo**. È una **buona idea** ma **non semplice e dispendiosa da implementare**:

- **con supporto hardware**: un **contatore nella CPU** e campi relativi nella tabella delle pagine;
- **via software**: si approssima con l'algoritmo **NFU** oppure con l'algoritmo di **Aging**.

### Perché LRU costa tanto

LRU richiede di sapere, per ogni pagina, *quando* è stata usata l'ultima volta — non *se* è stata usata. Le conseguenze:

- la differenza con NRU è tutta qui: NRU si accontenta del bit  $R$  (“usata o no dopo l'ultimo azzeramento”), LRU vuole un **ordinamento completo** delle pagine per istante d'uso;
- con il supporto hardware serve un **contatore** incrementato ad *ogni* accesso alla memoria, e un campo in ogni voce della tabella delle pagine in cui copiarlo. È un'**operazione di scrittura in memoria ad ogni accesso**: rovinoso per le prestazioni;
- con un contatore a 64 bit si torna al problema della dimensione delle voci (capitolo 12), e il contatore va comunque *aggiornato* ad ogni accesso.

Da qui nasce la necessità delle due approssimazioni software.

### 13.6 Algoritmo NFU

#### NFU — Not Frequently Used

È un'approssimazione via software di LRU. Si tiene un **contatore in ogni voce della tabella delle pagine**; periodicamente il valore del **bit R**, prima di essere azzerato, viene sommato a tale contatore. Viene rimossa la pagina con il **contatore più basso**. [SO-03 p24]

Il problema, che la slide enuncia con precisione: *può erroneamente privilegiare pagine che sono state molto utilizzate in passato ma che invece sono scarsamente usate di recente: queste lo saranno, probabilmente, anche nel prossimo futuro*. [SO-03 p24]

#### Il difetto di NFU: il contatore non dimentica

NFU conta le volte che una pagina è stata usata, ma **non dà peso a quando** è stata usata. Una pagina usata intensamente all'inizio dell'esecuzione del processo ha un contatore alto, e resta “protetta” per sempre, anche se da allora non è stata toccata. La memoria del contatore è *infinita*, mentre ciò che interessa è l'uso *recente*. È esattamente l'errore opposto rispetto a FIFO: FIFO ricorda solo l'*età* (ignora l'uso), NFU ricorda solo l'*uso* (ignora quando). L'algoritmo di aging risolve entrambi i difetti.

### 13.7 L'algoritmo di Aging

#### Aging

È la correzione di NFU. Ad ogni scadenza del clock: [SO-03 p25]

1. **shift a destra** del contatore associato ad ogni pagina;
2. **accostamento a sinistra** (come bit più significativo) del **bit R**.

| $t$        | 7 | 6     | 5     | 4     | 3     | 2     | 1     | 0     |
|------------|---|-------|-------|-------|-------|-------|-------|-------|
| R          | 1 | 0     | 1     | 1     | 0     | 0     | 1     | 1     |
| valore     |   | 1     | 0     | 1     | 1     | 0     | 0     | 1     |
| peso       |   | $2^7$ | $2^6$ | $2^5$ | $2^4$ | $2^3$ | $2^2$ | $2^1$ |
| contributo |   | 128   | 32    | 16    | 8     | 2     | 1     |       |

#### Il senso del contatore: una storia recente, pesata

È l'algoritmo che l'esame chiede più spesso di spiegare, e la spiegazione giusta è questa. Il contatore di  $n$  bit contiene la **storia degli ultimi  $n$  azzeramenti del bit R**, con un peso che **decresce esponenzialmente** andando indietro:

- il bit più significativo è l'**ultimo** periodo osservato;
- il bit meno significativo è quello di  $n$  periodi fa, e pesa  $2^{-(n-1)}$  rispetto al primo.

Il risultato è che una pagina usata *molto ma molto tempo fa* ha un contatore basso, perché i suoi 1 sono “scivolati” verso destra. È esattamente ciò che manca a NFU: l'**oblio**. La pagina

si “dimentica” del passato remoto, e questo è il comportamento voluto, perché il modello di località dice che ciò che conta è ciò che si è usato di recente.

Il numero di bit del contatore è quindi il numero di periodi che l'algoritmo ricorda, e il periodo è l'intervallo fra due azzeramenti del bit R: sono due parametri da scegliere, ed è qui che l'aging assomiglia alla media esponenziale dello SPN (capitolo 9).

## 13.8 Il confronto delle prestazioni

La metrica scelta è il **numero di page fault**. Le condizioni sono fissate: **3 frame** e la sequenza compatta delle pagine [SO-03 p26]

7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1

I risultati dichiarati dalle slides:

| Algoritmo | Fault     | Fonte       |
|-----------|-----------|-------------|
| OPT       | <b>9</b>  | [SO-03 p26] |
| LRU       | <b>12</b> | [SO-03 p28] |
| FIFO      | <b>15</b> | [SO-03 p27] |

### I tre numeri, verificati

Questi tre valori sono riprodotti dallo script che accompagna la guida eseguendo la simulazione, non ricopiati dalla slide: eseguendolo si ottengono 9, 12 e 15 esattamente come sopra. Vale la pena notare la gerarchia che ne risulta: **OPT (9) < LRU (12) < FIFO (15)**, cioè l'approssimazione software di LRU sta in mezzo fra l'ideale irraggiungibile e l'algoritmo ingenuo. È il modo in cui la slide costruisce l'argomento: ogni algoritmo si giudica *quanto si avvicina a OPT*.

### 13.8.1 L'anomalia di Belady

[SO-03 p27]

La slide presenta il FIFO e poi dice: *ma c'è qualcosa di strano*. Con la sequenza

1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5

si osserva che **aumentando i frame disponibili aumentano i page fault**. È l'**anomalia di Belady**. [SO-03 p27]

| Algoritmo | 3 frame | 4 frame   |
|-----------|---------|-----------|
| FIFO      | 9       | <b>10</b> |
| LRU       | 10      | 8         |

### Perché è “anomala”: la proprietà di inclusione

È il punto di teoria più importante della sezione, e la slide lo esplicita. LRU non soffre dell'anomalia perché gode della **proprietà di inclusione**:

$$B_t(n) \subseteq B_t(n+1) \quad \forall t, n$$

cioè l'insieme di pagine caricate avendo  $n$  frame è **sempre contenuto** in quello che si avrebbe con  $n+1$  frame. [SO-03 p28]

L'intuizione: se una pagina è fra le  $n$  “più meritevoli” con  $n$  frame, lo è a maggior ragione quando i frame sono  $n + 1$  (con LRU, “più meritevole” significa “usata più di recente”, e l'ordine degli ultimi usi non dipende da quanti frame ci sono). Con questo algoritmo avere più memoria non può far stare peggio. FIFO invece **non** ha la proprietà: l'ordine di espulsione dipende dall'ordine di *caricamento*, e questo cambia quando cambia il numero di frame. Aggiungere un frame cambia *quali* pagine vengono buttate, e può cambiare in peggio.

### 13.8.2 Chi soffre dell'anomalia

[SO-03 p28]

| Algoritmo             | Anomalia di Belady                                                                         |
|-----------------------|--------------------------------------------------------------------------------------------|
| LRU, NFU, Aging       | <b>no</b> : godono della proprietà di inclusione (NFU e aging sono approssimazioni di LRU) |
| seconda chance, clock | <b>sì</b> : riducono a FIFO                                                                |
| NRU                   | <b>sì</b> : riduce a FIFO                                                                  |

#### La regola in una riga

**Soffrono dell'anomalia tutti gli algoritmi che, in qualche caso, degenerano nel FIFO.** La seconda chance lo fa quando tutte le pagine hanno  $R = 1$  (il bit non protegge nessuno, e resta l'ordine di caricamento); NRU lo fa quando tutte le pagine finiscono nella stessa classe. È un criterio che vale la pena ricordare come *regola*, invece di imparare a memoria l'elenco: chiedersi “questo algoritmo può ridursi a buttare via la più vecchia?” basta a rispondere.

## 13.9 Riepilogo sugli algoritmi

È la tabella conclusiva delle slides, e conviene usarla come traccia di ripasso. [SO-03 p29]

| Algoritmo       | Giudizio delle slides                                                                            |
|-----------------|--------------------------------------------------------------------------------------------------|
| OPT             | non implementabile, ma utile come <b>termine di paragone</b>                                     |
| NRU*            | approssimazione <b>rozza</b> di LRU                                                              |
| FIFO*           | può portare all'eliminazione di <b>pagine importanti</b>                                         |
| Seconda chance* | un <b>netto miglioramento</b> rispetto a FIFO                                                    |
| Clock*          | come la seconda chance ma <b>più efficiente</b>                                                  |
| LRU             | <b>eccellente idea</b> (vicina a quella ottima) ma difficilmente realizzabile se non in hardware |
| NFU             | approssimazione software <b>abbastanza rozza</b> di LRU                                          |
| Aging           | <b>buona</b> approssimazione di LRU con implementazione software efficiente                      |

È la slide a segnalare con l'asterisco gli algoritmi che soffrono dell'anomalia di Belady.

#### La struttura da ricordare, invece dell'elenco

Gli otto algoritmi non sono otto cose diverse: sono **tre famiglie** e una progressione.

- **basati sull'età**: FIFO e le sue varianti (seconda chance, clock, e per certi versi NRU). Semplici, economici, ma buttano via pagine importanti e soffrono dell'anomalia.

- **basati sull'uso recente:** LRU e le sue approssimazioni software (NFU, aging). Migliori, ma più costosi; LRU non è realizzabile in pratica, e le approssimazioni perdono qualcosa.
- **i due estremi:** OPT, che è il limite teorico, e FIFO, che è il limite della semplicità.

L'osservazione che tiene insieme tutto: **NRU e aging usano gli stessi due bit** (R e M) e la differenza sta solo in *come* li combinano — NRU in quattro classi, aging in un contatore pesato. Non sono due idee diverse, sono due usi diversi della stessa informazione.

#### Come lo dici all'orale

“Mi parli degli algoritmi di sostituzione delle pagine.” La scaletta migliore è la **progressione dal peggiore al migliore**, con i numeri:

1. il **problema**: scegliere una vittima quando non ci sono frame liberi, minimizzando i fault *futuri* — e quindi serve *stimare* il futuro;
2. **OPT** come limite teorico (9 fault sull'esempio), e il fatto che serve solo come termine di paragone;
3. la famiglia dell'**età**: FIFO (15 fault, soffre l'anomalia), seconda chance, clock — dicendo che il clock è la stessa idea della seconda chance ma senza spostare elementi;
4. la famiglia dell'**uso**: LRU (12 fault, eccellente ma costoso), NFU (approssimazione che *non dimentica*: ecco il difetto), aging (la correzione, con lo shift e il bit R in testa);
5. l'**anomalia di Belady** e il criterio della **proprietà di inclusione**: chi degenera nel FIFO ne soffre, chi approssima LRU no.

Se il docente ti chiede “perché il clock e non la seconda chance, se fanno la stessa cosa?”, la risposta è: per il costo di implementazione, non per il comportamento — la seconda chance deve spostare le pagine nella lista, il clock muove solo un indice. È una domanda che distingue chi ha capito l'algoritmo da chi ha capito il sistema.

## Capitolo 14

### Altri aspetti della gestione della memoria

Il capitolo precedente ha stabilito *come* si traduce un indirizzo e *quale* pagina si butta via. Restano le domande di governo: **quanti** frame si danno a ciascun processo, **come** si evita che il sistema crolli, **quanto** deve essere grande una pagina, e **cosa** si può condividere. Chiude il capitolo la gestione della memoria *interna al kernel*, che ubbidisce a regole diverse da quelle dei processi utente.

#### 14.1 L'allocazione dei frame

##### Quanti frame a ciascun processo?

Si parte dalla **paginazione su richiesta** (*pure demand paging*). La domanda è quanti frame assegnare ad ogni processo, e la risposta ha un **minimo** e un **massimo**: [SO-03 p30]

- **minimo**: il **minimo strutturale**, cioè il numero di pagine che servono perché *una sola istruzione* possa essere eseguita — il set di istruzioni e i livelli di indirizzamento indiretto che quell'istruzione attraversa mentre è in corso;
- **massimo**: la **memoria libera** disponibile.

##### Che cosa vuol dire “minimo strutturale”

È un concetto che sembra astratto e non lo è, ed è la prima cosa da spiegare bene. Il punto è che *un'istruzione può toccare più pagine contemporaneamente*: un'istruzione come “copia il contenuto della locazione puntata da A nella locazione puntata da B” è un indirizzamento indiretto, e le pagine coinvolte sono tre (l'istruzione stessa, la pagina di A, la pagina di B). Se anche una sola di quelle pagine non è in memoria, l'istruzione va interrotta a metà — e se il sistema non ha abbastanza frame per tenerle tutte, il processo **non può avanzare di un'istruzione**, per quante volte lo si riprenda. Va in **thrashing** patologico: ogni tentativo genera un fault, ogni fault espelle una pagina che serve all'istruzione stessa. Per questo il minimo non è “qualche pagina”, ma un numero calcolato sull'**architettura dell'insieme di istruzioni**. Ed è anche il motivo per cui questa macchina, che sembra teoria pura, è invece il parametro che il sistema operativo deve conoscere davvero.

##### 14.1.1 Le strategie di allocazione

[SO-03 p30]

| Strategia            | Criterio                                                                         |
|----------------------|----------------------------------------------------------------------------------|
| <b>equa</b>          | lo stesso numero di frame a tutti i processi, senza distinzioni                  |
| <b>proporzionale</b> | i frame sono distribuiti in proporzione alla dimensione del processo             |
| <b>per priorità</b>  | si tiene conto della <i>priorità</i> del processo, non solo della sua dimensione |

L'allocazione proporzionale si esprime con una formula precisa: al processo *i*-esimo di dimensione  $s_i$  si assegnano [SO-03 p30]

$$a_i = \frac{s_i}{S} \times m \quad \text{con} \quad S = \sum_i s_i$$

dove  $m$  è il numero totale di frame disponibili.

**Che cosa non dice la formula — e le slides invece sì**

Le slides aggiungono subito dopo due avvertenze che una formula da sola non trasmette: **adeguamenti al livello di multiprogrammazione**. Il numero  $m$  di frame disponibili *cambia* quando un processo entra o esce dall'elenco dei processi attivi, e quindi l'allocazione proporzionale non è una decisione presa una volta per tutte ma qualcosa che va *ricalcolato*. La formula dà l'istantanea, non la politica.

**14.1.2 Allocazione locale e globale****Locale o globale**

Quando serve una pagina vittima, la scelta può cadere: [SO-03 p31]

- solo sulle pagine dello stesso processo → **allocazione locale**;
- su tutte le pagine, anche di altri processi → **allocazione globale**.

**Il compromesso fra le due**

Non è una scelta tecnica neutra, è una scelta di *politica fra processi*:

- con l'allocazione **locale** le prestazioni di un processo dipendono solo da quante pagine ha, e non da cosa stanno facendo gli altri: **deterministico e prevedibile**, ma un processo ricco di frame tiene fermi dei frame che un processo affamato userebbe volentieri;
- con l'allocazione **globale** i frame si spostano verso chi ne ha bisogno: **più efficiente in media**, ma le prestazioni di un processo dipendono dagli altri — e chi ha un *page fault rate* alto si prende i frame di chi ce l'ha basso, che paradossalmente può iniziare a faultare a sua volta. È il meccanismo che porta al **thrashing** descritto fra poco.

Il compromesso si ritrova in molti altri punti del corso: la stessa tensione fra “politica locale” e “politica globale” è quella fra scheduling a code separate e a coda unica (capitolo 10).

**14.1.3 Thrashing e sovraccarico**

[SO-03 p31]

**Thrashing**

Se a un processo restano **pochi frame**, si distingue:

- **sotto il minimo strutturale**: il processo viene **sospeso** e si fa **swapping** su disco — è l'unica cosa sensata da fare;
- **un po' sopra il minimo**: il processo può andare in **thrashing**, cioè passare la maggior parte del tempo a gestire page fault invece che a eseguire.

Quando il thrashing riguarda **tutti** i processi si parla di sistema in **sovraccarico**, causato da un **eccessivo livello di multiprogrammazione**.

**L'errore da non fare: “aggiungo processi, quindi sfrutto meglio la CPU”**

È il paradosso che vale la pena raccontare all'orale, perché va contro l'intuizione. Aggiungere un processo dovrebbe *aumentare* l'utilizzazione della CPU (c'è sempre qualcuno pronto).

Oltre un certo punto succede il contrario, e il meccanismo è questo:

1. l'allocazione è globale, quindi i frame si spostano verso chi faulta di più;
2. ma chi faulta di più è chi ha *meno* frame, cioè tipicamente il processo appena arrivato — che non ha ancora caricato il suo working set;
3. i frame vengono tolti a processi che stavano lavorando bene, e questi cominciano a faultare a loro volta;
4. la CPU passa il tempo a **servire page fault**, cioè a fare I/O, e l'utilizzazione *crolla*.

Il rimedio “a forza bruta” è ridurre il livello di multiprogrammazione (sospendere processi e fare swapping). Il rimedio “intelligente” è assegnare i frame in modo commisurato alle **necessità** del processo — e per sapere quali siano le necessità serve il **modello di località**, che è esattamente ciò che le slides introducono subito dopo con il concetto di *località* e la *strategia di base* del working set. [SO-03 p31]

## 14.2 Il working set

### Working set

Per ogni processo si mantiene il suo **working set**: l'insieme delle **pagine usate negli ultimi  $\Delta$  accessi alla memoria**. [SO-03 p32]

Il parametro  $\Delta$  va scelto **per adattarsi alla località corrente**. Conoscendo il working set attuale di ogni processo si può confrontare la **richiesta globale** di frame con la memoria disponibile: [SO-03 p32]

$$D = \sum_i WSS_i \quad \text{contro} \quad \text{memoria disponibile}$$

e da questo confronto si ottiene la **prevenzione del thrashing**.

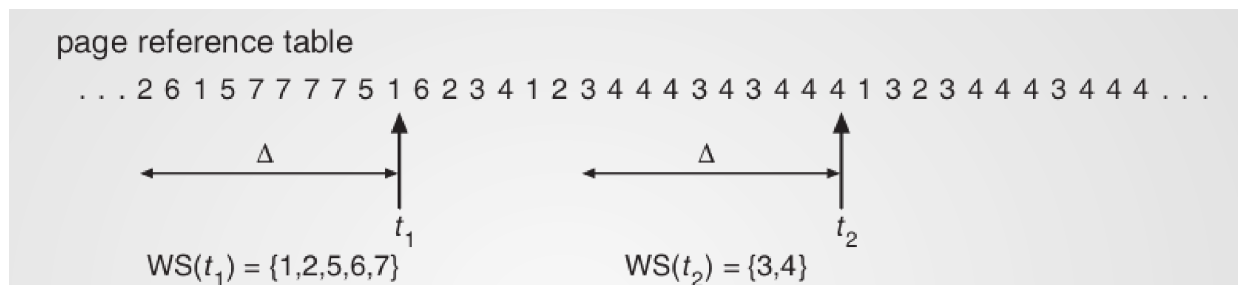


Figura 14.1: Il working set è la finestra degli ultimi  $\Delta$  accessi. A  $t_1$  la finestra contiene 2, 6, 1, 5, 7, 7, 7, 7, 5, 1 e il working set è  $\{1, 2, 5, 6, 7\}$ ; a  $t_2$ , dopo che il processo è passato a lavorare su altre pagine, la stessa finestra contiene solo  $\{3, 4\}$ : il working set *si sposta*, ed è proprio questo che il parametro  $\Delta$  deve inseguire

Fonte: proiezioni del corso, SO-03 p32

### I due numeri della figura, verificati

Vale la pena controllare che i due insiemi della figura siano davvero quelli che una finestra di  $\Delta = 10$  produce sulla sequenza mostrata, perché è il tipo di verifica che si può chiedere all'esame. La sequenza è

2 6 1 5 7 7 7 7 5 1 6 2 3 4 1 2 3 4 4 4 3 4 3 4 4 4

Gli ultimi 10 riferimenti prima di  $t_1$  sono 2, 6, 1, 5, 7, 7, 7, 7, 5, 1 e le pagine distinte sono



$\{1, 2, 5, 6, 7\}$ : **cinque** pagine. Gli ultimi 10 prima di  $t_2$  sono 3, 4, 4, 4, 3, 4, 3, 4, 4, 4 e le pagine distinte sono  $\{3, 4\}$ : **due** pagine. Il working set è passato da cinque a due pagine, cioè il processo ha cambiato località.

### 14.2.1 Il calcolo in pratica

Il working set non si può calcolare “tenendo una lista”: servirebbe conoscere esattamente gli ultimi  $\Delta$  accessi. Si **approssima** con tre strumenti. [SO-03 p32]

- **interrupt periodici**: un timer che si attiva a intervalli regolari;
- il **bit di referenziamento R**, già visto per gli algoritmi di sostituzione;
- un **log** che conserva la “storia di R” in base al parametro  $\Delta$ .

#### Come si combinano i tre

La costruzione è la stessa idea dell’algoritmo di aging (capitolo 13), applicata all’insieme invece che alla singola pagina. Ad ogni interrupt:

1. si legge il bit R di tutte le pagine;
2. lo si registra nel log, in testa;
3. si azzerava R.

Il log diventa così **la sequenza degli accessi a grana grossa**: “nell’intervallo 1 la pagina 7 è stata usata, nell’intervallo 2 no”, e via dicendo. Il working set approssimato è l’insieme delle pagine che hanno **almeno un 1 negli ultimi  $\Delta$  intervalli** registrati nel log.

Il costo è la lunghezza del log: serve una **matrice pagine  $\times \Delta$**  — per esempio 1000 pagine per 100 intervalli sono 100 000 bit, circa 12 kB. È un costo che si paga volentieri, ma è bene ricordare che è un costo, ed è il motivo per cui i sistemi reali spesso usano la tecnica più economica del paragrafo successivo.

## 14.3 La Page Fault Frequency

### PFF

È un modello **più diretto** per affrontare il thrashing: invece di ricostruire il working set, si **monitora direttamente la Page Fault Frequency** dei processi. Serve a **caratterizzare i sistemi in sovraccarico**. [SO-03 p33]

Le slides osservano che **i due modelli sono in relazione**: il working set e la frequenza dei page fault misurano la stessa cosa da due lati. [SO-03 p33]

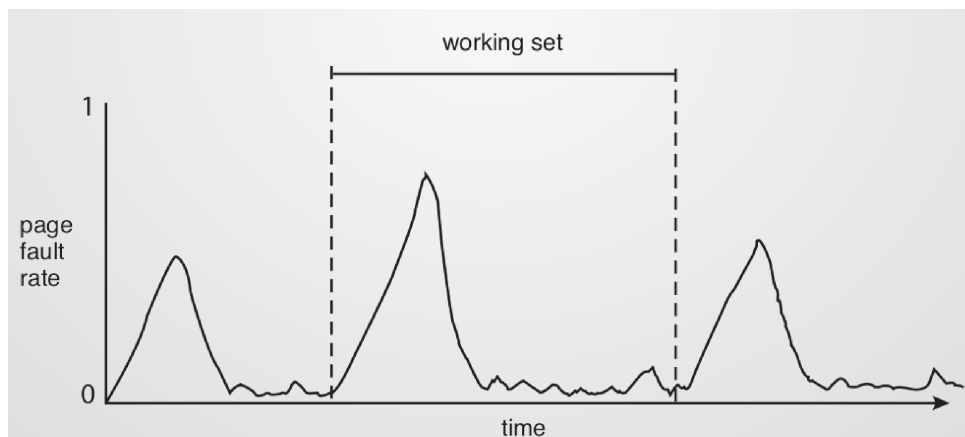


Figura 14.2: La frequenza dei page fault nel tempo. La curva sta quasi a zero nei periodi in cui il working set è in memoria, e *esplode* quando il processo cambia località e le pagine che gli servono non ci sono più: la parentesi tratteggiata è la “working set” delle slides, cioè la finestra in cui il modello classico lavorerebbe

Fonte: proiezioni del corso, SO-03 p33

#### Il vantaggio di PFF: misurare invece di ricostruire

È la domanda “perché due modelli?” che vale la pena sapersi rispondere. Il working set è **descrittivo**: dice *quante* pagine servono a un processo, e da quel numero si *deduce* se il sistema è in sovraccarico (confrontando  $\sum WSS_i$  con la memoria). PFF è **diagnostico**: si guarda direttamente *quanto male sta andando* il processo, e si interviene subito.

In pratica PFF si usa così: si fissa una **soglia superiore** e una **inferiore**.

- se la frequenza supera la soglia alta, il processo ha **troppo pochi frame**: gliene si danno altri (eventualmente togliendoli a chi ne ha di troppo);
- se sta sotto la soglia bassa, il processo ha **frame in eccesso**: gliene si tolgono;
- un processo che faulta e non ha più frame da ricevere è un candidato allo **swapping**.

Il bello è che non serve *nessuna* struttura dati ausiliaria: il contatore dei page fault lo si mantiene comunque. È il motivo per cui PFF, pur essendo meno “elegante” del working set, è ciò che si trova implementato.

## 14.4 La politica di pulitura

### Paging daemon

Il meccanismo di gestione dei page fault è efficiente **soprattutto se ci sono frame liberi sempre disponibili**: avere un frame già libero velocizza la gestione. Da qui il **paging daemon**: un **processo di servizio** che [SO-03 p34]

- controlla lo **stato di occupazione globale** dei frame del sistema;
- **seleziona, libera ed eventualmente pulisce** pagine;
- mantiene un **pool di frame liberi**;
- consente il **ripescaggio dal pool** in caso di richiesta.

È usato per esempio su **Linux** e **Windows**.

### Perché serve un processo che non fa nulla di utile

Il paging daemon è un processo che, in apparenza, *spreca* tempo di CPU a buttare via pagine che forse nessuno chiederà. La ragione per cui conviene è che sposta il costo fuori dal momento critico:

- se il frame libero non c'è, il page fault deve *aspettare* che si trovi una vittima, e se la vittima è **sporca** (modificata) bisogna *anche* scriverla su disco prima di riusare il frame. Sono due attese in serie, dentro la gestione del fault;
- con il daemon, quando il fault arriva il frame è **già libero e già pulito**: l'attesa si è già pagata in anticipo, quando la CPU non aveva nulla di meglio da fare.

È la stessa logica della **pulitura anticipata** (*cleaning*) rispetto alla pulitura su richiesta: è la distinzione, che le slides avevano già nominato parlando degli algoritmi di sostituzione, fra scrivere una pagina quando si è costretti e scriverla quando è comodo. Si noti anche che il “ripescaggio dal pool” è ciò che rende il pool *recuperabile*: il daemon non butta via il contenuto definitivamente, e se la pagina viene richiesta subito dopo non serve rileggerla dal disco.

## 14.5 La dimensione della pagina

È un parametro di progetto con **vantaggi opposti** ai due estremi, e l'esame lo chiede spesso come “confronto”. [SO-03 p35]

| Pagina grande                                                 | Pagina piccola                                                           |
|---------------------------------------------------------------|--------------------------------------------------------------------------|
| tabella delle pagine più piccola                              | minore frammentazione interna                                            |
| migliore efficienza nel trasferimento I/O                     | migliore risoluzione nel definire il working set (meno memoria sprecata) |
| tende a minimizzare il numero di page fault (minore overhead) |                                                                          |

### Il filo che tiene insieme i tre vantaggi “grande”

Non sono tre cose scollegate: è lo **stesso** effetto contato in tre modi.

- la tabella è piccola perché con pagine grandi servono *meno voci* per coprire la stessa memoria;
- il trasferimento I/O è più efficiente perché il disco lavora meglio su blocchi grandi, e l'overhead per accesso si ammortizza su più byte;
- i page fault sono **meno** perché ogni fault porta in memoria più roba: è il **principio di località** che lavora a favore — le pagine vicine a quella richiesta sono probabilmente quelle che serviranno subito dopo, e vengono caricate “gratis” insieme a lei.

È *quest'ultimo* il vantaggio che conta di più, ed è anche quello meno ovvio: si paga meno *overhead* di gestione, non meno I/O in assoluto.

### Il filo che tiene insieme i due vantaggi “piccola”

Anche qui sono lo stesso effetto visto due volte.

- la **frammentazione interna** è, per definizione (capitolo 11), lo spazio sprecato *dentro* la pagina: in media metà pagina per processo. Con pagine da 4 kB sono 2 kB sprecati

per processo, con pagine da 64 kB sono 32 kB — e questo vale anche per il *codice*, dove l'ultima pagina di un segmento di testo è quasi sempre mezza vuota;

- la **risoluzione del working set** è la stessa cosa detta dal lato opposto: una pagina che è stata toccata *una volta sola* finisce per intero nel working set, e con pagine grandi significa che il working set *sembra* più grande di quanto sia. Il modello perde di precisione, e la stima di frame necessari (che è ciò che previene il thrashing) sbaglia per eccesso.

La slides chiudono segnalando la **relazione con la dimensione del blocco su disco**: i due parametri non si scelgono indipendentemente. [SO-03 p35]

#### Il compromesso in una riga

La tensione di fondo è questa: **la pagina è l'unità di trasferimento e l'unità di protezione, ma anche l'unità di spreco**. Tutto ciò che si guadagna ingrandendola, lo si paga in spazio sprecato e in precisione del modello. I sistemi reali scelgono tipicamente pagine di 4 kB, con pagine “grandi” opzionali (2 MB o 1 GB) usate su richiesta per le regioni che lo meritano — ma questa è una nota di contesto, non un contenuto delle slides.

## 14.6 Le pagine condivise

I processi possono **condividere alcune pagine**, in due modi diversi che vanno tenuti distinti. [SO-03 p36]

#### Condivisione in sola lettura e in lettura/scrittura

[SO-03 p36]

- **solo lettura**: è il caso del **codice eseguibile condiviso**, detto **codice rientrante**. Più processi eseguono lo stesso programma e possono usare *le stesse* pagine di codice, perché nessuno lo modifica.
- **lettura/scrittura**: è l'**IPC tramite memoria condivisa**. Più processi mappano le stesse pagine nel proprio spazio e le usano per *comunicare*.

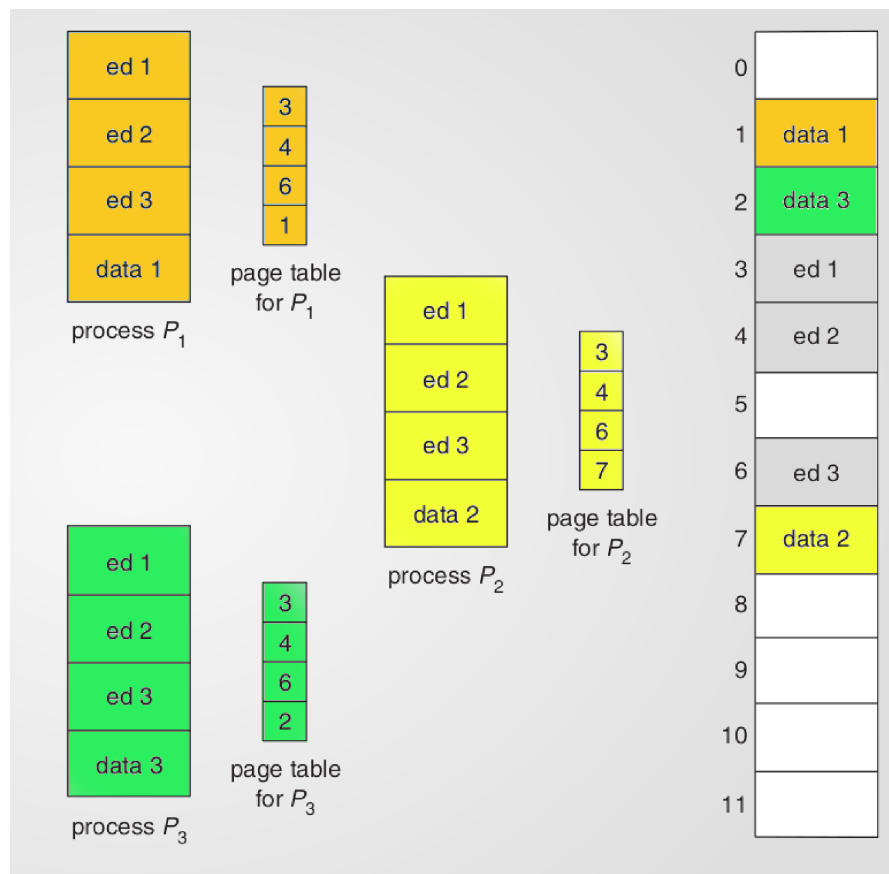


Figura 14.3: Pagine condivise fra tre processi. Le pagine di codice **ed 1**, **ed 2**, **ed 3** sono le *stesse* per  $P_1$ ,  $P_2$  e  $P_3$ : le tre tabelle delle pagine le mappano sui frame 3, 4 e 6. Le pagine di dati sono invece private, e ciascun processo le mappa su un frame diverso (1, 7 e 2)

Fonte: proiezioni del corso, SO-03 p36

#### La figura spiega da sola l'implementazione

Da questa figura si legge perché l'implementazione è **semplice ed efficiente** su una tabella delle pagine ordinaria o multilivello [SO-03 p36]: non serve *nulla* di speciale. La condivisione consiste soltanto nel fare in modo che **due voci, in due tabelle diverse, contengano lo stesso numero di frame**. Non ci sono strutture aggiuntive, non ci sono formati speciali: è la normalità. È anche la ragione per cui si è disposti a sopportare le complicazioni del paragrafo successivo, che sono di *cache* e non di traduzione.

È utile notare anche cosa rimane *privato*: i dati. Ogni processo ha la sua **data**, su un frame suo. La condivisione del codice non implica la condivisione dei dati, ed è proprio ciò che si vuole: l'alternativa — una copia del codice per ogni processo — è lo spreco che il linking dinamico e le librerie condivise eliminano.

#### 14.6.1 Le difficoltà: la cache

##### Il problema che rende la condivisione non banale

La difficoltà nasce dalla **cache**, non dalla tabella delle pagine. [SO-03 p37] Il punto è che una cache può essere indicizzata su **indirizzi virtuali**: se due processi mappano la *stessa* pagina fisica su indirizzi virtuali *diversi*, la cache vede due indirizzi diversi e li tratta come due dati diversi — ma sono lo stesso dato. Si creano così due copie della stessa pagina in cache, e una scrittura su una delle due **non si vede** dall'altra. Le slides notano che il problema sussiste

**anche usando gli ASID** (gli identificatori di spazio degli indirizzi), perché due processi *condividono* legittimamente la pagina e gli ASID servono a *separare* spazi, non a unirli.

Le due soluzioni elencate dalle slides [SO-03 p37]:

1. **disabilitare la cache sulle pagine condivise;**
2. usare **cache con ricerca basata su indirizzi virtuali e tag fisici**: la cache ricerca **in parallelo con la TLB** sulla base dell'indirizzo virtuale, ma **per capire se si tratta di un duplicato** bisogna aspettare che la TLB dia in output l'indirizzo fisico.

#### Le due soluzioni e il loro prezzo

Sono due modi di pagare lo stesso conto, e conviene saper dire quale costa cosa:

- la **prima** è semplice ma **butta via le prestazioni**: le pagine condivise — che sono tipicamente le pagine di *codice*, quelle eseguite di continuo da tutti i processi — diventano le più lente di tutte. È una soluzione “corretta ma inaccettabile” nella pratica;
- la **seconda** è la soluzione vera, e il suo costo è la **latenza**: si guadagna il fatto di poter cercare in cache *mentre* la TLB traduce, ma la conferma che non si tratti di un duplicato arriva solo dopo. È un esempio da manuale di come nell'hardware moderno si sovrappongano le operazioni per nascondere le latenze, e di quanto sia delicato farlo.

#### Curiosità: la cache è la ragione del problema

È interessante notare *dove* nasce il problema della condivisione. La traduzione degli indirizzi non ne soffre affatto: la figura precedente mostra che condividere è banale. Il problema nasce solo perché la cache introduce un livello di indirectione in più, e quel livello può essere indicizzato su un indirizzo che non è quello fisico. È esattamente il fenomeno già incontrato nel capitolo 12, dove la cache e la memoria virtuale venivano presentate come due meccanismi con *conseguenze reciproche*.

### 14.6.2 Le difficoltà: la tabella delle pagine invertita

[SO-03 p37]

#### Condivisione con tabella invertita

La tabella invertita (capitolo 12) ha **una voce per frame**, non per pagina. Condividere una pagina significa che più indirizzi virtuali devono puntare allo stesso frame, e questo va gestito in modo diverso a seconda dell'hardware:

- **singolo core**: si altera la tabella al **context switch** o al **page fault**;
- **multi core**: è **difficilmente gestibile**, se non con *teoriche* tabelle delle pagine invertite con **corrispondenze multi-a-uno**.

#### Perché la tabella invertita è incompatibile con la condivisione

È una domanda che unisce due argomenti e per questo piace molto. Il ragionamento:

1. nella tabella **normale** la relazione è “da pagina virtuale a frame”, e è una **funzione**: ogni pagina virtuale ha *un* frame. Che due pagine virtuali diverse puntino allo stesso frame non crea alcun conflitto — è esattamente la figura di sopra;

2. nella tabella **invertita** la voce è **una per frame** e contiene *l'indirizzo* virtuale che quel frame sta servendo. La relazione si è **rovesciata**, e ora il vincolo “un solo indirizzo virtuale per voce” è un **limite strutturale**: il secondo processo che vuole condividere la pagina *non ha dove scrivere* il proprio indirizzo, se non **cambiando la voce** — ed è quello che fa la soluzione *singolo core* (cambia la voce al momento giusto).
3. su **multi core** l'inganno non regge: due core possono eseguire i due processi *contemporaneamente*, e allora cambiare la voce “al momento giusto” non basta, perché non esiste *un momento giusto* per entrambi. Servirebbero tabelle che ammettano più indirizzi virtuali per la stessa voce, cioè corrispondenze **multi-a-uno**: le slides le chiamano *teoriche* perché nella pratica non si realizzano.

È un buon esempio del perché la tabella invertita sia rimasta una curiosità storica: il suo risparmio di memoria si paga con l'impossibilità pratica di condividere, che nei sistemi reali serve *sempre*.

## 14.7 Copy-on-write e zero-fill-on-demand

Due **ottimizzazioni** che sfruttano la stessa idea di fondo: non fare adesso un lavoro che forse non servirà mai. [SO-03 p38]

### Copy-on-write

Si **condivide finché possibile** fra processi **tutti i tipi di pagine**, sia codice sia dati. La copia vera avviene solo quando uno dei processi **scrive** nella pagina. [SO-03 p38]

### Zero-fill-on-demand

Il **principio di base** è che le **nuove pagine sono vuote e allocate su richiesta**. Le slides ne elencano le ragioni: [SO-03 p38]

- **azzeramento efficiente** gestito dal kernel;
- **sicurezza**;
- **pool di pagine vuote**;
- **copy-on-write su una read-only static zero page**.

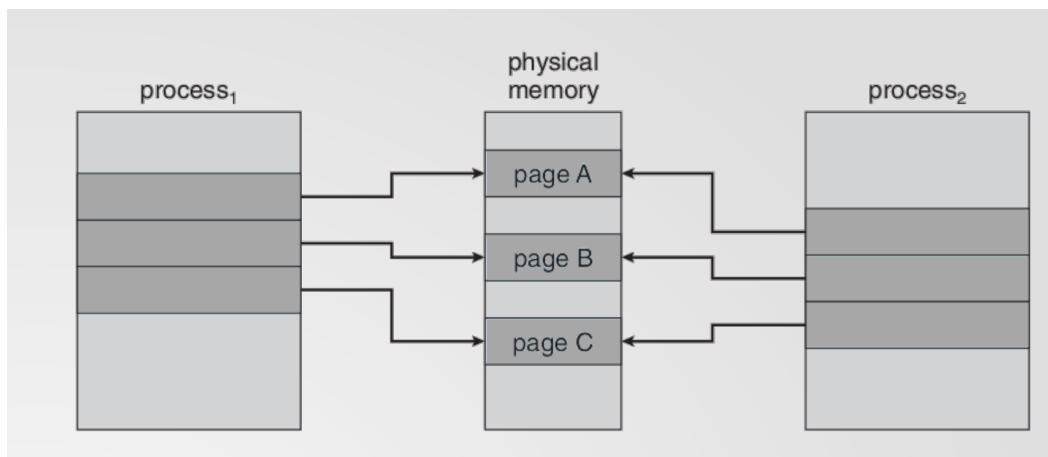


Figura 14.4: Copy-on-write: i due processi partono condividendo le stesse pagine fisiche *A*, *B* e *C*. La copia non si fa all’inizio ma *al primo tentativo di scrittura*, e solo della pagina che viene effettivamente modificata

Fonte: proiezioni del corso, SO-03 p38

#### Copy-on-write: dove si usa davvero

È la tecnica che rende possibile **fork** economico. Dopo una **fork** il figlio è una copia del padre, e copiare l’intero spazio degli indirizzi sarebbe costosissimo — ma quasi sempre il figlio esegue subito una **exec**, che *sostituisce* tutto lo spazio con un programma nuovo. Copiare sarebbe stato lavoro sprecato.

Il meccanismo: le pagine sono marcate **sola lettura** e condivise; il primo che scrive genera un **fault di protezione**, e il kernel risolve la cosa copiando *quella* pagina. Da notare che serve il **bit di modifica M** di cui si è parlato nel capitolo 13: lo stato della pagina va tracciato per sapere quando la copia è necessaria.

#### Zero-fill-on-demand: due ragioni, non una

È facile liquidarla come “un’ottimizzazione”, ma le slides ne elencano ragioni di natura diversa e conviene non confonderle:

- l’**efficienza**: azzerare una pagina è un’operazione che si può rimandare a quando serve davvero, e il kernel la fa con istruzioni ottimizzate;
- la **sicurezza**: se si riciclasse una pagina *non* azzerata, un processo leggerebbe i dati di un processo precedente. È un caso da manuale di “innocuo errore di ottimizzazione che diventa vulnerabilità”;
- il **pool di pagine vuote**: un insieme di pagine già azzerate tenute da parte, così che l’azzeramento avvenga fuori dal momento critico — lo stesso principio del paging daemon.

L’ultimo punto dell’elenco delle slides è il più elegante: **copy-on-write su una read-only static zero page**. È una pagina fisica sola, condivisa da tutti i processi, che contiene tutti zeri e non viene mai scritta. Ogni pagina “nuova” comincia come riferimento a *quella*; chi vuole scriverci sopra non la modifica ma ne riceve una tutta sua, azzerata. Così **non si azzerà nulla finché non si scrive** — e se un processo chiede un megabyte di memoria e poi non lo usa, non ha occupato nulla.



## 14.8 Librerie condivise e file mappati

### Linking statico e dinamico

[SO-03 p39]

- **linking statico:** **inclusione del codice** della libreria nel programma in fase di *linking*;
- **linking dinamico:** **collegamento e caricamento a run-time** di **librerie condivise**.

I vantaggi dichiarati del linking dinamico:

- **risparmio di spazio** su disco *e* in RAM;
- **sviluppo indipendente** e **facilità di aggiornamento**.

### I due risparmi non sono lo stesso risparmio

Vale la pena distinguerli, perché sono di natura diversa:

- il risparmio **su disco** deriva dal fatto che la libreria esiste una volta sola, invece di essere ricopiata dentro ogni eseguibile che la usa;
- il risparmio **in RAM** deriva dalla **condivisione delle pagine** descritta poco sopra: le pagine di codice della libreria vengono mappate da tutti i processi che la usano, e occupano *un* posto solo. È qui che le librerie condivise toccano il tema delle pagine condivise: ne sono l'applicazione principale.

Il “facilità di aggiornamento” è una conseguenza di progetto: si sostituisce *un* file invece di ricompilare tutti i programmi.

### File mappati

È un **modello alternativo di I/O su file**: al posto di *read/write*, il file viene **mappato nello spazio degli indirizzi** del processo, e i suoi accessi diventano normali accessi alla memoria. Ne discende la **possibilità di condivisione**. [SO-03 p39]

Le slides elencano cosa usano i file mappati per **gestire automaticamente**: [SO-03 p39]

- le **librerie condivise**;
- il **caricamento del codice eseguibile**;
- il **caricamento dei dati statici**.

### Il file mappato è il meccanismo sottostante

Il punto che unisce questo paragrafo al resto del capitolo: i file mappati *non sono una funzionalità fra le tante*, sono **il meccanismo con cui tutto il resto funziona**. Caricare un eseguibile significa mapparne le pagine di codice; caricare le librerie condivise significa mapparle in sola lettura; caricare i dati statici significa mappare la parte inizializzata del segmento dati. Il caricamento di un programma non è più “leggere il file dall’inizio e copiarlo in memoria”: è **installare una traduzione** da indirizzi virtuali a pagine del file.

Il vantaggio pratico si vede bene con il **paging su richiesta**: di un eseguibile da 100 MB si caricano solo le pagine che servono davvero. E siccome il file rimane il *contenitore* delle pagine, una pagina di codice non modificata non va mai scritta: si butta via e si rilegge dal file — che è esattamente la distinzione fra pagina pulita e sporca della sostituzione delle

pagine (capitolo 13).

## 14.9 L'allocazione della memoria per il kernel

### Utente e kernel: due problemi diversi

[SO-03 p40]

- la **memoria dei processi utente** è **paginata**, con la conseguente **frammentazione interna**;
- per la **memoria interna al kernel** l'obiettivo è una **frammentazione interna minima o nulla**, e in alcuni casi **l'allocazione non si può paginare**.

### Perché il kernel non può usare le pagine come i processi

È la domanda giusta da farsi davanti a questa slide. La ragione è la stessa del minimo strutturale di inizio capitolo, portata all'estremo:

- il kernel alloca **strutture dati piccole e numerosissime** (descrittori di processo, descrittori di file, *inode* in memoria, punti di montaggio): una tabella delle pagine con una voce da 4 kB per un descrittore da 200 byte spreca il **95%** della memoria;
- la struttura deve essere **fisicamente contigua** in molti casi, perché l'hardware (per esempio un controller DMA) deve poterla indirizzare senza passare dalla MMU;
- la frammentazione interna è **inaccettabile** proprio perché il kernel non può contare su "tanta" memoria sacrificabile: il kernel è la parte che deve funzionare sempre.

Da notare l'asimmetria, che è la cosa da dire: nel processo utente la frammentazione interna è il *prezzo accettato* della paginazione; nel kernel è il *problema da eliminare*.

### 14.9.1 Lo slab allocator

#### Slab allocator (Linux)

[SO-03 p40]

- uno **slab** è una **sequenza di pagine contigue**;
- una **cache** è **uno o più slab**;
- c'è **una cache per tipo di struttura dati interna omogenea**.

Lo **stato** di uno slab può essere **pieno, vuoto o parziale**, e la gestione è **dinamica**. I **vantaggi**: **niente spreco ed efficienza**.

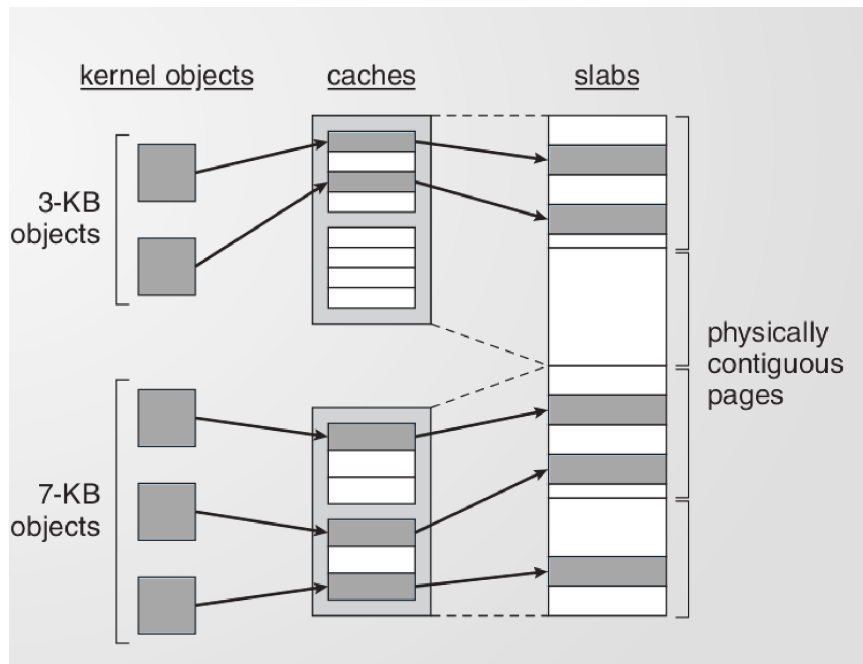


Figura 14.5: Lo slab allocator. Oggetti dello stesso tipo sono raggruppati in *cache*; ogni cache è fatta di *slab*, cioè di sequenze di pagine fisicamente contigue. Gli oggetti grandi (7 kB) hanno la loro cache, quelli piccoli (3 kB) la loro

Fonte: proiezioni del corso, SO-03 p40

#### Come fa lo slab a eliminare la frammentazione interna

È la domanda che chiude il capitolo, e la risposta sta nella parola *omogenea* dell'elenco delle slides. Il trucco è che **tutti gli oggetti di una cache hanno la stessa dimensione**, e quindi:

- lo slab si può dividere in un **numero intero esatto** di slot della taglia giusta. Non c'è nessun "resto" da sprecare: è il vantaggio che le slides chiamano **niente spreco**;
- allocare un oggetto è prendere il primo slot libero di uno slab *parziale*; se non ce ne sono, si prende uno slab vuoto; se non ce ne sono, se ne chiede uno nuovo. È una lista di slot liberi, **non una ricerca**: da qui l'**efficienza**;
- gli **stati** pieno/vuoto/parziale servono proprio a questo: si tiene una lista degli slab *parziali*, che sono quelli da riempire per primi. Uno slab vuoto è memoria pronta ma non sprecata, e può essere restituito al sistema.

Il confronto con l'allocazione dei processi è illuminante: là si **spreca** per non dover gestire la frammentazione esterna (capitolo 11); qui si **rinuncia alla generalità** — una cache serve un solo tipo di oggetto — per non sprecare affatto.

#### Come lo dici all'orale

“Mi parli degli altri aspetti della gestione della memoria.” Il capitolo è lungo e va **raggruppato in tre domande**, altrimenti ci si perde:

1. **Quanti frame?** Le strategie (equa, proporzionale con  $a_i = s_i/S \times m$ , per priorità), la distinzione fra allocazione **locale** e **globale**, e il fenomeno del **thrashing** con il sistema in **sovraccarico**: più processi non significa più CPU utilizzata.
2. **Quando togliere i frame?** I due modelli: il **working set** (finestra di  $\Delta$  accessi,

richiesta globale  $D = \sum WSS_i$  contro memoria disponibile, approssimato con interrupt periodici + bit R + log) e la **PFF** (monitoraggio diretto, con soglie). E la **politica di pulitura** con il *paging daemon*: pagare la pulizia in anticipo per non pagarla nel momento critico.

3. **Cosa si condivide e cosa si spreca?** La **dimensione della pagina** (grande: tabella piccola, I/O efficiente, meno fault; piccola: meno frammentazione interna, working set più preciso), le **pagine condivise** con le complicazioni di *cache* e di *tabella invertita*, **copy-on-write** e **zero-fill-on-demand**, e infine le **librerie condivise** e i **file mappati**.

Chiude, se il docente te lo chiede, lo **slab allocator**: la memoria del kernel è paginata con frammentazione interna e il kernel *non* può permettersela, quindi usa cache omogenee fatte di slab, con frammentazione interna nulla.

Una domanda che distingue: “perché il copy-on-write non si fa *a priori* e non si copia tutto subito?” La risposta è che la maggior parte delle copie non viene mai modificata — l’esempio della **fork** seguita da **exec** — e copiare subito sarebbe lavoro quasi sempre sprecato.

**Parte VI**

**File system e dischi**

## Capitolo 15

### I file system

---

Con questo capitolo comincia l'ultima parte del programma, e cambia la natura del problema. Finora si è sempre parlato di *memoria centrale*: veloce, volatile, e gestita per far convivere processi. Il problema che si affronta adesso è diverso su tutti e tre i fronti — ed è il primo in cui la **persistenza** entra in gioco.

#### 15.1 Il problema di base

##### Che cosa deve risolvere un file system

Il **problema di base** è: **gestire grandi quantità di informazioni**, in modo **persistente** e **condiviso tra più processi**. [SO-04 p02]

##### Le tre parole del problema, una per una

Vale la pena soffermarsi su ciascuna, perché ognuna è una difficoltà a sé:

- **grandi quantità**: la memoria centrale è limitata e costosa, il disco è grande ed economico. Ma il disco non è indirizzabile dalla CPU: si accede per *blocchi*, con operazioni lente. “Grandi quantità” significa anche “organizzazione gerarchica”, cioè strutture dati che stiano su disco e non si possano tenere tutte in memoria;
- **in modo persistente**: i dati devono sopravvivere all'esecuzione del processo e allo spegnimento della macchina. Questo vuol dire che il formato di ciò che si scrive è una **interfaccia pubblica**: una volta che i dati sono su disco in un certo formato, cambiarlo significa migrare;
- **condiviso tra più processi**: e quindi servono **protezione** (chi può leggere e chi può scrivere) e **sincronizzazione** (cosa succede se due processi scrivono insieme). Sono gli stessi due problemi della memoria (capitolo 11) e della mutua esclusione (capitolo 6), riproposti su un supporto che non si azzerà.

##### 15.1.1 L'astrazione: file e directory

##### File e directory

L'astrazione con cui si affronta il problema è fatta di due concetti: [SO-04 p02]

- il **file**;
- la **directory**.

I **dettagli di gestione e implementazione** di questa astrazione costituiscono il **file system**.

Le slides elencano gli “esempi di dettagli” che il file system deve definire. È un elenco da ricordare, perché è la traccia con cui si struttura tutta la risposta all'orale. [SO-04 p02]

| Dettaglio                     | Che cosa deve decidere                                                                                        |
|-------------------------------|---------------------------------------------------------------------------------------------------------------|
| <b>nomenclatura</b>           | come si chiamano i file: quali caratteri, quale lunghezza massima, se distinguere maiuscole, quale estensione |
| <b>tipi di file</b>           | se il sistema distingue i tipi (eseguibile, testo, binario...) o li tratta tutti allo stesso modo             |
| <b>tipi di accesso</b>        | sequenziale, diretto, indicizzato                                                                             |
| <b>metadati (o attributi)</b> | proprietario, dimensione, date di creazione e modifica, permessi                                              |
| <b>operazioni</b>             | quali operazioni sono supportate sul file: creazione, apertura, lettura, scrittura, chiusura, cancellazione   |
| <b>strutture dati</b>         | come si tiene traccia dei file aperti: una struttura <b>globale</b> e una <b>per processo</b>                 |

#### L'ultimo dettaglio: globale e per processo

È il meno ovvio e il più interessante, e vale la pena spiegarlo bene perché collega il file system al capitolo sui processi (capitolo 4). Quando un processo apre un file, il sistema deve ricordare *due cose diverse*:

- **per processo**: quali file *questo* processo ha aperto, con quale modalità, e soprattutto **a che punto della lettura è arrivato**. La *posizione corrente* è informazione del processo, non del file: due processi che leggono lo stesso file hanno posizioni diverse;
- **globale**: quali file sono aperti *nel sistema*, con quanti riferimenti, la loro posizione su disco, i blocchi già letti in cache. È informazione condivisa: se due processi aprono lo stesso file, la parte globale è *la stessa*.

È la stessa separazione, in piccolo, fra la tabella dei processi e la tabella dei file aperti che si trova in ogni sistema reale. E spiega perché **fork** non duplica le posizioni di lettura nello stesso modo in cui duplica la memoria: le strutture per processo vengono copiate, quelle globali no.

#### 15.1.2 I lock sui file

Il file system deve anche definire le regole dell'**accesso condiviso ai file**: i **lock**. Le due coppie di alternative sono [SO-04 p02]

| Alternativa      | Significato                                                                                                         |
|------------------|---------------------------------------------------------------------------------------------------------------------|
| <b>shared</b>    | il lock è <i>condiviso</i> : più processi possono tenerlo contemporaneamente (in lettura)                           |
| <b>exclusive</b> | il lock è <i>esclusivo</i> : un solo processo può tenerlo, e nessun altro (in scrittura)                            |
| <b>mandatory</b> | è il <i>sistema</i> a far rispettare il lock: un processo che prova ad accedere senza il lock viene <b>bloccato</b> |
| <b>advisory</b>  | il lock è solo un' <i>indicazione</i> : il sistema lo offre, ma sta al processo decidere se rispettarlo             |

#### Non sono due coppie dello stesso tipo

È un errore facile da fare, e distinguerle è un buon modo per mostrare di aver capito:

- **shared/exclusive** riguarda *come* il lock si comporta: quanti possono tenerlo e in che

modo. È la stessa distinzione fra lettori e scrittori del capitolo 7;

- **mandatory/advisory** riguarda *chi lo fa rispettare*: il sistema operativo o il programma. È una scelta di *politica*, e ha conseguenze profonde.

L'**advisory** è ciò che si trova di solito nei sistemi Unix, e la ragione è pragmatica: far rispettare un lock obbligatorio costa, perché il sistema deve controllare *ogni* accesso, e rompe i programmi che non sono stati scritti prevedendolo. La conseguenza però è che l'**advisory non protegge da un processo che se ne frega**: è una convenzione, non una difesa. Va detto, perché è la differenza che conta in pratica.

## 15.2 La struttura di un file system

### Come è organizzato un disco

[SO-04 p03]

- **Master Boot Record (MBR)**: il primo settore del disco, contiene la tabella delle partizioni;
- **partizioni e boot record** (o *boot block*): il disco è diviso in partizioni, e ciascuna può contenere un settore di avvio;
- **superblocco**: descrive il file system della partizione;
- **layout moderno alternativo: GPT** (GUID Partition Table), definito dallo standard **EFI**.

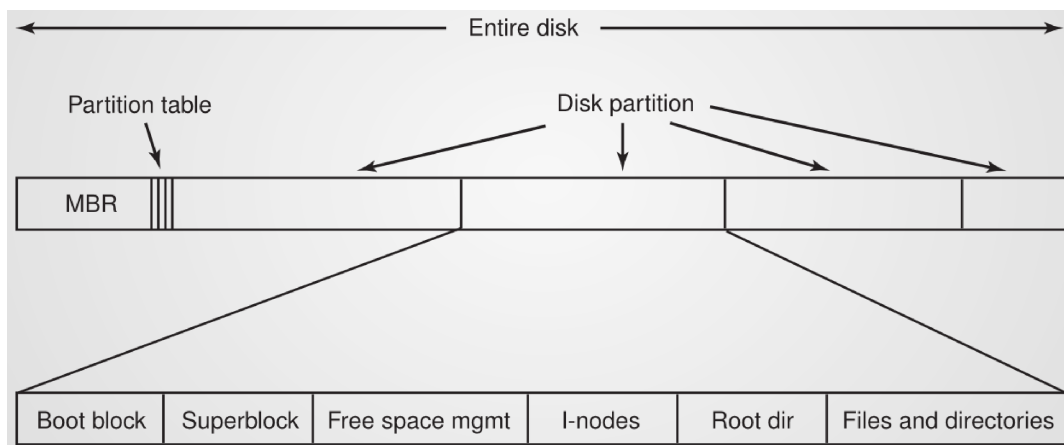


Figura 15.1: La struttura di un disco. In alto l'intero disco, con la *partition table* nel MBR e le partizioni; in basso l'ingrandimento di una partizione, con il *boot block*, il *superblock*, l'area di gestione dello spazio libero, gli *i-node* e infine la directory radice con i file e le directory

Fonte: proiezioni del corso, SO-04 p03

### Da leggere nella figura: l'ordine dei blocchi

La figura è più informativa di quanto sembri, se la si legge nell'ordine giusto. I blocchi di una partizione, dall'inizio:

1. il **boot block**: il codice per avviare il sistema operativo. Sta all'inizio perché l'hardware cerca il codice di avvio in una posizione *fissa*, e non può chiedere al file system dove sia;



2. il **superblock**: contiene i *parametri* del file system — quanti blocchi, quanto è grande un blocco, dove comincia l'area degli i-node, dove comincia quella dei dati. È il “punto di ingresso” di tutta la struttura: senza di esso, il resto è byte senza significato. È anche il blocco di cui si tiene spesso una **copia di riserva**, proprio perché perderlo significa perdere l'intero file system;
3. la **gestione dello spazio libero**: la bitmap o la lista dei blocchi liberi di cui si parla nel capitolo 16;
4. gli **i-node**: i metadati di tutti i file;
5. la **directory radice** e i dati veri e propri.

La cosa da notare è che **l'ordine non è casuale**: i metadati stanno tutti insieme, e le prestazioni di un file system dipendono in larga misura da quanto spesso si deve passare dall'area dei metadati a quella dei dati. È il problema che sta dietro alle scelte di progettazione del capitolo 16.

### MBR e GPT

La slides presentano GPT come **layout moderno alternativo**. Il motivo del cambiamento è un limite strutturale del MBR: i campi che contengono gli indirizzi dei settori sono di dimensione fissa e piccola, e il numero di partizioni è fissato a quattro (di cui una “estesa” usata come contenitore). GPT nasce dallo standard EFI per superare entrambi i limiti.

### Come lo dici all'orale

“Mi parli dei file system.” La traccia in tre passi:

1. il **problema**: grandi quantità, persistenza, condivisione. Le tre parole non sono decorative: ognuna porta con sé una difficoltà (l'accesso per blocchi, il formato come interfaccia pubblica, la protezione e la sincronizzazione);
2. l'**astrazione**: file e directory. E i *dettagli* che essa nasconde: nomenclatura, tipi, accesso, metadati, operazioni, strutture dati **globale e per processo**, e i **lock** con le due coppie shared/exclusive e mandatory/advisory;
3. la **struttura**: MBR, partizioni e boot record, superblocco, e il layout moderno GPT. Se hai la figura in testa, descrivi l'ordine dei blocchi nella partizione — boot block, superblocco, spazio libero, i-node, radice — e di' perché i metadati stanno insieme.

Il dettaglio che fa la differenza: spiegare la differenza fra la struttura dati **globale** e quella **per processo**, con l'esempio della posizione di lettura che appartiene al processo e non al file.

## Capitolo 16

### Progettazione di un file system

Il capitolo precedente ha presentato l'astrazione. Questo capitolo scende al livello delle strutture dati: **come si tiene traccia di quali blocchi formano un file**, **come si organizzano le directory**, **come si condivide un file** e **come si sa quali blocchi sono liberi**. Sono quattro decisioni di progetto indipendenti, e ognuna ha varianti con compromessi diversi.

#### 16.1 L'allocazione dei file

La prima domanda è: dato un file, come si registra *dove sono i suoi dati*? Le slides presentano quattro metodi, che sono anche una progressione storica. [SO-04 p04-p05]

##### 16.1.1 Allocazione contigua

[SO-04 p04]

Il file occupa una **sequenza di blocchi consecutivi** sul disco. Del file basta registrare **due numeri**: il blocco iniziale e la lunghezza.

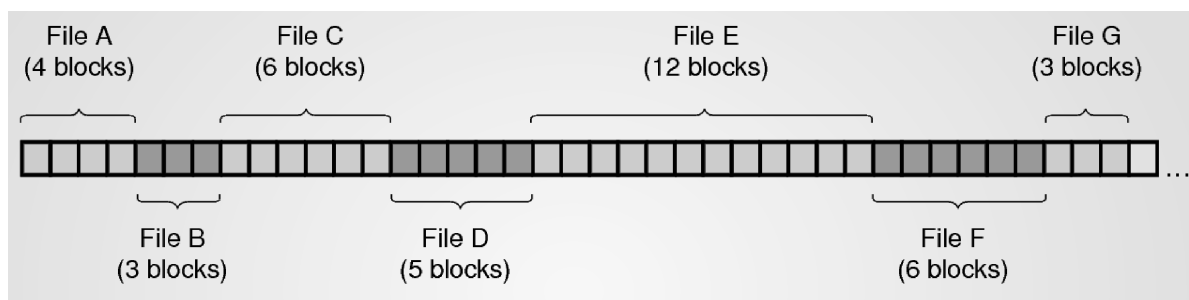


Figura 16.1: Allocazione contigua: ogni file occupa una sequenza di blocchi consecutivi, e di ciascuno bastano il blocco iniziale e la dimensione (“File A (4 blocchi)”, “File C (6 blocchi)”...). Si vede anche il difetto: i file si incastrano solo se hanno le dimensioni giuste al momento giusto

Fonte: proiezioni del corso, SO-04 p04

#### I due vantaggi e i due difetti

È il metodo da confrontare con tutti gli altri, quindi conviene avere chiaro *perché* è così attraente e *perché* non si usa quasi mai.

##### Vantaggi:

- **lettura semplicissima**: l' $n$ -esimo blocco del file sta alla posizione  $inizio + n$ . Il calcolo è un'addizione, e questo permette l'**accesso diretto** senza alcuna struttura intermedia;
- **prestazioni ottime**: i blocchi sono adiacenti, quindi la testina del disco si sposta pochissimo. È il caso migliore possibile per il tempo di seek di cui si parla nel capitolo 18.

##### Difetti:

- **frammentazione esterna**: è esattamente il problema del capitolo 11, riproposto sul disco. Nella figura si vede bene: i file occupano dimensioni diverse, e lo spazio che resta fra l'uno e l'altro non è riutilizzabile se non da un file della misura giusta;
- **il file non può crescere**: la sua dimensione è fissata alla creazione. Se il blocco successivo è già occupato, per estendere il file bisogna **spostarlo tutto**, e per un file

grande questo è un costo enorme. La figura mostra la conseguenza: i file sono disposti *calcolando* le dimensioni future, cioè sapendole in anticipo — cosa che in generale non si sa.

### 16.1.2 Allocazione concatenata

[SO-04 p04]

I blocchi del file sono **sparsi** sul disco, e ogni blocco contiene un **puntatore al successivo**. La lista dei blocchi liberi è quindi *dentro i blocchi stessi*.

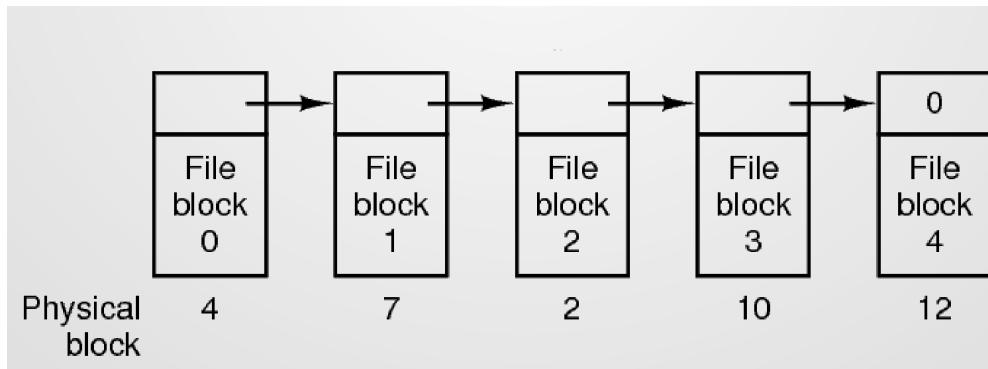


Figura 16.2: Allocazione concatenata: i blocchi logici del file (*File block* 0...4) si trovano nei blocchi fisici 4, 7, 2, 10 e 12, collegati da una catena di puntatori; lo 0 finale segnala la fine del file

Fonte: proiezioni del corso, SO-04 p04

#### Il difetto nascosto: la lettura è sequenziale per costrizione

Il vantaggio è che **la frammentazione esterna sparisce**: qualsiasi blocco libero va bene, e il file può crescere senza vincoli. Ma il prezzo è che **l'accesso diretto diventa impossibile**, e non per una scelta di progetto ma per una **conseguenza strutturale**:

per leggere il blocco  $n$  bisogna aver letto tutti i blocchi  $0, 1, \dots, n - 1$ .

L'unico modo di arrivare al blocco 100 è percorrere la catena, e ogni passo è un accesso al disco che potrebbe essere ovunque. Non c'è modo di "calcolare" la posizione. È il motivo per cui l'allocazione concatenata pura non si usa per i file, ma sopravvive in una forma migliorata — quella del paragrafo successivo.

Da notare anche un secondo difetto, meno visibile: ogni blocco **spreca i byte del puntatore**, e quei byte non sono più disponibili per i dati. Con blocchi da 512 byte e puntatori da 4, si perde quasi l'1% della capacità.

### 16.1.3 Allocazione tabellare: la FAT

#### File Allocation Table (FAT)

È l'allocazione **con liste collegate su una tabella di allocazione dei file**. L'idea è semplice e risolve il problema principale della concatenata: i puntatori **non stanno più nei blocchi dati** ma **in una tabella a parte**, che ha una voce per ogni blocco del disco.

[SO-04 p05]

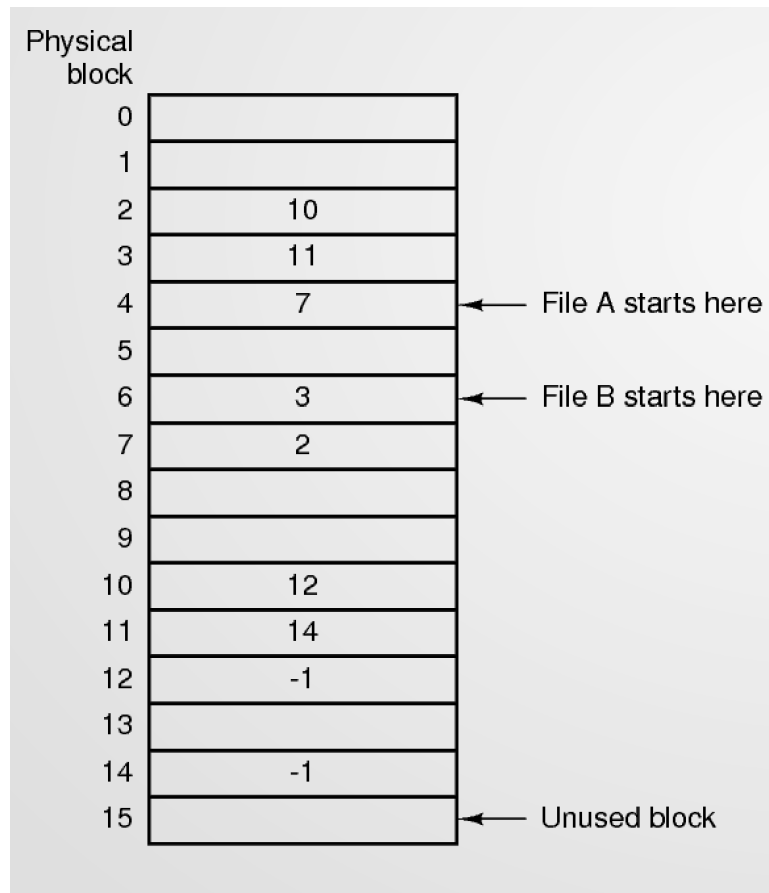


Figura 16.3: La FAT: la colonna *Physical block* è l'indice (il numero del blocco), e il valore contenuto è il *prossimo* blocco della catena. -1 segnala la fine del file; le voci vuote sono i blocchi liberi. “File A” comincia dal blocco 4 e “File B” dal blocco 6

Fonte: proiezioni del corso, SO-04 p05

#### Come si legge la figura, e perché è un miglioramento

Nella figura la voce 4 contiene 7, la 7 contiene 2, la 2 contiene 10, la 10 contiene 12 e la 12 contiene -1: ripercorrendo gli indici si ricostruisce la catena (4, 7, 2, 10, 12), che è esattamente la stessa catena della figura dell'allocazione concatenata. La differenza è **dove stanno i puntatori**, e da quella differenza discendono due conseguenze importanti:

- i blocchi dati contengono **solo dati**: non si spreca più byte per i puntatori;
- soprattutto, **la tabella si può tenere in memoria**. È questo il vero guadagno: se l'intera FAT è in RAM, seguire la catena non costa *nessun* accesso al disco. Si scarica la tabella una volta e poi si percorre a costo zero.

Il prezzo si paga in **spazio**: la tabella ha una voce per *ogni* blocco del disco, anche per quelli che non appartengono a nessun file. È il difetto che rende la FAT poco adatta ai dischi grandi — e infatti storicamente è rimasta legata ai supporti rimovibili, dove il disco è piccolo.

#### Un esempio da fare a voce: perché la FAT ha problemi in caso di append

È il punto che le slides segnalano esplicitamente parlando di condivisione: *usando una FAT: duplicare la lista con i riferimenti ai blocchi; problemi in caso di append*. [SO-04 p07]

Il caso è questo: due utenti vogliono condividere un file. Con la FAT non esiste un “file” come oggetto condiviso — esiste una **catena di voci**. Per far vedere lo stesso file a due utenti si

**duplica la catena:** due liste di riferimenti che, all'inizio, puntano agli stessi blocchi. A questo punto però **aggiungere dati al file** (append), un utente allunga la *sua* catena: gli ultimi blocchi sono ora diversi. Le due catene divergono, e i due utenti credono di avere lo stesso file ma ne hanno due. È un difetto strutturale, non un caso limite: la FAT non ha un modo di rappresentare “due nomi, un file”. La soluzione — il *contatore dei link* — richiede una struttura che la FAT non ha, ed è quella che introduce l'allocazione indicizzata.

#### 16.1.4 Allocazione indicizzata: gli i-node

##### Index node (i-node)

Si introduce una struttura per file, il **nodo indice** o **i-node**, che contiene **gli attributi** del file e i **puntatori ai suoi blocchi dati**. Le varianti elencate dalle slides sono tre: **collegata**, **multilivello**, **ibrida**. [SO-04 p05]

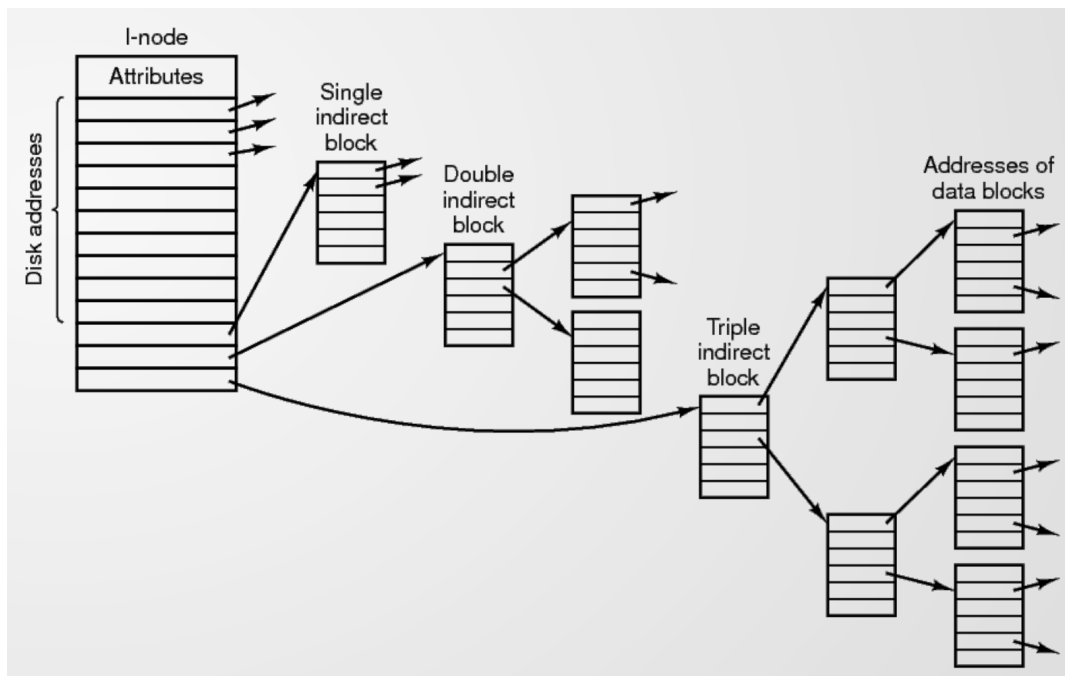


Figura 16.4: Un i-node: la voce contiene gli *Attributes* e una serie di indirizzi di blocco. Oltre agli indirizzi diretti, l'i-node punta a blocchi di indirizzi *single*, *double* e *triple indirect block*, che a loro volta puntano agli indirizzi dei blocchi dati

Fonte: proiezioni del corso, SO-04 p05

La figura si legge come un albero, e la variante “ibrida” è ciò che si vede

La struttura dei tre livelli di indirezione è un **albero**, e conviene descriverla per livelli:

- gli indirizzi **diretti** nell'i-node puntano *direttamente* ai blocchi di dati;
- il **single indirect block** è un blocco pieno di indirizzi che puntano a blocchi di dati: un livello in più;
- il **double indirect block** contiene indirizzi di blocchi che a loro volta contengono indirizzi di blocchi di dati: due livelli;
- il **triple** ne aggiunge un terzo.

È la variante che le slides chiamano **ibrida**: non si sceglie *un* metodo, si mettono insieme indirizzi *diretti* (per i file piccoli, che sono la maggioranza, e che così si leggono con un solo accesso) e indirezione a più livelli (per i file grandi). È una combinazione che sfrutta il fatto che la dimensione dei file è *fortemente sbilanciata*: moltissimi file piccoli, pochissimi file enormi.

Il conteggio vale la pena farlo a voce, perché mostra quanto si guadagni dall'indirezione multipla. Se un blocco contiene  $k$  indirizzi: i diretti coprono qualche blocco, il *single* ne copre  $k$ , il *double*  $k^2$ , il *triple*  $k^3$ . Con  $k = 256$  e blocchi da 1 kB, il solo triple copre  $256^3 \times 1 \text{ kB} = 16 \text{ GB}$ : è il meccanismo che permette a un file system con i-node di dimensione fissa di gestire file di dimensione arbitraria.

### Perché i-node e non FAT

Il confronto diretto fra i due metodi spiega come mai i sistemi Unix abbiano scelto gli i-node:

- nella **FAT** la tabella ha una voce per ogni blocco del *disco*: la struttura cresce con la dimensione del disco, e va tenuta tutta in memoria (o si perde il vantaggio);
- nell'**i-node** c'è una struttura per ogni *file*: la struttura cresce con il numero di file, e soprattutto **i blocchi di indirezione si caricano solo per i file grandi**. Per un file piccolo si legge un i-node e i suoi blocchi diretti, e basta.

È l'esempio di come un cambio di struttura cambi la scalabilità: la domanda non è “quanto costa in media” ma “come cresce al crescere del disco”.

## 16.2 L'implementazione delle directory

### Le tre domande delle directory

[SO-04 p06]

- **Dove memorizzare i metadati/attributi?** Dentro la voce di directory, oppure in una struttura separata a cui la voce punta;
- **Nomi lunghi**: come gestirli? Con **lunghezza variabile**, oppure **tramite heap**;
- **Prestazioni**: come rendere veloce la ricerca di un nome? Con una **tabella hash** e con una **cache**.

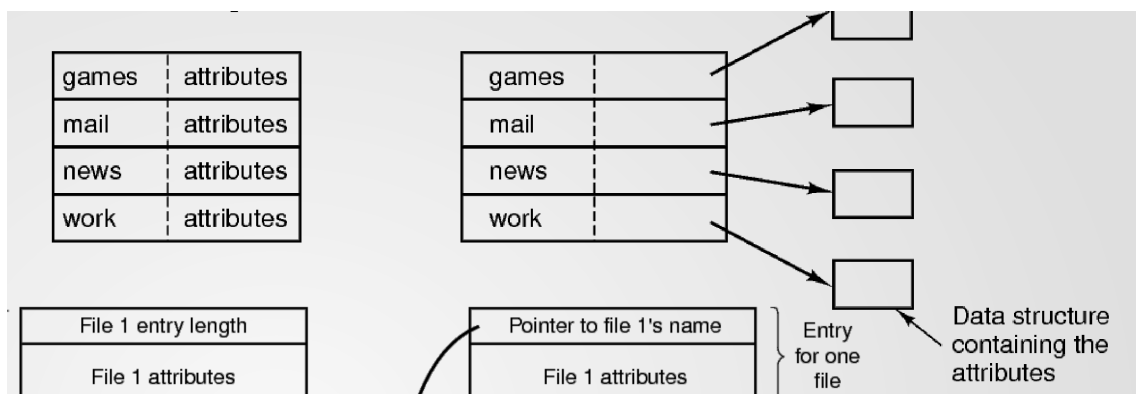


Figura 16.5: Dove stanno gli attributi. A sinistra la voce di directory contiene direttamente gli attributi; a destra la voce contiene un *puntatore* alla struttura che li contiene. Nel primo caso la directory è autosufficiente ma grande, nel secondo è piccola e gli attributi si caricano solo quando servono

Fonte: proiezioni del corso, SO-04 p06

### Perché la seconda soluzione è quella dei sistemi reali

Con la struttura separata si ottiene un effetto che vale la pena esplicitare: **la ricerca di un nome costa meno**. La ricerca in una directory consiste nel confrontare i nomi, e per farlo bisogna *leggere* le voci. Se ogni voce contiene gli attributi, si legge molto di più del necessario ogni volta; se contiene solo il nome e un puntatore, la directory sta in pochi blocchi e si scorre in fretta. Gli attributi si leggono *dopo*, una volta sola, per l'unico file che interessa. È lo stesso ragionamento che sta dietro all'organizzazione dei metadati nella partizione (capitolo 15): **tenere insieme ciò che si legge insieme**. Ed è anche la ragione per cui in Unix gli attributi vivono nell'i-node, che è esattamente “la struttura separata” della figura.

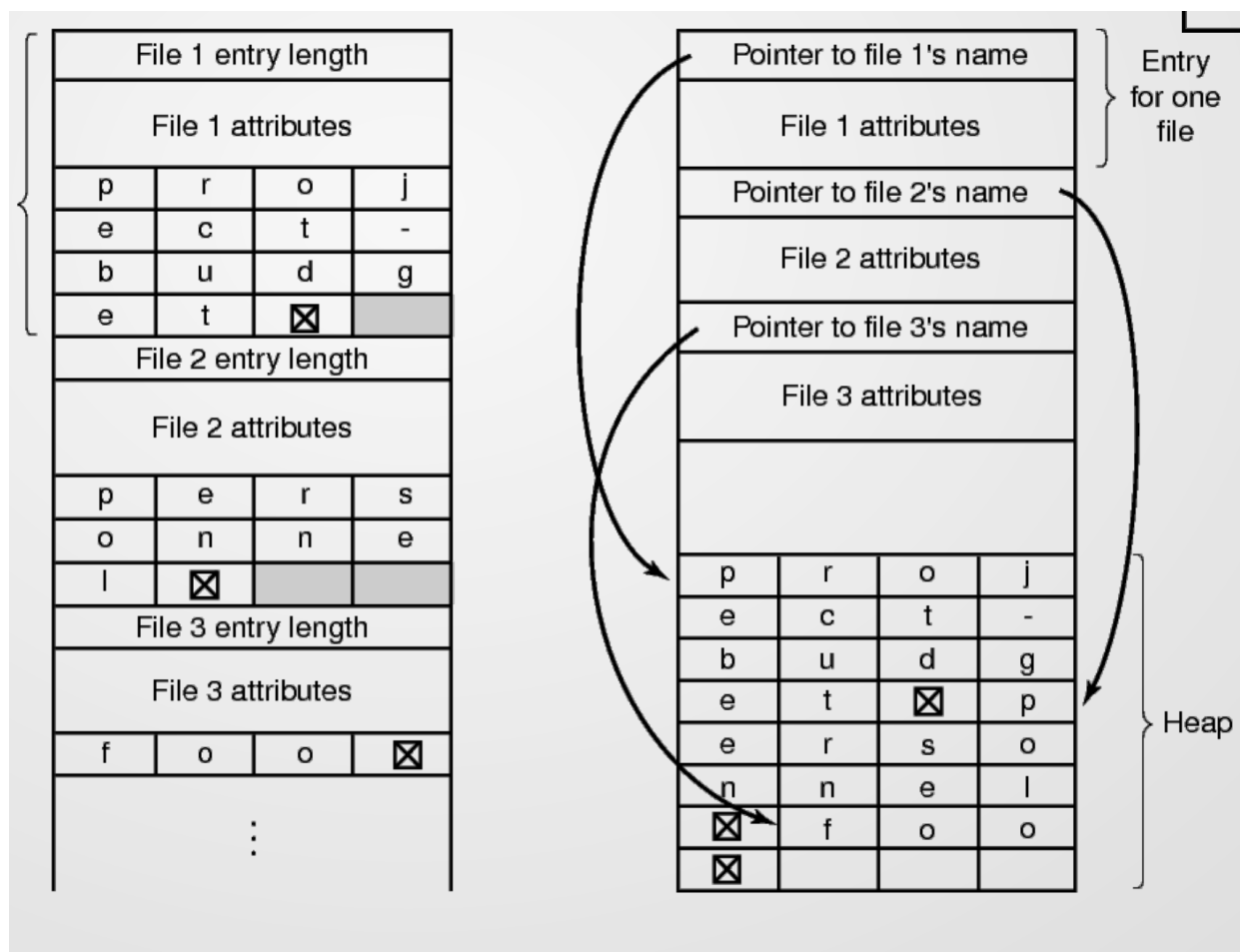


Figura 16.6: Due formati di voce per i nomi lunghi. A sinistra le voci hanno *lunghezza variabile* e contengono il nome per intero; a destra le voci hanno lunghezza fissa con un *puntatore al nome*, e i nomi sono accumulati in fondo in uno *heap*

Fonte: proiezioni del corso, SO-04 p06

### Il compromesso sui nomi lunghi

Le due soluzioni si oppongono esattamente come nel caso degli attributi:

- **lunghezza variabile:** non si spreca spazio, ma ogni voce ha una dimensione diversa. La conseguenza è che per passare alla voce successiva bisogna **leggere la voce corrente** — e cancellare un file lascia un buco di dimensione arbitraria. Nella figura si vede il prezzo pagato: ogni voce porta con sé il campo *File n entry length*;
- **heap:** le voci hanno **tutte la stessa dimensione**, quindi l'*n*-esima voce si trova con una moltiplicazione e i buchi si riusano banalmente (una voce cancellata è una voce con un puntatore nullo). I nomi stanno in fondo, e i puntatori devono **essere riordinati** quando lo heap si compatta — sono gli archi ricurvi che si vedono nella figura.

Il secondo schema è quello che vince nei sistemi moderni, ed è lo stesso motivo per cui si preferiscono le strutture a *dimensione fissa* quando si deve indicizzare: pagare un po' di spazio per avere un accesso calcolabile invece che sequenziale.

Alle due domande strutturali se ne aggiunge una sulle **prestazioni**: la **tabella hash**, per trovare un nome senza scorrere la directory, e la **cache**, per non rileggere dal disco le voci già consultate. [SO-04 p06]



### Perché servono entrambe

Non sono alternative, risolvono due parti diverse del problema:

- la **tabella hash** riduce il *numero di confronti* necessari per trovare un nome: da  $O(n)$  a  $O(1)$ . Ma il confronto va comunque fatto su un dato letto *da qualche parte*;
- la **cache** evita di rileggere dal disco le voci già lette. Anche con una ricerca in tempo costante, ogni ricerca che finisce su disco costa un accesso.

Si applicano così alla directory gli stessi due strumenti già visti per la traduzione degli indirizzi (capitolo 12): una struttura che riduce il *lavoro* e una memoria veloce che riduce le *attese*.

## 16.3 La condivisione di file

È il secondo problema in cui la scelta della struttura dati dell'allocazione ha conseguenze dirette.  
[SO-04 p07]

### Lo scenario e i tre approcci

**Scenario:** due o più utenti vogliono condividere un file. Gli approcci elencati dalle slides:

- **con una FAT:** **duplicare la lista** con i riferimenti ai blocchi — con i **problemi in caso di append** già visti;
- **con gli i-node:** **hard-link**, con il **contatore dei link** — e l'**anomalia con accounting**;
- **soft-link:** è **universale** e permette di fare riferimenti **anche al di fuori del file system**, ma comporta un **appesantimento nella gestione**.

Le slides segnalano infine che entrambi gli approcci presentano **problemi in fase di attraversamenti e backup**.

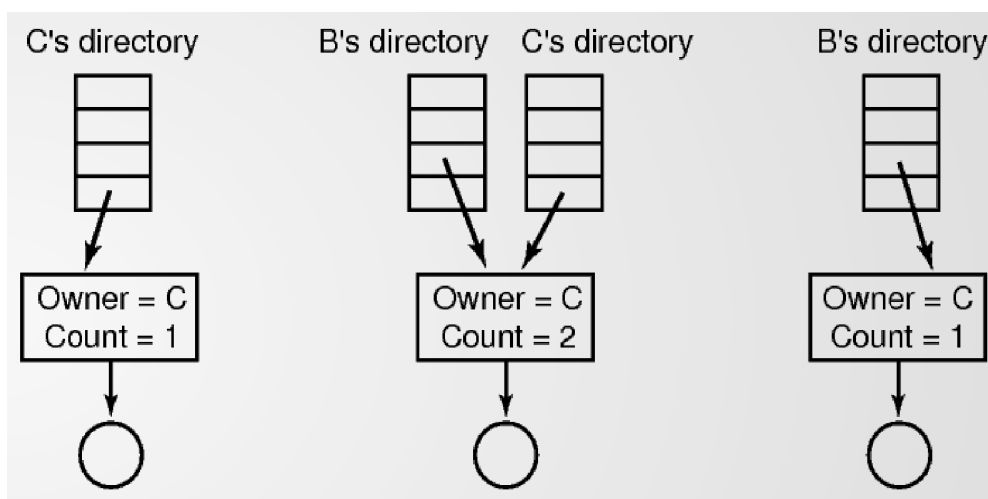


Figura 16.7: Hard link e link simbolico. In alto i due casi in cui due directory puntano allo *stesso* i-node (Count = 2): è l'hard link. In basso il caso del link simbolico: la voce della directory punta a una struttura che contiene *il nome* del file, non il file

Fonte: proiezioni del corso, SO-04 p07

### Hard link e soft-link: la differenza in una frase

La figura mostra la differenza in modo che si legge a colpo d'occhio:

- con l'**hard link** due nomi puntano allo **stesso i-node**, e il campo **Owner/Count** tiene il conto dei riferimenti. Non c'è nessuna "copia": c'è un oggetto con due nomi. Cancellare un nome decrementa il contatore; l'oggetto sparisce quando il contatore arriva a zero;
- con il **soft-link** (*link simbolico*) il secondo nome è un file a sé, che contiene una **stringa**: il percorso del file di destinazione. Il sistema, quando lo incontra, deve *ripercorrere* quel percorso da capo.

Da questa differenza discendono tutte le altre:

- il soft-link può puntare a un file di **un altro file system**, e perfino a un file che **non esiste** (un *dangling link*); l'hard link no, perché un i-node appartiene a un file system solo. È ciò che le slides chiamano **universalità**;
- il soft-link è più *caro* da attraversare, perché richiede di risolvere un percorso in più: è l'**appesantimento nella gestione** segnalato dalle slides.

### L'anomalia con accounting

È il punto più sottile del paragrafo, ed è quello che distingue una risposta completa. Il **contatore dei link** risolve la domanda "quando si può cancellare davvero il file?", ma non risolve la domanda "*a chi* appartiene lo spazio?".

Il caso: l'utente A crea un file (l'i-node registra A come proprietario) e l'utente B ne fa un hard link. Ora lo spazio occupato è **uno**, ma ci sono due utenti. Se si attribuisse lo spazio a entrambi, si conterebbe due volte e la somma degli spazi degli utenti supererebbe la capacità del disco — una cosa che i sistemi che fanno *accounting* (le quote disco, per esempio) non possono permettersi. Se lo si attribuisse solo ad A, allora B potrebbe riempire il disco attraverso il link senza che nulla venga addebitato a lui.

È un caso in cui una struttura dati *corretta* per un problema (cancellare) è *inadatta* per un altro (contabilizzare): la stessa informazione non basta a rispondere a entrambe le domande. Ed è anche il motivo per cui le slides chiudono parlando di **problemi in fase di attraversamenti e backup**: un programma che percorre la gerarchia delle directory e incontra un hard link deve decidere se è un file nuovo o già visto — e con i soft-link deve decidere se seguire il collegamento o registrarlo come tale.

## 16.4 La gestione dei blocchi liberi

### I due metodi

[SO-04 p08–p09]

- **bitmap**: un bit per blocco, 0 se libero e 1 se occupato. Le slides la descrivono come **relativamente piccola**;
- **liste concatenate**: richiede più spazio, ma si sfruttano i **blocchi stessi liberi**; c'è la possibilità di inserire **contatori per blocchi contigui**.

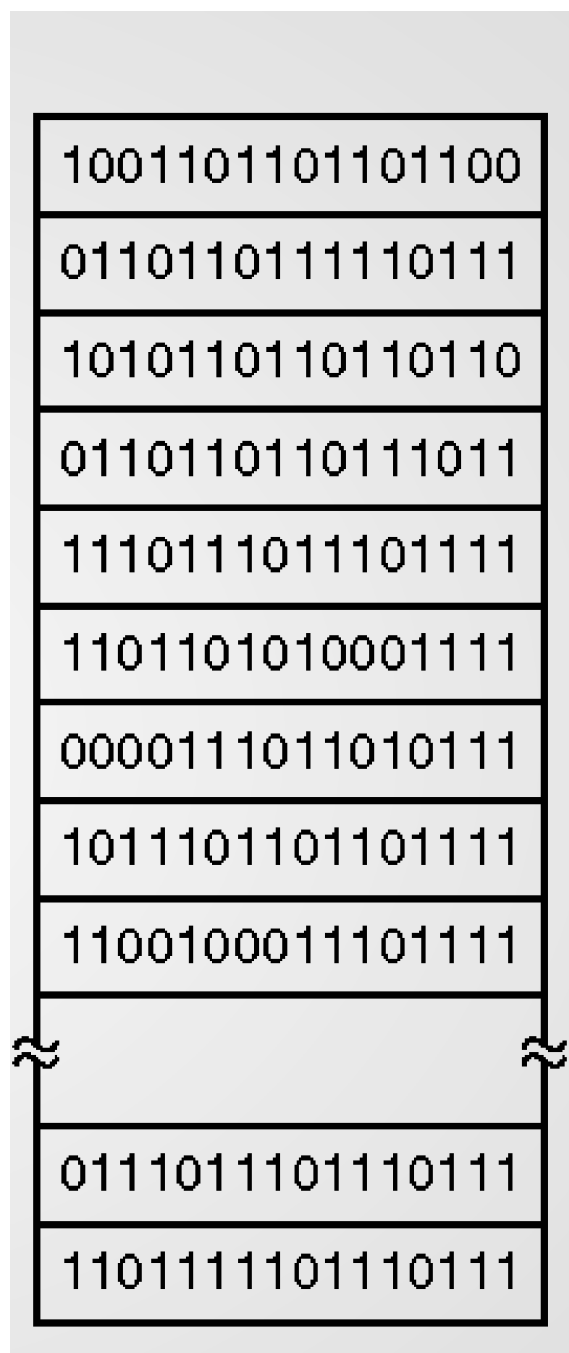


Figura 16.8: La bitmap dei blocchi liberi: una sequenza di bit, uno per blocco del disco. Il valore di ogni bit dice se il blocco corrispondente è libero o occupato

Fonte: proiezioni del corso, SO-04 p08

#### Quanto è grande una bitmap: il conto da saper fare

Le slides dicono che è “relativamente piccola”, e vale la pena quantificarlo, perché è un numero che si può chiedere. Un bit per blocco significa:

un disco da  $N$  blocchi richiede  $N$  bit =  $N/8$  byte

Con blocchi da 1 kB e un disco da 1 TB, i blocchi sono  $10^9$  e la bitmap è  $10^9/8 = 125$  MB. È molto o poco? Dipende dal confronto: è lo **0,012%** del disco, cioè **un bit per ogni 8192 bit di dati**. In proporzione è trascurabile, in assoluto è abbastanza grossa da porre un problema di gestione.

Da qui la domanda successiva delle slides: **tutta in memoria, o un blocco alla volta?** È la stessa alternativa di ogni struttura dati grande: tenerla tutta in RAM (veloce ma occupa memoria, e 125 MB non sono pochi) oppure leggere solo la parte che serve (economico ma ogni ricerca può costare un accesso). E la terza possibilità — **paginata con la memoria virtuale** — è la sintesi: la bitmap è una struttura dati come le altre, e si lascia gestire alla paginazione (capitolo 12), che la terrà in memoria solo per le parti che si stanno usando.

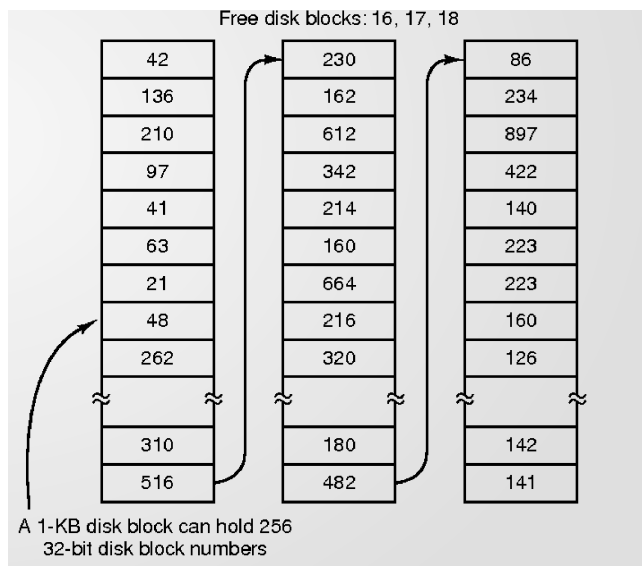


Figura 16.9: La lista dei blocchi liberi: i numeri dei blocchi liberi sono incatenati, e le catene passano *dentro i blocchi liberi stessi*. Le tre colonne sono tre blocchi: la catena finisce in uno di essi e ricomincia nel successivo

Fonte: proiezioni del corso, SO-04 p09

### Lista contro bitmap: due compromessi opposti

È il confronto che le slides impostano in due righe, e conviene scioglierlo. La frase chiave è *richiede più spazio, ma si sfruttano i blocchi stessi liberi*: sono due affermazioni che sembrano contraddirsi e invece dicono cose diverse.

- **richiede più spazio**: una lista di numeri di blocco occupa *molto* più di una bitmap. Con numeri da 32 bit e blocchi da 1 kB, un numero per blocco costa 4 byte contro 1 bit: **32 volte tanto**. E la figura lo mostra esplicitamente — “a 1-KB disk block can hold 256 32-bit disk block numbers”;
- **si sfruttano i blocchi stessi liberi**: quel costo però non è memoria *sprecata*, perché la lista vive nei blocchi che sono liberi comunque. Un blocco libero non serve a nessuno: se ci si scrive dentro la lista, non si sta togliendo spazio a nessuno. È un uso che non ha costo opportunità.

Da qui la differenza pratica: la lista è conveniente nei sistemi con **pochi blocchi liberi** (dove la lista è corta e la bitmap sarebbe comunque grande), la bitmap nei sistemi con **molti blocchi liberi**.

Infine, i **contatori per blocchi contigui**: un blocco libero raramente è solo, di solito fa parte di una sequenza di blocchi liberi consecutivi. Registrare “da qui, per 40 blocchi” invece di 40 numeri separati accorcia la lista in modo enorme — ed è anche informazione utile, perché sapere che esistono 40 blocchi contigui serve a chi cerca spazio per un file (ed è il collegamento al *first fit* e agli algoritmi di allocazione del capitolo 11).

**Come lo dici all'orale**

“Come si progetta un file system?” Il capitolo risponde a **quattro domande**, e conviene andarci in quest'ordine, dicendo per ognuna la progressione e il perché:

1. **Dove stanno i blocchi di un file?** **Contigua** (due numeri, accesso diretto, ma frammentazione esterna e file che non cresce) → **concatenata** (cresce sempre, ma lettura sequenziale per costrizione) → **FAT** (i puntatori in una tabella in memoria, ma la tabella ha una voce per *blocco del disco*) → **i-node** (una struttura per *file*, con indirizzione *ibrida*: diretta, singola, doppia, tripla);
2. **Come si organizzano le directory?** Dove stanno gli attributi (dentro la voce o in una struttura puntata), come si gestiscono i nomi lunghi (lunghezza variabile o heap), e come si rende veloce la ricerca (hash e cache);
3. **Come si condivide un file?** **Hard link** (stesso i-node, contatore dei link, l'anomalia con accounting) contro **soft-link** (universale, funziona fuori dal file system, ma costa di più da attraversare). E i problemi di attraversamento e backup;
4. **Quali blocchi sono liberi?** **Bitmap** (un bit per blocco, piccola e semplice) contro **lista concatenata** (32 volte più spazio, ma vive nei blocchi liberi stessi, e con i contatori per blocchi contigui diventa corta).

Se ti chiedono “quale useresti?”, la risposta che mostra comprensione è: *dipende da cosa si sta ottimizzando* — spazio degli i-node contro dimensione del disco per la FAT, velocità di ricerca contro spazio per i nomi, spazio contro flessibilità per i link.

## Capitolo 17

### Consistenza, journaling e cache

Il capitolo precedente ha mostrato come un file system è fatto. Questo affronta le due domande che restano: **cosa succede se si interrompe a metà** e **come si fa ad andare veloce** su un dispositivo lento. Sono i due problemi che dominano la progettazione dei file system reali, e le slides li trattano nell'ordine in cui si presentano: prima la robustezza, poi le prestazioni.

#### 17.1 I controlli di consistenza

##### Consistenza

A seguito di un crash del sistema i file system possono diventare inconsistenti.

Apposite utility possono effettuare dei controlli di consistenza: [SO-04 p10]

- sui blocchi;
- sui riferimenti agli i-node.

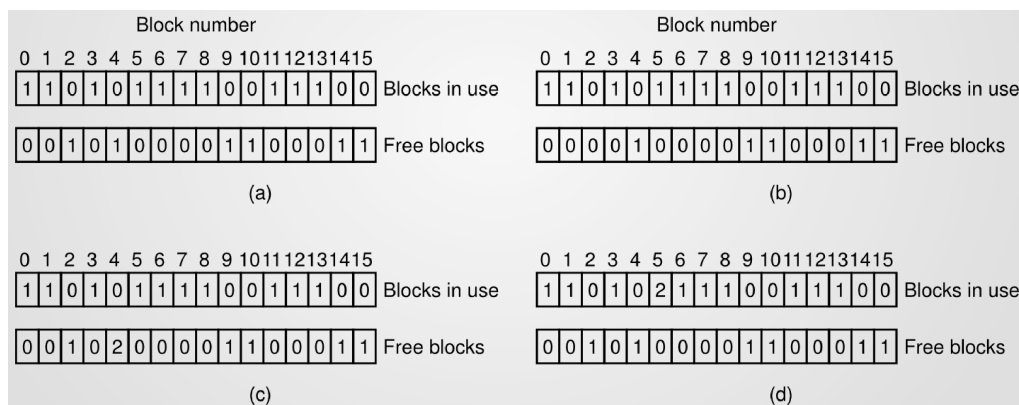


Figura 17.1: Il controllo di consistenza sui blocchi. Ogni coppia di righe confronta ciò che dice la *bitmap dei blocchi occupati* con ciò che dice la *bitmap dei blocchi liberi*. In (a) le due si completano; in (b) il blocco 2 non appartiene a nessuno; in (c) il blocco 4 è contato due volte fra i liberi; in (d) il blocco 5 è contato due volte fra gli occupati

Fonte: proiezioni del corso, SO-04 p10

##### Che cosa si controlla, e come

È utile saper *leggere* i quattro casi della figura, perché mostrano che “inconsistente” non significa una cosa sola. Il controllo si fa confrontando due strutture che **devono** descrivere la stessa partizione:

$\text{blocchi in uso} \cup \text{blocchi liberi} = \text{tutti i blocchi}$ , e le due liste sono disgiunte.

Dalla figura:

- (a) **consistente**: blocco per blocco, se uno è occupato l'altro è libero. La somma delle due righe dà 1 in ogni posizione;
- (b) **blocco perso**: nella posizione 2 entrambe le mappe dicono 0. Il blocco non appartiene a nessuno: è spazio sprecato, e nessuno lo riuserà mai (per il sistema è “occupato”, dato che non figura fra i liberi);

- **(c) doppio conteggio fra i liberi:** la posizione 4 vale 2 nella mappa dei liberi. Se il sistema alloca quello spazio due volte, due file diversi si sovrascrivono — è il caso **grave**;
- **(d) doppio conteggio fra gli occupati:** la posizione 5 vale 2 nella mappa degli occupati. Il blocco è condiviso da due file, e modificarne uno corrompe l'altro.

La gravità non è la stessa: il blocco perso è uno *spreco*, il doppio conteggio è una **corruzione**. È la ragione per cui questi controlli si fanno.

#### Il costo dei controlli di consistenza

Le utility di controllo sono la soluzione *storica*, e il loro problema principale non è l'efficacia ma il **costo**: per fare un controllo completo bisogna leggere **tutte** le strutture del file system, cioè percorrere tutti gli i-node e confrontarli con tutte le bitmap. La conseguenza pratica è il tempo: su un disco grande il controllo può richiedere **ore**.

È esattamente la critica che porta al **journaling** del paragrafo successivo. Vale la pena notare che i due controlli elencati dalle slides corrispondono ai due casi della figura:

- il controllo **sui blocchi** è quello della figura: le bitmap dei liberi e degli occupati si contraddicono;
- il controllo **sui riferimenti agli i-node** è l'analogo per i contatori: ogni i-node deve avere un numero di riferimenti che corrisponde a quante voci di directory lo puntano.

Il secondo è l'*anomalia con accounting* del capitolo 16 vista dal lato del riparatore: se il contatore dei link dice 2 e le directory che lo puntano sono 3, qualcosa è andato storto.

## 17.2 Il journaling

### La strategia del journaling

È l'alternativa ai controlli di consistenza *a posteriori*: [SO-04 p11]

- **le operazioni sui metadati sono preliminarmente appuntate in un log**, che viene poi **ripulito a posteriori** (subito dopo);
- **in caso di ripristino da crash**: si ripetono le operazioni in log.

Le slides elencano i **benefici**: [SO-04 p11]

- **maggiore robustezza dei metadati**;
- **veloce ripristino da crash/reboot**.

E aggiungono la condizione senza la quale il meccanismo non funziona: *affinché tutto funzioni bisogna operare con **operazioni idempotenti***. [SO-04 p11]

### Perché l'idempotenza è la condizione critica

È l'unica parola tecnica del paragrafo, e vale la pena spiegarla esattamente. Un'operazione è **idempotente** se eseguirla *due volte* dà lo stesso risultato che eseguirla una volta sola. Serve perché il ripristino non può sapere *dove* il crash è avvenuto:

1. il file system scrive nel log l'operazione che sta per fare;
2. esegue l'operazione sui metadati veri;

3. segna nel log che l'operazione è completata.

Se il crash avviene fra il passo 1 e il 2, al riavvio l'operazione non è stata fatta e va rifatta. Se avviene fra il 2 e il 3, l'operazione è stata fatta *ma il log dice di no*: e il ripristino, che non ha modo di distinguere i due casi, la **rifà comunque**.

Se l'operazione non fosse idempotente, rifarla sarebbe un errore. Un esempio: “incrementa il contatore dei link” *non* è idempotente — eseguita due volte dà 2 invece di 1. Nei journaling reali si usano perciò operazioni della forma “**scrivi il valore  $v$** ” invece di “*incrementa*”: scrivere lo stesso valore due volte dà lo stesso risultato. È un vincolo di progetto, non un dettaglio implementativo, ed è il tipo di osservazione che fa capire se il meccanismo lo si è capito o solo memorizzato.

#### Il guadagno: da “ore” a “secondi”

Vale la pena mettere a confronto diretto i due approcci, perché il secondo beneficio elencato dalle slides (“veloce ripristino”) si capisce solo così:

|                     | controllo di consistenza                  | journaling                                        |
|---------------------|-------------------------------------------|---------------------------------------------------|
| quando si lavora    | <i>dopo</i> il crash                      | <i>durante</i> il funzionamento                   |
| che cosa si legge   | <b>tutte</b> le strutture del file system | solo il <b>log</b>                                |
| tempo di ripristino | proporzionale alla dimensione del disco   | proporzionale alla dimensione del <i>log</i>      |
| costo ordinario     | nullo                                     | una scrittura in più per ogni operazione (il log) |

È il classico scambio fra costo *ordinario* e costo *in caso di guaio*: il journaling paga sempre qualcosa (la scrittura nel log) per non pagare mai il conto enorme. Ed è la stessa logica del *paging daemon* e del *pool di pagine vuote* del capitolo 14: spostare il lavoro fuori dal momento critico.

Da notare anche il limite dichiarato dalle slides: la robustezza riguarda i **metadati**, non i dati. Un journaling che protegge i metadati garantisce che il *file system* resti coerente, non che il contenuto di un file scritto a metà sia quello che l'utente voleva.

### 17.3 La cache del disco

#### Cache del disco (o buffer cache)

Per **migliorare le prestazioni dei dischi** si fa spesso uso di una **cache del disco**, detta anche **buffer cache**: è una **struttura basata su tabelle hash**. Le **strategie di gestione** elencate dalle slides sono [SO-04 p12]

- **LRU modificata**;
- **free-behind** e **read-ahead**;

e per la **scrittura** si distingue fra **sincrona** e **asincrona**.



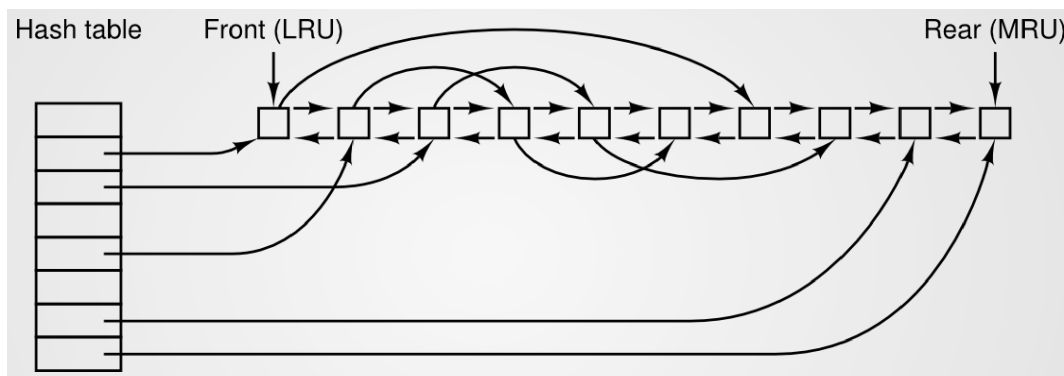


Figura 17.2: La cache dei blocchi. Una *hash table* indirizza i blocchi in cache; i blocchi sono tenuti in una lista doppiamente concatenata ordinata per uso, dal *Front* (LRU, il meno usato di recente) al *Rear* (MRU, il più usato di recente). Gli archi mostrano il riordino della lista ad ogni accesso

Fonte: proiezioni del corso, SO-04 p12

### Perché serve la cache, se c'è già la cache hardware

È la prima obiezione da superare, e la risposta è che i due livelli hanno funzioni diverse — come già visto per la cache della memoria rispetto alla TLB (capitolo 12):

- la cache **hardware** del disco (quella dentro il drive) è *piccola* e serve a riordinare gli accessi già in corso;
- la **buffer cache** è in memoria centrale, è grande (può essere centinaia di megabyte) e serve a non rileggere blocchi già letti. Ed è anche il posto dove il file system tiene i blocchi *modificati* prima di scriverli, il che permette di riordinare le scritture.

La **hash table** serve a trovare velocemente se un blocco è già in cache, e la **lista doppiamente concatenata** a sapere quale buttare via: è l'algoritmo LRU già visto per le pagine (capitolo 13), applicato ai blocchi.

### “LRU modificata”: cosa si modifica, e perché

Le slides parlano di **LRU modificata** e non di LRU pura, e la differenza è importante. LRU pura ordina i blocchi per istante dell'ultimo uso — ma nel file system c'è una complicazione che nella memoria non c'era: **i blocchi si possono scrivere**.

Un blocco **modificato** non è uguale a un blocco pulito: se lo si butta via, bisogna prima scriverlo su disco. La modifica alla LRU consiste nel tenere conto di questo, per esempio **proteggendo** i blocchi sporchi (per non doverli riscrivere subito) oppure al contrario **preferendoli** come vittime (per non accumulare scritture pendenti). Quello che si fa dipende dall'obiettivo: si sta ottimizzando il numero di accessi o la *regolarità* delle scritture su disco? È lo stesso bit di modifica (M) di cui si parlava a proposito delle pagine, e la stessa distinzione fra pagina pulita e sporca.

### Free-behind e read-ahead

Sono due strategie basate sull'**idea che l'accesso ai file sia sequenziale**, e vanno in direzioni opposte:

- **free-behind** (“libera dietro”): quando un blocco viene letto, i blocchi *precedenti* della stessa sequenza possono essere **liberati subito**. Se la lettura sta procedendo in avanti, il blocco appena superato non servirà più;
- **read-ahead** (“leggi avanti”): si **legge in anticipo** il blocco *successivo*, e anche più di

uno. Se la lettura procede in avanti, il blocco che servirà è già in cache — e la lettura dal disco era già stata avviata mentre la CPU lavorava.

#### Perché le due vanno insieme

Prese singolarmente, entrambe hanno un difetto che l'altra compensa:

- il **read-ahead** da solo *riempie* la cache di blocchi che forse non serviranno, e prima o poi la satura. Serve qualcosa che la svuoti;
- il **free-behind** da solo lascia la cache vuota e costringe a leggere tutto dal disco al momento del bisogno. Serve qualcosa che la riempi in anticipo.

Insieme formano una **finestra scorrevole** sulla sequenza dei blocchi letti: si tiene in cache ciò che sta davanti e si butta ciò che sta dietro. È l'idea di *località* del capitolo 14, ma applicata allo spazio (i blocchi vicini) invece che al tempo.

Il senso complessivo è che questi due meccanismi funzionano *solo* se l'accesso è sequenziale, o quasi. Su un accesso casuale il read-ahead spreca letture e il free-behind butta via blocchi che serviranno. È un buon esempio di ottimizzazione basata su un'ipotesi sul carico, e del fatto che le ipotesi sui carichi reali sono metà della progettazione.

#### Scrittura sincrona e asincrona

È una scelta che tocca direttamente ciò che l'utente percepisce, e conviene dire le conseguenze di entrambe le opzioni:

- **sincrona**: la scrittura non è conclusa finché il dato non è *davvero* sul disco. È **sicura** (se manca la corrente subito dopo, il dato c'è) ma **lenta** (la CPU o il processo aspettano un'operazione meccanica, con tempi di millisecondi);
- **asincrona**: la scrittura è conclusa quando il dato è in *cache*. È **veloce** (le scritture si accumulano e si riordinano, e il disco le esegue quando può) ma un crash prima che la cache venga svuotata **perde i dati**.

I sistemi reali usano entrambe: asincrona per l'ordinario, sincrona per ciò che deve sopravvivere per forza. Ed è una scelta che il programma può spesso richiedere esplicitamente — perché è il programma a sapere quali dati sono “importanti” e quali no.

## 17.4 Cosa usano i nostri sistemi operativi

La slide conclusiva del file system, che vale la pena ricordare come *cultura generale* — serve a mostrare che i concetti del capitolo non sono teoria, ma le scelte che i sistemi che si usano ogni giorno hanno fatto. [SO-04 p13]

| Sistema | File system                                                                                                                                                                                                              |
|---------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Windows | <b>exFAT</b> su unità rimovibili;<br><b>NTFS</b> su dischi fissi: file system moderno, <i>molto complesso</i> , con journaling, cifratura, compressione, copia shadow (CoW), dischi multipli (RAID)...                   |
| Linux   | <b>ext-4</b> : journaling, allocazione efficiente;<br><b>BTRFS</b> : file system recente ed evoluto — journaling, <i>check-sum</i> su dati e metadati, compressione, volumi, clonazione (CoW), dischi multipli (RAID)... |
| MacOS   | <b>HFS+</b> , recentemente soppiantato da <b>APFS</b>                                                                                                                                                                    |

#### Come si legge questa tabella

Non è un elenco di nomi da imparare a memoria: è la verifica che *tutti* i concetti del capitolo si ritrovano nei sistemi reali.

- il **journaling** c'è in NTFS, in ext-4 e in BTRFS: è la funzionalità che si è rivelata *indispensabile*;
- la **copia shadow** di NTFS e la **clonazione** di BTRFS sono entrambe **copy-on-write** (capitolo 14): la stessa idea vista nella memoria, qui applicata ai blocchi del disco;
- la **compressione** e la **cifratura** stanno nello strato del file system e non in quello dell'applicazione, perché è lì che si ha accesso a *tutti* i dati che passano;
- i **check-sum su dati e metadati** di BTRFS sono la risposta alla domanda “e se il disco restituisce dati sbagliati?” — un problema che il journaling *non* risolve, perché il journaling protegge dalla *interruzione*, non dalla *corruzione*;
- i **dischi multipli (RAID)** compaiono in tutte le voci, ed è il tema del capitolo 19.

La nota su MacOS (“HFS+ recentemente soppiantato da APFS”) dice che queste scelte non sono definitive: sono decisioni di progetto che vengono riviste quando cambiano i vincoli.

#### Come lo dici all'orale

“Come si mantiene affidabile e veloce un file system?” Il capitolo risponde con due problemi e quattro strumenti:

1. **se il sistema si interrompe a metà**: i **controlli di consistenza** (utility che confrontano le bitmap dei blocchi liberi e occupati, e i contatori dei riferimenti degli i-node) — efficaci ma lentissimi, perché leggono tutto il disco;
2. il **journaling**, che risolve lo stesso problema meglio: le operazioni sui metadati si scrivono *prima* in un log, e al riavvio si ripete il log. Veloce, robusto, ma richiede **operazioni idempotenti**. Va detto che protegge i *metadati*, non i dati;
3. le **prestazioni**: la **cache del disco** (hash table + lista LRU), la **LRU modificata** (che tiene conto dei blocchi sporchi), **free-behind** e **read-ahead** (finestra scorrevole sugli accessi sequenziali), e la scelta fra scrittura **sincrona** e **asincrona**;
4. la **verifica nei sistemi reali**: NTFS, exFAT, ext-4, BTRFS, HFS+/APFS. E qui la frase che chiude bene: journaling, CoW, compressione, cifratura, RAID e check-sum sono tutte funzionalità che si ritrovano *insieme*, perché rispondono a problemi diversi che si presentano tutti.

## Capitolo 18

### Scheduling del disco

---

Negli ultimi tre capitoli abbiamo costruito un file system: abbiamo deciso come si chiamano i file, dove stanno i loro blocchi, come si tiene traccia di quelli liberi e come si sopravvive a un guasto a metà scrittura. In tutto questo, però, abbiamo sempre dato per buono un passaggio: che leggere un blocco dal disco sia un'operazione di cui non ci si deve occupare.

Adesso ci si occupa di quel passaggio. E il motivo per cui merita un capitolo è che il disco è, fra tutti i componenti di un calcolatore, quello con la latenza più alta: la memoria centrale risponde in nanosecondi, il disco in millisecondi. Un millisecondo sembra poco, ma su un'operazione che ne dura cinque o dieci è la differenza fra un sistema che risponde e un sistema che sembra bloccato.

#### 18.1 Il problema

##### Scheduling del disco: gli obiettivi

Gli obiettivi dello scheduling del disco sono due: [SO-04 p14]

- **massimizzare il numero di richieste soddisfatte in una unità di tempo** (il throughput);
- **minimizzare il tempo medio di accesso.**

Da dove nasce la possibilità di scegliere? Dalla struttura del sistema. In un sistema, soprattutto se **multiprogrammato**, si vengono a creare varie richieste di I/O su disco, ma queste **tipicamente possono essere inviate al controller del disco solo una alla volta**. Si forma quindi una **coda di richieste pendenti**, e il sistema operativo può adottare **vari criteri di selezione della prossima richiesta da mandare**. [SO-04 p14]

È il punto chiave di tutto il capitolo, e vale la pena fermarsi: se le richieste potessero andare al controller tutte insieme, l'ordine sarebbe irrilevante. Siccome vanno **una alla volta** e formano una coda, l'ordine in cui le si estrae **cambia il tempo totale**. Riordinare una coda non costa nulla — è solo una scelta — e può far guadagnare un fattore due o tre.

##### Che cosa si può ottimizzare

[SO-04 p14]

- il **tempo di posizionamento** (*seek-time*): il tempo che la testina impiega a spostarsi sulla traccia giusta;
- la **latenza di rotazione**: il tempo che il settore giusto impiega a passare sotto la testina.

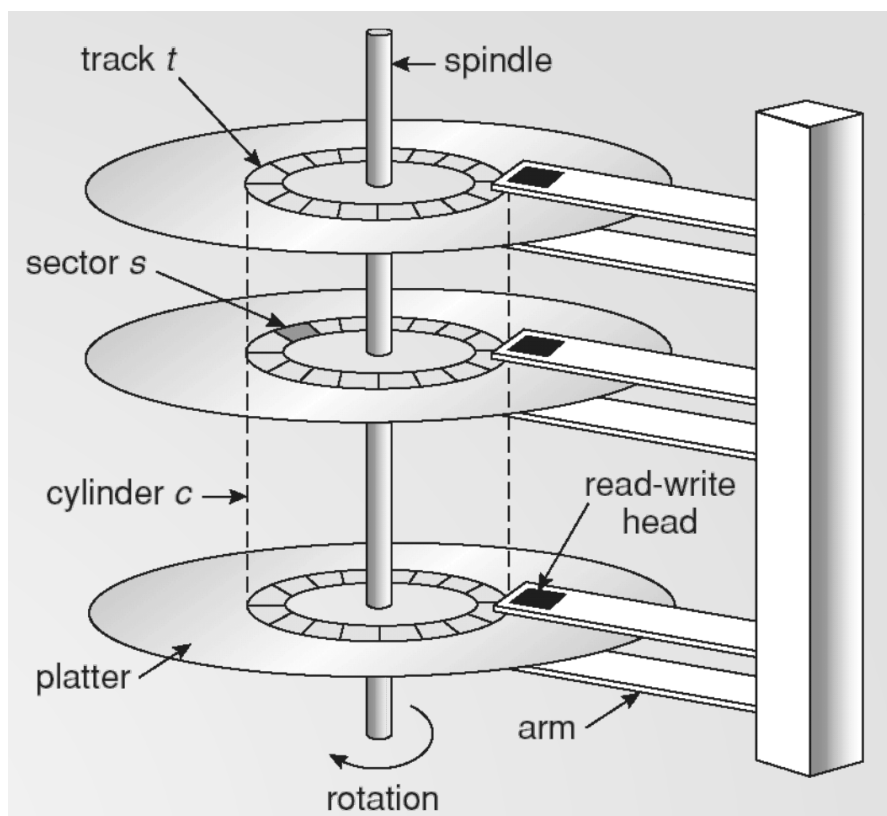


Figura 18.1: La geometria di un disco magnetico a piatti multipli. Le etichette sono quelle che servono per leggere tutto il capitolo: *platter* (il piatto), *spindle* (il perno su cui i piatti ruotano), *track* (la traccia), *sector* (il settore), *cylinder* (il cilindro), *read-write head* (la testina di lettura e scrittura), *arm* (il braccio), e la *rotation* che indica il verso di rotazione

Fonte: proiezioni del corso, SO-04 p14

### Cilindro: perché si ragiona per cilindri e non per tracce

La figura spiega da sola una scelta di vocabolario che altrimenti sembra gratuita. Un disco ha *più piatti*, e ogni piatto ha le sue tracce. Le testine sono solidali al braccio: **si muovono insieme**. Quindi le tracce che stanno alla stessa distanza dal perno su piatti diversi si trovano tutte sotto le rispettive testine *nello stesso istante*. Quell'insieme di tracce si chiama **cilindro**.

La conseguenza è che non ha senso parlare di “spostarsi sulla traccia *t* del piatto 3”: ci si sposta su un **cilindro**, e poi si sceglie quale testina usare. È per questo che nella figura l'etichetta *cylinder c* indica una superficie verticale che attraversa tutti i piatti, e che tutti gli esempi di questo capitolo sono espressi in “numero di cilindro”: **il seek-time dipende solo dal cilindro**, non da quale piatto.

### Su che cosa agisce davvero l'ordine delle richieste

È una precisazione che serve per non dire una cosa imprecisa all'orale. Riordinando la coda si agisce **direttamente sul tempo di posizionamento**: si sceglie quale cilindro raggiungere per primo, e quindi quanta strada fa la testina in tutto. Sulla **latenza di rotazione** l'ordine agisce in modo più indiretto: sapendo in che punto del giro si troverà il settore quando la testina arriva, il controller può preferire, fra due richieste sullo stesso cilindro, quella che passerà sotto la testina prima. È una raffinazione che i controller moderni fanno, ma il grosso del guadagno — quello che si vede negli esercizi — viene dal seek-time.

E c'è una cosa che **non si può** ottimizzare: il tempo di trasferimento vero e proprio. Dipende

dalla densità del disco e dalla velocità di rotazione, e nessun riordino lo cambia.

## 18.2 L'esempio di riferimento

Tutte e quattro le politiche che seguono sono presentate dalle slides **sullo stesso identico esempio**, e non è un caso: è ciò che permette di confrontarle. Conviene quindi fissarlo una volta per tutte. [SO-04 p15]

### L'esempio usato da tutte le slides

- **lista delle richieste**, in ordine di arrivo e per numero di cilindro:

98, 183, 37, 122, 14, 124, 65, 67

- **posizione iniziale della testina**: cilindro 53.

Le richieste si considerano **tutte già presenti**: è la formulazione dei libri di testo, ed è quella che produce i numeri dichiarati dalle slides. Nella realtà le richieste arrivano nel tempo, e questo cambia le cose (lo si vede nella verifica, in §18.8).

|                             | cilindro | 98  | 183 | 37 | 122 | 14 | 124 | 65 | 67 |
|-----------------------------|----------|-----|-----|----|-----|----|-----|----|----|
| distanza dalla testina (53) | 45       | 130 | 16  | 69 | 39  | 71 | 12  | 14 |    |

L'ultima riga è la distanza “in linea d'aria”, cioè quanto la testina dovrebbe spostarsi se andasse *direttamente* su quella richiesta. Non è il tempo di attesa di quella richiesta: è l'ingrediente con cui si calcola. Serve per seguire i conti delle prossime sezioni.

## 18.3 First Come First Served

### FCFS

Le richieste si servono **nell'ordine di arrivo**, senza riordinarle. Sull'esempio: [SO-04 p15]

- ordine usato: 98, 183, 37, 122, 14, 124, 65, 67;
- **distanza totale percorsa**: 640 tracce;
- **semplice da realizzare**;
- **equo**;
- **inefficiente**.

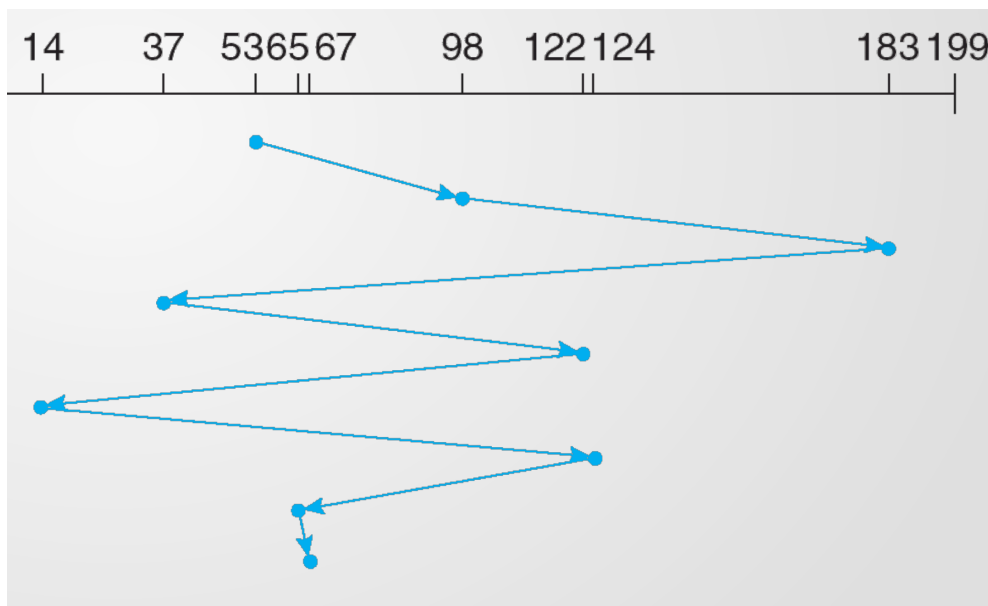


Figura 18.2: La traiettoria della testina con FCFS. Il tempo scorre verso il basso, l'asse orizzontale è il numero di cilindro: ogni segmento è uno spostamento della testina, e la lunghezza orizzontale del segmento è la distanza percorsa. La spezzata va da un capo all'altro del disco e ritorna, più volte

Fonte: proiezioni del corso, SO-04 p15

#### Da dove viene 640

La figura si legge contando i segmenti, ma il conto si fa in una riga. Le tappe della testina, con la distanza di ciascuna:

| da       | 53 | 98  | 183 | 37  | 122 | 14  | 124 | 65 |            |
|----------|----|-----|-----|-----|-----|-----|-----|----|------------|
| a        | 98 | 183 | 37  | 122 | 14  | 124 | 65  | 67 | totale     |
| distanza | 45 | 85  | 146 | 85  | 108 | 110 | 59  | 2  | <b>640</b> |

È il modo in cui va svolto un esercizio di questo tipo: una tabella con una riga per spostamento. Non serve ricordare 640 a memoria: si ricalcola.

#### Perché è “equo” e perché è “inefficiente”

Le due qualità hanno la stessa radice, e sono la cosa da dire all'orale.

- **equo**: nessuna richiesta può essere saltata. Una richiesta arrivata per prima viene servita per prima, *sempre*. In particolare nessuna richiesta può aspettare indefinitamente: il numero di richieste che la precedono è finito e non cresce. È l'assenza di **starvation**;
- **inefficiente**: proprio perché l'ordine è fissato dall'arrivo, la testina **non tiene conto di dove si trova**. La figura lo mostra bene: si va da 183 a 37 attraversando tutto il disco, e poi da 14 a 124 attraversandolo di nuovo, e poi ancora da 124 a 65. Sono passaggi sopra cilindri che *contenevano* richieste in attesa.

L'ultima osservazione è la chiave di tutto il capitolo: FCFS **passa sopra** le richieste pendenti senza servirle. Ogni politica successiva nasce dal tentativo di non sprecare quel passaggio.

18.4 Shortest Seek Time First

SSTF

Si serve sempre la richiesta **più vicina alla posizione attuale** della testina. Sull'esempio: [SO-04 p16]

- ordine usato: 65, 67, 37, 14, 98, 122, 124, 183;
- **distanza totale percorsa**: 236 tracce;
- **buone prestazioni**;
- **non equo** (*starvation*).

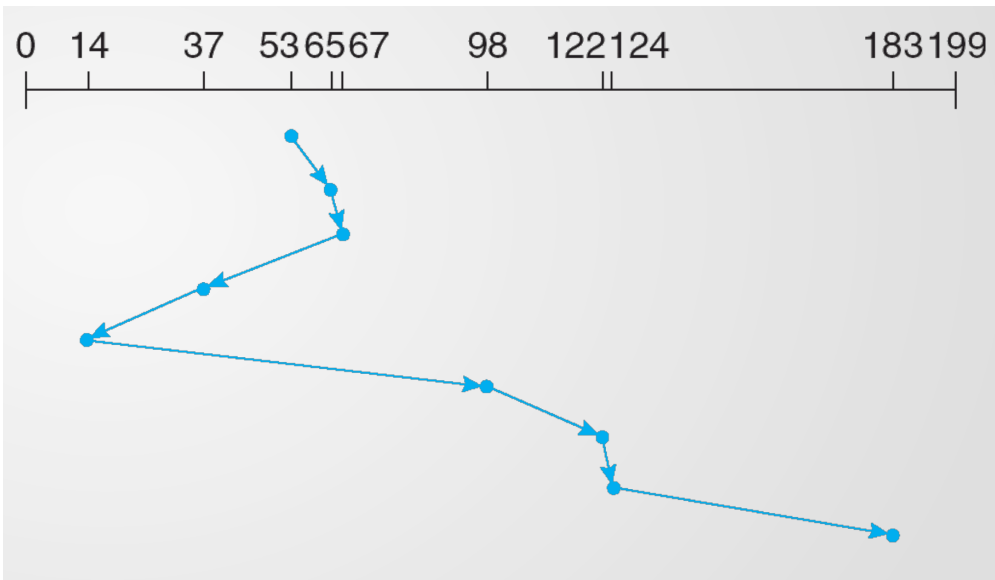


Figura 18.3: La traiettoria con SSTF. Il confronto con la figura di FCFS è immediato: i segmenti sono molto più corti e la spezzata resta compatta, invece di attraversare il disco da un capo all'altro  
Fonte: proiezioni del corso, SO-04 p16

Da dove viene 236, e come si costruisce l'ordine

L'ordine **non si indovina**: si costruisce un passo per volta, guardando ogni volta qual è la richiesta più vicina fra quelle ancora in attesa.

|                 |    |    |    |    |    |     |     |     |               |
|-----------------|----|----|----|----|----|-----|-----|-----|---------------|
| <b>da</b>       | 53 | 65 | 67 | 37 | 14 | 98  | 122 | 124 |               |
| <b>a</b>        | 65 | 67 | 37 | 14 | 98 | 122 | 124 | 183 | <b>totale</b> |
| <b>distanza</b> | 12 | 2  | 30 | 23 | 84 | 24  | 2   | 59  | <b>236</b>    |

Vale la pena notare *come* si forma l'ordine, perché è lì che si sbaglia. Partendo da 53 la richiesta più vicina è 65 (distanza 12), non 67 (14) e non 37 (16). Servita 65, la più vicina diventa 67 (distanza 2). Servita 67, le candidate sono 37 (30) e 98 (31): vince 37, e per un soffio. Da 37 la più vicina è 14 (23), che è *più lontana* di 98 (61): la testina prosegue quindi verso il basso anche se ciò la allontana da tutte le altre. Poi risale. Questo “cambio di verso a metà strada” è ciò che SSTF fa e che FCFS non farebbe mai.  
Confronto con FCFS: 640 → 236. La distanza si riduce di quasi tre volte, ed è esattamente il guadagno che deriva dal non sprecare i passaggi sopra le richieste pendenti.



**Starvation: il difetto che costa a SSTF il nome “buono”**

SSTF ottimizza *localmente*: sceglie sempre il passo più corto. Ma proprio per questo non ha memoria. Supponi che la testina si trovi stabilmente nella zona centrale del disco e che arrivino di continuo richieste per cilindri vicini ad essa. Una richiesta per il cilindro 199, ferma in coda da tempo, **non verrà mai scelta** finché continua ad arrivare qualcosa di più vicino. Non è un caso raro: è la situazione normale su un disco con accessi concentrati su un'area.

È la stessa patologia della politica a priorità a “priorità più bassa” del capitolo 9, e la cura è la stessa: introdurre qualcosa che rompa la monotonia — l'invecchiamento delle priorità di là, la **scansione** di qui. Il fatto che SSTF dichiari “buone prestazioni” e “non equo” non è una contraddizione: sono due giudizi su due cose diverse, la velocità media e la garanzia.

## 18.5 Scheduling per scansione (algoritmo dell'ascensore)

**Scansione**

Si **mantiene un verso** finché ci sono richieste in quel verso, e solo allora si inverte. Sull'esempio (inizio verso l'alto): [SO-04 p17]

- ordine usato: 65, 67, 98, 122, 124, 183, 37, 14;
- **distanza totale percorsa**: 299 tracce (più di SSTF);
- **scansione uniforme**;
- **garantisce comunque una attesa massima per ogni richiesta**.

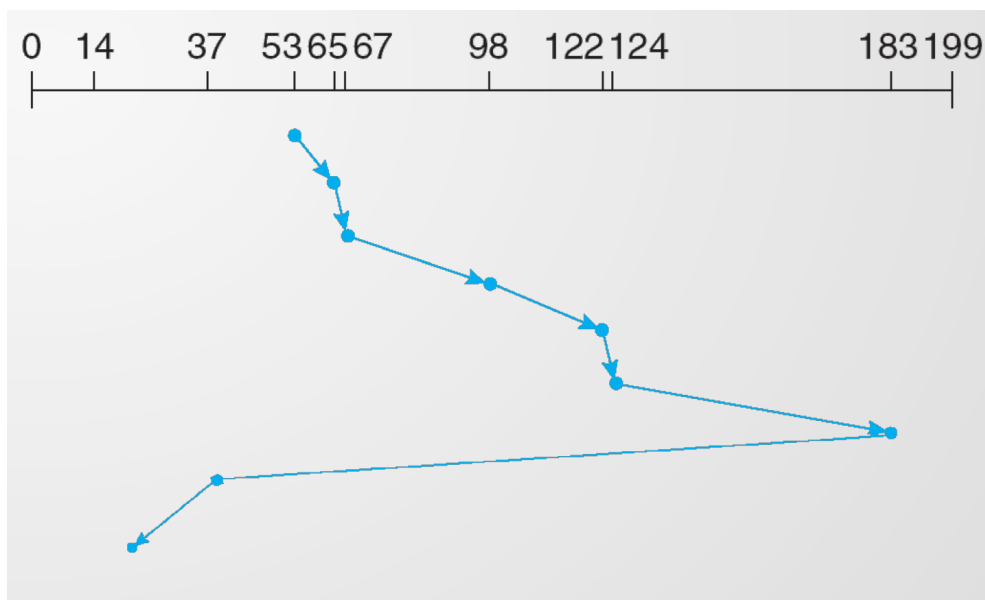


Figura 18.4: La traiettoria con la scansione. La forma è la firma dell'algoritmo: una lunga salita che serve tutto ciò che incontra, poi una discesa sola, senza tornare indietro

Fonte: proiezioni del corso, SO-04 p17

## Da dove viene 299

|                 |    |    |    |     |     |     |     |    |               |
|-----------------|----|----|----|-----|-----|-----|-----|----|---------------|
| <b>da</b>       | 53 | 65 | 67 | 98  | 122 | 124 | 183 | 37 |               |
| <b>a</b>        | 65 | 67 | 98 | 122 | 124 | 183 | 37  | 14 | <b>totale</b> |
| <b>distanza</b> | 12 | 2  | 31 | 24  | 2   | 59  | 146 | 23 | <b>299</b>    |

Le prime sei distanze sono *identiche* a quelle di SSTF, e non è un caso: finché la testina sale, “seguire il verso” e “andare al più vicino” coincidono, perché la richiesta più vicina è per forza la prossima salendo. La differenza sta **nella settimana**: da 183 SSTF tornava a 98 (distanza 85), mentre la scansione scende fino a 37 (distanza 146). La scansione perde quei 61 cilindri in più, e li perde di proposito.

## Attenzione al nome: qui “scansione” significa LOOK

È una trappola in cui si cade sia all’esame sia nei libri, e conviene averla chiara. La slide dice **“mantiene un verso fino all’ultima richiesta in tale direzione”**, e dichiara **299**. Le due cose sono coerenti fra loro: se la testina inverte *sull’ultima richiesta*, il conto è proprio 299. Verificato eseguendo il conto, non riletto.

Nel libro di testo, però, la parola *scansione* (SCAN) indica qualcosa di leggermente diverso: la testina prosegue **fino al fondo fisico del disco**, al cilindro 199, e *poi* inverte — anche se fra 183 e 199 non c’è nessuna richiesta da servire. In quel caso la distanza non è 299 ma  $299 + 16 + 16 = 331$ : 16 cilindri per arrivare a 199, e altri 16 per tornare indietro da 199 a 183, che la discesa deve comunque ripercorrere.

L’algoritmo che inverte all’ultima richiesta è quello che la letteratura chiama **LOOK**, e è quello che descrivono e usano le slides. La differenza non è solo terminologica: se in un esercizio la consegna dice “fino al fondo del disco” il numero da dare è 331, se dice “fino all’ultima richiesta” è 299. **Leggi la consegna.**

## Perché “garantisce una attesa massima”

È l’affermazione da capire, perché è il vero motivo per cui la scansione esiste malgrado costi più strada di SSTF.

Con SSTF una richiesta lontana può aspettare *senza limite*. Con la scansione no: in un giro la testina **percorre il disco interamente**, da un estremo all’altro. Quindi qualunque richiesta in attesa, *qualunque cilindro*, viene incontrata entro la fine del giro in corso. Il tempo di attesa peggiore possibile è quello di un sweep completo — un numero **finito e noto in anticipo**.

Questo introduce una distinzione che vale per tutto il capitolo e che va detta con queste parole: SSTF è migliore **in media**, la scansione è migliore **nel caso peggiore**. Non si può avere entrambi con una politica sola, ed è il motivo per cui la scelta finale (paragrafo 18.7) dipende dal carico.

## 18.6 Scheduling per scansione circolare

## Scansione circolare (C-SCAN)

Si considerano le posizioni come **collegate in modo circolare**: arrivata alla fine del disco, la testina **torna sul primo cilindro senza servire alcuna richiesta**. Sull’esempio: [SO-04 p18]

- ordine usato: 65, 67, 98, 122, 124, 183, 14, 37;



Verificato eseguendo: la simulazione dà esattamente 350 e l'ordine della slide.

#### Attenzione: un conto che non torna

La slide afferma che la scansione circolare “garantisce un tempo medio di attesa più basso **in presenza di tante richieste**”. Ho provato a riprodurre questa affermazione con una simulazione (in `verifica_esercizi.py`, sezione “Molte richieste”), e **non torna**: in *ogni* condizione di carico provata, la scansione circolare ha attesa media e attesa massima *peggiori* della scansione.

| politica                                         | attesa media | attesa massima | distanza |
|--------------------------------------------------|--------------|----------------|----------|
| <i>100 richieste, una ogni 20 unità di tempo</i> |              |                |          |
| FCFS                                             | 3493.7       | 6931.0         | 8911     |
| SSTF                                             | 125.9        | 245.0          | 2168     |
| scansione                                        | <b>111.8</b> | <b>189.0</b>   | 2150     |
| scansione circolare                              | 149.3        | 290.0          | 2251     |
| <i>200 richieste, una ogni 4 unità di tempo</i>  |              |                |          |
| FCFS                                             | 8535.6       | 16993.0        | 17789    |
| SSTF                                             | 222.3        | 434.0          | 1062     |
| scansione                                        | <b>114.0</b> | <b>228.0</b>   | 1012     |
| scansione circolare                              | 166.1        | 290.0          | 971      |
| <i>300 richieste casuali, media su 40 prove</i>  |              |                |          |
| SSTF                                             | 255.9        | 468.9          | 3235     |
| scansione                                        | <b>213.0</b> | <b>424.1</b>   | 3236     |
| scansione circolare                              | 319.3        | 572.5          | 3274     |

Il motivo è strutturale e si vede dall'implementazione: la scansione circolare percorre il disco **due volte per ogni giro utile** (una salendo a servire, una scendendo a vuoto), mentre la scansione scende *servendo*. Su un disco di 200 cilindri il rientro a vuoto costa quasi un giro intero, e il vantaggio che ne deriva — servire tutti i cilindri con la stessa frequenza — non basta a compensarlo.

Il vantaggio classico della scansione circolare è di **uniformità del servizio**: nessun cilindro è privilegiato, cosa che invece accade alla scansione, che serve due volte per giro i cilindri centrali e una sola volta quelli estremi. È però un argomento sulla *distribuzione* dei tempi di attesa, non sulla loro media.

**Come comportarsi all'esame:** la risposta attesa è quella della slide, e va data così com'è. Se vuoi mostrare di aver capito qualcosa in più, aggiungi che il vantaggio documentato della scansione circolare è l'uniformità dell'attesa, non la sua media. Non rispondere “la scansione circolare è peggiore”: la consegna chiede il programma del corso, non una discussione.

## 18.7 Che cosa scegliere

### La conclusione della slide

[SO-04 p18]

- scansione circolare ad alto carico;
- scansione o SSTF a basso carico.

Il criterio ha una logica riconoscibile: è lo stesso compromesso che si ritrova nello scheduling della CPU (capitolo 9). A **basso carico** la coda è corta, il sistema è quasi sempre in grado di servire subito la richiesta più vicina, e la cosa che conta è non fare strada inutile — quindi SSTF o scansione, che non pagano il rientro a vuoto. Ad **alto carico** la coda è sempre piena, il costo del rientro si ammortizza su molte richieste servite, e diventa dominante il rischio che qualche cilindro resti a lungo senza servizio: la circolarità garantisce che tutti i cilindri vengano raggiunti con la stessa frequenza.

#### Il collegamento con il capitolo sullo scheduling della CPU

Vale la pena rendersi conto che è lo *stesso* problema, con nomi diversi. Il disco è una **risorsa unica** che va assegnata a richieste concorrenti; la coda delle richieste pendenti è la coda dei processi pronti; il tempo di posizionamento è il tempo di servizio.

Le corrispondenze sono quasi uno a uno:

|          | scheduling della CPU                  | scheduling del disco                    |
|----------|---------------------------------------|-----------------------------------------|
| FCFS     | first come first served               | FCFS                                    |
| SJF      | il più corto prima                    | SSTF (il più vicino prima)              |
| RR       | round robin                           | scansione circolare                     |
| priorità | invecchiamento per evitare starvation | il verso imposto per evitare starvation |

Il parallelismo non è decorativo: le due famiglie di algoritmi soffrono degli stessi mali per le stesse ragioni. FCFS è equo e inefficiente in entrambi i casi; la scelta greedy (SJF, SSTF) è la migliore in media e affama qualcuno; la soluzione è imporre un ordinamento esterno — il quanto di tempo di là, il verso di qui.

## 18.8 La verifica: i numeri delle slides

Tutti i numeri dichiarati dalle slides di questo capitolo sono stati riprodotti con una simulazione, in `verifica_esercizi.py`. La tabella seguente mette a confronto l'ordine e la distanza dichiarati con quelli ottenuti eseguendo.

| politica            | ordine                            | distanza | esito           |
|---------------------|-----------------------------------|----------|-----------------|
| FCFS                | 98, 183, 37, 122, 14, 124, 65, 67 | 640      | coincide        |
| SSTF                | 65, 67, 37, 14, 98, 122, 124, 183 | 236      | coincide        |
| scansione           | 65, 67, 98, 122, 124, 183, 37, 14 | 299      | coincide        |
| scansione circolare | 65, 67, 98, 122, 124, 183, 14, 37 | 350      | ordine coincide |

Per la scansione circolare la slide non dichiara la distanza: il 350 è calcolato qui. Per la scansione, il 299 dichiarato dalla slide corrisponde all'inversione sull'ultima richiesta; andando fino al fondo fisico del disco la distanza sarebbe 331 (si veda la nota sul nome nella sezione della scansione).

Sulle stesse otto richieste, tutte presenti all'istante 0, si possono anche misurare le attese, che sono la cosa che realmente conta e che *non* coincidono con la distanza totale:

| politica            | distanza | attesa media | attesa massima |
|---------------------|----------|--------------|----------------|
| FCFS                | 640      | 392.2        | 640.0          |
| SSTF                | 236      | <b>109.5</b> | 236.0          |
| scansione           | 299      | 114.5        | 299.0          |
| scansione circolare | 350      | 127.2        | 350.0          |

### Che cosa si legge in questa tabella

Tre cose, e sono tutte esatte.

1. **la distanza non è l'attesa.** Confronta SSTF e scansione: SSTF percorre 236, la scansione 299. Ma l'attesa media è 109.5 contro 114.5, mentre l'attesa *massima* è 236 contro 299. Chi percorre meno strada non è automaticamente più rapido per tutti: dipende da *chi* viene servito tardi;
2. **con tutte le richieste presenti all'istante 0, l'attesa massima coincide con la distanza totale.** Non è un caso: la richiesta servita per ultima ha aspettato esattamente tutto il tempo in cui la testina si è mossa. È una scorciatoia utile per controllare un esercizio;
3. **SSTF vince in media, la scansione vince nel caso peggiore:** 109.5 contro 114.5 da una parte, 299 contro 236 dall'altra. È esattamente il compromesso spiegato nelle sezioni precedenti, e ora è un numero.

## 18.9 Riepilogo

| politica                       | idea                                           | giudizio della slide                                            | distanza |
|--------------------------------|------------------------------------------------|-----------------------------------------------------------------|----------|
| <b>FCFS</b>                    | in ordine di arrivo                            | semplice, equo,<br>inefficiente                                 | 640      |
| <b>SSTF</b>                    | la più vicina                                  | buone prestazioni, non<br>equo (starvation)                     | 236      |
| <b>scansione</b>               | mantiene il verso fino<br>all'ultima richiesta | scansione uniforme,<br>attesa massima garantita;<br>più di SSTF | 299      |
| <b>scansione<br/>circolare</b> | rientro a vuoto sul primo<br>cilindro          | attesa media più bassa ad<br>alto carico                        | 350      |

La riga che vale la pena ricordare è quella della scansione: è l'unica politica dell'elenco che **paga in distanza** quello che guadagna in garanzia. Tutte le altre scelgono uno dei due.

### Come lo dici all'orale

“Mi parli dello scheduling del disco.” La traccia:

1. il **problema**: massimizzare il throughput, minimizzare il tempo medio di accesso. Nasce dal fatto che le richieste si accodano e vanno al controller **una alla volta**; si può agire sul **seek-time** e sulla **latenza di rotazione**;
2. l'**esempio** — coda 98, 183, 37, 122, 14, 124, 65, 67, testina su 53 — e le **quattro politiche** con il loro ordine e la loro distanza: FCFS 640, SSTF 236, scansione 299, circolare 350. Se sai ricostruire le quattro tabelle delle distanze, sai rispondere a qualunque variante;
3. i **giudizi**: FCFS equo ma inefficiente; SSTF veloce ma affama; la scansione garantisce l'attesa massima perché percorre tutto il disco; la scansione circolare serve tutti i cilindri con la stessa frequenza;
4. la **scelta**: scansione circolare ad alto carico, scansione o SSTF a basso carico.

Il dettaglio che fa la differenza: dire che **scansione e scansione circolare differiscono solo nell'ultimo tratto** — la prima scende servendo, la seconda rientra a vuoto — e che per questo i loro ordini sull'esempio finiscono 37, 14 contro 14, 37.

Se ti chiedono un confronto con lo scheduling della CPU, la risposta che colpisce è la tabella delle corrispondenze: SSTF sta a SJF come la scansione circolare sta al round robin, e in entrambi i casi il problema è impedire che la scelta greedy affami qualcuno.

## Capitolo 19

### I sistemi RAID

Il capitolo precedente ha mostrato che l'ordine con cui si servono le richieste può ridurre di molto la strada percorsa dalla testina: su otto richieste, da 640 a 236 tracce. È un guadagno reale, ma ha un limite che nessun algoritmo può superare: **la testina è un oggetto meccanico**. Per spostarla bisogna fisicamente muovere un braccio, e per far passare il settore giusto sotto di essa bisogna aspettare che il disco giri. Il riordino decide *quale* strada fare, non *se* farla.

Questo capitolo presenta l'altra strada: **usare più dischi insieme**. Se il lavoro si può dividere fra più unità che lavorano contemporaneamente, il tempo non si riduce di un fattore costante — si riduce dividendo, ed è il tipo di guadagno che non si ottiene ottimizzando meglio un disco solo.

#### 19.1 L'idea: più dischi, visti come uno

##### RAID: lo striping

Un altro modo per aumentare le prestazioni è **sfruttare il parallelismo anche per l'I/O su disco**. [SO-04 p19]

- servono **più dischi indipendenti**;
- si suddividono i dati relativi a una **unità logica** (un file, in generale un **volume**) su più dischi: è lo **striping**;
- la suddivisione è **trasparente all'utente**.

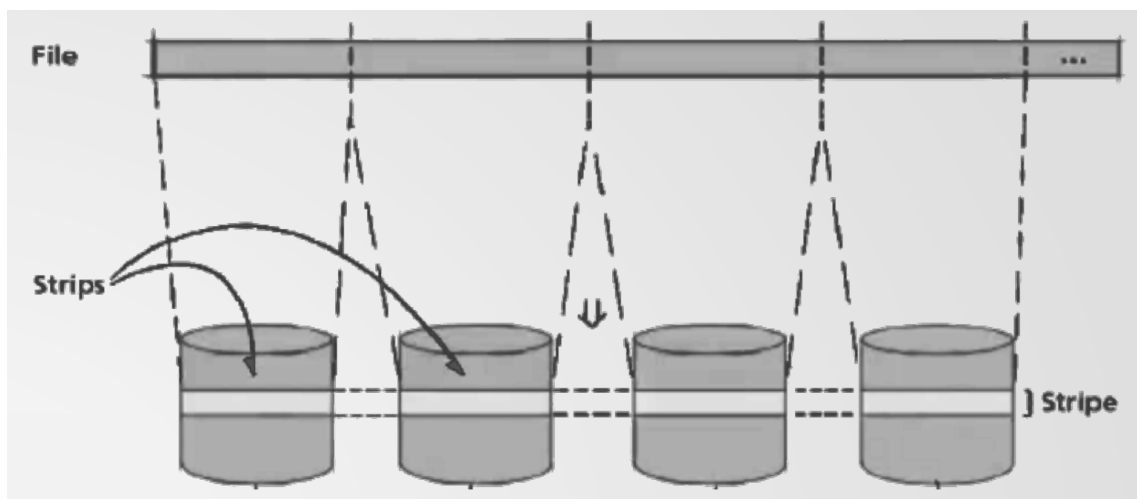


Figura 19.1: Lo striping. Il file in alto viene tagliato in strisce (*strips*), e le strisce vengono assegnate a dischi diversi seguendo le linee tratteggiate: la prima al primo disco, la seconda al secondo, e così via, tornando poi al primo. La graffa a destra indica che una striscia di ciascun disco forma, nel loro insieme, un'unità di dati più grande

Fonte: proiezioni del corso, SO-04 p19

##### Come si legge la figura, e che cosa significa “trasparente”

Seguiamo una striscia: la prima parte del file va sul disco 1, la seconda sul disco 2, la terza sul 3, la quarta sul 4, la quinta di nuovo sul disco 1. È una distribuzione **a rotazione**



(*round-robin*): il file occupa *tutti* i dischi, e nessun disco da solo contiene il file intero.

Il vantaggio si vede subito se si pensa a una lettura sequenziale: le quattro parti iniziali stanno su quattro dischi diversi, e possono essere lette **contemporaneamente**, una per disco. Il volume non è un disco grande: è un disco **più largo**, cioè più veloce.

“Trasparente all’utente” significa che questo non si vede: il file system continua a chiedere “leggi il blocco *k* del file”, e qualcuno — il controller RAID o il sistema operativo — traduce quell’indirizzo logico nell’indirizzo fisico del disco giusto. Chi apre il file non sa su quanti dischi è distribuito, esattamente come non sa su quale settore sta.

È la stessa idea della paginazione del capitolo 12: si introduce un livello di indirizzione fra “dove credo che sia il dato” e “dove sta davvero”. Là serviva a disaccoppiare lo spazio degli indirizzi dalla memoria fisica, qui a spalmare un file su più unità.

## 19.2 Il problema che lo striping crea

### Il prezzo del parallelismo

Lo striping **aumenta la probabilità che si verifichi un guasto sul volume logico RAID**. La soluzione è **aggiungere ridondanza** per ottenere **migliore affidabilità**; la sostituzione può essere automatica grazie a **dischi di riserva** (*spare*). [SO-04 p19]

### Perché mettere insieme più dischi peggiora l'affidabilità

È il punto concettualmente più importante del capitolo, e vale la pena rendersene conto bene, perché è controintuitivo.

Prendi un disco solo, con probabilità di guastarsi in un anno pari all’1%. Il dato è al sicuro al 99%. Ora metterne quattro in striping, e chiediti quando il *volume* perde i dati: il volume li perde se si guasta **uno qualunque** dei quattro dischi, perché ogni disco contiene una parte del file che nessun altro ha. Con quattro dischi la probabilità di perdere il volume è all’incirca **quattro volte** quella di perderne uno: dal 1% al 4%.

La regola generale è che il volume è affidabile quanto il **disco meno affidabile** che contiene, perché il guasto di uno solo è sufficiente. È la stessa logica per cui una catena si rompe dove è il suo anello più debole: mettere insieme più anelli *allunga* la catena e quindi **aumenta** la probabilità che uno di essi ceda.

Il guadagno in prestazioni si paga quindi con una perdita di affidabilità, ed è precisamente da questo che nasce la “R” di RAID: la ridondanza non è un’aggiunta, è la **compensazione necessaria** del parallelismo. Un array senza ridondanza non è un RAID — vedi il RAID 0, che infatti si chiama “0” per dire che la ridondanza è zero.

### Il nome, e le due realizzazioni

**RAID** sta per *Redundant Array of Inexpensive Disks*, ed è anche noto come *Redundant Array of Independent Disks*. [SO-04 p19] Le due letture raccontano la stessa storia da due lati: alla fine degli anni Ottanta l’idea era di sostituire un disco grande e costoso con molti dischi piccoli ed economici (la prima lettura); con il tempo, siccome la ridondanza serve comunque, quei dischi sono diventati unità indipendenti e affidabili (la seconda).

I RAID si realizzano in due modi [SO-04 p19]:

- **via hardware**: un controller RAID si presenta al sistema come *un solo disco*. Il sistema operativo non sa nulla della ridondanza: è **trasparente al S.O.**. Si paga il costo del controller;
- **via software**: è il sistema operativo a fare il lavoro, con un **carico sulla CPU**. Costa

meno e non richiede hardware dedicato, ma ogni scrittura passa per il processore e per la memoria principale invece che per un'*elettronica dedicata* accanto ai dischi.

La differenza ha una conseguenza che vale la pena dire: con il RAID hardware il sistema operativo **non può** accorgersi che qualcosa è cambiato se si sostituisce un disco, mentre con quello software la gestione degli *spare* e della ricostruzione è fatta dal sistema operativo stesso. È il motivo per cui i sistemi operativi moderni incorporano *mdadm* e simili.

### 19.3 RAID 0: striping puro

#### RAID 0

[SO-04 p20]

- fa striping in modalità **round-robin**;
- semplice, con **prestazioni ottimali con letture di grandi volumi**;
- **niente ridondanza**: maggiore vulnerabilità.

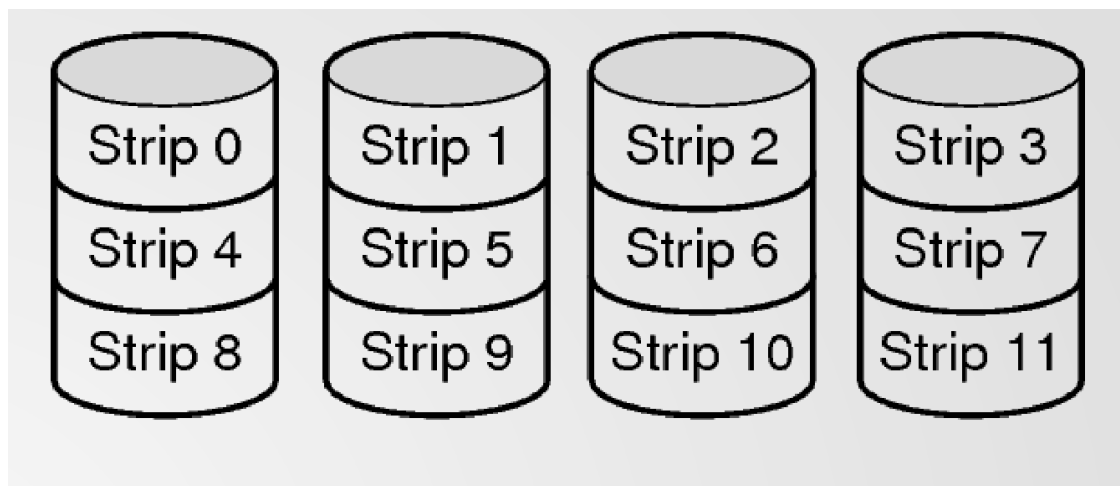


Figura 19.2: RAID 0 su quattro dischi. Gli strip vanno in rotazione — disco 1, 2, 3, 4, poi di nuovo disco 1 — e nessuna informazione è ripetuta. Ogni strip esiste su un disco solo

Fonte: proiezioni del corso, SO-04 p20

#### Perché “prestazioni ottimali con letture di grandi volumi”

La precisazione “di grandi volumi” non è un dettaglio: è il funzionamento stesso dello striping. Il guadagno c'è quando la lettura è **abbastanza lunga da occupare più dischi insieme**, così che il tempo di seek di ciascuno si sovrapponga a quello degli altri.

Per una lettura di *un blocco solo*, invece, il RAID 0 non offre niente: quel blocco sta su un disco preciso, e va letto da lì. Anzi, ci si mette il tempo di capire su quale disco sia, e non c'è nessun parallelismo da sfruttare. È la stessa condizione che rende utile la paginazione (capitolo 12): la località. Senza località, il parallelismo non ha nulla da parallelizzare.

## 19.4 RAID 1: mirroring

### RAID 1

[SO-04 p20]

- gli eventuali strip vengono **anche duplicati** (*mirroring*);
- può anche essere usato **senza striping**;
- **raddoppio delle prestazioni in lettura**;
- **migliore *fault tolerance***;
- **alto *overhead* di storage**.

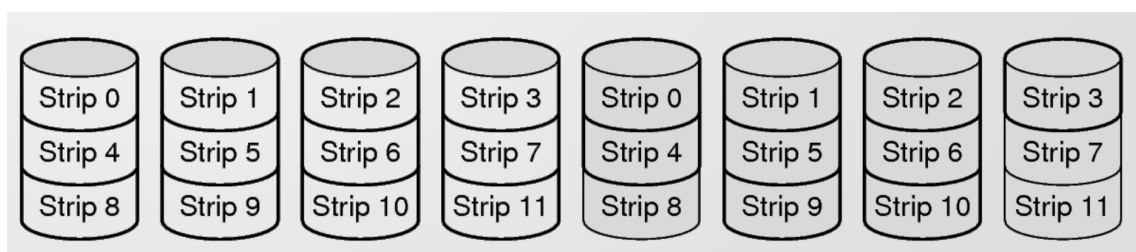


Figura 19.3: RAID 1 su otto dischi. Le prime quattro colonne contengono gli strip in rotazione, e le altre quattro ripetono **esattamente** la stessa sequenza: lo strip 0 sta sul disco 1 e sul disco 5, e così via. Ogni dato esiste in due copie

Fonte: proiezioni del corso, SO-04 p20

### Da dove viene il “raddoppio in lettura” e quanto costa

Le due voci vanno spiegate insieme, perché sono lo stesso fatto visto da due lati.

- **lettura:** ogni strip ha due copie, su due dischi diversi. Il controller può mandare due letture *contemporaneamente* ai due dischi e usare quella che arriva prima, oppure — meglio — distribuire le letture fra i due. Il numero di letture al secondo raddoppia perché raddoppiano i dischi che possono servirle;
- **scrittura:** no. Ogni scrittura va fatta *su entrambe* le copie, quindi il costo in scrittura non si riduce; anzi, la scrittura più lenta delle due determina il tempo.
- **overhead:** la capacità utile è **la metà** di quella dei dischi installati. Quattro dischi da due terabyte danno due terabyte utili. È il prezzo più alto fra tutti i livelli di questo capitolo, ed è il motivo per cui il RAID 1 si usa per i dati che contano davvero, non per tutto.

“Può anche essere usato senza striping” è la configurazione più semplice e comune: due dischi, uno la copia dell’altro. È quella che sui sistemi domestici si chiama *mirror*.

## 19.5 RAID 2: striping a livello di bit con ECC

### RAID 2

[SO-04 p21]

- lavora sulle **parole** applicando un **codice di correzione degli errori** — **ECC** (tipo *codice di Hamming* per singoli bit di errore);
- esempio: **4 bit dati + 3 bit di ridondanza**;
- **ottima *fault tolerance***;
- **serve sincronia nelle rotazioni dei dischi**.

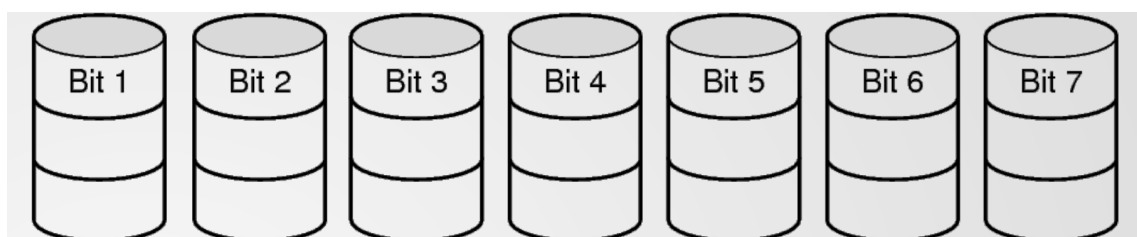


Figura 19.4: RAID 2. Ogni disco contiene uno solo dei bit di cui è composta una parola: i primi quattro i bit di dati, gli ultimi tre i bit di ridondanza calcolati dal codice. Non è un caso che i dischi siano **sette**: è l'esempio “4 bit dati + 3 bit ridondanza” della slide

Fonte: proiezioni del corso, SO-04 p21

#### Che cosa fa un codice di Hamming, in una frase

La differenza rispetto alla parità dei livelli successivi sta in questa parola: **correzione**.

La parità XOR (RAID 3, 4, 5) *rileva* che qualcosa non torna, e permette di ricostruire il dato **se si sa già quale disco è guasto**. Se il disco muore del tutto, il controller lo sa e la ricostruzione funziona. Ma se un bit si corrompe “in silenzio”, senza che il disco si dichiari guasto, con la sola parità non si sa dove guardare.

Un codice di Hamming, invece, **localizza** il bit sbagliato: i bit di ridondanza sono combinazioni dei bit dati calcolate in modo che l'insieme dei controlli che non tornano indichi *esattamente* quale posizione è errata, e quindi il bit si può **correggere**. È il motivo per cui la slide dice “ottima *fault tolerance*”: non tollera un guasto in più degli altri livelli, lo tollera *senza dover sapere in anticipo dove sia*.

Come si costruisce un codice del genere è mostrato nella verifica (§19.10): l'esempio “4 bit dati + 3 bit di ridondanza” “corregge davvero tutti gli errori su singolo bit, e non è un'affermazione da prendere sulla fiducia.

#### “Serve sincronia nelle rotazioni”: che cosa impedisce

È la voce che distingue davvero il RAID 2 e 3 dai RAID 4 e 5, e va capita bene.

Qui i bit di una stessa parola stanno su dischi **diversi**: il bit 1 sul disco 1, il bit 2 sul disco 2, e così via. Perché la parola sia leggibile, tutti quei bit devono arrivare **insieme**, nello stesso istante. Se i dischi girassero a velocità diverse, alcuni bit arriverebbero prima di altri e la parola non si potrebbe ricomporre. Serve quindi che i dischi siano **sincronizzati**, cioè che i loro settori passino sotto le testine contemporaneamente.

La conseguenza pratica è pesante: un array sincronizzato si comporta come **un disco solo ma veloce**. Tutte le testine si muovono insieme, e quindi *non si possono servire due richieste diverse in parallelo* — che era il motivo per cui si erano messi più dischi. Lo striping a livello

di bit dà larghezza di banda ma non indipendenza, ed è la ragione per cui i livelli 4 e 5, che lavorano sui blocchi, sono quelli che si usano davvero.

La slide chiude dicendo che fare striping a livello di bit è comunque **pesante se non gestito a livello hardware**: serve cioè un controller che ricomponga le parole in tempo reale.

## 19.6 RAID 3: striping a livello di bit con parità

### RAID 3

[SO-04 p21]

- usa **un solo disco** con raccolti i singoli **bit di parità**;
- in realtà permette anche di **recuperare i dati** e offre la **stessa capacità di *fault tolerance*** del RAID 2;
- **serve ancora la sincronia**.

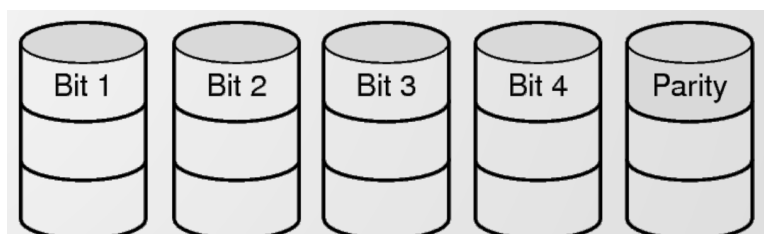


Figura 19.5: RAID 3 su cinque dischi: quattro bit di dati e un disco di parità. Rispetto al RAID 2, i tre bit di ridondanza sono sostituiti da **un solo** bit per parola, calcolato come parità dei quattro bit di dati

Fonte: proiezioni del corso, SO-04 p21

### Da sette dischi a cinque, senza perdere capacità di tolleranza

È l'osservazione interessante della slide: il RAID 3 tollera quanto il RAID 2, ma con **tre bit di ridondanza in meno per parola**.

La ragione è che un codice di Hamming spende quei tre bit per *localizzare* l'errore, cioè per sapere *quale* bit è sbagliato. Ma in un array di dischi il “quale” si sa già: se un disco si guasta, il controller se ne accorge e sa quale è. Una volta che si sa dov'è il buco, un solo bit di parità basta a riempirlo — è il calcolo “la parità di tutto il gruppo deve essere zero, quindi il bit mancante è la parità di tutti gli altri” che si è verificato eseguendo.

Il RAID 2 paga un bit in più per un'informazione che nel contesto dei dischi è *già disponibile*: la posizione del guasto. È il motivo per cui il RAID 3 lo sostituisce di fatto, pur essendo concettualmente più povero.

## 19.7 RAID 4: striping a blocchi con XOR sull'ultimo disco

### RAID 4

[SO-04 p22]

- basato sullo **striping a blocchi**;
- **disco extra = XOR degli strip**;
- **non necessita sincronia**;

- *ottima fault tolerance*;
- **aggiornamento lento in caso di modifica di un blocco**, con l'ottimizzazione: il nuovo blocco di parità si può calcolare dal blocco sovrascritto e dal vecchio blocco di parità.

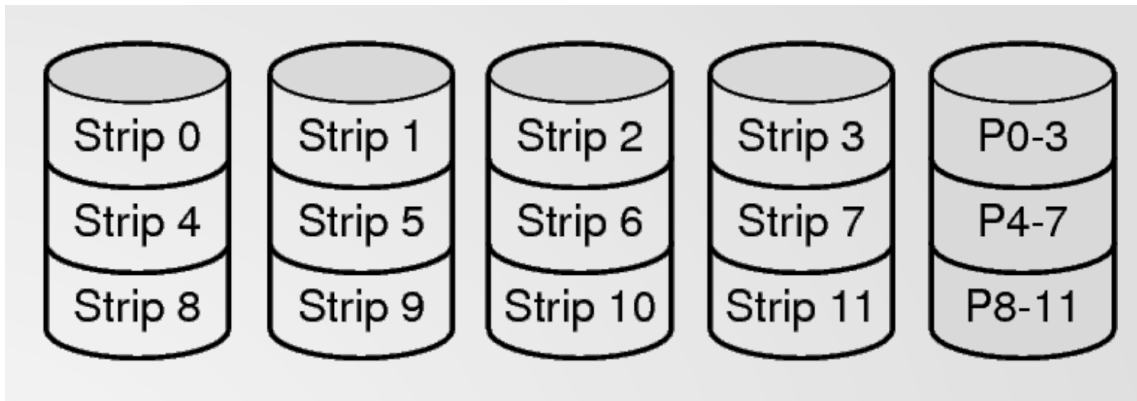


Figura 19.6: RAID 4. Lo striping è a livello di **blocchi** (non di bit), quindi un blocco intero sta su un disco solo, e la parità di ciascun gruppo sta raccolta sull'ultimo disco:  $P_{0-3}$ ,  $P_{4-7}$ ,  $P_{8-11}$

Fonte: proiezioni del corso, SO-04 p22

#### “Non necessita sincronia”: che cosa si guadagna

È l'altra faccia di ciò che si è detto per il RAID 2. Qui un blocco intero sta **su un disco solo**: per leggerlo basta quel disco, e non serve che gli altri siano allineati. I dischi possono quindi girare in modo indipendente, e — soprattutto — **richieste diverse si possono servire in parallelo** su dischi diversi.

Il guadagno non è solo di velocità massima: è che l'array si comporta come  $n$  dischi indipendenti per le richieste piccole, e come un disco solo grande per le letture lunghe. È esattamente ciò che serve a un file system, che fa entrambe le cose di continuo.

#### L'aggiornamento lento, e la scorciatoia

È il passaggio più “da esame” del capitolo, perché è un ragionamento su un costo.

Il modo ovvio di aggiornare un blocco in RAID 4 è: scrivo il blocco sul suo disco, poi **rileggo tutti i blocchi del gruppo** e ricalcolo la parità. Detto  $k$  il numero di blocchi del gruppo, sono  $k$  letture *più* la scrittura. Con gruppi grandi, quasi tutto il lavoro è rifare la parità.

La scorciatoia sfrutta il fatto che si è modificato **un blocco solo**. Si chiamino  $V$  il vecchio blocco,  $N$  il nuovo e  $P$  la vecchia parità. Vale

$$P_{\text{nuova}} = P \oplus V \oplus N$$

cioè: la parità nuova si ottiene dalla **vecchia parità**, dal **blocco vecchio** e dal **blocco nuovo** — tre valori, tutti già disponibili o che si stanno comunque scrivendo. Niente rilettura degli altri blocchi del gruppo.

Il costo passa da  $k$  letture a **due** (la vecchia parità e il blocco vecchio), e non dipende più da quanto è grande il gruppo. Nella verifica (§19.10) l'identità è controllata eseguendo: il ricalcolo completo e la scorciatoia danno lo stesso risultato.

| modo ovvio                                               | scorciatoia                                                                  |
|----------------------------------------------------------|------------------------------------------------------------------------------|
| rileggere tutti i $k$ blocchi del gruppo e rifare la XOR | leggere il blocco vecchio e la vecchia parità, e combinarli col blocco nuovo |
| costo: $k$ letture + 2 scritture                         | costo: 2 letture + 2 scritture                                               |
| indipendente da $k$ ? <b>no</b>                          | indipendente da $k$ ? <b>sì</b>                                              |

## 19.8 RAID 5: parità distribuita

### RAID 5

[SO-04 p22]

- i blocchi di parità del RAID 4 vengono distribuiti su tutti i dischi;
- di fatto sostituisce il RAID 4.

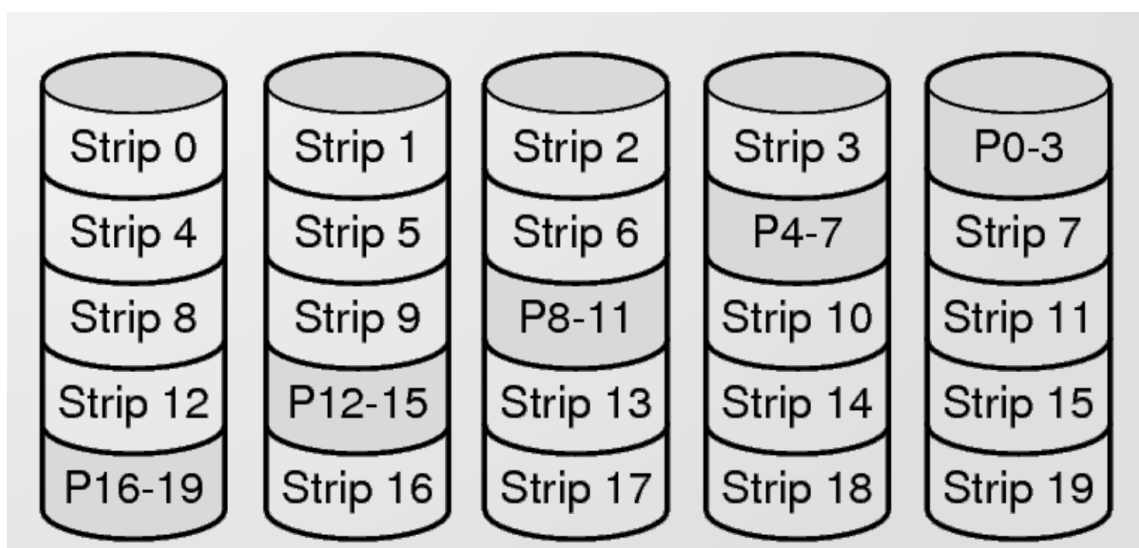


Figura 19.7: RAID 5. Gli strip sono tanti quanti nel RAID 4, ma la parità non sta più su un disco dedicato:  $P_{0-3}$  sta sull'ultima colonna,  $P_{4-7}$  sulla penultima,  $P_{8-11}$  sulla terzultima, e così via, ruotando

Fonte: proiezioni del corso, SO-04 p22

### Il difetto del RAID 4, che il RAID 5 elimina

Nel RAID 4 c'è un disco, l'ultimo, dove finisce **ogni** scrittura: perché ogni volta che si modifica un blocco bisogna aggiornare la parità, e quella parità sta sempre lì.

Il risultato è che i dischi dei dati vengono usati a rotazione, mentre **il disco di parità viene scritto a ogni operazione**. Diventa il collo di bottiglia dell'intero array: per quanto si aggiungano dischi ai dati, il numero di scritture al secondo non supera quello che quel disco solo riesce a fare.

Il RAID 5 risolve **spostando** la parità: invece di stare sempre sull'ultimo disco, i blocchi di parità *ruotano* su tutti. La prima parità sull'ultimo disco, la seconda sul penultimo, la terza sul terzultimo, e così via. Nessun disco è più quello delle scritture obbligate: il carico di parità si distribuisce come quello dei dati.

È per questo che la slide dice che il RAID 5 “di fatto sostituisce il RAID 4”: a parità di



spazio occupato — sempre un disco di ridondanza per gruppo — elimina il difetto peggiore del RAID 4. Il RAID 4 resta utile solo dove la parità su un disco separato è un vantaggio, cosa rara.

## 19.9 Riepilogo dei livelli

| livello | come organizza i dati                                                                  | prestazioni                                       | affidabilità e costo                                                   |
|---------|----------------------------------------------------------------------------------------|---------------------------------------------------|------------------------------------------------------------------------|
| 0       | striping round-robin, nessuna ridondanza                                               | ottime con letture di grandi volumi               | <b>nessuna tolleranza</b> ai guasti; capacità tutta utile              |
| 1       | dati duplicati su due dischi (mirroring), anche senza striping                         | <b>raddoppio in lettura</b> ; scrittura invariata | ottima; capacità utile <b>metà</b> del totale                          |
| 2       | striping <b>a bit</b> , con codice ECC (Hamming); esempio 4 bit dati + 3 di ridondanza | serve sincronia: l'array è un disco solo, veloce  | ottima, corregge anche senza sapere dov'è l'errore; ridondanza costosa |
| 3       | striping <b>a bit</b> , parità su <b>un disco dedicato</b>                             | serve ancora sincronia                            | come il RAID 2, ma con un solo bit di ridondanza per parola            |
| 4       | striping <b>a blocchi</b> , parità su <b>un disco dedicato</b>                         | <b>niente sincronia</b> : richieste in parallelo  | ottima; il disco di parità è il collo di bottiglia in scrittura        |
| 5       | striping a blocchi, <b>parità distribuita</b> su tutti i dischi                        | come il RAID 4, senza il collo di bottiglia       | come il RAID 4; <b>sostituisce il RAID 4</b>                           |

### La riga da ricordare

Al di là dei dettagli, la logica del capitolo sta in tre passi:

1. **lo striping** compra prestazioni e vende affidabilità;
2. **la ridondanza** ricompra l'affidabilità, e si paga in spazio: con la parità si paga *un disco per gruppo*; con il mirroring si paga *la metà*;
3. la scelta fra i livelli è la scelta di **dove mettere la parità** — nessun disco (0), su tutto (1), su un disco dedicato (4), distribuita (5) — e di **quanto fine** è lo striping — a bit (2, 3) o a blocchi (4, 5). Il primo decide il collo di bottiglia, il secondo decide se serve la sincronia.

## 19.10 La verifica

I due calcoli su cui poggia il capitolo — la parità XOR e il codice di Hamming dell'esempio — sono stati eseguiti, non riletti. Il codice è in `verifica_esercizi.py`, sezione "RAID".

### Che cosa è stato verificato, e che cosa dice

1. **la parità è una XOR**. Presi quattro strip  $1010_2$ ,  $0110_2$ ,  $1100_2$ ,  $1111_2$ , la loro parità vale  $1111_2$ , e la XOR di tutto (strip più parità) vale **0**. È la proprietà su cui si regge il controllo di consistenza: se il gruppo torna a zero, non manca nulla;



2. **uno strip perso si ricostruisce.** Il programma ha provato a *cancellare* ciascuno dei quattro strip, uno per volta, e a ricostruirlo dalla parità e dagli altri tre. In tutti e quattro i casi il valore ricostruito è identico a quello perso. È la verifica che “la parità recupera il dato” non è una formula imparata a memoria;
3. **il codice di Hamming corregge davvero.** Il programma ha costruito il codice  $4 + 3$  dell’esempio della slide, ha *guastato* uno per uno ciascuno dei sette bit di ciascuna delle 16 parole possibili, e ha controllato che la sindrome indicasse ogni volta la posizione giusta e che il bit corretto riportasse alla parola di partenza. Risultato: **112 errori su 112 corretti** ( $16 \text{ parole} \times 7 \text{ bit}$ ). La “ottima *fault tolerance*” della slide è quindi verificata sul suo stesso esempio;
4. **la scorciatoia del RAID 4 funziona.** Con la parità vecchia  $1011_2$ , il blocco vecchio  $1011_2$  e il blocco nuovo  $1110_2$ : il ricalcolo completo dà  $1110_2$ , e la scorciatoia  $P \oplus V \oplus N$  dà  $1110_2$ . I due valori **coincidono**, quindi si possono davvero risparmiare le  $k - 1$  riletture.

#### Come lo dici all’orale

“Mi parli dei sistemi RAID.” La traccia:

1. il **problema**: le prestazioni di un disco solo sono limitate dalla meccanica, quindi si sfrutta il parallelismo con più dischi indipendenti. Si suddivide l’unità logica in strip distribuiti a rotazione: **striping**, trasparente all’utente;
2. il **prezzo**: lo striping aumenta la probabilità di guasto del volume, perché basta un disco solo. Da qui la **ridondanza**, e i **dischi spare** per la sostituzione automatica;
3. i **livelli**: 0 striping puro, 1 mirroring, 2 striping a bit con ECC, 3 striping a bit con parità, 4 striping a blocchi con parità dedicata, 5 parità distribuita;
4. le **due domande che li distinguono**: lo striping è **a bit** (2 e 3, serve sincronia, l’array è un disco solo) o **a blocchi** (4 e 5, richieste in parallelo)? E la parità è **su un disco dedicato** (4) o **distribuita** (5)?

I due dettagli che fanno la differenza: spiegare che il RAID 3 usa un bit di ridondanza in meno del RAID 2 *perché nei dischi la posizione del guasto si conosce già*, e che il RAID 5 nasce per togliere il collo di bottiglia del disco di parità del RAID 4.

Se ti chiedono “quanto costa il RAID 1”, la risposta è: metà della capacità, ed è il prezzo più alto di tutti i livelli.

## Capitolo 20

### Memorie flash e dischi a stato solido

Tutto ciò che si è detto dal capitolo 15 in poi poggia su un presupposto preciso: che il supporto sia un **disco magnetico**, cioè un oggetto che gira, con una testina che si sposta e settori che passano. Da quel presupposto discendono il seek-time, la latenza di rotazione, gli algoritmi di scheduling del capitolo 18, il RAID del capitolo 19.

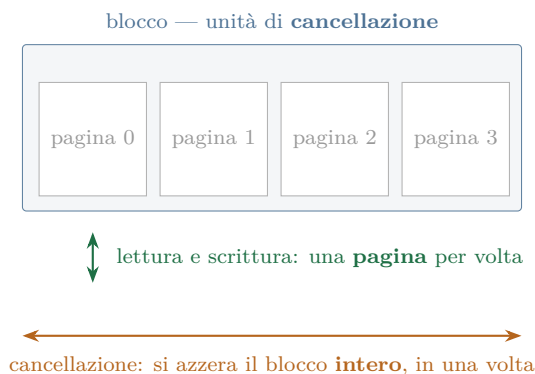
Questo capitolo finale descrive che cosa cambia quando il presupposto viene a mancare. Un **disco a stato solido** non ha né testina né piatti: è memoria flash. E la cosa interessante non è che sia “più veloce” — lo è, ma è un dettaglio — bensì che **non funziona come una memoria**: non si può riscrivere un byte dovunque si voglia. È una limitazione che obbliga a ripensare ciò che si è costruito nei capitoli precedenti.

#### 20.1 Che cosa sono

##### Dispositivi a stato solido

I dispositivi **SSD** sono basati su **memorie flash** (tecnologia **NAND**). [SO-04 p23]

- le **letture** sono **molto più veloci delle scritture**;
- un **blocco** si deve **cancellare prima di poter essere riscritto**;
- il **numero di cancellature** è **limitato** per ogni blocco;
- un **blocco** (unità di *cancellazione*) è composto da **pagine** (unità di *allocazione*);
- le **letture e le scritture** sono basate su **pagine**.



##### Schema, non figura delle slide

Il disegno qui sopra è un **schemi di servizio** della guida, non una figura delle proiezioni del corso: serve solo a fissare i due livelli di cui parla la slide. Non ha una fonte da citare, e va letto come una riscrittura di ciò che la slide dice a parole.

##### La gerarchia blocco/pagina, e perché è asimmetrica

Le due righe che contano sono queste: **si legge e si scrive per pagine, si cancella per blocchi**, e ogni blocco contiene più pagine. È un'asimmetria, ed è la radice di tutte le complicazioni di questo capitolo.

Il motivo è fisico e sta nel funzionamento della cella flash: scrivere una cella significa *caricarle*

una carica elettrica, e per rimetterla a zero bisogna scaricarla. La scarica non si può fare cella per cella, perché le celle di un blocco condividono il collegamento elettrico che le azzerava. Quindi:

- **scrivere** una pagina già scritta non si può: si può solo scrivere una pagina **già cancellata**;
- per modificare una pagina già scritta bisogna **cancellare l'intero blocco** che la contiene, e ricopiare dentro tutte le pagine che servono ancora;
- ogni ciclo di cancellatura **consuma** il blocco. Non è una questione di tempo: dopo un numero finito di cancellature il blocco non tiene più la carica, e la cella non è più affidabile. È l'usura di cui si parla quando si dice che gli SSD “hanno una vita”.

Ne segue una conseguenza che a prima vista sorprende: su un SSD, come dice la slide, **le letture sono molto più veloci delle scritture**. Non è un fatto di elettronica — l'elettronica è la stessa — ma del fatto che una scrittura può dover *innescare* la cancellazione e la ricopiatura di un blocco intero.

## 20.2 Perché i file system classici non vanno bene

### Che cosa serve

[SO-04 p23]

- i **file system classici** sono basati su **principi diversi**;
- servono file system **ad-hoc**: *Flash-Friendly File System (F2FS)*, *log-based fs*, ...;
- oppure un **controller** che rimappa i blocchi tramite un **Flash Translation Layer**.

### Quali “principi diversi”: la lettura dei capitoli 16 e 17

È il punto in cui questo capitolo incontra tutti i precedenti, e vale la pena vedere *esattamente* quali scelte entrano in conflitto.

Riprendi il capitolo 16. La **FAT** e gli **i-node** — e, in forma più blanda, l'allocazione contigua — sono *aggiornati in place*: si modifica una voce della tabella sulla quale si è appena scritto. Su un disco magnetico questo è il modo giusto: il blocco si riscrive lì dove sta. Su una memoria flash quell'aggiornamento è la cosa peggiore possibile, perché **il file system ha delle zone calde** — la tabella FAT, l'area degli i-node, la bitmap dei blocchi liberi, il superblocco — riscritte a ogni operazione. Sono esattamente le zone che finirebbero per esaurire le cancellature per prime, uccidendo il dispositivo in un punto solo.

Lo stesso vale per la **bitmap dei blocchi liberi** e per la **lista concatenata** del paragrafo sulla gestione dei blocchi liberi: entrambe richiedono riscritture frequenti in posizioni fisse. E c'è il problema della **journaling** del capitolo 17: il journal è, per definizione, un'area riscritta di continuo. Nasce per proteggere da un guasto a metà scrittura, e su flash diventa a sua volta un'area usurata.

Le due soluzioni elencate dalla slide attaccano il problema da due lati diversi:

- il **Flash Translation Layer** dentro il controller **nasconde** il problema al file system: il sistema operativo crede di scrivere il blocco 100 al suo posto, e il controller scrive invece da un'altra parte, rimappando la corrispondenza. È la stessa indirazione della paginazione (capitolo 12), applicata un livello più in basso: un *indirizzo logico* che non coincide con quello fisico;

- i file system **ad-hoc** come **F2FS** prendono atto della struttura e la sfruttano: **log-based**, cioè scrivono i dati in *append* invece di riscrivere sul posto. Il capitolo 17 ha già incontrato questa idea — il journaling è un file system log-based in piccolo — e qui diventa il principio organizzativo dell'intero file system.

## 20.3 Garbage collection e TRIM

### Le due operazioni che tengono in vita un SSD

[SO-04 p23]

- **garbage collection** e **operazione TRIM**:
- c'è un **degrado delle prestazioni nel tempo** se non vengono impiegate;
- richiedono un **adeguamento da parte del sistema operativo**.

### Il garbage collector: che cosa raccoglie, e perché non basta

Il **garbage collector** del controller esiste per risolvere un problema che il Flash Translation Layer crea da sé.

Aggiornare una pagina, si è detto, non si può: la si scrive *altrove*, e la vecchia copia viene marcata come non più valida. Dopo un po' il dispositivo si riempie di pagine "morte" sparse in blocchi altrimenti vivi. Non si possono cancellare, perché la cancellazione è per blocco intero e nel blocco ci sono anche pagine buone.

Il **garbage collector** fa allora questo: prende un blocco, **copia** le pagine ancora valide in un blocco libero, e infine **cancella** il blocco intero, restituendo spazio utilizzabile. È la stessa operazione che la slide descrive come "un blocco si deve cancellare prima di poter essere riscritto", vista dall'altro lato: per poter cancellare bisogna prima *liberare* il blocco.

Il costo si paga in prestazioni: copiare pagine valide è lavoro che non serve a nessuno se non a tenere in ordine la memoria. E il degrado di cui parla la slide nasce proprio da qui: più il dispositivo si riempie, più il garbage collector deve copiare per liberare posto, e più il tempo di scrittura percepito cresce. Su un dispositivo quasi pieno questa crescita può essere drammatica.

### TRIM: che cosa deve fare il sistema operativo

Il **garbage collector** da solo non basta, perché c'è una informazione che gli manca, ed è un'informazione che solo il sistema operativo possiede: **quali pagine non servono più**.

Si consideri un file cancellato. Il file system aggiorna le sue strutture, e per esso quei blocchi sono liberi. Ma il controller **non lo sa**: vede solo che quelle pagine non sono state sovrascritte, e le considera ancora valide. Al momento della garbage collection le copierà pazientemente in un blocco nuovo — copiando dati che nessuno vuole — e consumerà scritture e blocchi per farlo.

L'operazione **TRIM** è il messaggio con cui il sistema operativo avverte il controller: *queste pagine non servono più*. Da quel momento il controller sa di poterle buttare via senza copiarle, e la garbage collection diventa molto più economica.

È il senso dell'ultima riga della slide, "richiede adeguamento da parte del S.O.": TRIM non è una cosa che il dispositivo può fare da solo, perché il dispositivo **non sa** che cosa sia un file. È il file system a saperlo, ed è per questo che servono sia il comando TRIM nel sistema operativo sia il supporto nel controller. Senza di esso l'SSD continua a funzionare, ma invecchia prima e rallenta prima — e su questo la slide è esplicita.

### Il filo che lega i tre capitoli finali

Vale la pena rileggere il percorso, perché ha una direzione:

- il **capitolo 18** accetta il disco così com'è — meccanico, lento, con seek-time — e cerca di *non sprecare* il movimento, riordinando le richieste;
- il **capitolo 19** accetta il singolo disco e ne mette *molti*, comprando prestazioni col parallelismo e pagando la ridondanza per riavere l'affidabilità;
- il **capitolo 20** non accetta il supporto: toglie la meccanica di mezzo e si trova un problema nuovo — non più “come muovere la testina”, ma “come scrivere su una memoria che non si riscrive”.

La cosa da notare è che il terzo capitolo **non cancella** i due precedenti: li rende inutili *per quel supporto*. Gli algoritmi di scheduling del disco non hanno senso su un SSD, dove non c'è nessuna testina da spostare; il RAID invece sopravvive benissimo, anzi è più semplice, perché non ci sono testine da sincronizzare. Sapere *quale* parte di ciò che si è studiato dipende dalla meccanica e quale no: è questa, più che l'elenco delle sigle, la cosa che l'orale vuole sentire.

### Come lo dici all'orale

“Mi parli degli SSD.” La traccia:

1. che cosa sono: dispositivi basati su **memoria flash NAND**. Le tre proprietà che contano: letture molto più veloci delle scritture; un blocco va **cancellato prima** di essere riscritto; il numero di cancellature è **limitato**. E la gerarchia: il **blocco** è l'unità di cancellazione, la **pagina** quella di lettura e scrittura;
2. perché i file system classici non vanno bene: le FAT e gli i-node sono **aggiornati sul posto**, e quelle sono le zone calde. Le due strade sono il **Flash Translation Layer** nel controller, che rimappa e nasconde il problema, oppure un file system **ad-hoc** e **log-based** come **F2FS**;
3. **garbage collection** e **TRIM**: la prima copia le pagine valide e cancella il blocco; il secondo è il messaggio con cui il sistema operativo dice al controller quali pagine sono libere. Senza TRIM le prestazioni degradano nel tempo.

Il dettaglio che fa la differenza: dire dove si rompe l'analogia con il disco. Un SSD **non** è “un disco veloce”: è una memoria con regole di scrittura diverse. Per questo lo scheduling del disco perde senso (non c'è seek-time) mentre il RAID sopravvive (e non serve più la sincronia fra le rotazioni, che era il vincolo dei livelli 2 e 3).

## Parte VII

# Esercizi e colloquio orale

## Capitolo 21

### Il compito tipo svolto

---

Il corso pubblica **un solo** esempio di prova di teoria. Si chiama “*Stralcio di un compito d’esame*”, sta nella pagina del docente, ed è il documento su cui è costruito questo capitolo: le sette domande che seguono sono le sue, riportate come sono scritte lì, e ne sono l’unica fonte.

#### Che cos’è, esattamente, questa fonte

Tre cose vanno dette subito, perché riguardano l’attendibilità di tutto il capitolo.

- È uno **stralcio**, cioè un estratto: non è una prova completa, e non sappiamo quante domande avesse l’esame vero né quanto valessero. Sappiamo però con certezza *come sono fatte* le domande, ed è quello che serve;
- porta un **codice**: 20110524-01-7509-3. Il prefisso 20110524 è una data, 24 maggio 2011, e quel codice serviva allo studente per accedere alla propria valutazione. È quindi un compito vecchio, ripubblicato come esempio: i numeri sono quelli, e non sono stati aggiornati;
- nella domanda 6 compare nel testo originale la parola **[duplicato]**. È un residuo dell’impaginazione del docente, non parte della domanda: qui è omesso.

Le domande sono **sette**: due a risposta multipla e cinque di calcolo. Il capitolo le svolge tutte, con lo stesso ordine e la stessa numerazione del documento originale. Ogni risultato numerico è stato prodotto da uno script e verificato eseguendolo; quando un conto merita attenzione perché si presta a un errore, il riquadro apposito lo dice.

#### 21.1 Domanda 1 — l’aggiornamento del contatore di Aging

##### Esercizio 1 — L’algoritmo di Aging (compito tipo, domanda 1 — risposta multipla)

**Domanda reale:** Testo della domanda ripreso dallo *Stralcio di un compito d’esame* pubblicato nella pagina del corso ([compito-tipo.pdf](#)).

L’algoritmo di Aging per la sostituzione delle pagine prevede di mantenere un contatore  $C$  associato ad ogni pagina caricata in memoria. Tale contatore viene consultato nel momento in cui si deve scegliere quale pagina rimuovere dalla memoria: viene scelta quella con il contatore più basso.

Indicare esattamente qual è l’aggiornamento periodico che viene effettuato su tale contatore.

- A. Somma del bit di referenziamento  $R$  al contatore  $C$ , con seguente shift a sinistra.
- B. Shift a sinistra di  $C$  e somma del bit di referenziamento  $R$ .
- C. Shift a sinistra di  $C$  ed inserimento del bit di referenziamento  $R$  come bit più significativo.
- D. Shift a destra di  $C$  ed inserimento del bit di referenziamento  $R$  come bit più significativo.
- E. Shift a destra di  $C$  ed inserimento del bit di modifica  $M$  come bit meno significativo.

**Svolgimento.** La risposta è la **D**. La regola è enunciata in due righe nella slide dedicata:

### Algoritmo di Aging

[SO-03 p25] Ad ogni scadenza del clock:

- **shift a destra** del contatore associato ad ogni pagina;
- **accostamento a sinistra** (come bit più significativo) del bit  $R$ .

Le due opzioni che contengono entrambe le metà della regola sono la **C** e la **D**, e si distinguono solo per il *verso dello shift*: a sinistra nella C, a destra nella D. Tutte le altre sbagliano almeno una delle due metà — la A e la B fanno lo shift a sinistra, la E usa il bit di *modifica*  $M$  al posto del bit di *referenziamento*  $R$ , che nella Aging non c'entra nulla.

Il motivo per cui il verso è quello e non l'altro si capisce guardando che cosa deve fare l'algoritmo: il contatore è una memoria *che invecchia*. Ad ogni battito di clock il passato deve pesare un po' meno, e il verso dello shift è esattamente il modo in cui si ottiene questo effetto: spostando i bit a destra il valore viene dimezzato, quindi un riferimento vecchio di  $k$  battiti vale  $2^{-k}$  volte un riferimento recente.

Con lo shift a sinistra succede l'opposto: il valore *raddoppia* ad ogni battito e le informazioni vecchie si accumulano invece di sbiadire. Peggio: il bit appena inserito finisce per pesare meno di tutti, e siccome la pagina da eliminare è quella con il contatore *più basso*, l'algoritmo finirebbe per eliminare la pagina usata per ultima — il contrario di quello che deve fare.

### La prova che il verso conta, fatta eseguendo

Un'affermazione come quella qui sopra si può anche verificare invece di argomentarla, ed è quello che ho fatto. Ho implementato l'Aging tre volte, cambiando solo la riga dell'aggiornamento, e l'ho fatto girare sulla stessa sequenza di riferimenti che le slide usano per confrontare gli algoritmi di sostituzione (7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1, con 3 frame):

| Regola di aggiornamento                              | Fault di pagina |
|------------------------------------------------------|-----------------|
| LRU (slide SO-03 p28)                                | 12              |
| shift a destra, $R$ come bit più significativo (D)   | <b>12</b>       |
| shift a sinistra, $R$ come bit meno significativo    | 14              |
| shift a sinistra, $R$ come bit più significativo (C) | 13              |

La regola della slide, e solo quella, riproduce *esattamente* il comportamento dell'LRU: 12 fault, come l'LRU. Le altre due se ne allontanano, e la variante con lo shift a sinistra fa due fault in più proprio dove l'LRU ne fa uno in meno.

Il secondo controllo è più diretto. Dopo otto riferimenti (0, 1, 2, 3, 0, 1, 2, 3) su quattro pagine caricate, i contatori valgono:

| Regola                    | Contatori                         | Conseguenza                           |
|---------------------------|-----------------------------------|---------------------------------------|
| destra, $R$ in cima (D)   | $C_0=17, C_1=34, C_2=68, C_3=136$ | elimina la più vecchia                |
| sinistra, $R$ in fondo    | $C_0=136, C_1=68, C_2=34, C_3=17$ | elimina la pagina 3, <b>l'ultima</b>  |
| sinistra, $R$ in cima (C) | $C_0=0, C_1=0, C_2=0, C_3=128$    | pagine 0, 1, 2 <b>indistinguibili</b> |

Si legga la seconda riga: con lo shift a sinistra il contatore più basso è quello della pagina appena riferita — e siccome si elimina la pagina col contatore più basso, l'algoritmo eliminerebbe proprio quella. È l'opposto esatto dell'obiettivo dichiarato dell'LRU, di cui l'Aging deve essere una approssimazione. La terza riga mostra un guasto diverso ma altrettanto grave: il contatore azzerava tutte le pagine non riferite nell'ultimo battito, quindi l'algoritmo perde la storia che serviva a distinguerle.



**L’errore in cui si cade**

Il candidato che non ricorda la regola tende a scegliere la **C**, perché “shift e inserimento del bit più significativo” è la parte che si ricorda meglio, e il verso dello shift sembra un dettaglio. Non lo è: è l’unica cosa che distingue la risposta giusta da quella sbagliata. La regola da mandare a memoria è una frase sola: *il contatore si sposta a destra e il bit R entra in cima*.

**21.2 Domanda 2 — l’affermazione falsa sull’allocazione dei file****Esercizio 2 — Allocazione dei file** (compito tipo, domanda 2 — risposta multipla)

**Domanda reale:** Testo della domanda ripreso dallo *Stralcio di un compito d’esame* pubblicato nella pagina del corso ([compito-tipo.pdf](#)).

Con riferimento alle tecniche che abbiamo visto per memorizzare il contenuto dei file sui blocchi del disco e di come il file system ne tenga traccia, individuare quale tra le seguenti affermazioni è **falsa**.

- A. Nell’allocazione contigua è necessario conoscere a priori la dimensione massima del file in fase di creazione.
- B. Nell’allocazione concatenata (con liste collegate) è presente una certa perdita di spazio dovuto alla frammentazione interna.
- C. L’allocazione contigua è la soluzione che richiede meno memoria RAM ed il minor numero di accessi al disco per determinare il blocco in cui è memorizzato un arbitrario contenuto all’interno di un file.
- D. Usando una FAT per tenere traccia dei blocchi dei file non è necessario mantenere una ulteriore bitmap per tenere traccia dei blocchi liberi.
- E. Nell’allocazione che fa uso della tabella di allocazione dei file (FAT) la capacità del singolo blocco su disco può essere solo parzialmente sfruttata per memorizzare i contenuti del file, dovendo memorizzare il numero del blocco successivo.

**Svolgimento.** La risposta è la **E**.

Il modo giusto di affrontare questa domanda non è cercare l’affermazione che “suona male”, ma **verificarne una per una** contro i quattro schemi di allocazione del capitolo 16. Le opzioni sono scritte in modo da sembrare tutte plausibili, e tre di esse sono effettivamente vere.

|   | Opzione | Verifica                                                                                                                                                                           |
|---|---------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| A | vera    | È il difetto classico dell'allocazione contigua: lo spazio si prenota in blocco, quindi la dimensione va decisa alla creazione (capitolo 16).                                      |
| B | vera    | In una catena di blocchi l'ultimo blocco è occupato solo in parte, ed è spazio sprecato <i>dentro</i> un blocco assegnato.                                                         |
| C | vera    | Contigua significa che il blocco si calcola con una divisione, senza consultare nulla: nessuna tabella da tenere in RAM e un solo accesso al disco. È il suo vantaggio principale. |
| D | vera    | Nella FAT la voce di un blocco libero vale 0: l'informazione sta già nella tabella, e una bitmap separata sarebbe una seconda copia della stessa cosa (capitolo 16).               |
| E | falsa   | Nella FAT i puntatori stanno <b>nelle voci della tabella</b> , che vive in RAM. I blocchi su disco contengono <i>solo</i> dati del file: nessun byte viene loro sottratto.         |

#### Perché la E è falsa, e perché è la trappola meglio costruita

La frase dell'opzione E descrive una cosa vera — ma di un *altro* schema di allocazione. È la descrizione esatta dell'allocazione **concatenata** (con liste collegate), dove il puntatore al blocco successivo sta dentro il blocco stesso e quindi gli toglie spazio. Il capitolo 16 lo dice esplicitamente: in quel caso “i byte del puntatore non sono più disponibili per i dati”.

La FAT nasce precisamente per **togliere di mezzo quel difetto**: le slide del corso la presentano come “allocazione con liste collegate *su una tabella* di allocazione dei file” [SO-04 p5]. La lista c'è ancora, ma è una lista di *indici*, e vive in memoria invece che nei blocchi. Chi confonde i due schemi sceglie la E, o non sceglie affatto.

Il modo rapido di vederlo è chiedersi, per ogni opzione: *che cosa sta dentro il blocco su disco?* Nell'allocazione concatenata: dati e puntatore. Nella FAT: solo dati. Nella contigua: solo dati. Negli i-node: solo dati, con i puntatori nell'i-node. Solo una delle quattro ha il puntatore dentro il blocco, e l'opzione E lo attribuisce alla FAT.

#### Sull'opzione B, che è quella che mette in dubbio

Guardando questa domanda capita di fermarsi sulla B: “frammentazione interna” è un termine che nel capitolo 11 indica una cosa precisa, e in una catena di blocchi la perdita di spazio è di due tipi diversi — il puntatore che ruba byte in *ogni* blocco, e l'ultimo blocco riempito solo in parte.

Vale la pena saperlo, perché il dubbio non è infondato. Il testo da cui le slides del corso prendono le figure usa proprio questa parola per la concatenata: dice che in quello schema ogni blocco del disco può essere usato, e che *nessuno spazio si perde per frammentazione del disco, tranne la frammentazione interna nell'ultimo blocco*. La B è quindi la traduzione di una frase del libro, ed è **vera**.

Il criterio per sciogliere il dubbio, comunque, non è discutere la B in astratto: è usare il fatto che la domanda chiede *quale* è falsa, e una sola. La E è falsa in modo netto e verificabile; la B è vera, per quanto formulata in modo impreciso. Davanti a un'opzione “chiaramente falsa” e a una “forse imprecisa”, la risposta è la prima.

**Come si è arrivati a questa risposta**

Vale la pena dirlo, perché riguarda tutto il capitolo: il corso **non pubblica le soluzioni** del compito tipo. La risposta a questa domanda, e a tutte le altre di questo capitolo, è ricostruita **dalle fonti** — le slides, il testo da cui le slides prendono le figure, e il capitolo 16 di questa guida — non copiata da una chiave di correzione, che non esiste.

Per la E la prova è diretta: la figura della FAT [SO-04 p5] mostra i numeri dei blocchi successivi incolonnati *nella tabella* ( $4 \rightarrow 7 \rightarrow 2 \rightarrow 10 \rightarrow 12$ ), non dentro i blocchi. Per la B la prova è la frase del testo citata qui sopra: l'autore chiama “frammentazione interna” quella dell'ultimo blocco, e la domanda usa la sua parola.

**21.3 Domanda 3 — quanti frame fisici****Esercizio 3 — Memoria virtuale** (compito tipo, domanda 3)

**Domanda reale:** Testo della domanda ripreso dallo *Stralcio di un compito d'esame* pubblicato nella pagina del corso ([compito-tipo.pdf](#)).

Consideriamo un sistema che fa uso di memoria virtuale con le seguenti caratteristiche: uno spazio di indirizzamento virtuale da 1 Gb, un numero di pagina virtuale a 22 bit e un indirizzo fisico a 20 bit. Determinare esattamente quanti frame fisici ci sono in memoria.

**Svolgimento.** La risposta è **4096 frame**.

L'esercizio si risolve tutto con le potenze di due, e il modo di impostarlo è scrivere per esteso i pezzi dell'indirizzo. Il numero di pagina virtuale (VPN) occupa 22 bit, e lo spazio virtuale è da 1 Gb:

| Grandezza                   | Conto                   | Valore      |
|-----------------------------|-------------------------|-------------|
| bit dell'indirizzo virtuale | $1 \text{ Gb} = 2^{30}$ | 30          |
| bit di VPN                  | dato                    | 22          |
| bit di offset nella pagina  | $30 - 22$               | 8           |
| dimensione della pagina     | $2^8$                   | 256 byte    |
| bit dell'indirizzo fisico   | dato                    | 20          |
| bit del numero di frame     | $20 - 8$                | 12          |
| <b>numero di frame</b>      | $2^{12}$                | <b>4096</b> |

Il procedimento ha sempre questa forma: l'**offset dentro la pagina** è il pezzo che le due parti hanno in comune — la pagina virtuale e il frame fisico sono grandi uguali, altrimenti la traduzione non potrebbe limitarsi a cambiare il numero — quindi si ricava una volta sola e si sottrae da entrambe le lunghezze. Quello che avanza nell'indirizzo fisico è il numero di bit del frame, e il numero di frame è 2 elevato a quei bit.

Un controllo che vale la pena fare, e che conferma il risultato per una via del tutto diversa: i frame sono la memoria fisica divisa per la dimensione della pagina,

$$\frac{2^{20}}{2^8} = \frac{1\,048\,576}{256} = 4096.$$

Lo stesso conto, letto come “quante pagine ci stanno nella memoria fisica”.

**Due errori che si vedono spesso**

Il primo è partire dai **bit** invece che dai **byte**: 1 Gb è  $2^{30}$  byte, non  $2^{30}$  bit, e trattarlo come  $2^{30}$  bit porterebbe a una risposta sbagliata di un fattore otto. In questa materia “Gb”

significa quasi sempre  $2^{30}$  byte.

Il secondo è usare i 22 bit di VPN come se fossero i bit dell'indirizzo virtuale. Sono due cose diverse: l'indirizzo virtuale ha 30 bit, dei quali 22 indirizzano la pagina e 8 l'offset. Chi usa  $22 - 20$  o simili si accorge subito che il conto non torna.

## 21.4 Domanda 4 — la trace dei semafori

### Esercizio 4 — Tre processi e tre semafori (compito tipo, domanda 4)

**Domanda reale:** Testo della domanda ripreso dallo *Stralcio di un compito d'esame* pubblicato nella pagina del corso ([compito-tipo.pdf](#)).

Supponiamo di avere 3 processi che condividono una variabile  $x$  e che i loro pseudo-codici siano i seguenti:

|                                                                                          |                                                              |                                                                          |
|------------------------------------------------------------------------------------------|--------------------------------------------------------------|--------------------------------------------------------------------------|
| <b>P1:</b><br>wait(S)<br>$x = x - 2$<br>signal(T)<br>wait(S)<br>$x = x - 1$<br>signal(T) | <b>P2:</b><br>wait(R)<br>$x = x + 2$<br>signal(T)<br>wait(R) | <b>P3:</b><br>wait(T)<br>if ( $x < 0$ ) signal(R)<br>wait(T)<br>print(x) |
|------------------------------------------------------------------------------------------|--------------------------------------------------------------|--------------------------------------------------------------------------|

Determinare l'output del processo P3 assumendo che il valore iniziale di  $x$  è 1 e che i 3 semafori abbiano i seguenti valori iniziali:  $S = 1$ ,  $R = 0$ ,  $T = 0$ .

**Svolgimento.** P3 stampa **1**, e lo stampa in *ogni* esecuzione possibile. Non è una risposta probabile: è l'unico valore che può uscire.

Questi esercizi non si risolvono scegliendo un interleaving e seguendolo fino in fondo — quello produce una risposta, non la risposta. Si risolvono **esplorando tutti gli interleaving** e raccogliendo l'insieme dei valori che **print** può ricevere. Se l'insieme ha un solo elemento, l'output è determinato; se ne ha più di uno, la domanda va risolta dicendo quali sono e perché.

Qui l'insieme ha un solo elemento, e la ragione è una catena di conseguenze che vale la pena seguire per intero.

### Primo passo: quante volte si può passare da **signal(T)**

P3 ha bisogno di **due** **wait(T)** andati a buon fine prima di poter stampare. Ogni **wait(T)** consuma un **signal(T)**, e i **signal(T)** possibili sono soltanto due in tutto il programma:

- quello di **P1**, che segue  $x = x - 2$ ;
- quello di **P2**, che segue  $x = x + 2$ .

Guardiamo il resto di ciascun processo:

- **P1** ha un secondo **wait(S)**, ma  $S$  non viene *mai* segnalato da nessuno: il suo valore iniziale è 1, P1 lo consuma subito, e non esiste nel programma un **signal(S)**. Quindi il secondo **wait(S)** di P1 non si sblocca mai, e P1 non arriva mai al secondo **signal(T)**;
- **P2** ha un secondo **wait(R)**.  $R$  viene segnalato una volta sola, da P3, e nel punto in cui P2 lo aspetta la seconda volta non c'è più nessuno che possa segnalarlo di nuovo. Anche P2 si ferma lì.

Dunque i **signal(T)** sono esattamente due, e per forza entrambi: quello di P1 e quello di P2.

### Secondo passo: che valore ha $x$ quando P3 stampa

Che P1 e P2 eseguano entrambi le loro istruzioni su  $x$  non è una possibilità fra tante: è una condizione necessaria. Perché P3 stampi servono due `signal(T)`, e l'unico modo di averli è che P1 abbia eseguito  $x = x - 2$  e P2 abbia eseguito  $x = x + 2$ . Quindi:

$$x = 1 - 2 + 2 = 1.$$

Sempre. Qualunque sia l'ordine in cui le due cose avvengono, il risultato è lo stesso, perché sono una sottrazione e un'addizione sullo stesso valore iniziale.

C'è un punto delicato da controllare, ed è il `signal(R)` di P3: P3 lo esegue solo *se*  $x < 0$  nel momento in cui passa di lì. Se non lo eseguisse, P2 non partirebbe mai, non ci sarebbe il suo `signal(T)` e P3 non stamperebbe. Vediamo che invece lo esegue sempre: il primo `wait(T)` di P3 può essere soddisfatto solo dal `signal(T)` di P1, perché P2 è ancora fermo sul primo `wait(R)` in attesa di un `signal(R)` che solo P3 può dare. A quel punto P1 ha già eseguito  $x = x - 2$ , quindi  $x = 1 - 2 = -1 < 0$ , e il `signal(R)` viene eseguito. La catena si chiude: P3 sblocca P2, P2 produce il secondo `signal(T)`, P3 stampa.

### Terzo passo: chi resta bloccato

Il programma **non termina**: P1 resta fermo sul secondo `wait(S)` e P2 sul secondo `wait(R)`. Non è un errore della soluzione ed è anzi la ragione per cui l'esercizio è istruttivo: il valore viene stampato *prima* che i due processi si blocchino, e la domanda chiede l'output di P3, non la terminazione del sistema. In un esame vero, dire “stampa 1 e poi P1 e P2 restano bloccati” è una risposta migliore di “stampa 1” e basta.

#### Il metodo, in generale

Davanti a una trace di semafori, l'ordine dei passi è sempre questo:

1. contare quante volte ogni `signal` può essere eseguito — è il *budget* delle risorse disponibili;
2. guardare quante volte ciascun `wait` deve andare a buon fine perché il `print` venga raggiunto;
3. se i due conti coincidono, il cammino è **obbligato**: non serve esplorare nulla, basta seguirlo. È il caso di questo esercizio, dove i due `signal(T)` disponibili sono esattamente i due `wait(T)` necessari;
4. se i `signal` sono di più dei `wait` necessari, allora c'è davvero una scelta, e si esplorano i casi.

Il passo 3 è quello che fa risparmiare tempo: accorgersi che il cammino è obbligato trasforma un esercizio di esplorazione in una lettura.

#### La verifica eseguita

Il risultato qui sopra non è stato ottenuto seguendo a mano un interleaving, ma con un programma che li esplora tutti. Il programma riporta un solo valore stampabile, 1, e un solo insieme di processi bloccati,  $\{P1, P2\}$ . Se tu, svolgendo l'esercizio, trovassi un interleaving che fa stampare un valore diverso da 1, avresti trovato un errore nel programma di verifica: vale la pena provarci.

## 21.5 Domanda 5 — un offset dentro un file su FAT

### Esercizio 5 — FAT e offset (compito tipo, domanda 5)

**Domanda reale:** Testo della domanda ripreso dallo *Stralcio di un compito d'esame* pubblicato nella pagina del corso ([compito-tipo.pdf](#)).

Supponiamo di avere un file system che utilizza per tenere traccia dei file in esso memorizzati la seguente FAT, e che prevede le seguenti voci all'interno della cartella radice:

| FAT    |        |                 |              |
|--------|--------|-----------------|--------------|
| indice | valore | cartella radice |              |
| 1      | 4      | nome            | primo blocco |
| 2      | 3      | pippo.txt       | 7            |
| 3      | 15     | ...             | ...          |
| 4      | 5      |                 |              |
| 5      | 10     |                 |              |
| 6      | 12     |                 |              |
| 7      | 1      |                 |              |
| 8      | 2      |                 |              |
| 9      | 3      |                 |              |
| 10     | −1     |                 |              |
| ...    |        |                 |              |

Indicare esattamente in quale blocco del disco è localizzabile l'offset 10100 all'interno del file `pippo.txt`. Indicare qual è la dimensione minima presunta in byte dello stesso file. In tale calcolo tenere conto del fatto che un blocco del file system è grande 4 kB.

*Nota:* gli offset sono espressi in byte e partono da 0.

**Svolgimento.** L'offset 10100 cade nel **blocco 4** (catena  $7 \rightarrow 1 \rightarrow 4$ ), e la dimensione minima presunta del file è **12288 byte**, cioè 3 blocchi.

### Il metodo

Il primo passo è sempre lo stesso: **ricostruire la catena del file**. La cartella radice dice che `pippo.txt` comincia dal blocco 7; poi si segue la FAT finché non si incontra il valore  $-1$ , che significa “fine del file”.

| Passo            | Voce letta  | Significato                         |
|------------------|-------------|-------------------------------------|
| partenza         | indice 7    | primo blocco del file               |
| $7 \rightarrow$  | valore 1    | il blocco successivo è il <b>1</b>  |
| $1 \rightarrow$  | valore 4    | il blocco successivo è il <b>4</b>  |
| $4 \rightarrow$  | valore 5    | il blocco successivo è il <b>5</b>  |
| $5 \rightarrow$  | valore 10   | il blocco successivo è il <b>10</b> |
| $10 \rightarrow$ | valore $-1$ | <b>fine del file</b>                |

La catena è quindi  $[7, 1, 4, 5, 10]$ : cinque blocchi, nell'ordine in cui i byte del file li occupano.

### Quale blocco contiene l'offset 10100

Con blocchi da 4096 byte, il byte di offset  $o$  si trova nel blocco di indice  $\lfloor o/4096 \rfloor$  della catena. Qui:

$$\left\lfloor \frac{10100}{4096} \right\rfloor = 2.$$

L'indice **2** della catena, contando da zero, è il **terzo** blocco:  $[0] \rightarrow 7$ ,  $[1] \rightarrow 1$ ,  $[2] \rightarrow 4$ . L'offset 10100 sta nel **blocco 4**.

Il controllo che conviene fare a mente, per non sbagliare di uno:

$$2 \times 4096 = 8192 \leq 10100 < 12288 = 3 \times 4096,$$

quindi sì, il byte 10100 sta nel terzo blocco, che comincia al byte 8192 e finisce al byte 12287.

### La dimensione minima del file

Qui la domanda ha una parola pesante: *presunta*. Noi non conosciamo la dimensione del file: sappiamo solo che **almeno** arriva a coprire l'offset 10100. La dimensione minima va quindi dedotta da quello che sappiamo dell'allocazione, e ci sono due letture possibili:

|   | Lettura                                                                                                       | Valore                                |
|---|---------------------------------------------------------------------------------------------------------------|---------------------------------------|
| A | il file occupa blocchi <b>interi</b> : l'offset 10100 sta nel terzo blocco, quindi il file ne occupa almeno 3 | $3 \times 4096 = \mathbf{12288}$ byte |
| B | il file contiene <b>almeno</b> il byte di offset 10100, quindi almeno 10101 byte                              | $10100 + 1 = 10101$ byte              |

È la lettura **A** quella da dare. Il motivo è che un file su un file system a blocchi *occupa sempre blocchi interi*: la lettura B darebbe una dimensione che il file system non è in grado di rappresentare, e il file system non può allocare “due blocchi e un terzo di blocco”. Il byte 10100 si trova dentro il terzo blocco, e il terzo blocco è allocato per intero, quindi il file è grande almeno 12288 byte.

#### Le due risposte, e che cosa scrivere

Entrambe le letture sono difendibili *a parole*, ed è per questo che la domanda dice “presunta”: non chiede un dato che si può conoscere, chiede la deduzione più stretta coerente con le regole dell'allocazione. La risposta da dare è **12288 byte**.

Se vuoi essere al sicuro all'esame, scrivi il ragionamento e non solo il numero: “l'offset 10100 cade nel terzo blocco della catena, e i file occupano blocchi interi, quindi la dimensione minima è  $3 \times 4096 = 12288$  byte”. Una frase così mostra che hai capito *perché* i blocchi sono interi, ed è quello che distingue una risposta completa.

#### Gli errori da evitare

**Contare da uno invece che da zero.** Gli indici della catena partono da 0 e la domanda lo ricorda esplicitamente. L'indice 2 è il *terzo* blocco: se il conto parte da uno, si finisce sul blocco 1 — sbagliato.

**Fermarsi alla prima voce della FAT senza seguire la catena.** La voce della FAT all'indice 7 vale 1, ma quello è il *secondo* blocco, non la risposta. La risposta non è una voce della tabella: è un blocco raggiunto dopo aver percorso la catena.

**Dividere per 1000.** “4 kB” in questa materia significa 4096 byte. Con 4000 il conto darebbe  $10100/4000 = 2$  per fortuna, ma i limiti dei blocchi sarebbero sbagliati e la seconda parte dell'esercizio verrebbe fuori diversa.

**Esercizio 6 — i-node e blocchi indiretti** (compito tipo, domanda 6)

**Domanda reale:** Testo della domanda ripreso dallo *Stralcio di un compito d'esame* pubblicato nella pagina del corso ([compito-tipo.pdf](#)).

Consideriamo un file system UNIX basato su i-node: l'i-node di un file contiene, oltre ad una serie di metadati, un certo numero di voci che servono ad individuare i blocchi del disco su cui è memorizzato il contenuto del file stesso. In un i-node standard ci sono 13 di queste voci: le ultime 3 sono usate per indicare, rispettivamente, un blocco indiretto singolo, un blocco indiretto doppio ed, per ultimo, un blocco indiretto triplo. Prendiamo come esempio il seguente i-node ed il contenuto di alcuni blocchi sul disco (di alcuni blocchi dati sono indicate solo le word preliminari e finali):

| i-node 54 | blocco 112 | blocco 444 | blocco 333 | blocco 233 | blocco 322 |
|-----------|------------|------------|------------|------------|------------|
| meta-dati | 16         | 200        | 233        | 821        | 323        |
| 321       | 544        | 288        | 322        | 822        | 212        |
| 322       | 20         | 201        | 444        | 915        | 999        |
| 239       | 555        | 280        | 530        | 50         | 0          |
| 234       | 922        | 399        | 742        | 51         | 843        |
| 235       | 942        | 400        | 221        | 53         | 212        |
| 236       | ...        | ...        | ...        | ...        | ...        |
| 14        | ...        | ...        | ...        | ...        | ...        |
| 21        | ...        | ...        | ...        | ...        | ...        |
| 233       | ...        | ...        | ...        | ...        | ...        |
| 12        | 132        | 899        | -1         | 881        | 233        |
| 112       | 134        | 900        | -1         | 882        | 0          |
| 333       |            |            |            |            |            |
| -1        |            |            |            |            |            |

Tenendo conto del fatto che i blocchi usati dal file system sono da 4 kB e che i numeri di blocco sono a 32 bit: individuare in quali blocchi del disco risiedono i byte di offset XXX, YYY, ZZZ del contenuto del file a cui si riferisce l'i-node.

*Nota:* gli offset sono espressi in byte e partono da 0.

**Svolgimento.** Gli offset nella traccia sono tre **segnaposto**: XXX, YYY, ZZZ non sono numeri, e il compito non li definisce. Vanno quindi scelti, uno per zona, per mostrare tutti e tre i meccanismi. Nella soluzione che segue ho preso:

|     | Zona che si vuole esercitare        | Offset scelto |
|-----|-------------------------------------|---------------|
| XXX | zona <b>diretta</b> (prime 10 voci) | 16484 byte    |
| YYY | <b>indiretto semplice</b>           | 49159 byte    |
| ZZZ | <b>indiretto doppio</b>             | 16814080 byte |

**Prima di tutto: la mappa del file**

L'i-node ha 13 voci: le prime 10 puntano direttamente a blocchi di dati, l'undicesima a un blocco di puntatori, la dodicesima a un blocco di puntatori a puntatori, la tredicesima a tre livelli. Con blocchi da 4096 byte e numeri di blocco a 32 bit, ogni blocco di puntatori contiene

$$\frac{4096}{4} = 1024 \text{ voci.}$$

Quindi il file è diviso in quattro zone, e i confini si calcolano una volta sola:

| Zona               | Dimensione           | Intervallo di offset        |
|--------------------|----------------------|-----------------------------|
| diretta            | $10 \times 4096$     | 0 ... 40959                 |
| indiretto semplice | $1024 \times 4096$   | 40960 ... 4 235 263         |
| indiretto doppio   | $1024^2 \times 4096$ | 4 235 264 ... 4 299 202 559 |



I confini si leggono così: la zona diretta copre i primi 10 blocchi; quando finisce comincia il blocco indiretto semplice, che con le sue 1024 voci copre altri 1024 blocchi; quando finisce anche quello comincia il doppio. Il trucco per non sbagliare è tenere presente che l'indiretto semplice copre  $1024 \times 4096$  byte, non 1024 byte: ogni voce vale un blocco intero.

**XXX = 16484 byte**

L'offset è piccolo, quindi siamo nella **zona diretta**:

$$\left\lfloor \frac{16484}{4096} \right\rfloor = 4.$$

La voce di indice 4 dell'i-node (la quinta, contando da zero) vale **235**. L'offset 16484 sta nel **blocco 235**.

**YYY = 49159 byte**

L'offset supera i 40959 byte della zona diretta, quindi siamo nella zona **indiretta semplice**. Prima si toglie la parte già consumata dalla zona diretta, poi si divide:

$$49159 - 40960 = 8199, \quad \left\lfloor \frac{8199}{4096} \right\rfloor = 2.$$

La voce di indice 2 del blocco indiretto è la **terza**. Il blocco indiretto semplice è quello indicato dall'undicesima voce dell'i-node, cioè il **blocco 112**; la sua terza voce vale 20. L'offset 49159 sta quindi nel **blocco 20**.

**ZZZ = 16814080 byte**

L'offset supera anche la zona indiretta semplice, quindi siamo nel **doppio indiretto**, e qui *servono due indici*: il primo per scegliere il blocco di secondo livello, il secondo per scegliere la voce dentro quel blocco.

$$16814080 - 40960 - 1024 \times 4096 = 16814080 - 4235264 = 12578816$$

$$i = \left\lfloor \frac{12578816}{1024 \times 4096} \right\rfloor = 2, \quad j = \left\lfloor \frac{12578816 \bmod (1024 \times 4096)}{4096} \right\rfloor = 1023.$$

Ora si segue la catena dei puntatori:

1. la dodicesima voce dell'i-node è il blocco **333**, il blocco di *primo* livello del doppio indiretto;
2. la voce di indice  $i = 2$  del blocco 333 è la terza, e vale **444**: è il blocco di *secondo* livello;
3. la voce di indice  $j = 1023$  del blocco 444 è l'ultima, e vale **900**: è il blocco di dati.

L'offset 16814080 sta nel **blocco 900**.

#### Il metodo in una riga

Per ogni offset: **sottrai le zone che hai superato, poi dividi per la dimensione di un blocco per ottenere l'indice**. Il numero di divisioni è il numero di livelli di indirezione attraversati — una nella zona diretta, due nell'indiretto semplice (una per la voce, una per il blocco), tre nel doppio. Se ti trovi a fare più divisioni di così, stai contando due volte qualcosa.

### Gli errori da evitare

**Dividere per 4 al posto di 4096.** I 4 byte sono la dimensione di un *puntatore*, e servono solo a calcolare quante voci ci stanno in un blocco ( $4096/4 = 1024$ ). L'offset si divide per 4096, non per 4.

**Contare i livelli di indirezione sulla voce sbagliata.** La tredicesima voce dell'i-node vale  $-1$ , cioè il triplo indiretto non è usato. Un offset che cadesse oltre il doppio indiretto apparterebbe a un file più grande di questo, e la risposta giusta sarebbe “quel blocco non esiste per questo file”.

**Confondere il blocco 333 con un blocco di dati.** Il 333 compare sia come voce dell'i-node sia come *contenitore* di altri numeri di blocco. È un blocco di puntatori: il suo contenuto non sono dati del file, sono numeri. Il blocco di dati, alla fine della catena, è il 900.

## 21.7 Domanda 7 — lo scheduling del disco

### Esercizio 7 — LOOK su 200 tracce (compito tipo, domanda 7)

**Domanda reale:** Testo della domanda ripreso dallo *Stralcio di un compito d'esame* pubblicato nella pagina del corso ([compito-tipo.pdf](#)).

Supponiamo di avere un disco con 200 tracce (numerate da 0 a 199) la cui velocità di seek è di 1 traccia per ms. All'istante  $t = 0$  il sistema operativo sta servendo una richiesta sulla traccia 100 e in coda ci sono già le seguenti richieste per le tracce (50, 115, 180). Successivamente arrivano altre richieste all'istante  $t = 70$  per la traccia 150 e all'istante  $t = 130$  per la traccia 90. Si calcoli il tempo di ricerca complessivo (in ms) per servire tutte le richieste secondo la politica LOOK, iniziando in ordine ascendente (dalla traccia 0 verso la traccia 199) e trascurando la latenza rotazionale e il tempo di trasferimento. Indicare anche la sequenza di scheduling considerata.

**Svolgimento.** Tempo di ricerca complessivo: **210 ms**. Sequenza considerata:

**115, 180, 150, 90, 50.**

Quello che rende questo esercizio diverso dagli altri è che le richieste **non ci sono tutte all'istante zero**: due arrivano mentre il servizio è già cominciato, e questo cambia quali richieste sono disponibili quando la testina decide dove andare. È la parte che si sbaglia più spesso, perché si tende a ordinare tutte e cinque le richieste come se fossero presenti dall'inizio.

### Passo per passo

Si parte dalla traccia 100, in verso ascendente. La velocità è di 1 traccia al ms, quindi la distanza percorsa e il tempo trascorso sono lo stesso numero: possiamo seguire l'orologio contando le tracce.

| Istante | Testina | Richieste in coda | Scelta     | Perché                          | Tempo       |
|---------|---------|-------------------|------------|---------------------------------|-------------|
| 0       | 100     | 50, 115, 180      | <b>115</b> | ascendente: la prima oltre 100  | $+15 = 15$  |
| 15      | 115     | 50, 180           | <b>180</b> | ancora ascendente               | $+65 = 80$  |
| 70      | —       | <i>arriva 150</i> | —          | <i>la testina è già oltre</i>   | —           |
| 80      | 180     | 50, 150           | <b>150</b> | inversione: si scende           | $+30 = 110$ |
| 110     | 150     | 50, 90            | <b>90</b>  | 90 è già arrivata ( $t = 130$ ) | $+60 = 170$ |
| 170     | 90      | 50                | <b>50</b>  | ultima rimasta                  | $+40 = 210$ |

Il totale è confermato dalla somma dei singoli spostamenti:

$$15 + 65 + 30 + 60 + 40 = 210 \text{ ms.}$$

### Il punto critico: la richiesta 150

Vale la pena fermarsi su questa, perché è l'unica cosa davvero difficile dell'esercizio.

Al tempo  $t = 70$  arriva la richiesta per la traccia 150. In quell'istante la testina è partita da 115 e sta andando verso 180; a  $t = 70$  si trova sulla traccia  $115 + (70 - 15) = 170$ . La traccia 150 è quindi **dietro** la testina, nel verso in cui sta andando.

La reazione sbagliata è dire “allora la servo subito, torno indietro”: così facendo si cambierebbe direzione a metà strada, e non sarebbe più né LOOK né niente altro. La LOOK è così fatta: man mano che si scende, serve tutte le richieste che incontra. La richiesta 150 viene servita **al ritorno**, quando la testina, dopo aver toccato 180, scende e la incrocia. E infatti la sequenza è ..., 180, 150, 90, 50: la 150 è la *seconda* del ritorno, non la prima cosa che si fa.

### Perché si chiama LOOK, e che cosa cambierebbe con SCAN

Nella domanda c'è scritto “LOOK”, che è il nome della politica che inverte il verso **sull'ultima richiesta incontrata** in quella direzione, cioè sulla traccia 180, senza andare fino in fondo al disco. La variante che invece prosegue fino alla traccia 199 (il fondo fisico) prima di invertire si chiama SCAN, e sullo stesso esercizio dà un risultato diverso:

| Politica                     | Sequenza              | Tempo         |
|------------------------------|-----------------------|---------------|
| LOOK                         | 115, 180, 150, 90, 50 | <b>210 ms</b> |
| SCAN (fino alla traccia 199) | 115, 180, 150, 90, 50 | 248 ms        |

Le due sequenze sono identiche, ma il tempo no: la SCAN paga le 19 tracce da 180 a 199 e ritorno. È il motivo per cui la domanda dice *LOOK* e non *SCAN*: con la parola sbagliata il risultato cambia di 38 ms.

#### Attenzione al nome, di nuovo

Questa confusione fra LOOK e SCAN non è un'invenzione di questa guida: se ne parla nel capitolo 18, dove si vede che le slide del corso chiamano “scansione (algoritmo dell'ascensore)” proprio la politica che inverte sull'ultima richiesta — cioè la LOOK di questo esercizio. Nella domanda 7 del compito tipo, però, la parola usata è “LOOK”, e la risposta dichiarata qui sopra (210 ms) è quella della LOOK. All'esame conviene attenersi alla parola che compare nel testo della domanda.

#### Che cosa guarda il correttore

Due cose, e la seconda pesa quanto la prima:

1. il **numero**, cioè 210 ms;
2. la **sequenza**, che la domanda chiede esplicitamente (“indicare anche la sequenza di scheduling considerata”). Senza la sequenza, un 210 corretto può essere stato ottenuto per caso; con la sequenza, il correttore vede se hai capito come si costruisce.

Il consiglio pratico: scrivi la tabella degli spostamenti, uno per riga. Costa poco e rende impossibile sbagliare la somma.

## 21.8 Che cosa insegna questo compito

Rileggendo le sette domande tutte insieme, salta all'occhio una cosa che non si vede guardandole una per una: **il compito tipo copre una parte sola del programma**. Delle sette domande,

cinque riguardano la gestione della memoria e dei file (memoria virtuale, FAT, i-node, sostituzione delle pagine, scheduling del disco) e due riguardano la concorrenza (semafori) e l'allocazione su disco.

Restano fuori, per esempio, i processi e i loro stati, i thread, gli algoritmi di scheduling della CPU, la struttura del sistema operativo, il RAID e gli SSD. Non è una lacuna del compito tipo: è che è uno *stralcio*, cioè un pezzo di una prova più lunga. Ma è un'informazione importante per chi studia, e conviene dirla chiara:

#### Come usare il compito tipo

Il compito tipo dice **come** vengono chieste le cose, non **quanto** programma viene chiesto. Le due domande a risposta multipla e i cinque esercizi mostrano la *forma* delle domande, e la forma si ripete: le cinque famiglie di calcolo sono quelle elencate nella premessa *Com'è fatta la prova di teoria*, e conviene saperle risolvere a occhi chiusi.

Ma il resto del programma — i venti argomenti dei capitoli da 1 a 20 — va studiato lo stesso, perché una prova di teoria su nove CFU non può fermarsi a cinque esercizi. La parte che il compito tipo non tocca è proprio quella che si chiede all'orale.

I capitoli che seguono prendono le cinque famiglie del compito tipo e le esercitano su numeri nuovi: ogni esercizio dichiara se è una domanda reale o un esercizio costruito, e ogni numero è stato calcolato e verificato eseguendo lo script, non ricopiato.

## Capitolo 22

### Varianti: gli stessi esercizi con numeri nuovi

Il capitolo 21 ha svolto le sette domande del compito tipo. Questo capitolo ne propone di nuove, dello stesso tipo e della stessa difficoltà, ma con numeri diversi e scelti per coprire i casi che il compito tipo non tocca.

#### Che cosa sono, e che cosa non sono

Gli esercizi di questo capitolo sono tutti **costruiti**: non vengono da un compito d'esame, li ho scritti io sul modello di quello ufficiale. Ognuno lo dichiara con la riga arancione in testa. Il motivo per cui esistono è semplice: il compito tipo ha *sette* domande, e le sette domande che ha toccano ogni famiglia *una volta sola*, nel caso più semplice. Un conto fatto una volta sola si impara a memoria; un conto fatto su cinque tracce diverse si impara davvero. Dove un esercizio mette alla prova un caso limite — un offset esattamente sul confine fra due zone, un file che finisce esattamente su un blocco — lo dico nel testo, perché è lì che si impara qualcosa.

Come per il compito tipo, tutti i numeri sono prodotti eseguendo lo script di verifica, non calcolati a mano.

### 22.1 Memoria virtuale

#### Esercizio 1 — Un indirizzamento con numeri nuovi

**Esercizio costruito sul modello del compito tipo.** Variante della domanda 3 del compito tipo. I numeri sono cambiati, il procedimento è lo stesso.

Un sistema usa memoria virtuale con uno spazio di indirizzamento virtuale da **4 Gb**, un numero di pagina virtuale a **20 bit** e un indirizzo fisico a **26 bit**. Determinare:

- a. la dimensione di una pagina, in byte;
- b. il numero di pagine virtuali;
- c. il numero di frame fisici.

**Svolgimento.** a)  $4 \text{ Gb} = 2^{32} \text{ byte}$ , quindi l'indirizzo virtuale ha 32 bit. Togliendo i 20 bit di VPN restano 12 bit di offset:

$$2^{12} = 4096 \text{ byte} = 4 \text{ kB.}$$

b) Le pagine virtuali sono  $2^{\text{bit di VPN}} = 2^{20} = 1\,048\,576$ .

c) L'offset è di 12 bit, e l'indirizzo fisico ne ha 26:

$$26 - 12 = 14 \text{ bit di frame,} \quad 2^{14} = 16384 \text{ frame.}$$

Il controllo per la terza via: memoria fisica divisa per la pagina,

$$\frac{2^{26}}{2^{12}} = \frac{67\,108\,864}{4096} = 16384.$$

#### Che cosa cambia rispetto al compito tipo

I numeri sono diversi ma la struttura è identica, ed è proprio questo il punto: cambiando i dati, l'unica cosa che cambia è *quale* delle tre grandezze si ricava per sottrazione. Qui la

pagina è da 4 kB, nel compito tipo era da 256 byte, e il numero di frame è 16384 invece di 4096. Se hai capito il procedimento, la variazione non aggiunge difficoltà; se l'hai imparato a memoria sul compito tipo, qui non funziona più.

## 22.2 Effective Access Time

### Esercizio 2 — Quanto costa davvero un accesso con la TLB

**Esercizio costruito sul modello del compito tipo.** Variante dell'esempio della slide SO-03 p15, con tempi e hit ratio diversi.

Un sistema ha un tempo di accesso alla memoria di 80 ns e un tempo di accesso alla TLB di 10 ns. L'hit ratio della TLB è del 90%. Calcolare:

- il tempo di accesso in caso di *hit*;
- il tempo di accesso in caso di *miss*;
- l'*Effective Access Time* (EAT);
- quale hit ratio servirebbe per scendere sotto i 92 ns.

**Svolgimento.** a) Con la TLB che risponde, serve un solo accesso alla memoria (perché la TLB restituisce direttamente il numero di frame):

$$80 + 10 = 90 \text{ ns.}$$

b) Senza risposta dalla TLB servono due accessi alla memoria — uno per leggere la voce nella tabella delle pagine, uno per il dato — più il tempo della TLB che ha già speso:

$$2 \times 80 + 10 = 170 \text{ ns.}$$

c) La media pesata dei due casi:

$$\text{EAT} = 0,90 \times 90 + 0,10 \times 170 = 81 + 17 = \mathbf{98 \text{ ns.}}$$

d) L'EAT dipende dall'hit ratio  $\varepsilon$  in modo lineare:

$$\text{EAT}(\varepsilon) = \varepsilon \cdot 90 + (1 - \varepsilon) \cdot 170 = 170 - 80\varepsilon.$$

Ponendo  $\text{EAT}(\varepsilon) = 92$ :

$$\varepsilon = \frac{170 - 92}{80} = 0,975.$$

Serve un hit ratio del **97,5%**.

#### La domanda che vale la pena farsi

Chiedersi “quanto deve migliorare l'hardware perché l'EAT scenda sotto una soglia” è un esercizio che si vede spesso, e ha una trappola: esistono soglie **irraggiungibili**. Il valore minimo possibile dell'EAT è quello con hit ratio al 100%, cioè  $\alpha + \beta = 90$  ns: nessun miglioramento della TLB può scendere sotto il costo di un accesso in memoria.

Infatti per un EAT di 85 ns il conto darebbe  $\varepsilon = (170 - 85)/80 = 1,062$ , cioè un hit ratio del 106,2%: impossibile. Un esercizio così va risposto dicendo *perché* è impossibile, non forzando un numero.

## 22.3 Trace di semafori

### Esercizio 3 — Una stampa non sincronizzata

**Esercizio costruito sul modello del compito tipo.** Variante della domanda 4 del compito tipo, che ha invece un output unico. Qui l'output **non** è unico, e capire perché è il punto dell'esercizio.

Due processi condividono la variabile  $x$ , che vale inizialmente 0, e il semaforo  $S$ , che vale inizialmente 1.

```
P1:          P2:
wait(S)      print(x)
x = x + 1
signal(S)
wait(S)
x = x + 2
signal(S)
```

Determinare l'output del processo P2.

**Svolgimento.** P2 può stampare **0**, **1** oppure **3**. Non c'è una risposta giusta sola, e la risposta da dare all'esame è l'*insieme* dei valori possibili, con la spiegazione.

Il punto è che **P2 non prende nessun semaforo**. Il semaforo  $S$  protegge  $x$  dalle interferenze fra P1 e sé stesso — P1 lo prende e lo rilascia due volte — ma P2 stampa senza passare da lì, quindi il suo `print` può cadere in qualunque istante, compreso in mezzo alle operazioni di P1. Il valore che legge dipende da quanto di P1 è già avvenuto:

| Output | Quando                            | Spiegazione                                                  |
|--------|-----------------------------------|--------------------------------------------------------------|
| 0      | P2 parte per primo                | P1 non ha ancora toccato $x$ : si stampa il valore iniziale. |
| 1      | P2 stampa fra i due blocchi di P1 | P1 ha eseguito il primo incremento ma non il secondo.        |
| 3      | P2 stampa per ultimo              | P1 ha finito: $0 + 1 + 2 = 3$ .                              |

#### Perché questo esercizio vale più di uno con la risposta unica

La traccia del compito tipo si chiude con due processi bloccati, e qui non si blocca nessuno. Sono due modi diversi in cui una traccia di semafori può essere “strana”, e vanno riconosciuti entrambi:

- nel **compito tipo** il valore è unico ma il programma non termina: la difficoltà è accorgersi che i `signal` disponibili sono esattamente quelli necessari;
- **qui** il programma termina sempre, ma il valore non è unico: la difficoltà è accorgersi che un processo che non prende semafori è fuori dalla protezione, e non c'è nessun ordine privilegiato da seguire.

Il filo comune è il metodo: *non* scegliere un interleaving e seguirlo. Chiedersi prima quali sono i valori che `print` può vedere, e poi quali di quelli sono raggiungibili.

All'esame, una risposta come “stampa 3” a questa traccia sarebbe sbagliata, e sarebbe sbagliata anche “stampa 0, 1 o 3” senza spiegazione: quello che il correttore cerca è il *perché* i valori sono tre, cioè il fatto che P2 non è sincronizzato.

## 22.4 FAT

### Esercizio 4 — Un file con una catena più lunga

**Esercizio costruito sul modello del compito tipo.** Variante della domanda 5 del compito tipo. Qui la catena ha cinque blocchi e si chiedono anche i casi di confine.

Un file system usa blocchi da 4096 byte e una FAT. La cartella radice contiene la voce `paperino.txt` con primo blocco 5. La FAT contiene, fra le altre, le voci seguenti:

| indice | 1 | 2  | 3 | 4 | 5 | 6 | 7  | 8 | 9 | 10 | 11 |
|--------|---|----|---|---|---|---|----|---|---|----|----|
| valore | 0 | 11 | 0 | 0 | 9 | 0 | -1 | 0 | 2 | 0  | 7  |

- ricostruire la catena dei blocchi di `paperino.txt`;
- dire in quale blocco cadono gli offset 0, 4095, 4096, 13000, 16383 e 16384;
- dire qual è la dimensione minima presunta del file, sapendo che contiene l'offset 13000;
- elencare i blocchi liberi.

**Svolgimento.** a) Si parte dal blocco 5 e si segue la FAT fino al valore -1:

5  $\xrightarrow{\text{voce } 5=9}$  9  $\xrightarrow{\text{voce } 9=2}$  2  $\xrightarrow{\text{voce } 2=11}$  11  $\xrightarrow{\text{voce } 11=7}$  7  $\xrightarrow{\text{voce } 7=-1}$  fine.

La catena è [5, 9, 2, 11, 7]: cinque blocchi.

b) Si divide l'offset per 4096 e si prende l'indice corrispondente della catena:

| Offset | Indice $\lfloor o/4096 \rfloor$ | Blocco                             |
|--------|---------------------------------|------------------------------------|
| 0      | 0                               | 5                                  |
| 4095   | 0                               | 5 (ultimo byte del primo blocco)   |
| 4096   | 1                               | 9 (primo byte del secondo)         |
| 13000  | 3                               | 11                                 |
| 16383  | 3                               | 11 (ultimo byte del quarto blocco) |
| 16384  | 4                               | 7 (primo byte del quinto)          |

I tre casi di confine sono quelli che vale la pena guardare. L'offset 4095 e l'offset 4096 sono *adiacenti* e cadono in blocchi diversi: il primo è l'ultimo byte del blocco 5, il secondo il primo byte del blocco 9. Lo stesso vale per 16383 e 16384, che separano il blocco 11 dal blocco 7. È il tipo di conto che si sbaglia di uno, e l'unico modo di non sbagliarlo è ricordare che l'offset  $o$  sta nel blocco  $\lfloor o/4096 \rfloor$  — divisione *intera*, e gli offset partono da zero.

c) L'offset 13000 sta nell'indice 3, cioè nel **quarto** blocco. Se il file occupa blocchi interi, la dimensione minima è

$$4 \times 4096 = \mathbf{16384} \text{ byte.}$$

d) I blocchi liberi sono quelli con la voce a 0: {1, 3, 4, 6, 8, 10}. Si leggono dalla FAT stessa, senza bisogno di nessun'altra struttura: è esattamente il motivo per cui l'opzione D della domanda 2 del compito tipo è vera.

#### Un dettaglio sulla tabella di questa traccia

La tabella qui sopra si ferma all'indice 11, e la traccia dice “la FAT contiene, *fra le altre*, le voci seguenti”: è un estratto, come sono sempre le tabelle delle tracce vere. Le voci che servono a rispondere ci sono tutte — quelle della catena e quelle a 0 — ma questo è il punto da controllare ogni volta: *prima* di dire “il blocco 40 è libero”, verificare che la voce 40



sia nella tabella. Un blocco di cui la traccia non parla non è né libero né occupato: è non determinabile, e se all'esame serve quel dato la risposta giusta è dirlo.

## 22.5 i-node

### Esercizio 5 — I tre confini di zona

**Esercizio costruito sul modello del compito tipo.** Variante della domanda 6 del compito tipo, sullo stesso i-node. Gli offset sono scelti sui **confini** fra le zone, che è il caso in cui si sbaglia.

Si usi lo stesso i-node e gli stessi blocchi della domanda 6 del compito tipo (i-node 54, blocchi da 4096 byte, numeri di blocco a 32 bit, 13 voci delle quali le ultime tre per l'indiretto semplice, doppio e triplo; la voce tripla vale  $-1$ ).

Determinare in quali blocchi del disco risiedono i byte di offset:

- a. 40959;
- b. 40960;
- c. 4 235 263;
- d. 4 235 264.

**Svolgimento.** Prima di tutto la mappa delle zone, che è la stessa del capitolo 21: con  $4096/4 = 1024$  puntatori per blocco,

| Zona               | Estensione           | Intervallo di offset        |
|--------------------|----------------------|-----------------------------|
| diretta            | $10 \times 4096$     | 0 ... 40959                 |
| indiretto semplice | $1024 \times 4096$   | 40960 ... 4 235 263         |
| indiretto doppio   | $1024^2 \times 4096$ | 4 235 264 ... 4 299 202 559 |

I quattro offset sono **esattamente** i due estremi delle due zone centrali: l'ultimo byte della diretta e il primo della indiretta semplice; l'ultimo byte della indiretta semplice e il primo della indiretta doppia.

**a) Offset 40959** — è l'ultimo byte della zona diretta, quindi l'ultima voce diretta dell'i-node, cioè la decima (indice 9), che vale 12. Il byte sta nel **blocco 12**.

**b) Offset 40960** — è il primo byte della zona indiretta semplice. Si sottrae la zona diretta, e resta 0:

$$40960 - 40960 = 0, \quad \left\lfloor \frac{0}{4096} \right\rfloor = 0.$$

La **prima** voce del blocco indiretto semplice (blocco 112) vale 16. Il byte sta nel **blocco 16**.

**c) Offset 4 235 263** — è l'ultimo byte della zona indiretta semplice:

$$4235263 - 40960 = 4\,194\,303, \quad \left\lfloor \frac{4194303}{4096} \right\rfloor = 1023.$$

La voce di indice 1023, cioè l'**ultima** del blocco 112, vale 134. Il byte sta nel **blocco 134**.

**d) Offset 4 235 264** — è il primo byte della zona indiretta *doppia*. Si sottraggono entrambe le zone già percorse:

$$4235264 - 40960 - 1024 \times 4096 = 4235264 - 4235264 = 0.$$

Quindi  $i = 0$  e  $j = 0$ : la **prima** voce del blocco di primo livello (il 333) vale 233, e la **prima** voce del blocco 233 vale 821. Il byte sta nel **blocco 821**.

### Perché proprio questi quattro offset

I confini sono il posto dove gli errori di uno si vedono. Guarda la differenza fra (a) e (b): due offset *adiacenti*, 40959 e 40960, che stanno in blocchi completamente diversi — il 12, ultima voce diretta, e il 16, prima voce indiretta. Chi sbaglia di uno fra i due finisce nella zona sbagliata, e da lì in poi è tutto sbagliato.

Lo stesso vale per (c) e (d): 4 235 263 sta nell'indiretto semplice (blocco 134), 4 235 264 nel doppio (blocco 821). Sono due byte adiacenti che appartengono a strutture di indirizzione diverse.

Il modo per non sbagliarli è non contare “a occhio” ma scrivere sempre la sottrazione delle zone percorse: quando il risultato è esattamente 0, sei sul primo byte della zona, e l'indice è 0.

## 22.6 Scheduling del disco

### Esercizio 6 — Nove richieste, tutte presenti

**Esercizio costruito sul modello del compito tipo.** Variante della domanda 7 del compito tipo. Qui le richieste sono tutte già in coda all'istante zero, e si confrontano le politiche invece di applicarne una sola.

Un disco ha 200 tracce (da 0 a 199) e una velocità di seek di 1 traccia/ms. La testina si trova sulla traccia **143** e sta salendo. In coda ci sono, tutte presenti all'istante 0, le richieste per le tracce:

86, 147, 91, 177, 94, 150, 102, 175, 130.

Si calcoli il tempo di ricerca complessivo e la sequenza di servizio per ciascuna di queste politiche:

- FCFS;
- SSTF;
- LOOK (verso ascendente);
- SCAN, cioè fino al fondo fisico del disco;
- C-SCAN e C-LOOK.

Trascurare latenza rotazionale e tempo di trasferimento.

#### Svolgimento.

| Politica  | Sequenza                                 | Distanza (ms) |
|-----------|------------------------------------------|---------------|
| a) FCFS   | 86, 147, 91, 177, 94, 150, 102, 175, 130 | 565           |
| b) SSTF   | 147, 150, 130, 102, 94, 91, 86, 175, 177 | 162           |
| c) LOOK   | 147, 150, 175, 177, 130, 102, 94, 91, 86 | <b>125</b>    |
| d) SCAN   | 147, 150, 175, 177, 130, 102, 94, 91, 86 | 169           |
| e) C-LOOK | 147, 150, 175, 177, 86, 91, 94, 102, 130 | 169           |
| e) C-SCAN | 147, 150, 175, 177, 86, 91, 94, 102, 130 | 341           |

Come si leggono questi numeri:

- **FCFS** serve le richieste nell'ordine di arrivo, senza guardare dove si trova la testina. Qui costa 565 ms, oltre quattro volte la LOOK: è il prezzo di non riordinare nulla;
- **SSTF** sceglie sempre la richiesta più vicina. Parte salendo (147 prima di 86) e poi scende raccogliendo tutto ciò che incontra, ma alla fine deve risalire da 86 a 175 e 177: 162 ms;

- **LOOK** sale servendo 147, 150, 175, 177, e poi scende servendo tutto il resto fino a 86. Non va oltre l'ultima richiesta incontrata, quindi non paga né il viaggio fino a 199 né il rientro da 0: 125 ms, il migliore dei sei;
- **SCAN** fa lo stesso percorso della LOOK ma prosegue fino alla traccia 199 prima di invertire. Le richieste servite sono le stesse, nell'ordine stesso, ma il tempo sale a 169 ms: la differenza sono le tracce da 177 a 199 e ritorno;
- **C-LOOK** e **C-SCAN** trattano il disco come circolare: dopo l'ultima richiesta in alto saltano direttamente al blocco più basso invece di scendere servendo. Serve 86 per prima al ritorno, e le altre in ordine crescente.

#### Due cose che questi numeri dicono, e una che non dicono

La prima: **LOOK e SCAN hanno la stessa sequenza ma tempi diversi**. È la trappola della domanda 7 del compito tipo, dove la parola usata nel testo decide il risultato. Qui la differenza è di 44 ms su un solo esercizio, che è tanto.

La seconda: **SSTF non è la migliore**. Sembra controintuitivo, perché SSTF minimizza lo spostamento *del prossimo passo*. Ma minimizzare ogni passo non minimizza il totale: scegliendo sempre il vicino, SSTF si lascia dietro le richieste lontane e alla fine deve tornare a prenderle. Qui paga 37 ms in più della LOOK. È lo stesso fenomeno per cui SJF non è ottimale con arrivi scaglionati (capitolo 9).

La cosa che questi numeri **non** dicono: che la scansione circolare sia la politica migliore. Le misurazioni di questa guida non lo confermano, e il capitolo 18 spiega perché e che cosa rispondere all'esame.

## 22.7 Sostituzione delle pagine

### Esercizio 7 — Belady su una sequenza nuova

**Esercizio costruito sul modello del compito tipo.** Variante dell'esempio delle slide SO-03 p26-p28. La sequenza è nuova e serve a far vedere l'anomalia di Belady su un caso diverso da quello della slide.

Si consideri la sequenza di riferimenti

3, 2, 1, 0, 3, 2, 4, 3, 2, 1, 0, 4.

Si calcoli il numero di fault di pagina con 3 e con 4 frame, per gli algoritmi OPT, FIFO e LRU.

**Svolgimento.**

| Algoritmo | 3 frame | 4 frame | Effetto                             |
|-----------|---------|---------|-------------------------------------|
| OPT       | 7       | 6       | migliora                            |
| FIFO      | 9       | 10      | <b>peggiora:</b> anomalia di Belady |
| LRU       | 10      | 8       | migliora                            |

Il caso da guardare è il FIFO: passando da 3 a 4 frame i fault *aumentano*, da 9 a 10. È l'**anomalia di Belady**, e non è un errore di calcolo: con più memoria a disposizione l'algoritmo lavora peggio.

#### Che cosa c'è dietro l'anomalia

FIFO non gode della **proprietà di inclusione**: l'insieme delle pagine presenti con  $n$  frame *non* è contenuto in quello con  $n + 1$  frame. Con 4 frame, il FIFO può trovarsi in uno stato

in cui sta per eliminare una pagina che con 3 frame non avrebbe mai caricato, e da lì la sequenza di scelte diverge in peggio.

LRU e OPT invece **godono** della proprietà di inclusione, ed è per questo che i loro numeri scendono sempre passando a più frame:  $10 \rightarrow 8$  per LRU,  $7 \rightarrow 6$  per OPT. Le slide lo dicono esplicitamente per LRU [SO-03 p28], e aggiungono che anche NFU e Aging la rispettano, mentre NRU, seconda chance e clock non la rispettano perché “riducono a FIFO”.

Quindi la regola da ricordare non è “più frame = meno fault” — quello vale solo per gli algoritmi che godono della proprietà di inclusione. La regola vera è: *l'anomalia di Belady ce l'hanno quegli algoritmi che degradano a FIFO*.

## 22.8 RAID

### Esercizio 8 — Parità e ricostruzione su uno strip nuovo

**Esercizio costruito sul modello del compito tipo.** Variante dell'esempio di parità del capitolo 19.

Un array RAID 4 ha quattro dischi dati più un disco di parità. Su un certo gruppo di strip i quattro dischi dati contengono i valori

$$V_1 = 1011_2, \quad V_2 = 0111_2, \quad V_3 = 1101_2, \quad V_4 = 1110_2.$$

- calcolare il contenuto  $P$  dello strip di parità e verificare che lo XOR di tutti e cinque gli strip faccia 0;
- supponendo di aver perso ciascuno dei quattro strip dati, uno per volta, ricostruirne il contenuto e verificare che venga quello di partenza.

**Svolgimento.** a) La parità è lo XOR dei quattro valori dati. Calcolando colonne di bit a colonne,

$$P = V_1 \oplus V_2 \oplus V_3 \oplus V_4 = 1011 \oplus 0111 \oplus 1101 \oplus 1110 = \mathbf{1111}_2.$$

Il controllo: lo XOR di tutti e cinque gli strip, parità inclusa, deve fare zero. E infatti

$$V_1 \oplus V_2 \oplus V_3 \oplus V_4 \oplus P = 1111 \oplus 1111 = 0000.$$

È l'identità su cui si regge tutto il RAID con parità: la parità è il valore che *annulla* la somma, e per questo si può ricavare dall'insieme degli altri.

b) Per ricostruire lo strip mancante si rifà lo XOR di tutti quelli che restano, parità compresa:

$$V_i = P \oplus \bigoplus_{k \neq i} V_k.$$

Applicandolo a ciascun caso:

| Strip perso  | Ricostruito con                      | Risultato | Esito    |
|--------------|--------------------------------------|-----------|----------|
| $V_1 = 1011$ | $P \oplus V_2 \oplus V_3 \oplus V_4$ | 1011      | corretto |
| $V_2 = 0111$ | $P \oplus V_1 \oplus V_3 \oplus V_4$ | 0111      | corretto |
| $V_3 = 1101$ | $P \oplus V_1 \oplus V_2 \oplus V_4$ | 1101      | corretto |
| $V_4 = 1110$ | $P \oplus V_1 \oplus V_2 \oplus V_3$ | 1110      | corretto |

Tutti e quattro gli strip si ricostruiscono esattamente. È il senso della tolleranza a guasto singolo: qualunque *uno* dei dischi ceda, il suo contenuto si ricalcola dagli altri.

### Il guasto singolo, e il plurale

La ricostruzione funziona per **uno** strip perso alla volta. Se ne cadono due nello stesso gruppo, la formula dà un risultato ma non è più quello giusto: due incognite e una equazione non si risolvono.

È la ragione per cui RAID 4 tollera il guasto di un solo disco, e per cui nella pratica si usano livelli con più ridondanza. Un esercizio che chiede “ricostruire due strip persi” è un esercizio senza soluzione, e vale la pena accorgersene invece di calcolare.

È anche il motivo per cui la parità non è una copia: nessuno dei cinque strip contiene l’informazione di un altro, ma l’insieme dei **cinque** la contiene tutta.

## 22.9 Come usare queste varianti

### Il modo giusto di allenarsi

Questi esercizi non servono a essere *letti*. Il modo in cui si preparano le cinque famiglie di calcolo è:

1. coprire lo svolgimento;
2. provare a farlo su un foglio, senza guardare;
3. confrontare non solo il numero finale, ma *il procedimento*: se il numero torna ma il procedimento era diverso, quasi sempre è un caso;
4. ripetere l’esercizio cambiando un dato a caso — un offset diverso, una traccia diversa, un hit ratio diverso — e rifarlo.

Il quarto passo è quello che conta più di tutti, ed è anche quello che si salta. Risolvere un esercizio con i numeri della traccia è riconoscerlo; risolverlo con numeri inventati è saperlo fare.

## Capitolo 23

### Domande a risposta multipla

---

#### Come usare questo capitolo

Ci sono **sessanta domande**, tre per ciascuno dei venti argomenti del programma. Servono a una cosa sola: scoprire *quale* capitolo non hai capito, prima che lo scopra il compito.

Il modo giusto di usarle è questo. Scegli un capitolo, leggi le sue tre domande e rispondi **senza guardare**. Solo dopo confronti con il riquadro delle risposte, che sta in fondo al gruppo e riporta, per ogni domanda, la lettera giusta e una riga di motivazione. Se sbagli, il riquadro ti dice anche *dove* andare a rileggere.

Le risposte sono raggruppate per capitolo, e non messe subito sotto ogni domanda, di proposito: con la risposta sotto gli occhi si finisce per leggerla invece di cercarla, e l'autoverifica non vale più niente.

#### Una differenza di formato rispetto al compito vero

Nel compito tipo le domande a risposta multipla hanno **cinque** opzioni, da A a E, e una delle due chiede addirittura quale affermazione sia *falsa*. Qui le domande hanno **quattro** opzioni e chiedono quasi sempre quale affermazione sia *corretta*.

La differenza non cambia niente nella sostanza — una domanda a scelta multipla si vince sapendo o non sapendo, e il numero di distrattori non sposta quel confine — ma vale la pena saperlo, perché con cinque opzioni la probabilità di indovinare scende e la lettura delle alternative diventa parte del lavoro. In fondo al capitolo trovi due domande scritte *esattamente* nel formato del compito, per prendere confidenza anche con quello.

### 23.1 Parte I — Il sistema operativo e l'hardware

#### 23.1.1 Capitolo 1 — Introduzione al concetto di sistema operativo

- 1.1 Il capitolo distingue due forme di *multiplexing*. Quale delle seguenti coppie di esempi corrisponde al multiplexing *nello spazio*?
- A. la CPU e la stampante
  - B. la memoria centrale e il disco
  - C. la CPU e la memoria centrale
  - D. la stampante e il disco
- 1.2 Secondo il capitolo, come avviene il passaggio da una modalità di esecuzione all'altra?
- A. Tramite normali chiamate di funzione, che restano in modalità utente.
  - B. Tramite chiamate di sistema (*system call*) e interrupt hardware.
  - C. Solo alla fine dell'esecuzione del processo, quando il kernel ne recupera le risorse.
  - D. Solo su richiesta esplicita dell'utente, tramite l'interfaccia del sistema operativo.
- 1.3 Quale affermazione sui sistemi embedded e sui sistemi real-time è sostenuta dal capitolo?
- A. Ogni sistema embedded è anche real-time, perché ha funzionalità fisse.
  - B. Ogni sistema real-time è anche embedded, perché ha vincoli di spazio molto stretti.
  - C. Un sistema embedded non è necessariamente real-time, e un sistema real-time non è necessariamente embedded.

- D. Un sistema real-time è semplicemente un sistema molto veloce, senza scadenze temporali da rispettare.

### Risposte del capitolo 1

- 1.1 B — nel tempo la risorsa è assegnata a turno (CPU, stampante); nello spazio è suddivisa fisicamente fra più richiedenti (memoria centrale, disco).
- 1.2 B — il capitolo indica due vie di passaggio: la chiamata di sistema, che è volontaria, e l'interrupt hardware.
- 1.3 C — le due categorie sono indipendenti: il sistema embedded non ha per forza scadenze critiche, e il controllo real-time può girare su hardware non embedded.

### 23.1.2 Capitolo 2 — Richiami sull'architettura degli elaboratori

2.1 Il capitolo descrive il multithreading (o *hyperthreading*) come:

- A. una tecnica che raddoppia le prestazioni, perché la CPU esegue davvero due thread in parallelo.
- B. una tecnica in cui la CPU tiene al suo interno lo stato di due thread: non c'è esecuzione parallela vera e propria, ma dal punto di vista del SO le unità logiche eseguibili sembrano due.
- C. la presenza di più processori veri nello stesso sistema, con vantaggi di throughput, economia di scala e affidabilità.
- D. la presenza di più nuclei integrati nello stesso chip, oppure di molti nuclei specializzati per il calcolo parallelo su dati.

2.2 Secondo il capitolo, quanto vale 1 GiB?

- A.  $10^9$  byte
- B.  $10^6$  byte
- C.  $2^{30}$  byte
- D.  $2^{10}$  byte

2.3 Un controllore dedicato trasferisce i dati direttamente fra dispositivo e memoria centrale, senza far passare ogni byte dal processore, e la CPU viene avvisata solo a trasferimento concluso. Di quale modalità di I/O si tratta?

- A. Busy waiting
- B. Con programmazione di interrupt
- C. Con uso del DMA
- D. La mappatura in memoria dei registri del controller (*memory-mapped I/O*)

### Risposte del capitolo 2

- 2.1 B — l'hyperthreading non raddoppia le prestazioni: tiene lo stato di due thread per riempire gli stalli della pipeline, non per eseguire due istruzioni nello stesso istante.
- 2.2 C — GiB è un multiplo in base 2, non in base 10:  $10^9$  byte è il GB,  $2^{10}$  byte è il KiB.
- 2.3 C — è la definizione del DMA; con l'interrupt la CPU non resta bloccata ma i dati passano comunque dal processore, e la mappatura in memoria riguarda i registri del controller, non il trasferimento.

### 23.1.3 Capitolo 3 — Struttura di un sistema operativo

#### 3.1 Qual è il difetto principale della struttura a livelli, secondo il capitolo?

- A. I problemi di prestazioni dovuti alle chiamate nidificate e al relativo overhead: per arrivare al livello più basso una richiesta deve attraversare tutti i livelli intermedi.
- B. Un errore in un punto qualunque, per esempio in un driver, può compromettere l'intero sistema.
- C. Il costo dello scambio di messaggi fra componenti, con due passaggi di contesto per ogni operazione.
- D. L'assenza di barriere interne: il sistema operativo è un unico programma senza alcuna struttura.

#### 3.2 Che cosa distingue la struttura a moduli dal microkernel?

- A. Nel microkernel i moduli stanno nel kernel e si invocano direttamente; nella struttura a moduli i componenti stanno in spazio utente e comunicano a messaggi.
- B. Nella struttura a moduli ogni modulo può invocare qualunque altro modulo direttamente e non ci sono messaggi; nel microkernel la comunicazione fra i componenti avviene attraverso messaggi.
- C. Nella struttura a moduli il kernel si occupa soltanto di scheduling, memoria e IPC, mentre nel microkernel ogni componente può richiamare direttamente tutti gli altri.
- D. Non c'è una differenza sostanziale: entrambe spostano i driver a livello utente e li fanno comunicare a messaggi.

#### 3.3 Un hypervisor di tipo 2 è:

- A. un hypervisor che gira direttamente sull'hardware, come VMware ESX/ESXi, KVM su Linux e Microsoft Hyper-V hypervisor.
- B. un processo dentro un sistema operativo *host*, come VMware Workstation e VirtualBox.
- C. il supporto hardware dedicato alla virtualizzazione, senza il quale ogni istruzione privilegiata dell'ospite va intercettata e simulata via software.
- D. il sistema operativo ospite che gira dentro la macchina virtuale.

#### Risposte del capitolo 3

**3.1 A** — la struttura a livelli è la più pulita da progettare, ma la *meno* efficiente: l'overhead sta nelle chiamate nidificate.

**3.2 B** — il confronto si espone in termini di come comunicano i componenti: moduli nel kernel con chiamate dirette e nessun messaggio, microkernel con componenti in spazio utente e scambio di messaggi.

**3.3 B** — l'hypervisor di tipo 2 è un processo dentro un SO *host*; l'opzione A descrive il tipo 1.

### 23.1.4 Capitolo 4 — I processi

#### 4.1 In UNIX, che cosa fa il processo figlio creato con `fork`?

- A. Esegue un nuovo programma, indicato dal padre come parametro di `fork`.
- B. È una copia del padre e sta eseguendo lo stesso codice del padre, dalla stessa riga.
- C. Esegue lo stesso codice del padre, ma riparte dalla prima istruzione del programma.
- D. Viene creato direttamente per eseguire un programma nuovo, in un solo passo, come `CreateProcess` su Windows.



**4.2** Qual è la differenza fra lo stato *Pronto* e lo stato *In attesa*?

- A. In *Pronto* il processo aspetta la CPU; in *In attesa* aspetta un evento esterno.
- B. In *Pronto* il processo aspetta un evento esterno; in *In attesa* aspetta solo di ottenere la CPU.
- C. *Pronto* è uno degli stati addizionali, mentre *In attesa* è uno dei tre stati principali.
- D. In *In attesa* il processo sta effettivamente usando la CPU, mentre in *Pronto* no.

**4.3** Come sono organizzate le code in cui il sistema operativo tiene i processi?

- A. Una coda dei processi pronti per ciascun processore e una sola coda dei dispositivi.
- B. Un'unica coda dei processi pronti e una coda dei dispositivi per ciascun dispositivo di I/O.
- C. Un'unica coda che contiene sia i processi pronti sia quelli in attesa di I/O.
- D. Una coda per ciascuno stato del processo, compresi gli stati Nuovo e Terminato.

#### Risposte del capitolo 4

- 4.1 B** — `fork` duplica il processo: il figlio non fa partire un programma nuovo, e non riparte dall'inizio, ma dalla stessa riga del padre.
- 4.2 A** — *Pronto* aspetta la CPU, e l'evento che lo sblocca è una decisione dello scheduler; *In attesa* aspetta un evento esterno. È per questo che da *In attesa* non si passa mai direttamente a *In esecuzione*.
- 4.3 B** — le strutture collegate sui PCB sono due tipi: la coda dei processi pronti, che è unica, e le code dei dispositivi, una per ciascun dispositivo di I/O.

### 23.1.5 Capitolo 5 — I thread

**5.1** Che cosa è proprio di un singolo thread e non è condiviso con gli altri thread dello stesso processo?

- A. Lo spazio degli indirizzi
- B. I file aperti
- C. PC, registri, stack e stato
- D. Gli allarmi pendenti

**5.2** Quale operazione fa sì che un thread si blocchi aspettando che un altro thread sia terminato?

- A. `thread_create`
- B. `thread_exit`
- C. `thread_yield`
- D. `thread_join`

**5.3** Nel modello ibrido (*molti-a-molti*), come sono organizzati i thread?

- A. Molti thread utente su un solo thread del kernel: il *dispatching* è una semplice chiamata di libreria, ma una chiamata bloccante ferma tutti.
- B. Un thread del kernel per ogni thread utente: è più robusto, ma il cambio di contesto richiede un trap nel kernel.
- C. Un certo numero di thread del kernel, e ognuno di essi viene assegnato a un certo numero di thread utente, eventualmente uno solo; l'assegnazione è decisa dal programmatore.
- D. Il numero di thread utente è sempre uguale al numero di thread del kernel, e l'assegnazione la decide il kernel.

## Risposte del capitolo 5

- 5.1 **C** — il thread è caratterizzato da PC, registri, stack e stato, e condivide tutto il resto (spazio di indirizzi, file aperti, allarmi) con gli altri thread del processo.
- 5.2 **D** — `thread_join` è un'attesa: il thread si blocca finché l'altro non è terminato; `thread_yield` invece rilascia volontariamente la CPU e il thread resta vivo.
- 5.3 **C** — nel modello multi-a-molti si hanno  $n$  thread utente su  $m$  thread del kernel con  $m < n$ , e l'assegnazione è decisa dal programmatore; A descrive l'1-a-molti e B l'1-a-1.

## 23.2 Parte II — Processi, thread e sincronizzazione

## 23.2.1 Capitolo 6 — Sezioni critiche e mutua esclusione

- 6.1 L'alternanza stretta (la variabile `turn`) è una soluzione con attesa attiva al problema della sezione critica. Quale delle quattro condizioni di una buona soluzione, secondo il capitolo, viene violata da questa soluzione?
- A. La condizione 1: due processi possono trovarsi insieme nella propria sezione critica.
  - B. La condizione 2: la soluzione presuppone che i processi siano due, su una sola CPU e a velocità simile.
  - C. La condizione 3: un processo fuori dalla propria sezione critica può bloccare un altro processo.
  - D. La condizione 4: un processo resta all'infinito in attesa di entrare nella propria sezione critica.
- 6.2 Che cosa rende speciale l'istruzione `TSL` (*Test and Set Lock*)?
- A. Legge il valore di `lock` nel registro e contemporaneamente vi scrive 1: l'operazione è atomica e blocca il bus di memoria, quindi nessun altro processore può accedere alla memoria fra la lettura e la scrittura.
  - B. Elimina anche il busy waiting, perché il processo che non ottiene il lock viene rimosso dai processi pronti.
  - C. È l'unico modo per realizzare in modo atomico le operazioni su un semaforo.
  - D. Funziona solo sui sistemi con una sola CPU, perché richiede che gli interrupt siano disabilitati.
- 6.3 Con la semantica *signal & continue* (il monitor di Mesa, quello usato in Java), perché il capitolo prescrive di ricontrollare la condizione in un ciclo `while` invece di usare un `if`?
- A. Perché chi ha segnalato sospende sé stesso e cede il monitor al processo risvegliato, che riprenderà dopo.
  - B. Perché chi ha segnalato continua a eseguire, e il processo risvegliato dovrà riacquisire il monitor: nel frattempo la condizione potrebbe essere cambiata.
  - C. Perché il processo risvegliato non deve riacquisire il monitor, quindi può osservare uno stato inconsistente.
  - D. Perché in Java ogni variabile condizione non ha una coda di attesa propria, e la `signal` può risvegliare il processo sbagliato.

## Risposte del capitolo 6

- 6.1 **C** — l'alternanza stretta non rompe la mutua esclusione: il difetto sono i rigidi turni, per cui un processo che sta fuori dalla sezione critica blocca l'altro.

- 6.2 A** — TSL legge il lock nel registro e vi scrive 1 in un'unica operazione atomica, che blocca il bus di memoria. Risolve la mutua esclusione ma **non** il busy waiting.
- 6.3 B** — con Mesa chi segnala continua a eseguire, quindi fra la **signal** e la ripresa del risvegliato lo stato può cambiare: serve il **while**.

### 23.2.2 Capitolo 7 — I problemi classici

- 7.1** Nella soluzione 1 del problema dei filosofi (ogni filosofo prende prima la forchetta alla propria sinistra, poi quella alla destra), qual è il difetto che il capitolo indica?
- A.** I filosofi prendono le forchette in modo non atomico, quindi due filosofi possono impossessarsi della stessa forchetta.
  - B.** Ogni filosofo trattiene ciò che ha già preso: se tutti e cinque afferrano contemporaneamente la forchetta alla propria sinistra, ognuno aspetta quella del vicino, e il difetto sta nell'ordine di acquisizione delle risorse.
  - C.** Manca la mutua esclusione sulle forchette: due filosofi adiacenti possono usarle insieme.
  - D.** La soluzione non si blocca mai, ma è inefficiente perché i filosofi riprovano in continuazione (*livelock*).
- 7.2** Qual è il difetto che il capitolo segnala nella soluzione con i semafori del problema dei lettori e scrittori?
- A.** È un deadlock: i lettori e lo scrittore si aspettano a vicenda e il sistema resta fermo.
  - B.** I lettori non possono accedere al database contemporaneamente, quindi il vantaggio della lettura concorrente va perso.
  - C.** Lo scrittore potrebbe attendere per un tempo indefinito: se i lettori arrivano di continuo, **rc** non torna mai a zero e **db** non viene mai rilasciato — è una *starvation*, non un deadlock.
  - D.** Due scrittori possono modificare il database nello stesso momento.
- 7.3** Nella versione a monitor del problema dei lettori e scrittori, che cosa distingue la soluzione n. 3 dalla n. 2?
- A.** In **end\_write** si preferisce risvegliare uno scrittore (**in\_queue(write)**) invece di un lettore: si dà la precedenza agli scrittori.
  - B.** In **start\_read** la condizione di attesa diventa **busy\_on\_write OR in\_queue(write)**.
  - C.** Si introduce la variabile **busy\_on\_write**, che nella versione n. 2 non esisteva.
  - D.** Si elimina il contatore **rc**: il numero dei lettori attivi si ricava da **in\_queue(read)**.

#### Risposte del capitolo 7

- 7.1 B** — il difetto non è la mutua esclusione (le forchette sono correttamente inaccessibili a due filosofi insieme) né l'atomicità: ogni filosofo trattiene la forchetta già presa, e si ha un deadlock per l'ordine di acquisizione.
- 7.2 C** — il meccanismo “il primo lettore prende **db**, l'ultimo lo rilascia” non dà garanzie sull'ordine di arrivo: lo scrittore può attendere all'infinito. È una *starvation*, non un deadlock, perché il sistema prosegue.
- 7.3 A** — la n. 2→n. 3 cambia una sola riga, in **end\_write**: si preferisce svegliare uno scrittore, ed è il passo che dà la precedenza agli scrittori.

## 23.3 Parte III — Scheduling

### 23.3.1 Capitolo 8 — Scheduler e dispatcher

8.1 Che cos'è la latenza di dispatch?

- A. Il tempo che lo scheduler impiega a decidere quale processo mandare in esecuzione.
- B. Il tempo che un processo passa nella coda dei pronti senza essere eseguito.
- C. L'intervallo fra l'arrivo di un processo e il primo istante in cui ottiene la CPU.
- D. Il tempo necessario a effettuare il passaggio di contesto da un processo all'altro: salvare lo stato del processo uscente, caricare quello del processo entrante e, con la memoria virtuale, riprogrammare la MMU. È tempo speso dal sistema e non dal programma.

8.2 Secondo il capitolo, come si ottiene il tempo di attesa di un processo?

- A.  $\text{attesa} = \text{turnaround} + \text{durata del burst}$
- B.  $\text{attesa} = \text{istante di conclusione} - \text{istante di arrivo}$
- C.  $\text{attesa} = \text{turnaround} - \text{durata del burst}$
- D.  $\text{attesa} = \text{tempo di risposta} - \text{durata del burst}$

8.3 Quali sono gli obiettivi tipici dei sistemi *real-time*, secondo il capitolo?

- A. Il rispetto delle scadenze (*deadline*) e la prevedibilità.
- B. Massimizzare il throughput e minimizzare il tempo di turnaround.
- C. Minimizzare il tempo di risposta.
- D. L'equità nell'assegnazione della CPU e il bilanciamento nell'uso delle risorse.

#### Risposte del capitolo 8

- 8.1 **D** — la latenza di dispatch è il tempo del passaggio di contesto: salvataggio e caricamento dello stato, più la riprogrammazione della MMU. È tempo del sistema, non del programma.
- 8.2 **C** — è l'unica formula del capitolo che vale la pena mandare a memoria: l'attesa è il turnaround meno il tempo in cui il processo ha effettivamente lavorato.
- 8.3 **A** — scadenze e prevedibilità sono gli obiettivi dei real-time; throughput e turnaround sono dei batch, il tempo di risposta degli interattivi, equità e bilanciamento sono obiettivi comuni.

### 23.3.2 Capitolo 9 — Gli algoritmi di scheduling

9.1 Il capitolo mette in evidenza che lo SJF è ottimale solo a una condizione. Quale?

- A. Solo se tutti i lavori hanno la stessa durata.
- B. Solo se i lavori sono tutti CPU-bounded.
- C. Solo se il sistema è interattivo e non batch.
- D. Solo se i lavori sono tutti disponibili subito.

9.2 Nel Round-Robin, che cosa comporta un quanto di tempo *troppo piccolo*?

- A. Il comportamento degenera verso il FCFS: chi arriva prima tiene la CPU a lungo.
- B. Il sistema è reattivo, ma il costo del cambio di contesto diventa una frazione enorme del tempo: con un quanto di 1 ms e un passaggio di contesto di 0,1 ms, il 10% della CPU se ne va in contabilità.
- C. Il costo del cambio di contesto diventa trascurabile rispetto al quanto.
- D. Il tempo di risposta peggiora, perché i processi tornano in fondo alla coda più spesso.

**9.3** Nello scheduling a priorità, che cosa sono la *starvation* e l'*aging*?

- A. La starvation è un'attesa circolare fra processi che si aspettano a vicenda (A aspetta B, B aspetta A) e il sistema resta fermo; l'aging è il tempo che un processo trascorre in coda.
- B. La starvation è il difetto del FCFS che penalizza i processi corti; l'aging è l'effetto convoglio che ne deriva.
- C. La starvation riguarda solo i problemi di sincronizzazione (come lo scrittore nei lettori-scrittori) e non lo scheduling; l'aging è una tecnica di scheduling dei sistemi real-time.
- D. La starvation è l'attesa indefinita di un processo a bassa priorità, che non ottiene mai la CPU perché c'è sempre qualche processo a priorità più alta pronto; l'aging è la soluzione, e consiste nell'aumentare progressivamente la priorità dei processi che attendono da troppo tempo.

**Risposte del capitolo 9**

- 9.1 D — all'istante 0 lo SJF non può sapere che più tardi arriveranno lavori più corti. È la condizione che manca nell'esempio delle slides, dove lo SJF produce un'attesa media di 2,8 mentre una sequenza alternativa arriva a 2,6.
- 9.2 B — un quanto troppo piccolo rende il sistema reattivo ma fa esplodere il peso del cambio di contesto; un quanto troppo *grande* è ciò che fa degenerare il RR verso il FCFS.
- 9.3 D — starvation è attesa indefinita (il sistema prosegue, non c'è nessun ciclo di attese: non è un deadlock); l'aging è la cura, e consiste nell'alzare progressivamente la priorità di chi attende da troppo tempo.

**23.3.3 Capitolo 10 — Scheduling dei thread e su sistemi multi-processore****10.1** Con i thread utente, perché il sistema run-time della libreria può usare solo algoritmi non-preemptive?

- A. Perché la prelazione richiede un interrupt, e l'interrupt arriva al processo (cioè al kernel), che non ha modo di sapere che il processo contiene più thread: la libreria riceve il controllo solo quando un thread glielo cede volontariamente.
- B. Perché i thread utente non hanno una pila propria e quindi non possono essere sospesi.
- C. Perché il kernel conosce i thread ma non sa a quale processo appartengono.
- D. Perché solo gli algoritmi non-preemptive garantiscono la mutua esclusione fra i thread dello stesso processo.

**10.2** Nella multielaborazione simmetrica (SMP), quando è necessario il bilanciamento del carico?

- A. Sempre: sia con la coda unificata sia con le code separate.
- B. Solo in presenza di code distinte, una per processore: con la coda unificata il bilanciamento è automatico, perché un processore libero prende il primo processo disponibile.
- C. Solo con la coda unificata, che essendo una struttura condivisa richiede sincronizzazione a ogni accesso.
- D. Mai: ogni processore può schedare da sé, quindi il carico si distribuisce da solo.

**10.3** Secondo la parte conclusiva del capitolo, che cosa caratterizza Linux?

- A. Il *Mach scheduler*, basato su code di priorità con euristiche.
- B. Euristiche per migliorare il servizio dei processi interattivi, in particolare quelli in *foreground*, e per evitare il problema dell'inversione di priorità.
- C. Lo scheduling basato sui *task*, intesi come generalizzazione di processi e thread, e il *Completely Fair Scheduler* (CFS), un moderno scheduler garantito.

D. La multielaborazione asimmetrica: uno dei processori assume il ruolo di *master server*.

#### Risposte del capitolo 10

- 10.1 A — la prelazione richiede un interrupt, e l'interrupt arriva al processo: il kernel vede un solo processo, e la libreria in spazio utente riceve il controllo solo quando un thread le cede volontariamente.
- 10.2 B — il bilanciamento serve solo con le code distinte, dove il carico non si bilancia da solo; con la coda unificata il difetto è invece la contesa sulla struttura condivisa.
- 10.3 C — Linux ragiona in termini di *task* (generalizzazione di processi e thread) e usa il CFS; le code di priorità con euristiche sono di altri sistemi, e la multielaborazione asimmetrica è solo il modello più semplice fra i due approcci di base.

## 23.4 Parte IV — La memoria

### 23.4.1 Capitolo 11 — Gestione della memoria

- 11.1 Nella rilocalizzazione dinamica, che cosa realizzano rispettivamente il registro *base* e il registro *limite*?
- A. Il base realizza la protezione, il limite realizza la rilocalizzazione.
  - B. Il base realizza la rilocalizzazione, il limite realizza la protezione.
  - C. Entrambi realizzano la protezione, e la rilocalizzazione è compito della MMU.
  - D. Il base indica quanto è grande lo spazio del processo, il limite dove comincia.
- 11.2 Quale tipo di frammentazione è tipico delle partizioni a dimensione fissa?
- A. La frammentazione interna, cioè lo spazio sprecato *dentro* la partizione assegnata al processo.
  - B. La frammentazione esterna, cioè i buchi liberi sparsi che nessuno è grande abbastanza per riempire.
  - C. Nessuna delle due: le partizioni a dimensione fissa eliminano la frammentazione.
  - D. Entrambe: dipende solo dalla dimensione della partizione scelta.
- 11.3 Che cosa distingue l'algoritmo *next fit* dal *first fit* nella ricerca di un blocco libero?
- A. Il next fit sceglie il blocco *più piccolo* fra quelli abbastanza grandi.
  - B. Il next fit riprende la ricerca da dove si era fermata l'ultima volta.
  - C. Il next fit tiene la lista ordinata per dimensione invece che per indirizzo.
  - D. Il next fit sceglie il blocco *più grande* disponibile.

#### Risposte del capitolo 11

- 11.1 B — i due registri svolgono due funzioni diverse: uno trasforma gli indirizzi (rilocalizzazione), l'altro impedisce al processo di uscire dal proprio spazio (protezione).
- 11.2 A — la partizione fissa viene assegnata interamente al processo, e ciò che avanza resta inutilizzabile da altri.
- 11.3 B — il next fit è il first fit con la ricerca che non riparte ogni volta dall'inizio della lista.

### 23.4.2 Capitolo 12 — La memoria virtuale

- 12.1** Nella traduzione di un indirizzo virtuale in indirizzo fisico, quale parte dell'indirizzo rimane invariata?
- A. Il numero di pagina virtuale, che viene sostituito dal numero di frame.
  - B. L'offset, cioè i bit meno significativi dell'indirizzo virtuale.
  - C. Il bit di presenza, che viene copiato nella voce del frame.
  - D. Tutto l'indirizzo, che viene solo incrementato del valore del registro base.
- 12.2** Perché nella TLB si usano gli *address-space identifiers* (ASID)?
- A. Perché una voce della TLB appartiene a un processo, e senza di essi bisognerebbe svuotare la TLB a ogni cambio di contesto.
  - B. Perché la TLB contiene un numero ridotto di registri e non basta a coprire tutte le pagine di un processo.
  - C. Perché gli ASID velocizzano la ricerca associativa all'interno della TLB.
  - D. Perché servono a far condividere le stesse pagine a processi diversi.
- 12.3** Perché le tabelle delle pagine multilivello fanno risparmiare memoria?
- A. Perché riducono la dimensione *totale* della tabella anche se tutte le pagine fossero in uso.
  - B. Perché permettono di *non allocare* le parti di tabella che non servono.
  - C. Perché eliminano la necessità della TLB.
  - D. Perché con due livelli servono solo due accessi alla memoria invece di tre.

#### Risposte del capitolo 12

- 12.1 B** — si sostituisce la parte alta dell'indirizzo, quella bassa resta dov'è: il byte dentro la pagina non si sposta.
- 12.2 A** — l'alternativa agli ASID è svuotare la TLB a ogni cambio di contesto, e il flush si paga ogni volta anche se i due processi usano le stesse pagine.
- 12.3 B** — il risparmio non è sul totale, ma sul fatto che le tabelle di secondo livello delle zone vuote semplicemente non esistono.

### 23.4.3 Capitolo 13 — Gli algoritmi di sostituzione delle pagine

- 13.1** In NRU l'ordine delle classi mette la classe 1 (R=0, M=1) prima della classe 2 (R=1, M=0). Che cosa dice questa scelta?
- A. Che una pagina modificata ma non usata di recente è considerata più sacrificabile di una non modificata ma usata di recente.
  - B. Che il bit di modifica conta più del bit di referenziamento.
  - C. Che le pagine della classe 0 vanno salvate su disco prima di essere rimosse.
  - D. Che l'ordine delle classi è arbitrario e non ha significato.
- 13.2** Perché nei sistemi reali si trova il *clock* e non la seconda chance, se i due algoritmi si comportano allo stesso modo?
- A. Perché il clock rimuove pagine diverse da quelle rimosse dalla seconda chance.
  - B. Perché il clock non richiede di spostare elementi: si muove solo un indice.
  - C. Perché il clock non ha bisogno del bit di referenziamento R.
  - D. Perché il clock non soffre dell'anomalia di Belady.

- 13.3** LRU non soffre dell'anomalia di Belady perché gode della *proprietà di inclusione*. Che cosa afferma questa proprietà?
- A. Che l'insieme delle pagine caricate avendo  $n$  frame è sempre contenuto in quello che si avrebbe con  $n + 1$  frame.
  - B. Che l'insieme delle pagine caricate con  $n + 1$  frame è sempre contenuto in quello che si avrebbe con  $n$  frame.
  - C. Che l'ordine di espulsione dipende dall'ordine di *caricamento* delle pagine.
  - D. Che aggiungendo frame il numero di page fault diminuisce sempre, qualunque sia l'algoritmo.

#### Risposte del capitolo 13

- 13.1** A — si butta per prima la pagina che non serve, e fra le due si preferisce perdere quella il cui salvataggio costa, non quella che vale: il costo del salvataggio conta meno del valore d'uso.
- 13.2** B — i due algoritmi sono equivalenti per comportamento, e la differenza è tutta nel costo di implementazione.
- 13.3** A — con più frame non si può stare peggio, perché l'insieme delle pagine con  $n$  frame è sempre un sottoinsieme di quello con  $n + 1$  frame.

#### 23.4.4 Capitolo 14 — Altri aspetti della gestione della memoria

- 14.1** Perché, oltre un certo punto, aumentare il livello di multiprogrammazione fa *crollare* l'utilizzazione della CPU?
- A. Perché con l'allocazione globale i frame si spostano verso il processo appena arrivato, che non ha ancora caricato il suo working set, e vengono tolti a processi che stavano lavorando bene.
  - B. Perché ogni processo in più richiede l'aggiornamento dei registri base e limite a ogni context switch.
  - C. Perché il working set di ogni processo va ricalcolato a ogni interrupt periodico.
  - D. Perché il paging daemon non riesce più a garantire il pool di frame liberi.
- 14.2** Quale vantaggio è associato alle pagine di dimensione *piccola*?
- A. Una tabella delle pagine più piccola.
  - B. Minore frammentazione interna e migliore risoluzione nel definire il working set.
  - C. Un trasferimento I/O più efficiente.
  - D. Un numero minore di page fault.
- 14.3** Perché il copy-on-write rende *economica* la creazione di un processo con **fork**?
- A. Perché le pagine sono condivise e marcate sola lettura, e la copia avviene solo alla prima scrittura: quasi sempre il figlio esegue subito una **exec**, che sostituisce tutto lo spazio.
  - B. Perché le pagine condivise vengono copiate subito, così il figlio non deve aspettare al primo accesso.
  - C. Perché le nuove pagine sono vuote e allocate su richiesta, e quindi non occupano memoria finché non si scrive.
  - D. Perché la copia delle pagine avviene durante il context switch, quando la CPU non ha altro da fare.



**Risposte del capitolo 14**

- 14.1 A** — chi faulta di più prende i frame di chi faultava di meno, che comincia a faultare a sua volta, e la CPU passa il tempo a servire page fault.
- 14.2 B** — con pagine piccole si spreca meno spazio dentro la pagina e il working set stimato è più preciso.
- 14.3 A** — copiare subito l'intero spazio degli indirizzi sarebbe costosissimo, e quasi sempre sarebbe lavoro sprecato.

**23.5 Parte V — I file system****23.5.1 Capitolo 15 — I file system**

- 15.1** Che cosa significa che un lock su un file è *advisory*?
- A. Che è il sistema a farlo rispettare: chi prova ad accedere senza il lock viene bloccato.
  - B. Che il sistema lo offre, ma sta al processo decidere se rispettarlo.
  - C. Che più processi possono tenerlo contemporaneamente, in lettura.
  - D. Che un solo processo può tenerlo, e nessun altro, in scrittura.
- 15.2** Un processo ha aperto un file e ne ha letto metà del contenuto. A chi appartiene la posizione corrente di lettura?
- A. Al file: è uno dei suoi metadati, uguale per tutti i processi che lo aprono.
  - B. Al processo, insieme alla modalità di apertura: due processi che leggono lo stesso file hanno posizioni diverse.
  - C. Alla struttura dati globale del sistema, che tiene traccia dei file aperti.
  - D. Alla cache dei blocchi già letti, che sa a che punto è arrivata la lettura.
- 15.3** Che cosa contiene il *superblocco* di una partizione?
- A. Il codice per avviare il sistema operativo.
  - B. I parametri del file system: quanti blocchi, quanto è grande un blocco, dove comincia l'area degli i-node.
  - C. La tabella delle partizioni del disco.
  - D. I metadati di tutti i file.

**Risposte del capitolo 15**

- 15.1 B** — l'*advisory* è una convenzione, non una difesa: non protegge da un processo che se ne frega.
- 15.2 B** — la posizione corrente è informazione del processo, non del file: due processi che leggono lo stesso file hanno posizioni diverse.
- 15.3 B** — il superblocco contiene i parametri del file system, ed è il “punto di ingresso” della struttura: se ne tiene spesso una copia di riserva.

**23.5.2 Capitolo 16 — Progettazione di un file system**

- 16.1** Quale metodo di allocazione tiene i puntatori in una tabella a parte, che ha una voce per *ogni* blocco del disco, anche per quelli che non appartengono a nessun file?
- A. L'allocazione contigua, che registra il blocco iniziale e la lunghezza

- B. L'allocazione concatenata, che mette in ogni blocco un puntatore al blocco successivo
  - C. L'allocazione tabellare, cioè la FAT
  - D. L'allocazione indicizzata, che ha una struttura per ogni file
- 16.2** Perché conviene tenere gli attributi in una struttura separata puntata dalla voce di directory, invece che dentro la voce stessa?
- A. Perché la ricerca di un nome legge solo il nome e un puntatore, e la directory sta in pochi blocchi
  - B. Perché così i nomi lunghi possono avere lunghezza variabile dentro la voce
  - C. Perché così gli attributi non servono più quando si apre il file
  - D. Perché così la directory non ha bisogno né di tabella hash né di cache
- 16.3** Che cosa può fare un soft-link che un hard link non può fare?
- A. Mantenere un contatore dei link che dice quanti nomi puntano allo stesso i-node
  - B. Puntare a un file di un altro file system, e perfino a un file che non esiste
  - C. Evitare che lo spazio del file venga contato due volte dall'accounting
  - D. Fare in modo che cancellare un nome decrementi il contatore dei riferimenti del file

#### Risposte del capitolo 16

- 16.1 C** — la tabella della FAT ha una voce per ogni blocco del disco, anche per quelli che non appartengono a nessun file: è il difetto che la fa crescere con il disco invece che con i file.
- 16.2 A** — se la voce contiene solo il nome e un puntatore, la ricerca confronta meno dati e la directory sta in pochi blocchi, quindi si scorre in fretta.
- 16.3 B** — l'universalità del soft-link: può puntare fuori dal file system, e anche a un file inesistente (collegamento pendente). È il prezzo che paga in velocità di attraversamento rispetto all'hard link.

### 23.5.3 Capitolo 17 — Consistenza, journaling e cache

- 17.1** Nel controllo di consistenza sui blocchi, in quale caso due file diversi possono finire per sovrasciversi?
- A. Quando un blocco non appartiene a nessuno, perché entrambe le mappe dicono 0
  - B. Quando un blocco è contato due volte nella mappa dei blocchi liberi
  - C. Quando un blocco è contato due volte nella mappa dei blocchi occupati
  - D. Quando il contatore dei link di un i-node non corrisponde al numero di voci di directory che lo puntano
- 17.2** Perché il journaling richiede operazioni idempotenti?
- A. Perché al riavvio il ripristino non sa se l'operazione è già stata eseguita, e la rifà comunque
  - B. Perché il log viene ripulito a posteriori, subito dopo ogni operazione
  - C. Perché il journaling deve proteggere anche il contenuto dei file, non solo i metadati
  - D. Perché anche i dati, e non solo i metadati, vengono appuntati nel log
- 17.3** Qual è il rischio della scrittura asincrona?
- A. Un crash prima che la cache venga svuotata perde i dati
  - B. Il processo resta in attesa di un'operazione meccanica che dura millisecondi
  - C. Le scritture non possono essere riordinate e si accumulano nella cache
  - D. Il dato arriva davvero sul disco, ma il file system non può più riordinare le scritture

**Risposte del capitolo 17**

- 17.1 B** — il doppio conteggio fra i blocchi liberi è il caso grave: se lo spazio viene allocato due volte, due file diversi si sovrascrivono.
- 17.2 A** — il ripristino non ha modo di distinguere i due casi e ripete l'operazione: se l'operazione non fosse idempotente, rifarla sarebbe un errore.
- 17.3 A** — la scrittura asincrona si considera conclusa quando il dato è in cache: è veloce, ma un crash prima che la cache venga svuotata perde i dati.

**23.6 Parte VI — Dispositivi e affidabilità****23.6.1 Capitolo 18 — Scheduling del disco**

- 18.1** Da che cosa dipende il tempo di posizionamento (*seek-time*) di un disco a piatti multipli?
- A. Dal cilindro, perché le testine sono solidali al braccio e si muovono insieme: non conta quale piatto
  - B. Dal piatto scelto, perché ogni piatto ha una testina con un braccio indipendente
  - C. Dalla velocità di rotazione, perché il settore giusto deve passare sotto la testina
  - D. Dalla densità del disco, cioè da quanti bit stanno su una traccia
- 18.2** Qual è il difetto di SSTF messo in evidenza dal capitolo?
- A. Percorre più strada di FCFS, perché non sfrutta l'ordine di arrivo delle richieste
  - B. Può lasciare una richiesta in attesa senza limite, perché sceglie sempre la richiesta più vicina a quella attuale
  - C. Non è semplice da realizzare, perché richiede di costruire l'ordine un passo per volta
  - D. Non tiene conto della posizione in cui si trova la testina
- 18.3** In che cosa differiscono, sull'esempio del capitolo, l'ordine della scansione e quello della scansione circolare?
- A. La scansione, dopo 183, scende servendo 37 e 14; la scansione circolare rientra a vuoto e serve 14 e 37 risalendo
  - B. La scansione serve 183 per ultimo, mentre la scansione circolare lo serve per primo
  - C. La scansione circolare non serve mai i cilindri più bassi di quello di partenza
  - D. La scansione inverte il verso a ogni richiesta, mentre la scansione circolare lo mantiene sempre

**Risposte del capitolo 18**

- 18.1 A** — lo spostamento della testina si misura in cilindri, non in piatti: le testine sono solidali al braccio.
- 18.2 B** — è la starvation: una richiesta lontana non viene mai scelta finché continua ad arrivare qualcosa di più vicino.
- 18.3 A** — è l'unica differenza fra i due ordini: la prima scende servendo, la seconda rientra a vuoto.

**23.6.2 Capitolo 19 — I sistemi RAID**

- 19.1** Che cosa succede all'affidabilità del volume se si mettono in striping quattro dischi che, singolarmente, si guastano con probabilità dell'1% in un anno?

- A. Resta l'1%, perché i guasti dei dischi sono eventi indipendenti
- B. Sale a circa il 4%, perché basta il guasto di uno qualunque dei quattro dischi
- C. Resta l'1%, come se ci fosse un disco solo, perché il volume è affidabile quanto il disco meno affidabile che contiene
- D. Scende sotto l'1%, perché nessun disco da solo contiene il file intero

**19.2** Nel RAID 1, che cosa raddoppia e che cosa resta invariato?

- A. Raddoppia il numero di letture al secondo, mentre il costo in scrittura non si riduce
- B. Raddoppia la velocità di scrittura, mentre le letture restano invariate
- C. Raddoppia la capacità utile, mentre l'affidabilità resta quella di un disco solo
- D. Raddoppiano sia le letture sia le scritture, mentre la capacità utile resta invariata

**19.3** Perché il RAID 5 “di fatto sostituisce” il RAID 4?

- A. Perché usa lo striping a livello di bit invece che a livello di blocchi
- B. Perché distribuisce i blocchi di parità su tutti i dischi, eliminando il collo di bottiglia del disco di parità scritto a ogni operazione
- C. Perché non ha bisogno di ridondanza, risparmiando un disco per gruppo
- D. Perché non richiede la sincronia delle rotazioni, che invece serviva nel RAID 4

#### Risposte del capitolo 19

- 19.1 B** — basta il guasto di un disco solo, quindi la probabilità di perdere il volume si moltiplica: con quattro dischi è all'incirca quattro volte quella di perderne uno.
- 19.2 A** — le due copie permettono di servire due letture in parallelo, ma ogni scrittura va fatta su entrambe: il numero di letture al secondo raddoppia, la scrittura no.
- 19.3 B** — la parità ruota su tutti i dischi, così nessun disco è quello delle scritture obbligate.

### 23.6.3 Capitolo 20 — Memorie flash e dischi a stato solido

**20.1** In una memoria flash, qual è l'unità di cancellazione e quale quella di lettura e scrittura?

- A. Si cancella per blocchi interi, mentre lettura e scrittura avvengono per pagine
- B. Si cancella per pagine, mentre lettura e scrittura avvengono per blocchi interi
- C. Si cancella e si legge sempre una pagina per volta
- D. Si cancella e si legge sempre un blocco intero per volta

**20.2** Perché i file system classici, basati sulla FAT o sugli i-node, sono inadatti a una memoria flash?

- A. Perché aggiornano le strutture “sul posto”, e le zone calde riscritte a ogni operazione esaurirebbero per prime le cancellature
- B. Perché non tengono traccia dei blocchi liberi con una bitmap
- C. Perché richiedono che ogni blocco sia cancellato prima di poterlo leggere
- D. Perché non prevedono alcuna cache dei blocchi, e su flash ogni lettura consuma una cancellazione

**20.3** Perché il garbage collector da solo non basta, e che cosa aggiunge l'operazione TRIM?

- A. Perché al controller manca l'informazione su quali pagine non servono più, e TRIM è il messaggio con cui il sistema operativo glielo dice
- B. Perché il garbage collector copia anche le pagine non più valide, e TRIM le cancella

- C. Perché il garbage collector non può cancellare un blocco intero, e TRIM abilita la cancellazione per pagina
- D. Perché il controller conosce i file ma non i blocchi, e TRIM gli manda la mappa dei blocchi liberi

#### Risposte del capitolo 20

- 20.1 A** — il blocco è l'unità di cancellazione e contiene più pagine, che sono l'unità di lettura e scrittura.
- 20.2 A** — le zone calde (FAT, area degli i-node, bitmap, superblocco) si usurano per prime: il dispositivo morirebbe in un punto solo.
- 20.3 A** — il dispositivo non sa che cosa sia un file: è il sistema operativo a dire al controller, con TRIM, quali pagine non servono più.

### 23.7 Due domande nel formato del compito

#### Perché queste due

Le sessanta domande qui sopra hanno quattro opzioni. Queste due ne hanno **cinque**, da A a E, come nel compito tipo — e la seconda, sempre come nel compito tipo, non chiede quale affermazione è corretta ma quale è **falsa**.

Sono due esercizi di lettura, prima che di conoscenza. Con cinque alternative la probabilità di indovinare scende, e nelle domande “quale è falsa” il lavoro si fa in due tempi: prima si cercano le affermazioni che si sanno riconoscere come vere, e solo su quelle che restano si applica il dubbio. È la strategia che rende una domanda a cinque opzioni affrontabile anche quando la risposta secca non viene in mente.

#### Domanda 1 — Quanti frame

Uno spazio di indirizzamento virtuale di **1 Gb** è organizzato in pagine da **8 kB**. L'indirizzo fisico della macchina è lungo **32 bit**. Quanti frame ha la memoria fisica?

- A. 65 536
- B. 131 072
- C. 262 144
- D. 524 288
- E. 1 048 576

#### Risposta: D

Il procedimento, che va scritto e non solo pensato:

1.  $1 \text{ Gb} = 2^{30} \text{ byte}$ , quindi lo spazio virtuale ha **30 bit**;
2. le pagine sono da  $8 \text{ kB} = 2^{13} \text{ byte}$ , quindi l'**offset** è di **13 bit**;
3. il numero di pagina virtuale vale  $30 - 13 = \mathbf{17 \text{ bit}}$ , e le pagine sono  $2^{17} = 131\,072$ ;
4. l'indirizzo fisico è di 32 bit e l'offset è lo stesso (non si traduce), quindi i bit di frame sono  $32 - 13 = \mathbf{19}$ ;
5. i frame sono  $2^{19} = \mathbf{524\,288}$ .

L'errore da evitare è fermarsi al passo 3: le pagine *virtuali* non sono i frame. Il numero di frame dipende dall'indirizzo *fisico*, che qui è più lungo dello spazio virtuale, e infatti i frame sono più delle pagine (capitolo 12).

### Domanda 2 — Quale affermazione è falsa

Quale delle seguenti affermazioni sulla sostituzione delle pagine è **falsa**?

- A. OPT è l'algoritmo ottimo, ma richiede di conoscere il futuro, quindi serve solo come termine di paragone.
- B. L'anomalia di Belady consiste nel fatto che aggiungendo frame il numero di page fault può aumentare invece che diminuire.
- C. FIFO soffre dell'anomalia di Belady, mentre LRU non ne soffre perché gode della proprietà di inclusione.
- D. Il clock e la seconda chance si comportano allo stesso modo, e il clock si preferisce perché non richiede di spostare elementi: muove solo un indice.
- E. NFU conta quante volte una pagina è stata usata di recente, e per questo è un'approssimazione migliore di LRU.

#### Risposta: E

È falsa per una parola di troppo: *di recente*. NFU conta quante volte una pagina è stata usata, ma **non dà peso a quando** è stata usata. Una pagina usata intensamente all'inizio dell'esecuzione ha un contatore alto e resta protetta per sempre, anche se da allora non è stata più toccata: la memoria del contatore è infinita, mentre ciò che interessa è l'uso recente. È esattamente l'errore opposto a quello di FIFO — FIFO ricorda solo l'*età* e ignora l'uso, NFU ricorda solo l'*uso* e ignora quando — ed è il difetto che l'**aging** corregge, con lo scorrimento del contatore e il bit *R* inserito in testa.

Le altre quattro sono vere come sono scritte, e vale la pena notare come sono costruite: la A è la definizione di OPT, la C è la coppia Belady/inclusione, la D è la differenza di *costo* e non di comportamento fra clock e seconda chance. Nessuna delle quattro esagera — ed è proprio l'esagerazione, in una domanda “quale è falsa”, il posto dove guardare (capitolo 13).

#### Che cosa fare adesso

Se hai sbagliato più di una domanda in un capitolo, quel capitolo va *riletto*, non ripassato: la differenza fra le due cose è che ripassare un capitolo che non si è capito non lo fa diventare capito.

Se invece le hai sbagliate sparse un po' ovunque, il problema non è un capitolo: è che manca la mappa. In quel caso conviene tornare al capitolo 24 e usare l'indice delle domande, che costringe a inquadrare ogni argomento prima di scendere nei dettagli.

## Capitolo 24

### Il colloquio orale

---

#### Come usare questo capitolo

Questo capitolo **non ripete le risposte**. Le risposte stanno già in fondo a ciascuno dei venti capitoli, nei riquadri viola *Come lo dici all'orale*: quelle sono le scalette complete, con la struttura, i termini giusti e le trappole.

Qui c'è l'altra metà del lavoro, quella che nessun capitolo può fare da solo: l'**indice delle domande** (per esercitarsi coprendo le risposte), il **metodo** che tutte quelle scalette hanno in comune, e soprattutto le domande che **attraversano più capitoli** — che sono quelle che all'orale vengono davvero chieste, perché a un colloquio di venti minuti non si chiede “mi parli della paginazione”, si chiede “mi parli della memoria virtuale”.

#### 24.1 Come si risponde

Rileggendo le venti scalette dei riquadri orali si vede che fanno tutte la stessa cosa. Vale la pena isolarla, perché è il metodo — e il metodo si impara una volta sola.

##### Le cinque regole

1. **Struttura prima dei dettagli.** Ogni risposta comincia dicendo in quante parti si divide l'argomento, e poi le percorre. “Gli stati di un processo sono cinque: tre principali e due addizionali” è già metà della risposta. Chi comincia dai dettagli non arriva mai in fondo.
2. **Racconta una progressione, non un elenco.** Quasi tutti gli argomenti del corso sono una storia di problemi e soluzioni: un tentativo ingenuo, il motivo per cui fallisce, la soluzione che lo corregge. La gestione della memoria è la progressione dalle partizioni fisse alla paginazione; la sostituzione delle pagine è la progressione da FIFO all'aging; l'allocazione dei file è la progressione da contigua a i-node.
3. **Nomina il fallimento con il termine giusto.** È la differenza più visibile fra chi ha capito e chi ha memorizzato: *deadlock* non è *starvation*, *frammentazione interna* non è *esterna*, *latenza di dispatch* non è *tempo di attesa*.
4. **Se ti chiedono “qual è il migliore?”, rispondi che dipende.** Scheduling della CPU e del disco, strutture dei sistemi operativi, allocazione dei file: in tutti questi casi la risposta corretta è che l'ottimo dipende dall'ambiente (batch, interattivo, real-time) e da *quale* metrica si sta ottimizzando. Dire “dipende” senza spiegare da cosa, però, è peggio che non dirlo.
5. **Porta un numero, se ce l'hai.** “LRU fa 12 fault contro i 15 di FIFO” vale più di un paragrafo di aggettivi. All'orale i numeri sono la prova che hai capito il meccanismo, non solo la definizione.

#### 24.2 L'indice delle domande

Per ogni capitolo, le domande che quel capitolo mette in grado di rispondere. Copri il riquadro viola del capitolo, rispondi a voce, poi confronta.

### Parte I — Il sistema operativo e l'hardware

- **1 — Introduzione.** Che cos'è un sistema operativo (e perché va risposto su due piani)? Che differenza c'è fra multiplexing nel tempo e nello spazio, e a quale dei due appartiene la memoria? Che cos'è la doppia modalità di esecuzione e da quali problemi nasce? Come si sono evoluti i sistemi operativi dalle valvole ai personal computer? Come si classificano i sistemi operativi (batch, time-sharing, real-time, embedded)?
- **2 — Architettura.** Com'è fatto un processore e che cosa fa la pipeline? Che cos'è una chiamata di sistema, e in che cosa differisce da una chiamata di funzione? Perché si usa il DMA? Quali sono le tre modalità di I/O e perché la terza è quella giusta? Che cos'è la gerarchia delle memorie e perché esiste? Che differenza c'è fra multiprocessore e multicore?
- **3 — Struttura dei SO.** Quali sono le strutture possibili di un sistema operativo, e quale è la meno efficiente? Che cos'è un microkernel e quali vantaggi porta? Che cosa sono i moduli caricabili? Mi parli delle macchine virtuali (definizione, scopi, hypervisor di tipo 1 e 2, supporto hardware).

### Parte II — Processi, thread e sincronizzazione

- **4 — Processi.** Che cos'è un processo, e in che cosa differisce da un programma? Mi descriva gli stati di un processo, con il criterio che li distingue e le transizioni possibili — e quella che *non* esiste, con il motivo. Come si crea un processo, e in che cosa differiscono `fork` ed `exec`? Come si rappresenta un processo nel kernel (PCB, contesto)?
- **5 — Thread.** Differenza fra processo e thread (unità di possesso contro unità di esecuzione)? Che cosa condividono i thread di uno stesso processo, e che cosa non condividono? Che cosa sono `thread_yield` e `thread_join`? Quali sono i cinque principi della programmazione multicore? Mi illustri i tre modelli di thread (1-a-molti, 1-a-1, molti-a-molti) con i loro compromessi.
- **6 — Mutua esclusione.** Mi parli della mutua esclusione (la corsa critica, le quattro condizioni, la successione dei tentativi con il motivo del fallimento di ciascuno, il salto di qualità a `sleep/wakeup`, i semafori, i monitor). Che cos'è la corsa del `wakeup` perso, e come la risolvono i semafori? Che differenza c'è fra mutex e semaforo? Mi illustri le tre semantiche della `signal`, e perché con la semantica di Mesa la condizione va ricontrollata in un ciclo? Che cos'è lo scambio di messaggi e quando conviene rispetto alla memoria condivisa?
- **7 — Problemi classici.** Mi parli dei cinque filosofi (che cosa modella, il tentativo ingenuo e il suo difetto, la soluzione corretta, la versione a monitor). Mi parli di lettori e scrittori, con il confronto fra le tre versioni. Perché non basta mettere `take_fork` dentro un mutex? Che differenza c'è fra deadlock e starvation, e quale dei due problemi presenta ciascuno dei due casi classici?

### Parte III — Scheduling

- **8 — Scheduler e dispatcher.** Differenza fra scheduler e dispatcher, e che cos'è la latenza di dispatch? Quando si attiva lo scheduler, e come si comporta nei due casi? Quali sono gli obiettivi di uno scheduler, e perché dipendono dall'ambiente? Che differenza c'è fra processo *I/O bound* e *CPU bound*, e perché conta?
- **9 — Algoritmi.** Mi illustri gli algoritmi di scheduling, distinguendo batch e interattivi.



Che cos'è l'effetto convoglio, e quale algoritmo lo elimina? Che cos'è la starvation in una politica a priorità, e come si cura? Nel confronto FCFS/SJF/SRTN, chi vince sulle medie e chi peggiora? Che cos'è il round robin e da che cosa dipende la scelta del quanto? Se ti chiedono “qual è il migliore?”, che cosa rispondi?

- **10 — Thread e multiprocessore.** Che differenza c'è fra lo scheduling di thread utente e di thread del kernel, e perché con i primi servono algoritmi non preemptive? Quali sono i due approcci allo scheduling multiprocessore, e che cosa cambia fra coda unica e code separate? Che cos'è la predilezione del processore, e perché è in conflitto con il bilanciamento del carico? Perché la riprogrammazione della MMU conta in questo capitolo?

#### Parte IV — La memoria

- **11 — Gestione della memoria.** Mi parli della gestione della memoria, dalla assenza di astrazione allo swapping (i due problemi del modello senza astrazione, i due tentativi di rilocazione e il loro limite, lo spazio degli indirizzi, base e limite, lo swapping con l'I/O pendente, frammentazione interna ed esterna, i quattro algoritmi di ricerca). Perché non si può fare swapping su un processo con I/O pendente, e quali sono i due rimedi? Che differenza c'è fra frammentazione interna ed esterna, e quale delle due non si risolve con la compaction?
- **12 — Memoria virtuale.** Mi parli della memoria virtuale (i due problemi, la paginazione e la traduzione, il doppio accesso e la TLB, il problema della dimensione e le tabelle multilivello, la tabella invertita, i due bit che contano). Che cosa significa che l'offset non si traduce? Che cosa contiene una voce della tabella delle pagine? A che cosa serve davvero la memoria virtuale, se non a “usare più memoria del fisico”?
- **13 — Sostituzione delle pagine.** Mi parli degli algoritmi di sostituzione, dal peggiore al migliore e con i numeri. Qual è il difetto di NFU, e in che senso è l'errore opposto a quello di FIFO? Che cos'è l'anomalia di Belady, e chi ne soffre? Che cos'è la proprietà di inclusione? Perché si usa il clock e non la seconda chance, se fanno la stessa cosa?
- **14 — Aspetti avanzati.** Quanti frame si danno a un processo, e che differenza c'è fra allocazione locale e globale? Che cos'è il thrashing, e perché aggiungere processi può peggiorare l'utilizzazione della CPU? Mi illustri il modello del working set e quello della page fault frequency. Che cos'è il paging daemon? Come si sceglie la dimensione della pagina? Perché la condivisione di pagine è resa complicata dalla cache? A che serve il copy-on-write, e perché non si copia subito? Che cos'è lo slab allocator e perché il kernel ne ha bisogno?

#### Parte V — I file system

- **15 — File system.** Mi parli dei file system (il problema nelle sue tre parole, l'astrazione di file e directory, i dettagli che nasconde, la struttura con superblocco e i-node, il layout moderno). Che differenza c'è fra la struttura dati globale e quella per processo, e a chi appartiene la posizione di lettura? Mi illustri i tipi di lock sui file.
- **16 — Progettazione.** Come si progetta un file system (le quattro domande: dove stanno i blocchi, come si organizzano le directory, come si condivide, quali blocchi sono liberi)? Mi illustri la progressione da allocazione contigua a i-node, dicendo per ciascuna il difetto che la successiva risolve. Che differenza c'è fra hard link e soft link?

Perché la FAT ha problemi in caso di append? Che cos'è l'anomalia con accounting?

- **17 — Consistenza e prestazioni.** Come si mantiene affidabile un file system? Che cosa fanno i controlli di consistenza, e perché sono lenti? Che cos'è il journaling, e perché richiede operazioni idempotenti? Che cosa protegge, e che cosa *non* protegge? Come si migliorano le prestazioni (cache del disco, LRU modificata, free-behind, read-ahead, scrittura sincrona e asincrona)?

#### Parte VI — Dispositivi e affidabilità

- **18 — Scheduling del disco.** Mi parli dello scheduling del disco (il problema, l'esempio con i quattro numeri, i giudizi, la scelta). Che differenza c'è fra scansione e scansione circolare? Perché SSTF affama, e qual è la cura? Che corrispondenze ci sono con lo scheduling della CPU?
- **19 — RAID.** Mi parli dei sistemi RAID (il problema, il prezzo dello striping, i livelli, le due domande che li distinguono). Perché il RAID 3 usa un bit di ridondanza in meno del RAID 2? Qual è il difetto del RAID 4, e come lo risolve il RAID 5? Quanto costa il RAID 1?
- **20 — SSD.** Mi parli degli SSD (che cosa sono, le tre proprietà che contano, perché i file system classici non vanno bene, garbage collection e TRIM). Perché un SSD non è “un disco veloce”? Che cosa cambia nello scheduling del disco e nel RAID? Che succede se manca il TRIM?

### 24.3 Le domande che attraversano i capitoli

Queste sono le domande che all'orale vengono chieste davvero. Sono le più difficili, perché non hanno una risposta in un capitolo solo: bisogna sapere *quali* capitoli convocare e in che ordine.

- **“Mi parli della memoria virtuale.”** È una domanda che copre quattro capitoli, e va percorsa come una storia: *perché* serve (capitolo 11: il modello senza astrazione non regge, servono rilocalizzazione e protezione) → *come* si traduce (capitolo 12: paginazione, offset che non si traduce, TLB e EAT, multilivello) → *cosa* si sostituisce quando i frame finiscono (capitolo 13: OPT come limite, FIFO e la sua anomalia, LRU e le sue approssimazioni) → *quanti* frame a chi e come si evita il thrashing (capitolo 14: working set, PFF, paging daemon).
- **“Mi parli della sincronizzazione.”** Due capitoli: *le primitive* (capitolo 6: corsa critica, quattro condizioni, i tentativi con attesa attiva e il loro fallimento, **sleep/wakeup** e la corsa del wakeup perso, semafori, monitor) e *gli usi* (capitolo 7: filosofi e lettori-scrittori, con deadlock e starvation come patologie distinte).
- **“Mi parli del file system.”** Tre capitoli: *l'astrazione* (capitolo 15: file, directory, metadati, strutture globale e per processo, lock) → *l'implementazione* (capitolo 16: le quattro domande, contigua/concatenata/FAT/i-node, link, blocchi liberi) → *affidabilità e prestazioni* (capitolo 17: consistenza, journaling, cache).
- **“Mi parli dell'I/O.”** Quattro capitoli, dal basso verso l'alto: *come* il processore parla ai dispositivi (capitolo 2: le tre modalità, il DMA) → *in che ordine* si servono le richieste (capitolo 18: FCFS, SSTF, scansione, circolare) → *come* si mettono insieme più dischi (capitolo 19: striping, ridondanza, i livelli) → *cosa cambia* quando il disco è flash (capitolo 20: cancellazione, FTL, TRIM).

- **“Mi parli dello scheduling.”** Tre capitoli: *che cosa* si sta ottimizzando e in quale ambiente (capitolo 8: gli obiettivi, e il fatto che dipendono dall’ambiente) → *con quali algoritmi* (capitolo 9: batch e interattivi, con i numeri) → *che cosa cambia* con i thread e con più processori (capitolo 10: thread utente contro kernel, SMP, predilezione e bilanciamento).
- **“Che cosa c’è fra il mio programma e l’hardware?”** È la domanda d’apertura del corso, e la risposta è una pila: l’*astrazione* (capitolo 1: la macchina estesa e il gestore delle risorse) → il *meccanismo* che la rende possibile (capitolo 2: doppia modalità, system call, trap) → la *forma* che quella pila prende (capitolo 3: monolitico, a livelli, microkernel, moduli, macchine virtuali).
- **“Come si passa da un file di codice alla sua esecuzione?”** Dal processo (capitolo 4: fork ed exec, gli stati, le code) ai flussi dentro il processo (capitolo 5: i thread, che cosa condividono e che cosa no), fino agli strumenti con cui quei flussi si coordinano (capitolo 6).

## 24.4 Le domande-trappola

Sono le domande che iniziano con “perché”. Chi ha memorizzato risponde con la definizione; chi ha capito risponde con il motivo. Ognuna di queste ha la risposta estesa in un riquadro del suo capitolo.

1. **Perché si usa il DMA?** *Non* perché è più veloce: perché senza DMA ogni byte passerebbe attraverso il processore, occupandolo per tutta la durata del trasferimento (capitolo 2).
2. **Perché la struttura a livelli non è la migliore?** Perché è la *meno* efficiente: l’overhead delle chiamate nidificate. È la più pulita da progettare, non la più veloce (capitolo 3).
3. **Da “in attesa” si può passare direttamente a “in esecuzione”?** No, sempre attraverso “pronto”: lo sbloccarsi di un evento esterno dà il diritto di *competere* per la CPU, non di ottenerla (capitolo 4).
4. **Un programma utente può accedere direttamente al disco?** No: deve passare da una chiamata di sistema, che è l’unico modo per chiedere al kernel un privilegio (capitolo 2).
5. **Perché non basta mettere le due `take_fork` dentro un mutex?** Perché il filosofo resterebbe bloccato *dentro* la sezione critica aspettando la seconda forchetta, impedendo agli altri di entrare e di rilasciare: si trasformerebbe un deadlock in una serializzazione completa (capitolo 7).
6. **Perché il clock e non la seconda chance, se fanno la stessa cosa?** Per il costo di implementazione: la seconda chance deve spostare le pagine nella lista, il clock muove solo un indice (capitolo 13).
7. **Perché il copy-on-write invece di copiare subito?** Perché la maggior parte delle copie non viene mai modificata — l’esempio è la fork seguita da exec, dove la copia sarebbe gettata via immediatamente (capitolo 14).
8. **Perché aggiungere processi può peggiorare l’utilizzazione della CPU?** È il thrashing: i frame si spostano verso chi faulta di più (il processo appena arrivato, che non ha ancora il suo working set), togliendoli a chi lavorava bene, finché la CPU passa il tempo a servire page fault (capitolo 14).
9. **Perché la FAT ha problemi in caso di append?** Perché non esiste un “file” condiviso, esiste una catena: per condividere si duplica la catena, e allungandone una le due divergono (capitolo 16).

10. **Perché l'hard link non risolve l'accounting?** Perché è una struttura *corretta* per rispondere a “quando si può cancellare?” ma *inadatta* a rispondere a “a chi appartiene lo spazio?": la stessa informazione non basta per entrambe le domande (capitolo 16).
11. **Il journaling mette al sicuro i dati?** No: protegge i *metadati*. È il fraintendimento più comune su questo argomento (capitolo 17).
12. **Perché il RAID 3 usa un bit di ridondanza in meno del RAID 2?** Perché nei dischi la posizione del guasto si conosce già, quindi non serve una codifica che la localizzi (capitolo 19).
13. **Perché il RAID 5 sostituisce il RAID 4?** Perché sposta la parità, togliendo il collo di bottiglia del disco dedicato — a parità di spazio occupato (capitolo 19).
14. **Perché un SSD non è “un disco veloce”?** Perché ha regole di scrittura diverse: un blocco va cancellato prima di essere riscritto, e le cancellature sono limitate. Per questo lo scheduling del disco perde senso mentre il RAID sopravvive (capitolo 20).
15. **Perché si scrive `while` e non `if` prima di una `wait`?** Per la semantica di Mesa (*signal & continue*): fra la `signal` e la ripresa qualcun altro può cambiare lo stato, quindi la condizione va ricontrollata in un ciclo (capitolo 6).

## 24.5 Gli errori da non fare

I capitoli hanno ciascuno il proprio riquadro rosso con l'errore tipico. Messi in fila, sono la lista delle cose da non dire.

### Confusioni di vocabolario

- **Deadlock contro starvation:** nel primo c'è una attesa circolare e nessuno prosegue; nella seconda il sistema va avanti e qualcuno è semplicemente sfortunato nella selezione.
- **Frammentazione interna contro esterna:** la prima è spazio sprecato *dentro* una partizione, la seconda è spazio libero ma inutilizzabile perché sparso. Solo la seconda si cura con la compaction.
- **Chiamata di sistema contro chiamata di funzione:** la seconda resta in modalità utente e non dà privilegi.
- **fork contro exec:** il primo duplica il processo *corrente*, il secondo sostituisce l'immagine. Dopo `fork` il figlio esegue lo stesso codice del padre.
- **thread\_yield contro thread\_join:** il primo è un passo indietro, il secondo un'attesa che l'altro finisca.
- **Multiplexing nel tempo contro nello spazio:** la CPU è a turno (tempo), la memoria è divisa in porzioni contemporanee (spazio).
- **MB contro MiB:** un disco “da 1 TB” ha  $10^{12}$  byte, che sono 931,32 GiB. Negli esercizi del compito le dimensioni sono in potenze di 2 e il conto torna comodo, ma il criterio va saputo.

### Risposte che suonano bene e sono sbagliate

- “La struttura a livelli è la più efficiente perché è ordinata.” *È la meno efficiente.*
- “Il DMA si usa perché è più veloce.” *Si usa perché libera il processore.*

- “Disabilitare gli interrupt è una buona soluzione alla mutua esclusione.” *Non scala a più CPU ed è un’istruzione privilegiata.*
- “La memoria virtuale serve a usare più memoria di quella fisica.” *Serve a dare a ogni processo uno spazio proprio, contiguo e isolato; il resto è una conseguenza.*
- “SSTF è il migliore perché ha le prestazioni migliori.” *Sono due giudizi su cose diverse: la velocità media e la garanzia. SSTF affama.*
- “Il RAID 1 è il migliore perché non perde dati.” *Costa metà della capacità, il prezzo più alto di tutti i livelli.*
- “La FAT è una struttura per file.” *È una tabella con una voce per blocco del disco: la sua dimensione cresce con il disco, non con i file.*
- “LRU è la soluzione ottima.” *È eccellente ma costosa da implementare; OPT è il limite teorico, LRU lo approssima bene ma non è l’ottimo.*

#### L’errore che non è un errore

Attenzione a una cosa: al colloquio non si perde punti per un dettaglio dimenticato, si perdono per una **struttura assente**. Una risposta come “LRU usa il passato recente come stima del futuro” detta con calma, anche se non ricorda il numero dei fault, vale più di un elenco di algoritmi recitato in ordine alfabetico senza dire perché ciascuno esiste. Il colloquio non verifica quanto hai letto: verifica se hai capito che ogni algoritmo del corso nasce come *risposta a un problema del suo predecessore*.

## 24.6 Gli esempi da saper rifare

All’orale capita che il docente chieda di rifare un esempio delle slide. Questi sono quelli che vale la pena avere in mano, con il numero finale — tutti presenti nei rispettivi capitoli e prodotti dal programma di verifica.

#### I sei numeri da ricordare

- **EAT con la TLB** (capitolo 12): accesso in memoria 100 ns, ricerca nella TLB 20 ns, hit ratio 80% → **140 ns**. Sapere rifare questo conto vale più che ricordare la formula.
- **Sostituzione delle pagine** (capitolo 13): sull’esempio delle slide, OPT fa **9** fault, FIFO **15**, LRU **12**. È il confronto che rende visibile quanto LRU si avvicini all’ottimo.
- **Anomalia di Belady** (capitolo 13): con FIFO si passa da **9** fault con 3 frame a **10** con 4 — più memoria, più fault. È l’unico caso in cui i numeri fanno notizia.
- **Effetto convoglio** (capitolo 9): un processo da 24 unità davanti a due da 3 dà un tempo medio di attesa di **17**, pur essendo il lavoro utile totale solo 30 unità.
- **Scheduling del disco** (capitolo 18): sulla coda dell’esempio, FCFS **640** cilindri di spostamento, SSTF **236**, scansione **299**, scansione circolare **350**. Saper ricostruire le quattro tabelle delle distanze significa saper rispondere a qualunque variante.
- **Ricostruzione dopo un guasto** (capitolo 19): con la parità si ricostruisce il disco mancante bit a bit; il conto è mostrato per esteso nel capitolo.

**Come si usa questo capitolo nelle ultime settimane**

Il colloquio si prepara **a voce**, non leggendo. Il metodo che funziona: prendi un capitolo, copri il riquadro viola, apri l'indice qui sopra, scegli una delle domande e rispondi ad alta voce per due minuti. Poi rileggi il riquadro e conta quante cose hai dimenticato.

Dopo i primi giri, smetti di preparare capitoli e comincia a preparare *domande che attraversano i capitoli*: quelle della quarta sezione. Sono le uniche che all'orale non si possono evitare, e sono anche l'ultima cosa che si impara.